

개발자 가이드

9.2.2.170

이 문서에 잘못된 정보가 있을 수 있습니다. 투비소프트는 이 문서가 제공하는 정보의 정확성을 유지하기 위해 노력하고 특별한 언급 없이 이 문서를 지속적으로 변경하고 보완할 것입니다. 그러나 이 문서에 잘못된 정보가 포함되어 있지 않다는 것을 보증하지 않습니다. 이 문서에 기술된 정보로 인해 발생할 수 있는 직접적인 또는 간접적인 손해, 데이터, 프로그램, 기타 무형의 재산에 관한 손실, 사용 이익의 손실 등에 대해 비록 이와 같은 손해 가능성에 대해 사전에 알고 있었다고 해도 손해 배상 등 기타 책임을 지지 않습니다.

사용자는 본 문서를 구입하거나, 전자 문서로 내려 받거나, 사용을 시작함으로써, 여기에 명시된 내용을 이해하며, 이에 동의하는 것으로 간주합니다.

각 회사의 제품명을 포함한 각 상표는 각 개발사의 등록 상표이며 특허법과 저작권법 등에 의해 보호를 받고 있습니다. 따라서 본 문서에 포함된 기타 모든 제품들과 회사 이름은 각각 해당 소유주의 상표로서 참조용으로만 사용됩니다.

발행처 | (주)투비소프트

발행일 | 2025/03/07

주소 | (06083) 서울시 강남구 봉은사로 617 인탑스빌딩 2-5층

전화 | 02-2140-7700

홈페이지 | www.tobesoft.com

고객지원센터 | support.tobesoft.co.kr

제품기술문의 | 1588-7895 (오전 10시부터 오후 5시까지)

유지보수정책 | 유지보수기간과 범위는 제품 라이선스 계약에 따라 다릅니다.

변경 이력

버전	변경일	내용
9.2.2.170	2017-05-30	Position & Anchor 항목을 추가했습니다.

차례

저작권 및 면책조항	ii
변경 이력	iii
차례	iv
1. XPLATFORM 응용프로그램의 구조	1
1.1 XPLATFORM 응용프로그램 파일들	1
1.1.1 ADL File	2
1.1.2 Type Definition File	4
1.1.3 Global Variable File	5
1.1.4 Theme File / Style File	6
1.1.5 FDL File	6
1.2 파일 로딩 순서	6
1.2.1 응용 프로그램 로딩 순서	7
1.2.2 Form 로딩 순서	7
1.3 주요 화면 요소	8
1.3.1 응용프로그램 모델	8
1.3.2 Frame을 이용한 화면배치	8
1.3.3 Form의 구성	11
1.4 서버와의 Data연동	12
1.4.1 X-API와 XPLATFORM Runtime	12
2. XML File들의 구조	13
2.1 표기법	13
2.2 ADL XML Format	14
2.3 FDL XML Format	16
2.4 Type Definition XML Format	17
2.5 Global Variable XML Format	17
3. 애니메이션 기능	18
3.1 애니메이션 기능의 개요	18
3.1.1 애니메이션 기능의 4가지 요소	18

3.1.2	간단한 애니메이션 기능 구현	19
3.1.3	애니메이션 적용 대상	24
3.2	애니메이션의 이해	24
3.2.1	애니메이션 용어정의	24
3.2.2	interpolation	26
3.3	Transition Animation	28
3.3.1	전환 전후 이미지들의 저장 및 실행	28
3.3.2	Transition Animation의 예	29
3.3.3	Transition Animation의 Property	33
3.3.4	Transition Animation의 Method	34
3.4	Property Animation	34
3.4.1	Transition Animation과 Property Animation의 차이점	34
3.4.2	Property Animation의 예	35
3.4.3	Property Animation의 Property	43
3.4.4	Property Animation의 Method	43
3.5	Composite Animation	43
3.5.1	Composite Animation의 예	44
3.5.2	Composite Animation의 Property	52
3.5.3	Composite Animation의 Method	53
3.6	향상된 Animation 기능의 사용	53
3.6.1	자동 Trigger사용하기	53
3.6.2	User Interpolation만들기	55
3.6.3	그 밖에 Animation적용	58
4.	Frame Tree	60
4.1	표기법	60
4.1.1	box	60
4.1.2	연결선	60
4.1.3	설명	60
4.1.4	용어정의	61
4.1.5	공통으로 사용하는 Syntax	61
4.2	Form	61
4.2.1	관계도	61
4.2.2	Script 예시 화면	63
4.2.3	component / invisible object / bind 의 Script 접근	63
4.2.4	container component에 Script 접근	64
4.2.5	form을 연결한 container component에 Script 접근	65
4.2.6	parent의 Script 사용	65
4.2.7	container component의 element사용	65
4.3	Single Frame 형식의 Application	66

4.3.1	관계도	66
4.3.2	Script 예시 화면	67
4.3.3	application에서 form의 script 접근	67
4.3.4	form에서 application의 script 접근	68
4.4	Multi Frame형식의 Application	68
4.4.1	관계도	69
4.4.2	Script 예시 화면	70
4.4.3	application에서 form의 script 접근	70
4.5	Modal, Modalesse 형식의 Form	71
4.5.1	관계도	71
4.5.2	nested ChildFrame에서 Script 사용법	72
4.5.3	modal/modalesse ChildFrame에서 Script 사용법	73
5.	Style(CSS)과 Theme의 관리	74
5.1	Style / Theme 개요	74
5.1.1	Style의 정의 및 적용	74
5.1.2	Theme의 정의	77
5.1.3	Style/Theme의 등록 및 적용	78
5.1.4	Style과 Theme의 적용 대상	78
5.2	Style의 적용	79
5.2.1	Style의 생성	79
5.2.2	Style의 편집	83
5.2.3	Style의 적용	88
5.2.4	Style Class의 적용	92
5.3	Theme의 적용	94
5.3.1	Theme의 생성	95
5.3.2	Theme의 편집	96
5.3.3	Theme의 적용	98
5.3.4	Deploy Theme	102
6.	Widget	105
6.1	Widget 개요	105
6.1.1	Widget의 구성	106
6.1.2	Widget Form의 제약사항	106
6.1.3	Widget와 Layered기능	106
6.1.4	Widget 만드는 방법	107
6.2	Widget Project로 만들기	108
6.2.1	Widget 프로젝트의 생성	108
6.2.2	background 이미지 등록	109
6.2.3	Widget Form의 등록	110
6.2.4	Widget의 실행	114

6.2.5 응용프로그램에 등록된 Widget 확인	115
6.3 일반 Project에서 Widget 만들기	117
6.4 Script로 Widget 만들기	117
6.4.1 동적 Widget생성을 위한 준비	117
6.4.2 Script로 동적 Widget 생성	117
6.4.3 Widget을 화면에 출력합니다.	118
6.4.4 Widget Object를 삭제합니다.	118
6.4.5 Script로 Widget에 접근 하기	119
7. Event Flow	120
7.1 Event의 개요	120
7.1.1 Event 처리 대상	120
7.1.2 Parent-Child간 Event 전이	121
7.2 표기법	122
7.2.1 Event Flow에서 사용하는 도형	122
7.3 Key event Flow	123
7.3.1 KeyPush module 정의	123
7.3.2 ApplicationMenu,Button,Calendar,CheckBox,Combo,Div,Edit,Form,ImageViewer,ListBox,MaskEdit,Menu,PopupDiv,PopupMenu,Radio,Spin,Tab,TabPage,TextArea	124
7.3.3 Grid	124
7.4 Edit event Flow	125
7.4.1 Edit module 정의	125
7.4.2 Edit, MaskEdit, TextArea, Combo, Calendar, Spin	126
7.4.3 Grid	126
7.5 Edit중 Focus Out Flow	127
7.5.1 Calendar/Combo/Spin, Edit/MaskEdit/TextArea	127
7.6 Mouse Move event Flow	128
7.6.1 Edit/MaskEdit/TextArea, Calendar/Combo/Spin, Tab/TabPage, ListBox/Radio, PopupMenu/Menu, Button, CheckBox, GraphicPath, GroupBox, ImageViewer, Panel, ProgressBar, Shape, Splitter, Static, Div, Form, Grid, PopupDiv	128
7.7 Mouse Click event Flow	129
7.7.1 MouseClick Module	129
7.7.2 MouseLbuttonDown Module	130
7.7.3 Button,CheckBox,Form,GraphicPath,ImageViewer,Panel,PopupDiv,ProgressBar,Shape,Static, Div/TabPage	131
7.7.4 Edit/MaskEdit/TextArea, Menu	131
7.7.5 Calendar	132
7.7.6 Combo	134
7.7.7 Spin	135
7.7.8 Tab	136

7.7.9	ListBox	137
7.7.10	Radio	138
7.7.11	Splitter	139
7.7.12	PopupDiv / PopupMenu	140
7.7.13	Grid	141
7.8	Drag event Flow	142
7.8.1	ApplicationMenu, Button, Calendar, CheckBox, Combo, Div, Edit, Form, Grid, ImageViewer, ListBox, MaskEdit, Menu, PopupDiv, PopupMenu, ProgressBar, Radio, Sign, Spin, Tab, TabPage, TextArea	142
7.9	Component Move event Flow	143
7.9.1	ApplicationMenu, Button, Calendar, Chart, CheckBox, Combo, Div, Edit, Form, GraphicPath, Grid, GroupBox, ImageViewer, ListBox, MaskEdit, Menu, Panel, PopupDiv, PopupMenu, ProgressBar, Radio, ScrollBar, Shape, Sign, Spin, Splitter, Static, Tab, TabPage, TextArea	143
7.10	Component Resize event Flow	143
7.10.1	ApplicationMenu, Button, Calendar, CheckBox, Combo, Div, Edit, Form, GraphicPath, Grid, GroupBox, ImageViewer, ListBox, MaskEdit, Menu, Panel, PopupDiv, PopupMenu, ProgressBar, Radio, ScrollBar, Shape, Sign, Spin, Splitter, Static, Tab, TabPage, TextArea	144
7.11	Dataset, Filtered Dataset Flow	145
7.12	Form event Flow	146
7.12.1	Form load / unload	146
7.13	Scroll event Flow	146
7.13.1	Div, Form, Grid, ListBox, PopupDiv, TabPage, TextArea	147
8.	Customized Object	148
8.1	Customized Object 란?	148
8.1.1	Binary Object란?	149
8.1.2	Customized Object와 Binary Object의 관계	149
8.1.3	Customized Object들의 비교	151
8.2	Composite 컴포넌트	151
8.2.1	Composite 컴포넌트 개념	152
8.2.2	Composite 컴포넌트 파일 만들기	153
8.2.3	Composite 컴포넌트 TypeDefinition에 등록하기	153
8.2.4	Composite 컴포넌트 사용하기	155
8.2.5	Composite 컴포넌트 예제	155
8.3	Form Inheritance	157
8.3.1	Form Inheritance 개념	157
8.3.2	Form을 TypeDefinition에 등록하기	159
8.3.3	상속 받아서 Form 파일 만들기	160
8.3.4	상속받은 form 사용하기	162
8.3.5	Function Override	163
8.3.6	Form Inheritance 예제	163

9. MLM(V9.2추가)	167
9.1 Project 생성 및 수정	168
9.1.1 Project생성	168
9.1.2 Screen정보의 수정	170
9.1.3 Layout Template등록	171
9.2 Form생성	172
9.3 Layout	175
9.3.1 Layout Tab	175
9.3.2 Layout 정보 수정	177
9.3.3 Properties Window	179
9.4 Sub Layout	180
9.4.1 Default Layout에서의 편집	181
9.4.2 Sub Layout에서의 편집	182
9.5 Step	186
9.5.1 Positionstep Property	187
9.5.2 컴포넌트 이동 및 Position 변환	188
9.6 InitValue	189
9.7 실행	190
9.8 XML 추가/변경 사항	191
9.8.1 XPRJ	191
9.8.2 XADL	192
9.8.3 XFDL	192
10. Position	195
10.1 positiontype 속성	195
10.2 Position & Anchor	195
10.2.1 default	196
10.2.2 left top 고정	196
10.2.3 속성값에 따라 변경되는 요소	197
10.3 Position2 (V9.2 추가)	198
10.3.1 좌표계 설명	199
10.3.2 컴포넌트의 Position2 설정	199
10.3.3 Form Design	201
Tracker	201
Ruler / Dot Grid	202
컴포넌트 Resize Info	202
Position Editor	203
10.3.4 New Project Wizard	204
10.3.5 Position2관련 주의사항	205

1.

XPLATFORM 응용프로그램의 구조

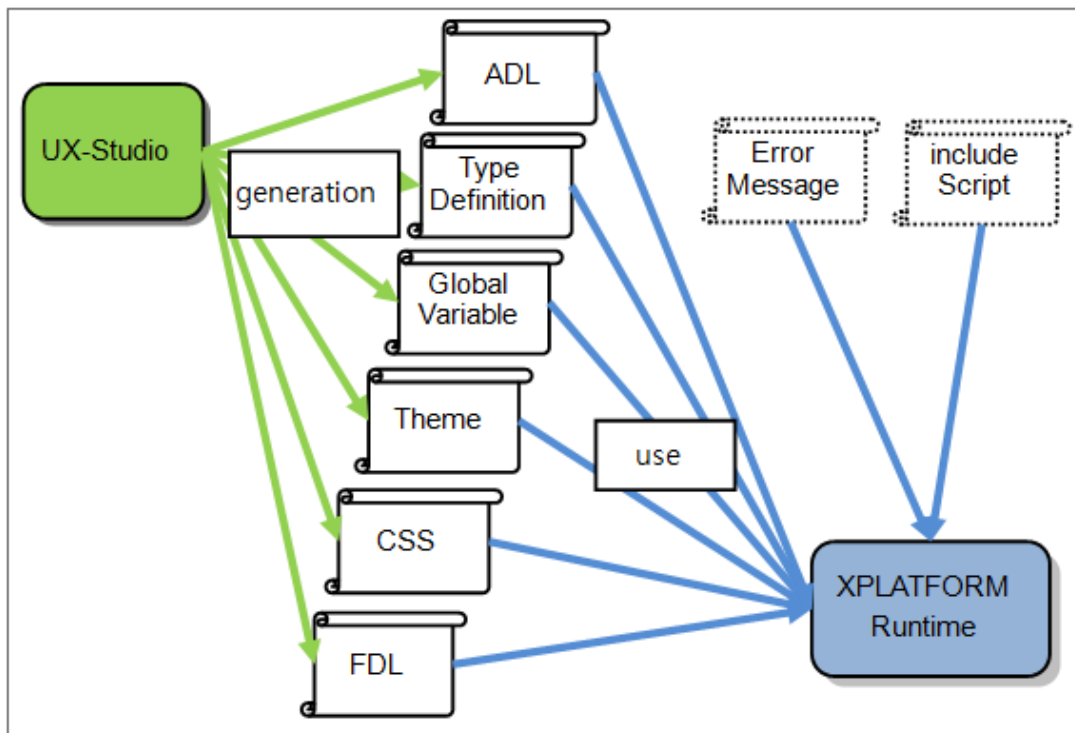
XPLATFORM Engine의 설치가 완료된 후, XPLATFORM Engine은 응용프로그램 수행을 위하여 개발자가 개발한 파일을 차례로 로딩합니다.

작동원리에 배포과정도 포함되지만, 여기서는 배포에 대한 부분은 언급하지 않습니다.

1.1 XPLATFORM 응용프로그램 파일들

개발자가 단 하나의 화면만 개발했다 하더라도, 그 화면을 응용프로그램으로 실행하기 위하여 추가적인 파일들을 필요로 합니다. 응용프로그램 실행을 위해 XPLATFORM Runtime이 필요로 하는 파일들은 UX-Studio가 자동으로 관리합니다.

다음은 XPLATFORM Runtime이 응용프로그램 수행을 위하여 필요한 파일들입니다.



XPLATFORM Runtime은 최초에 접근하는 ADL 파일로부터 나머지 파일들의 정보를 얻습니다.

1.1.1 ADL File

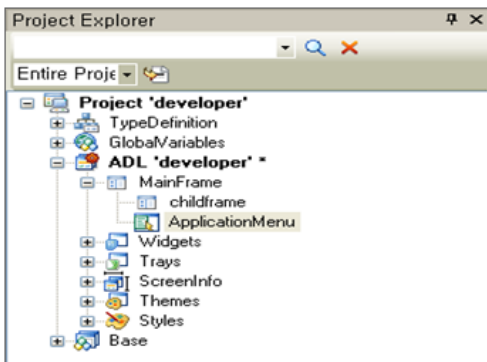
ADL파일은 Application Definition Language의 약자로, Application구성에 필요한 모든 요소들을 정의합니다.

ADL파일이 정의하는 주요 요소들은 아래와 같습니다.

요소	설명
Application Model	- 응용프로그램의 UI기본 틀을 정합니다. - Multi-Frame, Single Frame, Widget의 3가지 모델이 있습니다. - 이 모델들의 구성을 정의합니다.
기본 화면 배치	- 응용프로그램 화면의 가장 큰 틀인 MainFrame내에 Form들을 어떻게 배치할 것인지를 정의합니다.
메뉴	- 응용프로그램의 메뉴구성을 정의합니다.
Object 배포 (link)	- 응용프로그램 실행에 필요한 Object들(Dataset, Grid등)을 나열하고 버전 정보를 등록합니다. - 이 정보는 Type Definition 파일에 저장되고, ADL에서는 url 링크 정보만 관리합니다.
Theme 구성 (link)	- 응용프로그램의 화면에 보여지는 디자인 형식을 정의합니다. - Theme는 Image와 CSS의 조합으로 구성됩니다. - 이 정보는 Theme 파일에 저장되고, ADL에서는 url 링크 정보만 관리합니다.
CSS (link)	- 응용프로그램의 CSS를 정의합니다. - Theme에 등록한 CSS외에 추가로 CSS를 등록할 때 사용합니다. - 이 정보는 CSS 파일에 저장되고, ADL에서는 url 링크 정보만 관리합니다.
Global Variable (link)	- 응용프로그램내의 여러 화면들이 사용하는 공통 변수들을 등록합니다. - 공통 변수의 종류는 variant변수, Dataset등을 포함한 Object들과 Image가 있습니다. - 이 정보는 Global Variable 파일에 저장되고, ADL에서는 url 링크 정보만 관리합니다.
Global Script	- 응용프로그램이 사용하는 공통 Function들을 정의합니다. - 응용프로그램 자체의 Even Function을 구현합니다.

ADL파일은 UX-Studio에서 자동으로 관리합니다.

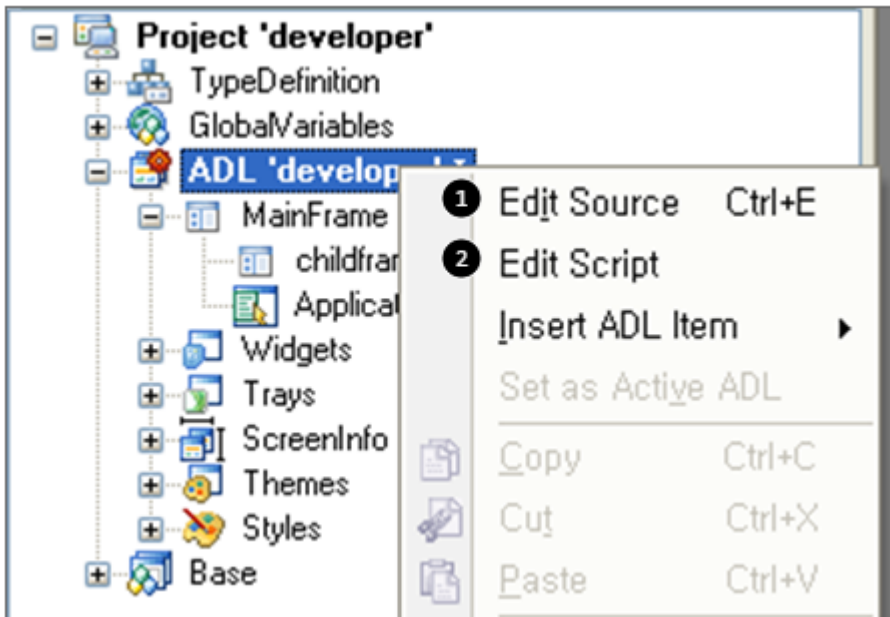
UX-Studio상의 Project Explorer창에서 다루는 내용들이 대부분 ADL파일에 저장되는 내용입니다. 다음은 UX-Studio에서 Project Explorer창의 화면입니다.



아이템	설명
Project 'developer'	프로젝트명입니다. 하나의 프로젝트에는 여러 개의 ADL이 등록될 수 있습니다.
TypeDefinition	Type Definition 정보를 저장합니다.
GlobalVariables	Global Variable 정보를 저장합니다.
ADL 'developer'	ADL명 입니다.
childframe	화면 배치입니다. ChildFrame 하나만 등록되었으므로 Single Frame 모델입니다.
ApplicationMenu	메뉴 구성을 저장합니다.
Widgets	Widget 구성을 저장합니다.
Trays	TrayIcon 구성을 저장합니다.
ScreenInfo	Screen 구성을 저장합니다. (V9.2 추가)
Themes	Theme 구성을 저장합니다.
Styles	CSS 구성을 저장합니다.
Base	Form들을 정의하고 등록합니다.

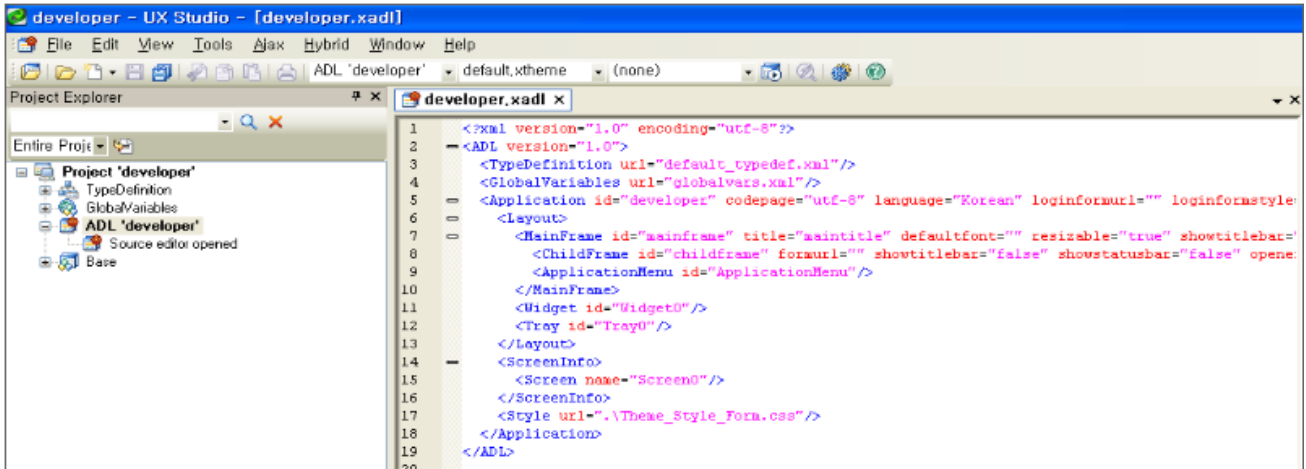
Project Explorer에서 정의한 내용들은 하나의 파일에만 저장되는 것이 아니라, 여러 개의 파일로 분리하여 저장됩니다. 그 중에 Root에 해당하는 파일이 ADL파일 입니다.

ADL 파일은 Project Explorer창 외에 Edit창에서 직접 편집이 가능합니다.



	메뉴	설명
1	Edit Source	ADL에서 마우스 오른쪽 버튼을 클릭한 후, [Edit Source] 메뉴를 클릭하면 ADL 내용을 편집할 수 있습니다.
2	Edit Script	ADL에서 마우스 오른쪽 버튼을 클릭한 후, [Edit Script] 메뉴를 클릭하면 ADL의 Global Script를 편집할 수 있습니다.

다음은 ADL의 내용을 편집하는 화면입니다.



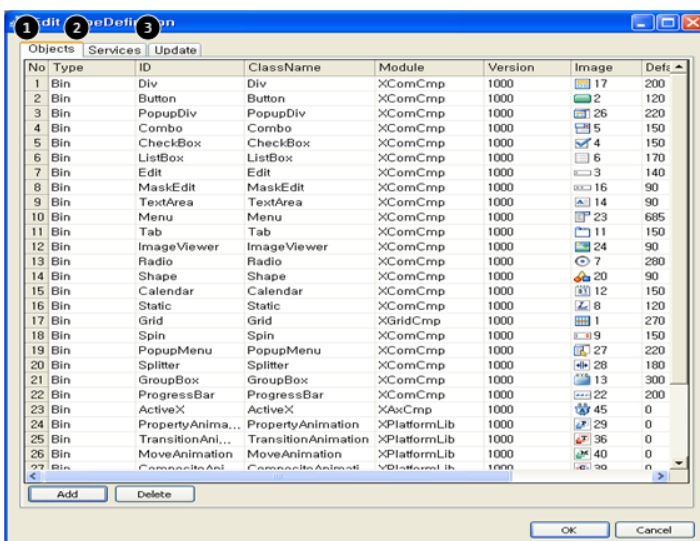
1.1.2 Type Definition File

Type Definition 파일은 XPLATFORM 응용프로그램이 필요로 모듈들의 배포 정보를 갖고 있습니다. 필요 모듈로는 Object들과 응용프로그램들이 있습니다.

Type Definition 파일이 정의하는 주요 요소들은 아래와 같습니다.

요소	설명
Object배포정보	- 응용프로그램 수행에 필요한 Object들의 정보를 담고 있습니다. - 여기에 등록된 Object들은 배포의 대상이 됩니다. - XPLATFORM이 제공하는 Object들 외에도 사용자가 개발한 Object들과 Protocol Adaptor도 그 대상이 됩니다.
Object배포URL	- 배포 대상이 되는 Object들의 URL을 OS별로 지정합니다.
응용프로그램 배포 URL	- 개발자가 개발한 응용프로그램들의 URL위치 및 Cash의 종류 지정합니다.

다음은 Type Definition 편집창의 화면입니다.



	메뉴	설명
1	Objects	Object 배포 정보
2	Services	애플리케이션 배포 URL
3	Update	Object 배포 URL

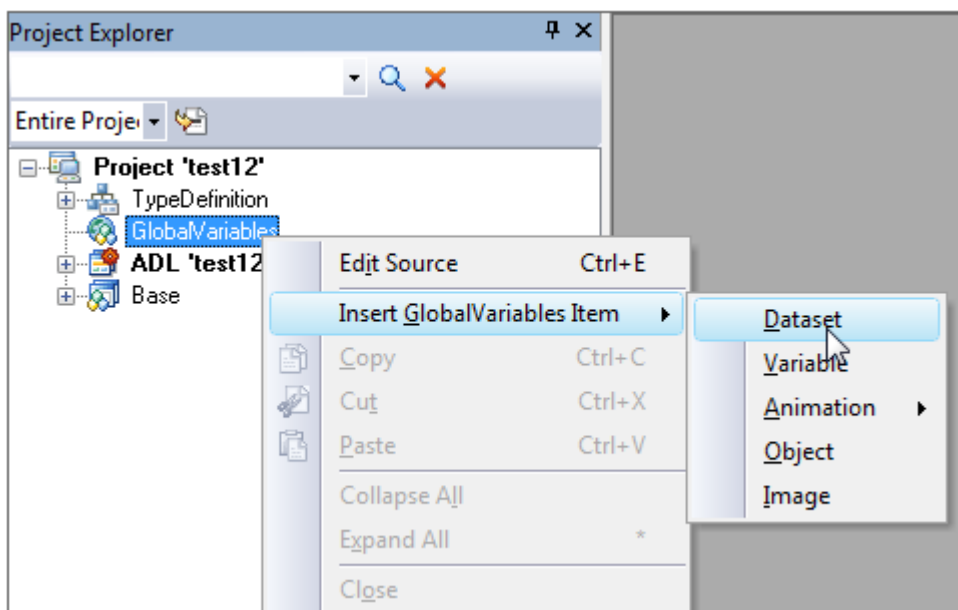
1.1.3 Global Variable File

Global Variable 파일은 XPLATFORM 응용프로그램의 여러 화면들이 사용하는 공통 변수들을 등록합니다. 공통 변수로 등록할 수 있는 변수의 종류로는 Variant 변수, Dataset등을 포함한 Object들과 Image가 있습니다.

Global Variable 파일이 정의하는 변수 들은 아래와 같습니다.

요소	설명
Dataset 변수	Dataset을 등록하고 element들을 설정합니다.
Variable 변수	Variant Type의 변수를 등록합니다.
Animation 변수	Animation Object를 등록하고 element들을 설정합니다.
Object 변수	Dataset, Animation이외의 invisible Object들을 등록하고, 해당 Object의 element들을 설정합니다.
Image 변수	Image들을 등록합니다.

다음은 Global Variable중 Dataset을 등록하는 화면입니다. Project Explorer 창에서 [Project > GlobalVariable > Insert GlobalVariables Item > Dataset] 메뉴를 선택해 Dataset을 등록합니다.



위의 화면에서 [Project > GlobalVariable > Edit Source]를 선택하여 GlobalVariable을 직접 편집을 할 수도 있습니다.

1.1.4 Theme File / Style File

XPLATFORM에서 Style과 Theme란 화면상에 보이는 UI요소들의 디자인을 정의하는 것을 의미합니다. 버튼을 예로 들면, 투명도, 폰트, 칼라, 그림자, 번짐과 기울임 등의 효과를 버튼 컴포넌트에게 주는 것을 의미합니다.

Theme와 Style의 차이점은 Image의 포함 여부입니다. Theme는 Style에 Image를 추가한 형태입니다. 그러므로 Image를 사용하는 Theme는 Style에 비해 좀 더 다양한 UI 디자인을 할 수 있습니다.

1.1.5 FDL File

FDL 파일은 사용자 화면 Form을 정의합니다. 사용자에게 보여지는 화면 배치뿐만 아니라 UI Logic수행을 위한 Script를 포함한 다양한 정보를 담고 있습니다. FDL파일은 ADL파일과 더불어 XPLATFORM 응용프로그램에서 가장 중요한 파일입니다.

FDL파일이 정의하는 주요 요소는 다음과 같습니다.

요소	설명
컴포넌트 배치	- Visible Object인 컴포넌트를 배치하는 정보를 갖고 있습니다. - 컴포넌트의 Property 및 Event를 설정합니다.
Object의 설정	- Dataset, Animation등 Invisible Object의 Property 및 Event를 설정합니다.
Script	- Form을 포함한 모든 Object들의 Event Function을 스크립트로 작성합니다. - Event Function외에도 필요한 Function들을 스크립트로 작성합니다.
Style	- Form에서 사용하는 Style을 등록합니다. 여기서 사용하는 Style은 ADL에서 등록한 Style보다 우선권을 갖습니다.
BindItem	- Form, 컴포넌트, Invisible Object들과 Dataset을 연결하는 BindItem Object를 설정합니다.

1.2 파일 로딩 순서

XPLATFORM이 응용프로그램을 실행할 때, 개발자가 개발한 파일들을 차례로 로딩합니다. 개발자는 단 하나의 화면만 개발했다 하더라도, 그 화면을 응용프로그램으로 실행시키기 위하여 UX-Studio는 많은 파일들을 생성합니다. 여기서는 그 파일들을 로딩하는 순서를 설명합니다.

1.2.1 응용 프로그램 로딩 순서

응용 프로그램을 실행하는 주체는 XPLATFORM Runtime입니다. XPLATFORM Runtime은 아래의 순서대로 파일들을 로딩하여 응용프로그램을 실행합니다.

순서	로딩 파일	설명
1	Error File	Error 파일을 로딩하여 이후에 작업에 발생하는 예외상황에 대한 Error 메시지를 준비합니다. 개발자가 별도로 지정하지 않으면, XPLATFORM Engine의 기본 파일을 사용합니다.
2	ADL File (xadl file)	응용프로그램의 기본 구조를 정의한 파일입니다. 이 파일을 먼저 로드 해야만 나머지 로딩파일들의 위치를 알 수 있습니다.
3	Type Definition File	응용프로그램의 환경정보를 갖고 있는 파일입니다. 응용프로그램이 사용하는 Object들의 정보, upgrade정보와 서비스 url정보를 담고 있습니다.
4	Global Variable File	응용프로그램이 사용하는 범용 변수들의 갖고 있습니다.
5	Theme File	응용프로그램이 사용하는 Theme 파일입니다.
6	CCS File	Theme외에 개발자가 추가로 등록한 CSS 파일입니다. CSS 파일은 Theme 파일보다 적용 우선순위가 높습니다.
7	Script include File	ADL에 등록된 Script에서 다른 Script 파일을 include했을 때, 해당 Script 파일을 의미합니다.
8	Form File	ADL의 MainFrame에서 사용하는 Form 파일입니다. 즉, 첫 화면을 보여주기 위한 Form파일입니다.

1.2.2 Form 로딩 순서

응용 프로그램 로딩의 마지막 단계인 Form 로딩을 다시 세분화하면 다음과 같습니다.

순서	로딩 파일	설명
1	Form File (xfdl file)	응용프로그램의 실질적인 화면UI인 폼을 정의하는 파일입니다.
2	Type Definition File	ADL로딩(응용프로그램의 로딩)시에 사용한 Type Definition파일을 재사용하거나, 별도의 Type Definition 파일을 연결할 수 있습니다. 즉, 특정 Form만의 Type Definition 파일을 연결할 수 있습니다. 일반적으로는 ADL의 Type Definition 파일을 그대로 사용합니다.
3	CSS File	Form만의 CSS를 별도로 사용할 때 사용합니다.
4	Object연관 File들	컴포넌트 또는 Invisible Object에서 필요로 하는 파일 (예 image, subform url, buffer, dataset file정보 등) 들 입니다.
5	Script include File	Form에 등록된 스크립트에서 다른 스크립트 파일을 include했을 때, 해당 스크립트 파일을 의미합니다.

1.3 주요 화면 요소

XPLATFORM Runtime은 고객의 화면에 응용프로그램 UI를 출력하기 위하여 개발자가 개발한 파일들을 차례로 로딩합니다.

1.3.1 응용프로그램 모델

XPLATFORM Runtime은 응용프로그램 화면 출력을 위하여 맨 처음으로 응용프로그램의 모델을 결정합니다. 응용 프로그램 모델이란 UI화면의 큰 틀을 의미합니다. 과거에는 MDI, SDI와 같은 용어로 큰 틀의 개념을 정의했었습니다. XPLATFORM은 고객의 다양한 요구사항을 수용할 수 있도록 보다 다양한 형태의 모델을 제시합니다.

XPLATFORM은 UI의 큰 틀의 정의를 보다 다양하게 할 수 있도록 Frame, Widget과 Tray의 3가지 기능을 제공합니다.

- [Frame 기능]
 - Frame은 일반 응용프로그램의 메인 화면 역할을 하고, Single Frame과 Multi Frame으로 분류합니다.
 - Frame은 하위 Frame을 가질 수 있으며, 최하위 Frame을 ChildFrame이라고 하고 최상위 Frame을 MainForm이라고 합니다.
 - ChildFrame은 더 이상 하위 Frame을 가질 수 없으며 Form만을 가질 수 있습니다.
- [Widget 기능]
 - Widget은 Frame처럼 여러 개의 Form들을 하나의 큰 틀에 배치하는 것이 아니고, 각각의 화면 Form들을 독립적으로 실행시켜서 그 배치를 사용자가 원하는 위치에 마음대로 배치할 수 있습니다.
 - Widget은 ChildFrame역할을 하므로 Form을 바로 가질 수 있습니다. 즉, 한 개의 Widget은 한 개의 Form을 갖습니다.
- [Tray 기능]
 - Tray는 Frame, Widget등의 화면들을 제어할 수 있는 TrayIcon을 제공합니다.

이 3가지 모델들을 적절히 조합하여, 화면 UI의 큰 틀을 조합할 수 있습니다. 즉, Frame기능을 사용할 때는 Widget기능을 못 쓰는 것이 아니라 함께 사용할 수 있습니다.

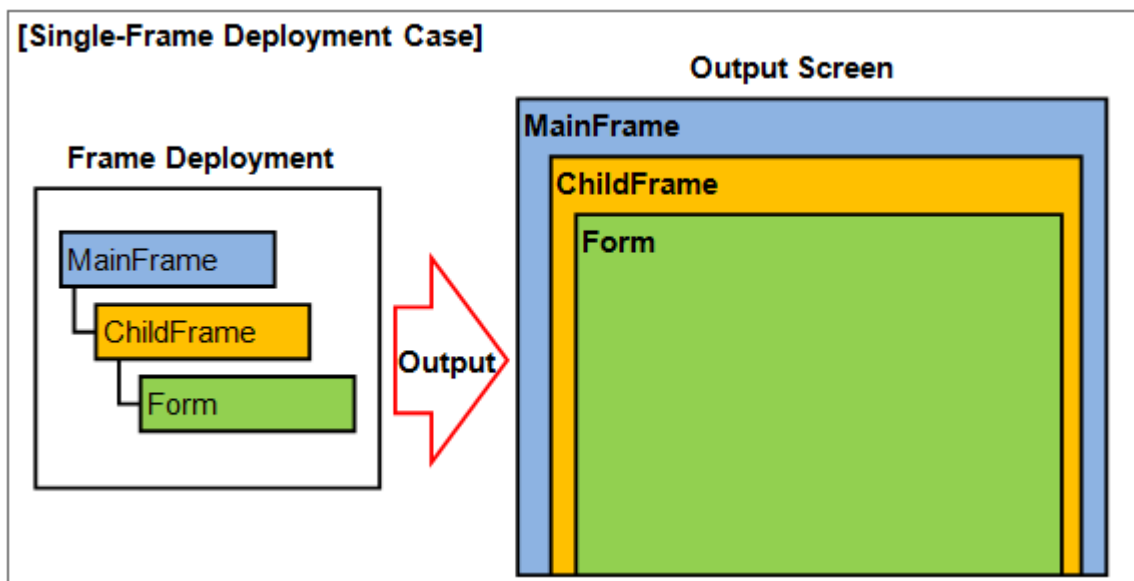
1.3.2 Frame을 이용한 화면배치

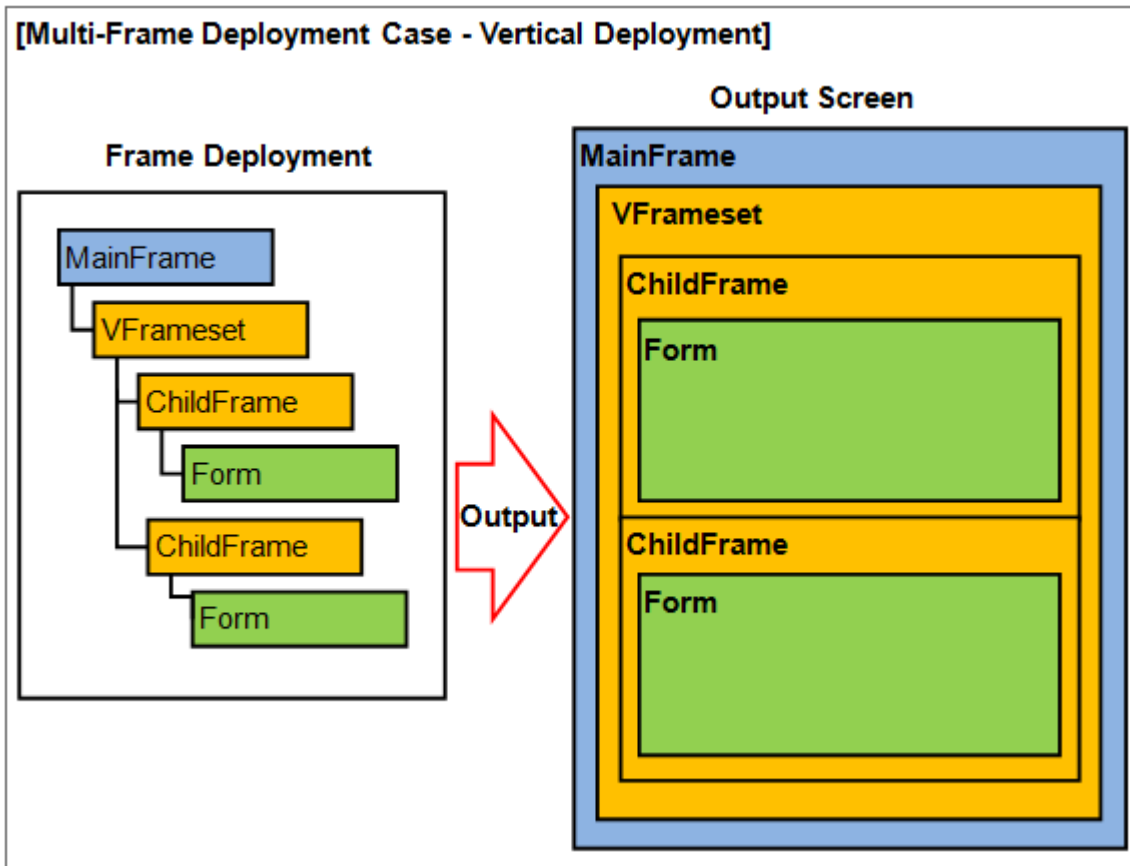
XPLATFORM 응용프로그램은 아무리 단순한 화면이라 할지라도 화면출력을 위해 Frame이 있어야만 합니다. 즉, Frame은 Form을 배치하는 판의 역할을 합니다. 그러나, Widget은 자체가 하나의 Frame역할을 하므로 Frame없이 Form을 화면에 출력합니다.

Frame은 크게 MainForm, Frameset, ChildFrame의 3가지 종류로 나뉘어 집니다.

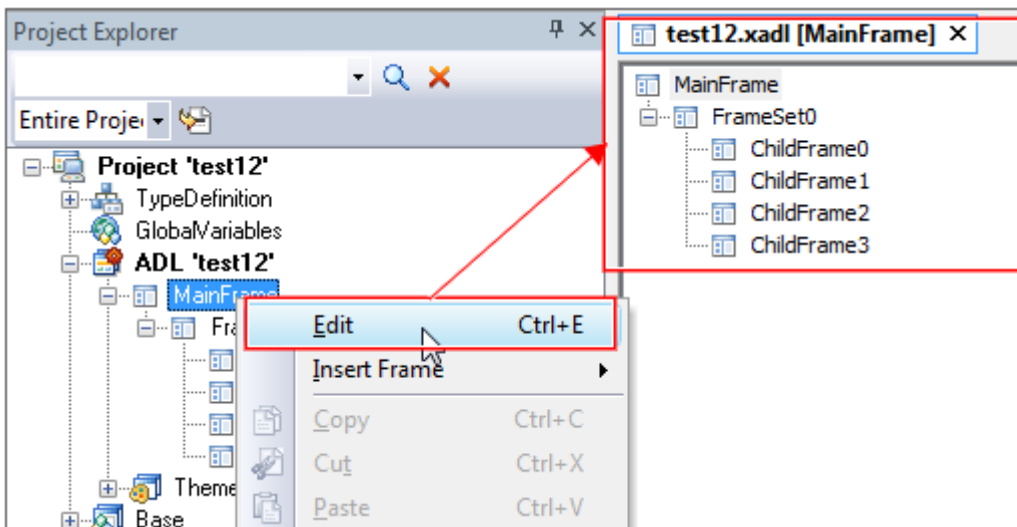
Frame 종류	Frame	설명
Root Frame	MainFrame	- 최상위 Frame입니다. 즉, Root Frame에 해당합니다. - 하위로 Frameset과 ChildFrame을 가질 수 있습니다.
Node Frame	Frameset	- 특별한 형태 없이 하위 Frame을 배치합니다. 하위 Frame들이 총계형태로 겹쳐서 배치됩니다. - 하위로 NodeFrame을 가질 수 있습니다. - 하위로 ChildFrame을 가질 수 있습니다.
	VFrameset	- 세로(vertical)형태로 하위 Frame을 배치합니다. - 하위로 NodeFrame을 가질 수 있습니다. - 하위로 ChildFrame을 가질 수 있습니다.
	HFrameset	- 가로(Horizontal)형태로 하위 Frame을 배치합니다. - 하위로 NodeFrame을 가질 수 있습니다. - 하위로 ChildFrame을 가질 수 있습니다.
	TileFrameset	- 바둑판 형태로 하위 Frame을 배치합니다. - 하위로 NodeFrame을 가질 수 있습니다. - 하위로 ChildFrame을 가질 수 있습니다.
	Tab Frame	- Multi Tab의 형태로 Frame을 배치합니다. - 하위로 Tab Frame Page만을 가질 수 있습니다.
Terminal Frame	ChildFrame	- 하위로 어떤 Frame도 가질 수 없습니다. - 하위로 하나의 Form만 가질 수 있습니다.
	Tab Frame Page	- 하위로 어떤 Frame도 가질 수 없습니다. - 하위로 하나의 Form만 가질 수 있습니다.

간략하게 Frame의 관계를 그림으로 표현하면 아래와 같습니다.

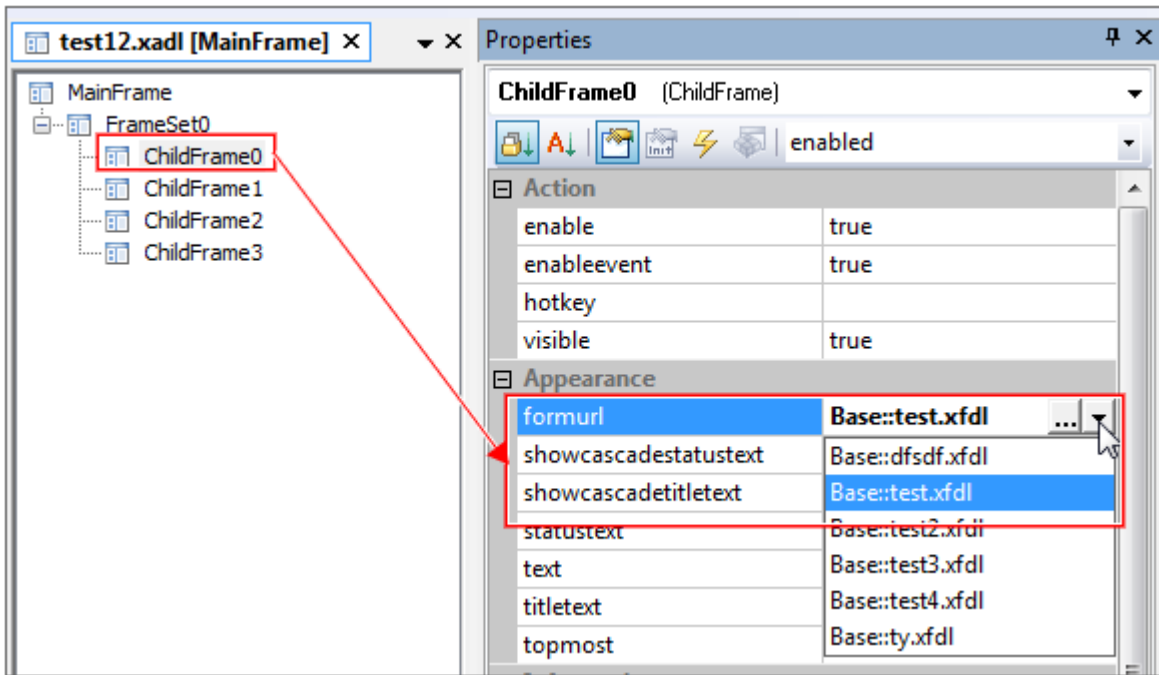




UX-Studio상에서는 Project Explorer를 통해 Frame을 편집합니다. [Project > ADL > MainFrame > Edit]메뉴를 선택하고 Frame 편집창을 실행합니다.



생성한 ChildFrame에 Form을 등록합니다. ChildFrame 선택 후 Properties 창에서 formurl 값을 등록합니다.

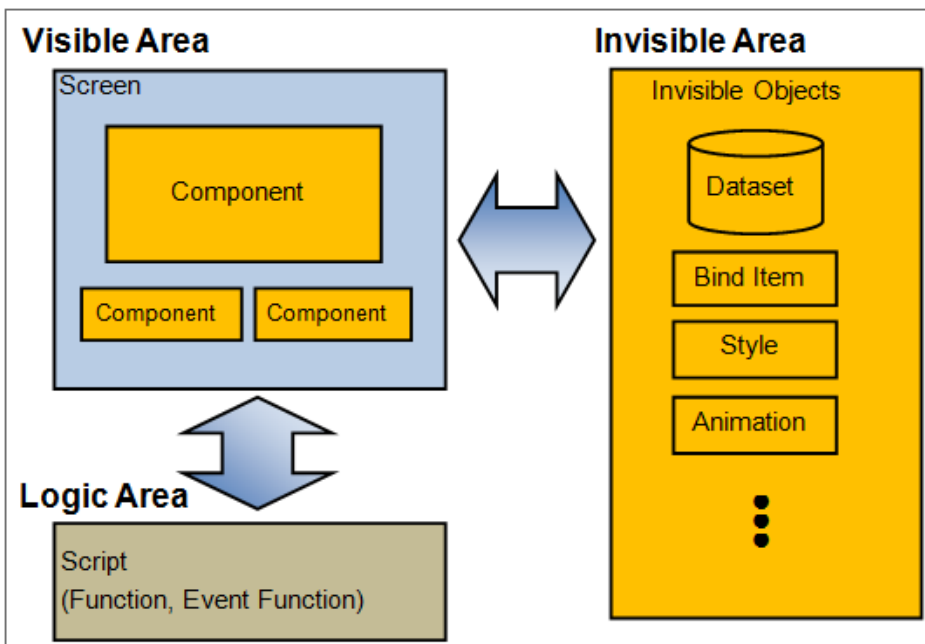


1.3.3 Form의 구성

Form은 실질적인 화면 UI를 구성하는 대상입니다. 이 Form들이 Frame 위에 올려져서 고객의 화면에 보여집니다.

Form은 Visible영역, Invisible 영역과 Logic영역으로 크게 3개의 영역으로 구성되어 있습니다.

- Visible영역은 화면에 보여지는 부분으로 visible Object(이하 컴포넌트)들의 배치로 만들어 집니다.
- Invisible영역은 Invisible Object들의 조합으로 만들어집니다.
- Logic영역은 Script로 직접 코딩 하는 부분으로 컴포넌트와 Invisible Object들을 제어하는 부분입니다.



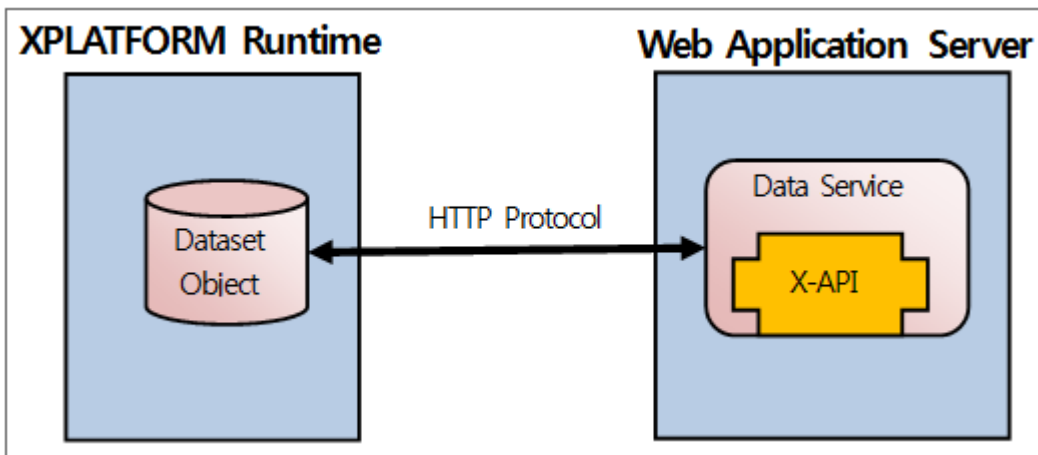
1.4 서버와의 Data연동

UX-Studio로 개발한 응용프로그램은 Data Server와 연동하면서 Data를 UI화면에 보여줍니다.

XPLATFORM은 Data Server와의 연동을 위하여 X-API를 제공합니다. 이 X-API는 JSP, ASP, Java Servlet등의 다양한 Server 환경에서 사용할 수 있습니다.

1.4.1 X-API와 XPLATFORM Runtime

X-API는 XPLATFORM Runtime이 사용하는 Data Format을 생성해주는 Library입니다. 그러므로 X-API를 사용하지 않아도 Data Format을 준수하는 생성하는 Data Service를 개발한다면, XPLATFORM Runtime이 해당 Service의 Data를 사용할 수 있습니다.



2.

XML File들의 구조

응용프로그램 실행을 위하여 XPLATFORM Runtime은 Form파일 외에 많은 파일들을 필요로 하고, UX-Studio는 이 파일들을 자동으로 생성 관리합니다.

이 파일들은 모두 XML Format으로 되어 있습니다. 여기서는 XPLATFORM이 제시하는 XML 구조를 설명합니다.

2.1 표기법

이 장에서 사용하는 표기법은 다음과 같습니다.

표기법	설명	사용 예 / 설명
#	N번 반복	<code><TabFramePages></code> <code><TabFramePage/>#</code> <code></TabFramePages></code> TabFramePages내에 여러 개의 TabFramePage가 올 수 있습니다.
##	N번 반복 또는 Recursive 반복	<code></Frames></code> <code><FrameSet/>##</code> <code></Frames></code> Frames내에 여러 개의 FrameSet이 올 수 있고, FrameSet내에 하위 Frame들이 또 올 수 있습니다.
or	배타적 선택	<code><MainFrame></code> <code><ChildFrame/></code> or <code><TabFrame/>#</code> <code></MainFrame></code> MainFrame내에 ChildFrame 또는 TabFrame이 올 수 있습니다. 그러나 둘 중 하나만 올 수 있습니다.

2.2 ADL XML Format

```

<ADL>
  <TypeDefinition /> ---- 별도파일 설명
  <GlobalVariables /> --- 별도파일 설명
  <Application></Application>
  <Script><![CDATA[   ...]]></Script>
</ADL>

```

```

<Application>
  <Style/>#
  <Layout>
    <MainFrame>
      <ApplicationMenu>
        <Dataset id="innerdataset">
          </Dataset>
        </ApplicationMenu>
      <ChildFrame/>
      or
      <TabFrame/>
      or
      <FrameSet/>##
      or
      <VFrameSet/>##
      or
      <HFrameSet/>##
      or
      <TileFrameSet/>##
    </MainFram>
    <Widget />#
    <Tray>
      <TrayPopupMenuItems>
        <TrayPopupMenuItem>#
      </TrayPopupMenuItems>
    </Tray>
  </Layout>
</Application>

```

```
<ChildFrame/> --- 하위 Frame이 없습니다.
```

```
<TabFrame>
  <TabFramePages>
    <TabFramePage/>#
  </TabFramePages>
</TabFrame>
```

```
<FrameSet>
  <Frames>
    <ChildFrame/>#
    <TabFrame/>##
    <FrameSet/>##
    <VFrameSet/>##
    <HFrameSet/>##
    <TileFrameSet/>##
  </Frames>
</FrameSet>
```

```
<VFrameSet>
  <Frames>
    <ChildFrame/>#
    <TabFrame/>##
    <FrameSet/>##
    <VFrameSet/>##
    <HFrameSet/>##
    <TileFrameSet/>##
  </Frames>
</VFrameSet>
```

```
<HFrameSet>
  <Frames>
    <ChildFrame/>#
    <TabFrame/>##
    <FrameSet/>##
    <VFrameSet/>##
    <HFrameSet/>##
    <TileFrameSet/>##
```

```

    </Frames>
</HFrameSet>

```

```

<TileFrameSet>
  <Frames>
    <ChildFrame/>#
    <TabFrame/>##
    <FrameSet/>##
    <VFrameSet/>##
    <HFrameSet/>##
    <TileFrameSet/>##
  </Frames>
</TileFrameSet>

```

2.3 FDL XML Format

```

<FDL>
  <TypeDefinition/> --- 별도파일 설명
  <Form>
    <Style/>#
    <Bind>
      <BindItem/>#
    </Bind>
    <Objects>
      <!-- 여기에 Invisible Object들이 나열됨-->
    </Objects>
    <Layout>
      <!-- 여기에 컴포넌트들이 나열됨-->
    </Layout>
    <Script><![CDATA[ ]]></Script>
  </Form>
</FDL>

```

2.4 Type Definition XML Format

```
<Typedeintion>
  <Components>
    <Component/>#
  </Components>
  <Services>
    <Service/>#
  </Services>
  <Update>
    <item/>#
  </Update>
</Typedefinition>
```

2.5 Global Variable XML Format

```
<GlobalVariables>
  <Images>
    <Image/>#
  </Images>
  <Dataset/>#
  <Variable/>#
  <!-- Animation, Color, Font등 다양한 Invisible Object들 -->#
</GlobalVariables>
```

3.

애니메이션 기능

애니메이션 기능은 XLATFORM 화면을 구성하는 화면요소들에게 애니메이션 효과를 주는 기능을 의미합니다. 애니메이션 효과를 줄 수 있는 화면 요소로는 비주얼 컴포넌트들, 화면 Form, Frame, Widget등이 있습니다.

3.1 애니메이션 기능의 개요

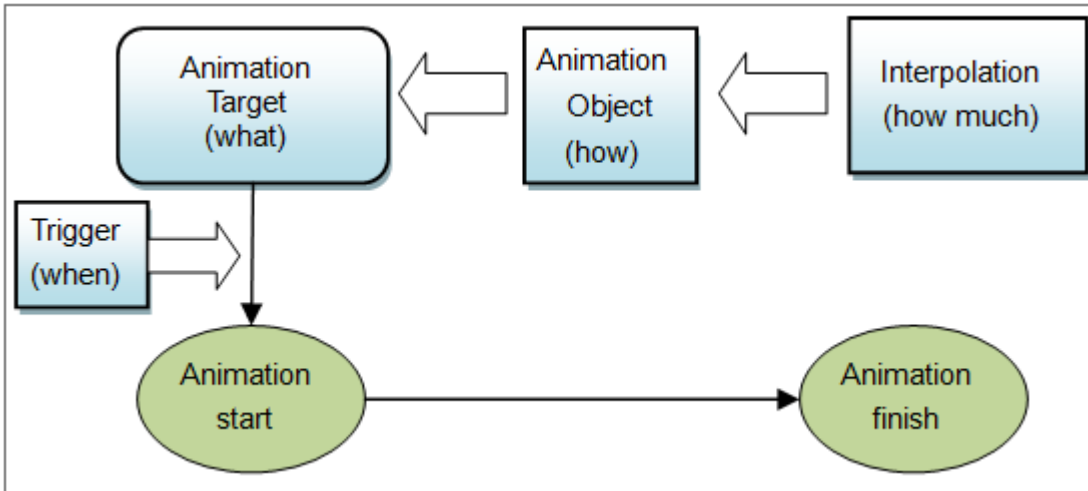
XPLATFORM에서 애니메이션 기능이란, 버튼이 회전 또는 이동하는 등 화면상에 보이는 UI요소들이 동적으로 변하는 것을 의미합니다.

애니메이션 종류는 크게 Transition Animation, Property Animation, Move Animation과 Composite Animation의 4가지가 있는데, Composition Animation은 Transition과 Property Animation을 조합한 것을 의미합니다.

3.1.1 애니메이션 기능의 4가지 요소

애니메이션 기능 사용을 위하여 크게 4가지 요소가 필요합니다.

- 무엇을 : 어떤 화면요소에게 애니메이션 효과를 줄 것인가를 결정합니다.
- 어떻게 : 어떤 애니메이션 효과를 줄 것인가를 결정합니다.
- 얼마나 : 결정된 애니메이션 효과를 얼마나 오랫동안 어떤 빠르기로 실행할 것인가를 결정합니다. interpolation은 애니메이션에서 시간변화의 장단을 나타내는 계산식을 의미합니다. 그러므로 “얼마나” 보다는 “어떻게”에 가깝다고도 할 수 있지만, 시간 개념이 들어가므로 “얼마나”로 분류했습니다.
- 언제 : 애니메이션 효과를 언제 시작할 지를 결정합니다.



3.1.2 간단한 애니메이션 기능 구현

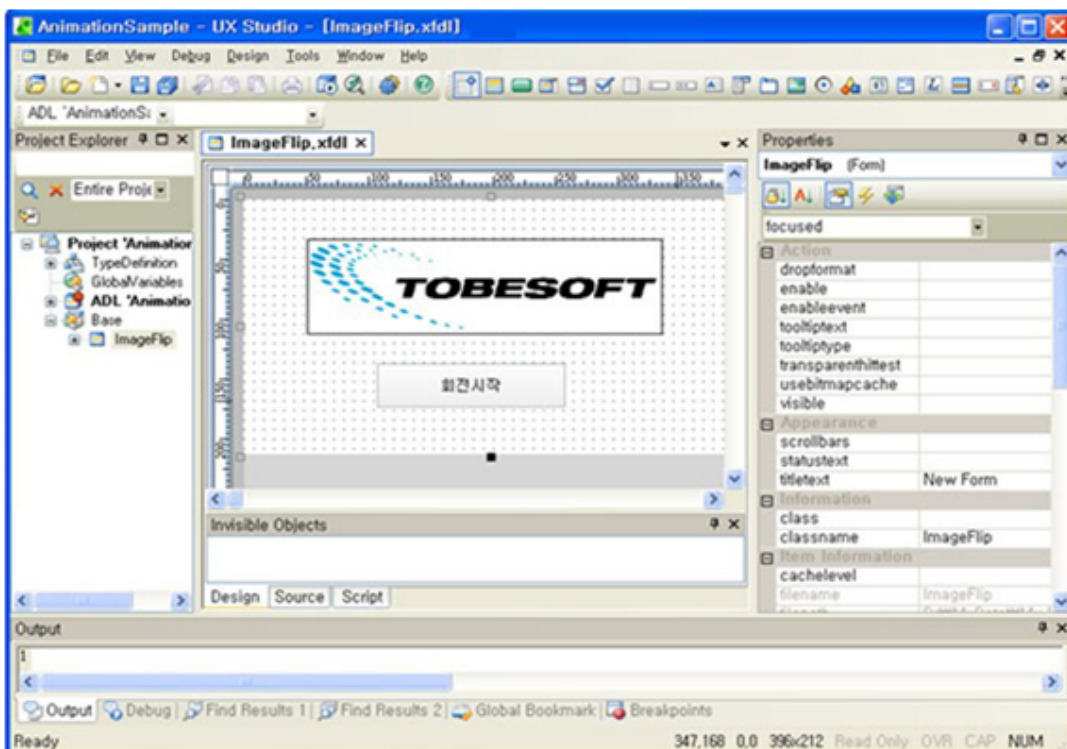
애니메이션 기능은 4가지 요소가 조합되면 작동 한다고 보면 됩니다.

여기서는 간단히 이미지를 회전시키는 애니메이션 효과를 예로 들어 설명하겠습니다.

프로젝트 생성 및 form의 생성과정은 이미 완료했다는 가정하에 설명을 전개 합니다. 대부분의 예제 화면은 UX-Studio의 화면입니다.

1단계 (무엇을): 회전 대상을 선정합니다. 즉, Animation Target을 선정합니다.

여기서는 TOBESOFT CI를 보여주는 ImageViewer 컴포넌트를 대상으로 선정했습니다.

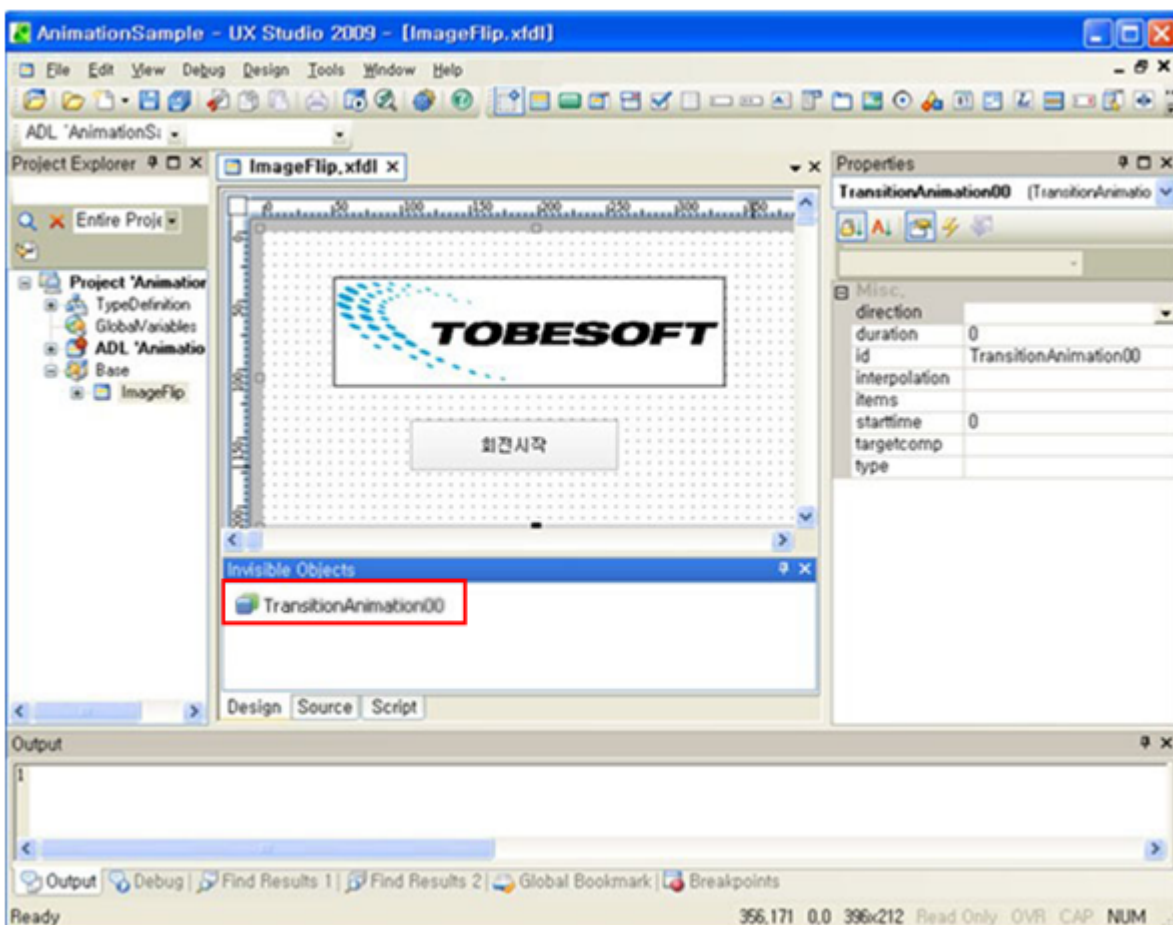


다음은 UX-Studio에서 Quick View로 실행한 화면입니다.



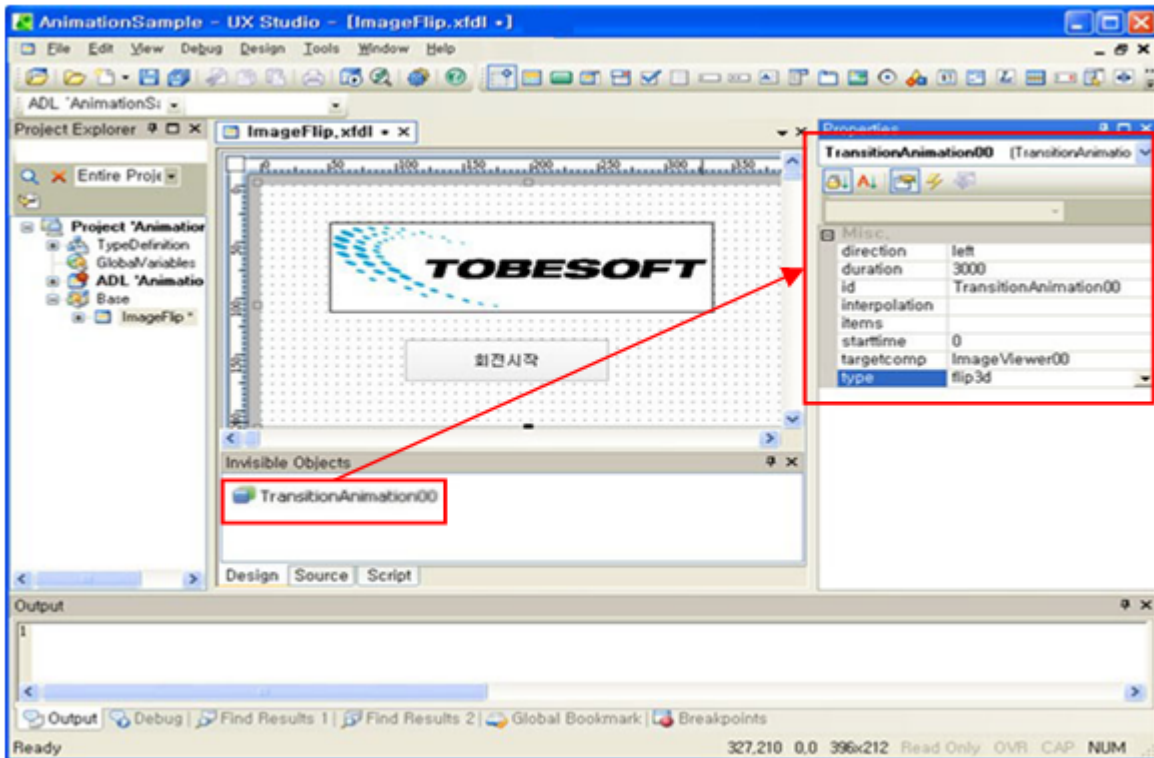
2단계 (어떻게): Animation Object를 생성하여 애니메이션 종류를 선정합니다.

여기서는 “TOBESOFT CI”를 회전시키는 효과를 선정했으므로 회전효과 기능이 있는 Transition Animation Object를 생성합니다.



3단계 (어떻게): Animation Object의 상세 값을 설정합니다.

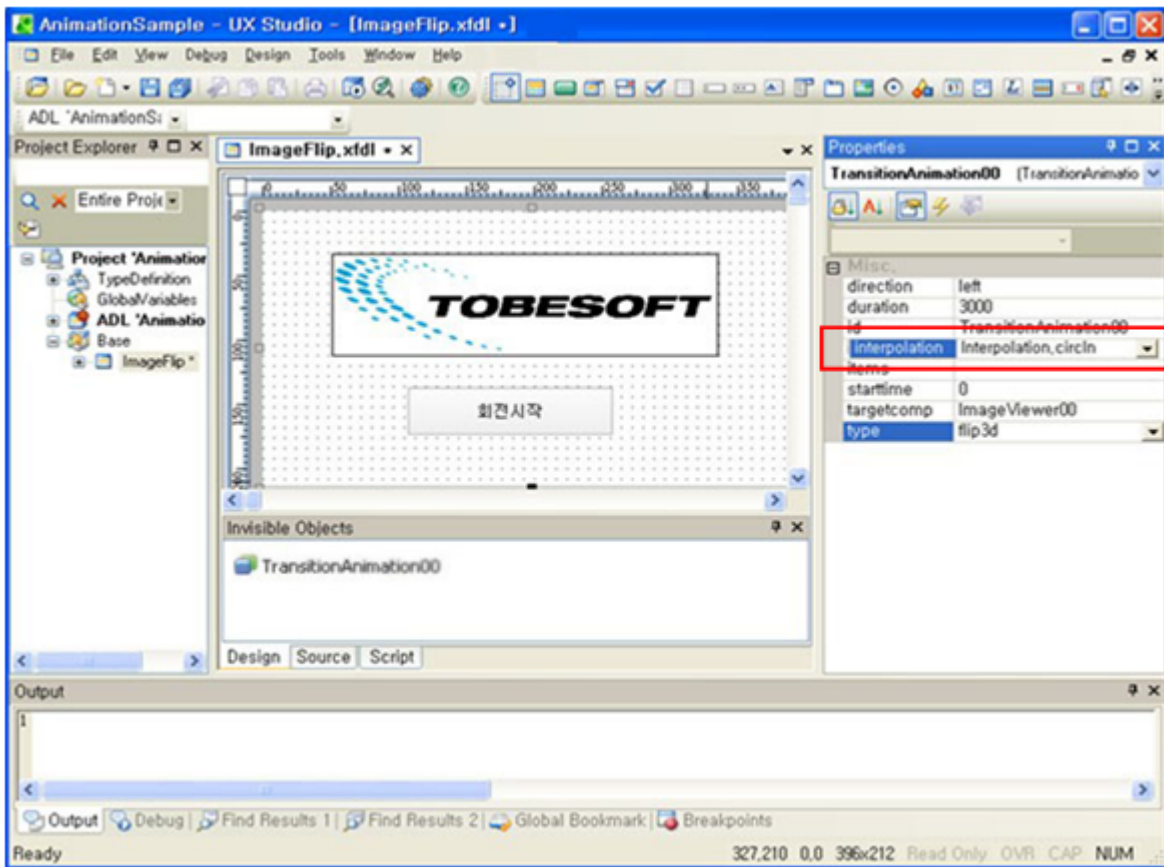
여기서 애니메이션 대상과 효과를 연결합니다.



Animation Object의 설정 값을 개략적으로 설명하면 아래와 같습니다.

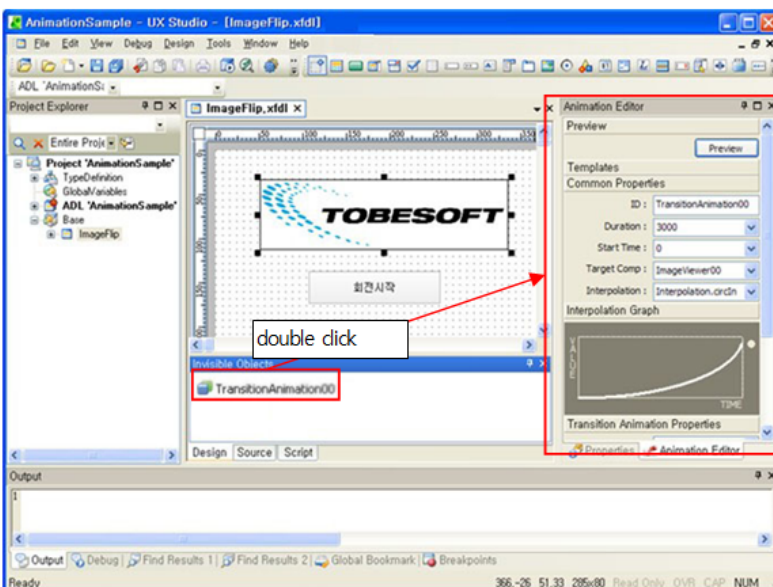
Property명	값	의미
direction	left	왼쪽에서 오른쪽으로 효과가 발생합니다.
duration	3000	애니메이션 효과를 3초간 보여줍니다. 정확히, 3초간 1바퀴 돕니다.
id	TransitionAnimation00	애니메이션 효과를 구분하는 id입니다.
starttime	0	트리거 이벤트 발생 이후 대기 시간입니다. 그러므로, 여기서는 곧바로 애니메이션 효과가 나타납니다.
targetcomp	ImageViewer0	ImageViewer00 컴포넌트를 대상으로 지정합니다.
type	flip3d	애니메이션 효과를 회전으로 지정합니다. Direction property값이 left이므로 왼쪽에서 오른쪽으로 회전효과가 발생합니다.

4단계 (얼마나): 애니메이션 효과를 어떤 시간변화로 작동할지를 결정합니다.

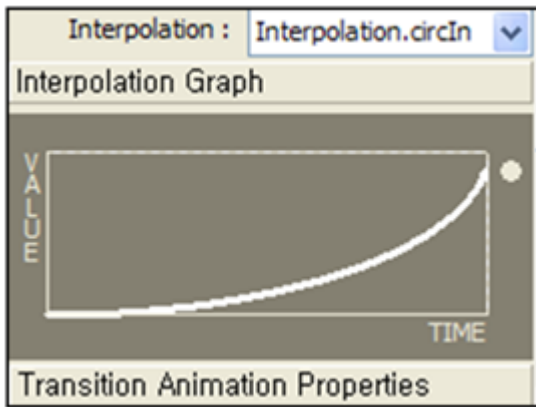


interpolation값을 “Interpolation.circIn”으로 지정한다. “circIn”은 처음에는 천천히 움직이다가 점차 빨리 움직이는 효과를 나타냅니다.

UX-Studio의 Animation Editor를 이용하면 좀 더 편리하게 애니메이션 효과를 정의할 수 있습니다. Animation Object를 더블클릭하면 해당 Animation Object에 대한 Animation Editor가 나타납니다.



Animation Editor는 위의 화면처럼 interpolation의 효과를 그래프 형태로 볼 수 있습니다.



예제로 보여지는 그래프는 시간이 지날수록 value 값이 급격히 커지는 효과를 보여줍니다. 즉, 처음에는 천천히 움직이다가 끝날 때쯤에 빠르게 움직이게 됩니다.

5단계 (언제): 애니메이션 트리거를 지정합니다.

이제 애니메이션에 대한 모든 정보입력을 끝냈고, 작동 스위치만 만들면 됩니다. 여기서는 “회전시작”이라는 버튼을 클릭하면 작동하도록 하겠습니다.

“회전시작”버튼의 click event에 다음 스크립트를 추가합니다.

```
function Button00_onclick(obj:Button, e:ClickEventInfo)
{
    TransitionAnimation00.beginTransition();
    TransitionAnimation00.endTransition();
    TransitionAnimation00.run();
}
```

이렇게 스크립트 상에서 run()을 호출하여 애니메이션을 시작시키는 트리거를 수동 트리거라고 합니다.

다음은 UX-Studio에서 Quick View를 실행한 후, “회전시작”버튼을 클릭했을 때 “TOBESOFT” CI 이미지가 회전하는 모습입니다.



“회전시작”버튼으로 회전을 시키면 처음에는 천천히 움직이다가 끝날 때쯤에 빠르게 움직이는 것을 알 수 있을 것입니다.

3.1.3 애니메이션 적용 대상

애니메이션의 적용대상은 단순한 컴포넌트 뿐만 아니라 다양한 화면요소에 적용됩니다. 다음은 그 적용 대상들을 표로 나타낸 것입니다.

다음은 애니메이션 적용이 가능한 대상들입니다.

종류	적용 대상
컴포넌트	Button, Calendar, CheckBox, Combo, Div, Edit, graphicpath, Grid, GroupBox, ImageViewer, ListBox, MaskEdit, Menu, Panel, progressbar, Radio, shape, Spin, Splitter, Static, Tab, TabPage, TextArea
화면 Frame	Form, WidgetForm, ChildFrame, FrameSet, HFrameSet, TabFrame, TileFrameSet, VFrameSet

3.2 애니메이션의 이해

앞에서 애니메이션 개요 및 사용법 간단하게 언급했습니다. 하지만, 실제 애니메이션 기능을 자유롭게 사용하려면 좀 더 많은 부분을 이해해야만 합니다. 여기서는 심도 있는 애니메이션 기능 구현을 위하여 사전에 알아야 할 지식을 설명합니다.

3.2.1 애니메이션 용어정의

- 트리거(Trigger)

애니메이션의 시작을 일으키는 사건을 의미합니다. 애니메이션 트리거에는 수동 트리거와 자동 트리거가 있습니다.

수동 트리거는 스크립트에서 직접 트리거를 걸어주는 것을 의미합니다. “5.1.2.장”에서 사용한 run() 호출이 여기에 속합니다.

자동 트리거는 화면 실행 중에 자동으로 발생하는 트리거를 의미합니다. 자동 트리거의 사용방법은 “5.4.1장”에서 상세하게 다룹니다.

- Interpolation

interpolation은 애니메이션 효과의 진행 속도의 장단을 나타내는 계산식입니다. interpolation은 계산식이므로 개발자가 직접 등록할 수도 있지만, XPLATFORM이 제공하는 interpolation을 사용하는 것이 좋습니다.

XPLATFORM은 약 30여 개의 interpolation을 제공합니다.

- 애니메이션의 시간과 값

용어	설 명
StartTime	트리거 발생 시간을 기준으로 애니메이션이 시작되는 시점입니다.
EndTime	트리거 발생 시간을 기준으로 애니메이션이 종료되는 시점입니다.
Duration	애니메이션의 총 진행 시간입니다. StartTime 이후 Duration에 지정한 시간이 지나면 애니메이션 종료됩니다. EndTime - StartTime값이 Duration값이 됩니다.
FromValue	애니메이션 시작 시점의 시작 값입니다. 주어지지 않으면 현재 값을 사용합니다.
ToValue	애니메이션 종료 시점의 종료 값입니다.
ByValue	FromValue부터의 상대값입니다. 만일, FromValue를 10으로 하고 ByValue를 20으로 하면 ToValue값을 30으로 지정한 것과 같습니다. 만일, ToValue와 FromValue를 둘 다 지정하면 ByValue값은 무시됩니다.
RunMethod	애니메이션 트리거를 발생 시킵니다.
StopMethod	애니메이션의 실행을 종료시킵니다.
EndingMode	애니메이션이 종료된 후에 갖게 될 최종 값에 대한 선택입니다. - “To”의 선택: 최종값이 ToValue값을 갖게 됩니다. - “From”의 선택 : 최종값이 FromValue값을 갖게 됩니다. - “Current”의 선택 : 최종값이 종료시점의 값을 갖게 됩니다. 즉, FromValue에 도달하기 전에 StopMethod로 애니메이션을 정지시키면 정지된 시점의 값이 최종값이 됩니다.

Time과 Value간의 관계를 예를 들어서 설명하겠습니다.

우선 아래와 같이 Time과 Value를 설정했다고 가정합니다.

StartTime=1000, Duration=5000, FromValue=20, ToValue=1

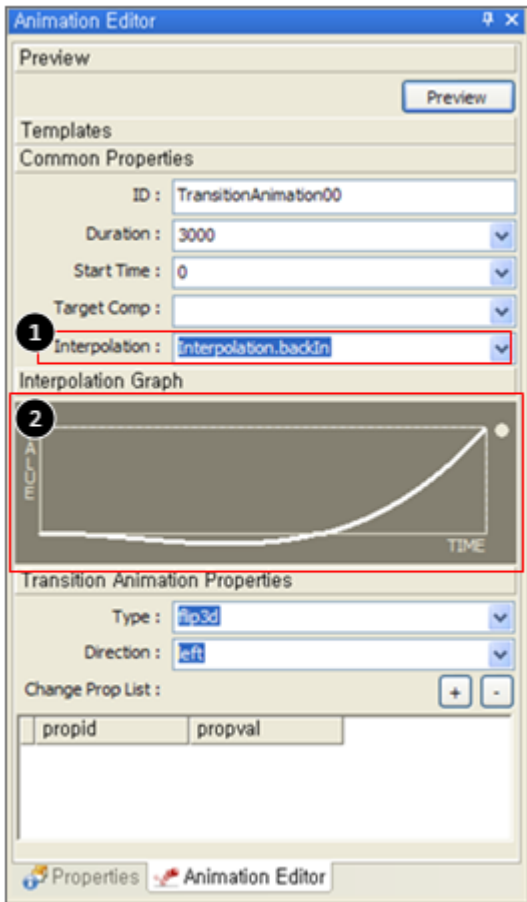
위와 같이 값을 설정하면 아래의 실행 결과를 예측할 수 있습니다.

- 트리거 발생시점으로부터 1초(1000ms)후에 애니메이션이 실행됩니다. 트리거는 자동 트리거를 사용할 수도 있고, run() 메소드를 사용한 수동 트리거를 사용할 수도 있습니다.
- 애니메이션 실행의 시작 시점의 Value는 20입니다.
- 애니메이션 실행 후, 5초 경과 후에 애니메이션이 종료됩니다.
- 애니메이션 종료후의 Value는 1입니다. 만일, EndingMode를 From으로 했다면 Value는 애니메이션 종료 후 20으로 되돌아 갑니다.
- 애니메이션을 실행하는 5초 동안 Value값은 20에서 1로 변경됩니다. 값은 보통 1초에 4씩 감소하게 됩니다. 만일, EndingMode값을 From으로 설정하면, Value값이 1이 되어도 애니메이션 실행이 종료된 이후의 Value값은 20으로 돌아가게 됩니다.

3.2.2 interpolation

interpolation은 애니메이션 효과의 진행 속도의 장단을 나타내는 계산식입니다. interpolation은 계산식이므로 개발자가 직접 등록할 수도 있지만, XPLATFORM이 제공하는 interpolation을 사용하는 것이 좋습니다. XPLATFORM은 약 30여 개의 interpolation을 제공합니다.

다음은 UX-Studio의 Animation Editor화면입니다.



1	약 30여 개 interpolation을 제공합니다.
2	선택한 interpolation의 계산 결과값을 그래프로 나타낸 것입니다.

몇 개의 interpolation을 Graph로 설명 드리겠습니다. 이 설명을 보시면 나머지 interpolation들도 Graph만 보고도 바로 그 특징을 알 수 있을 것 입니다.

interpolation	
linear	

interpolation	
	Time값이 증가하는 추세와 동일하게 Value값이 증가합니다. 그러므로, 가속이나 감속이 없이 동일한 속도를 Animation을 진행합니다.
circIn	Interpolation : Interpolation.circIn Interpolation Graph  Time값이 증가할수록 Value값의 증가 추세가 급격해 집니다. 그러므로, 시간이 지날수록 점차 가속하여 Animation을 진행합니다. 즉, 처음에는 천천히 움직이다가 마지막에는 빠르게 움직입니다.
circOut	Interpolation : Interpolation.circOut Interpolation Graph  Time값이 증가할수록 Value값의 증가 추세가 완만해 집니다. 그러므로, 시간이 지날수록 점차 감속하여 Animation을 진행합니다. 즉, 처음에는 빠르게 움직이다가 마지막에는 천천히 움직입니다.
bounceOut	Interpolation : Interpolation.bounceOut Interpolation Graph  Time값이 증가할수록 Value값의 bounce추세로 변합니다. 시간이 변화에 따라 Value값이 목표 값에 4번이 도달했습니다. 이러한 효과는 공을 바닥에 떨어 뜨리는 효과로 Animation을 진행할 경우에 많이 사용됩니다.

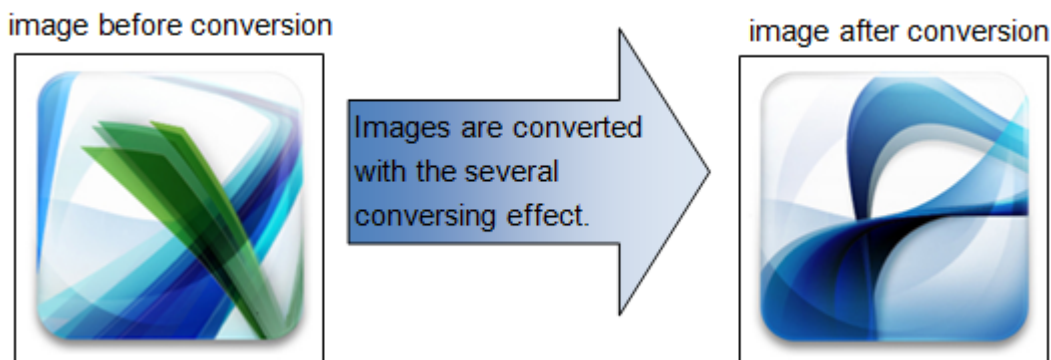
이 밖에도 많은 interpolation을 제공합니다. 모든 interpolation이 graph를 제공하므로 graph를 보고 interpolation의 특징을 이해할 수 있습니다.

3.3 Transition Animation

Transition Animation이란 이미지 전환 시에 사용하는 Animation효과를 의미합니다. TV에서 A장면에서 B장면으로 넘어가는 효과와 유사합니다. 즉, TV화면 전환할 때 장면을 회전시키거나 흐린 효과를 주면서 다른 장면으로 전환하는 효과를 의미합니다.

Transition Animation의 핵심단어는 “전환”이라고 할 수 있습니다. 그러므로 이 효과에서 가장 중요한 것이 “전환전의 이미지”와 “전환후의 이미지”입니다.

그림으로 표현하면 아래와 같습니다.



전환효과는 다음과 같은 것이 있습니다.

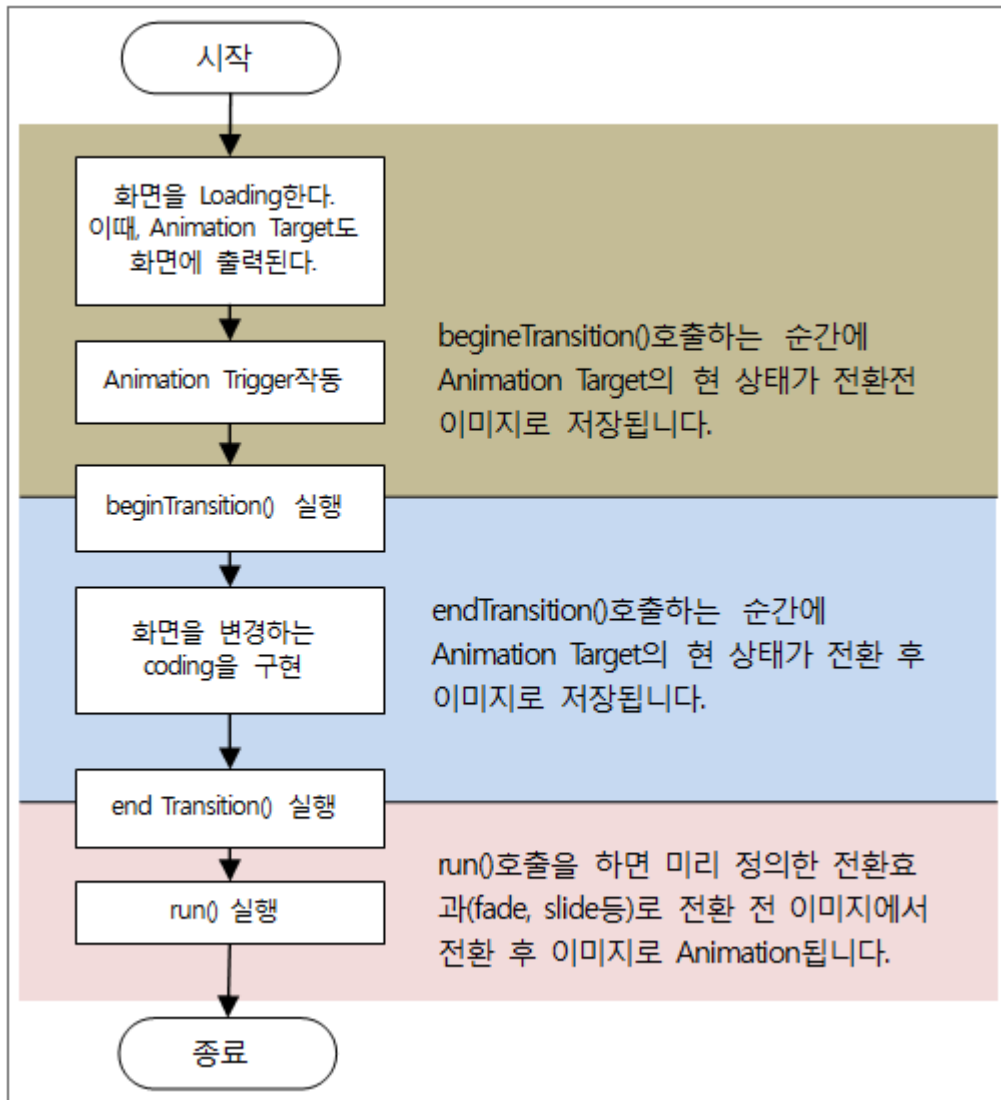
효과명	설명
fade	이미지가 흐려졌다가 밝아지면서 이미지가 전환됩니다.
slide	전환 후 이미지가 전환 전 이미지를 밀면서 전환됩니다.
wipe	전환 후 이미지가 전환 전 이미지를 덮어쓰면서 전환합니다.
flip3d	입체회전을 하면서 이미지가 전환됩니다.
cube3d	입체 Cube회전을 하면서 이미지가 전환됩니다.

3.3.1 전환 전후 이미지들의 저장 및 실행

Transition Animation의 구현의 핵심은 전환전과 전환후의 2개의 이미지를 어떻게 저장하느냐를 알면 됩니다. 그 방법은 아주 간단합니다.

전환 전 이미지는 현재의 이미지입니다. 전환 후 이미지는 beginTransition()함수와 endTransition()함수 사이의 구현내용입니다.

이를 도식화하면 아래와 같습니다.

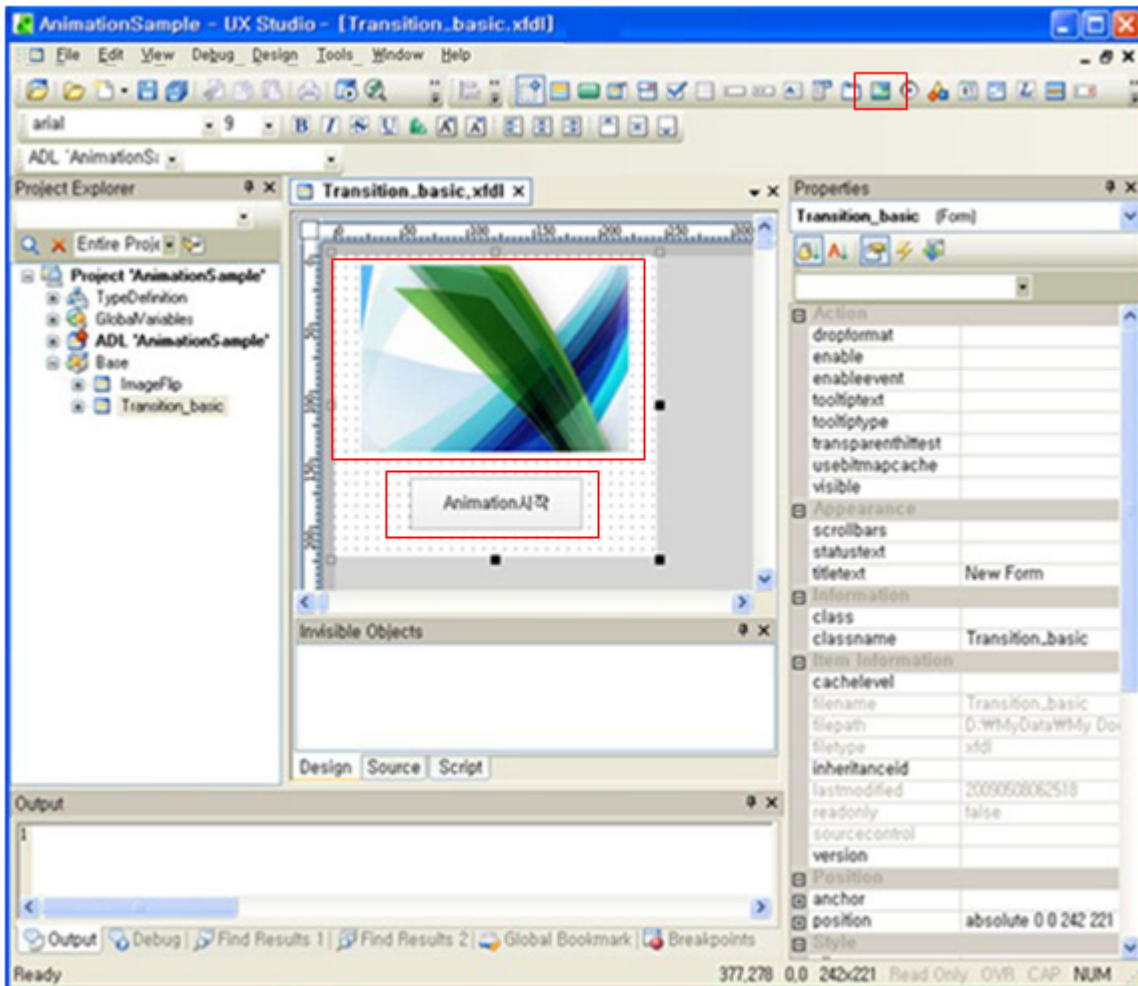


3.3.2 Transition Animation의 예

여기서는 ImageViewer 컴포넌트를 사용하여 한 이미지에서 다른 이미지를 변환하는 화면을 예로 만들어 보겠습니다. 전환효과는 입체 Cube회전으로 하겠습니다.

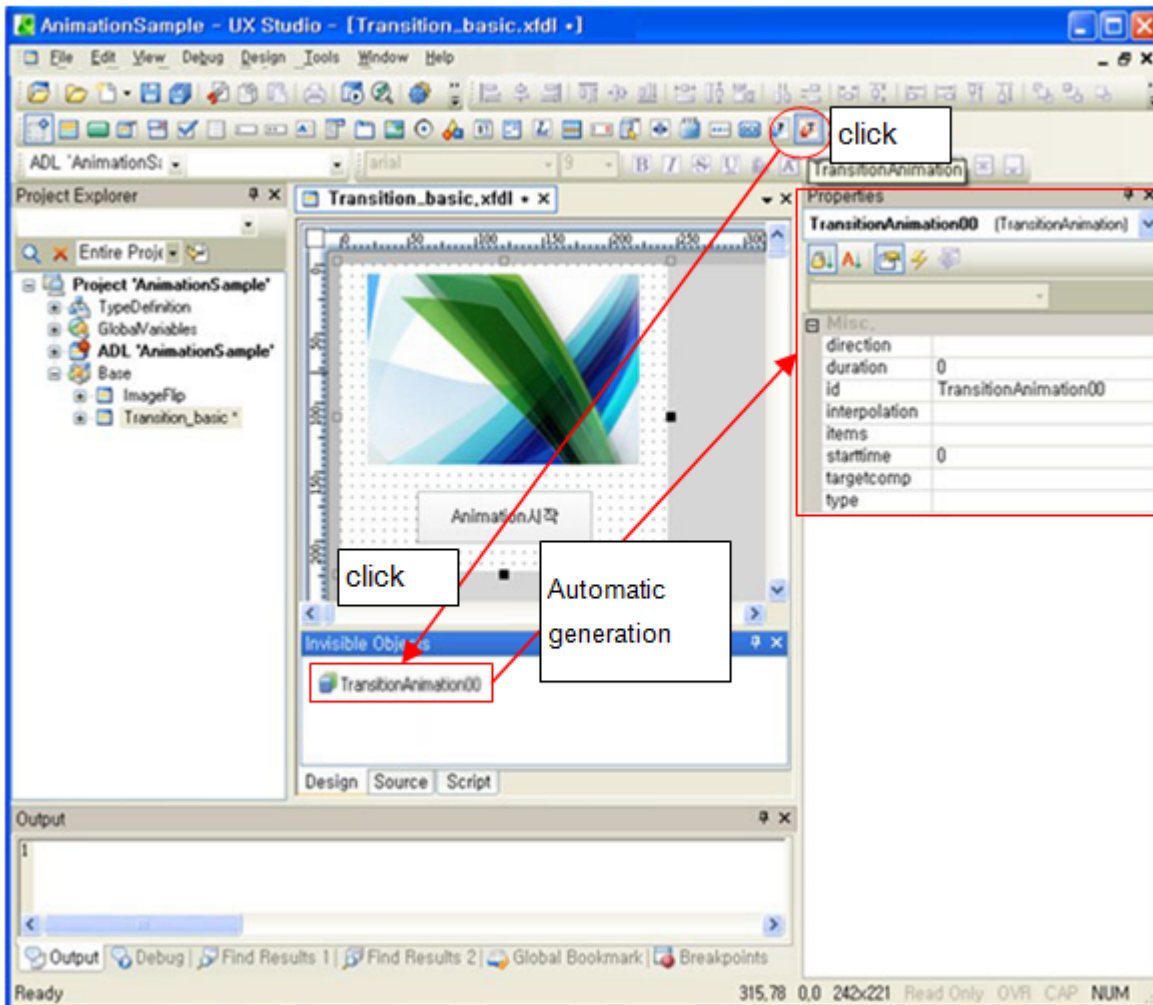
1단계: Animation Target을 선정합니다.

ImageViewer에 이미지를 연결하고, 버튼을 하나 두어 Animation Trigger의 준비를 합니다.

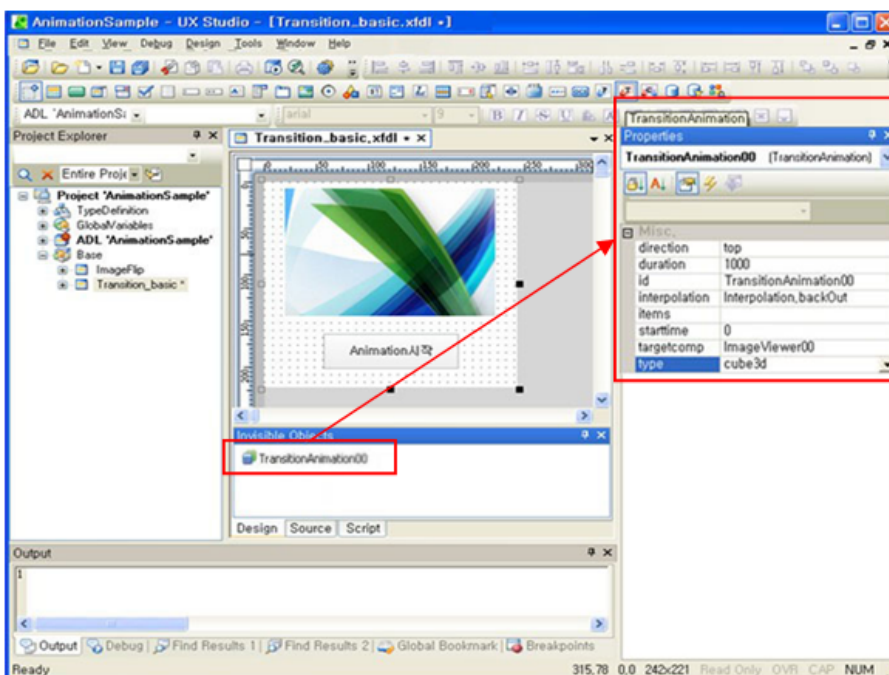


2단계 (어떻게): Transition Animation Object를 생성합니다.

MenuBar에서 TransitionAnimation을 클릭한 후 Invisible Object영역에서 클릭하면 Transition Animation Object가 생성됩니다. 생성과 동시에 Property영역에도 Transition Animation의 속성을 정의할 수 있는 화면이 열립니다.



3단계 (어떻게): Animation Object의 상세 값을 설정합니다.
여기서 Animation Target과 Animation효과를 연결합니다.



Animation Object의 설정 값을 개략적으로 설명하면 아래와 같습니다.

Property명	값	의미
direction	top	위에서 아래로 회전을 합니다.
duration	1000	Animation효과를 1초간 보여줍니다. 정확히, 1초간 1바퀴 돕니다.
id	TransitionAnimation00	Animation효과를 구분하는 id입니다.
interpolation	Interpolation.backOut	Animation의 시간 경과에 따른 변환속도를 나타냅니다.
starttime	0	Animation Triggering 이후 대기 시간입니다. 그러므로, 여기서 Animation Triggering이후 곧바로 Animation효과가 나타납니다.
targetcomp	ImageViewer0	Animation Target을 ImageViewer00 컴포넌트로 연결합니다.
type	cube3d	Animation효과를 3d cube회전으로 지정합니다.

4단계 (언제): Animation Trigger를 지정합니다.

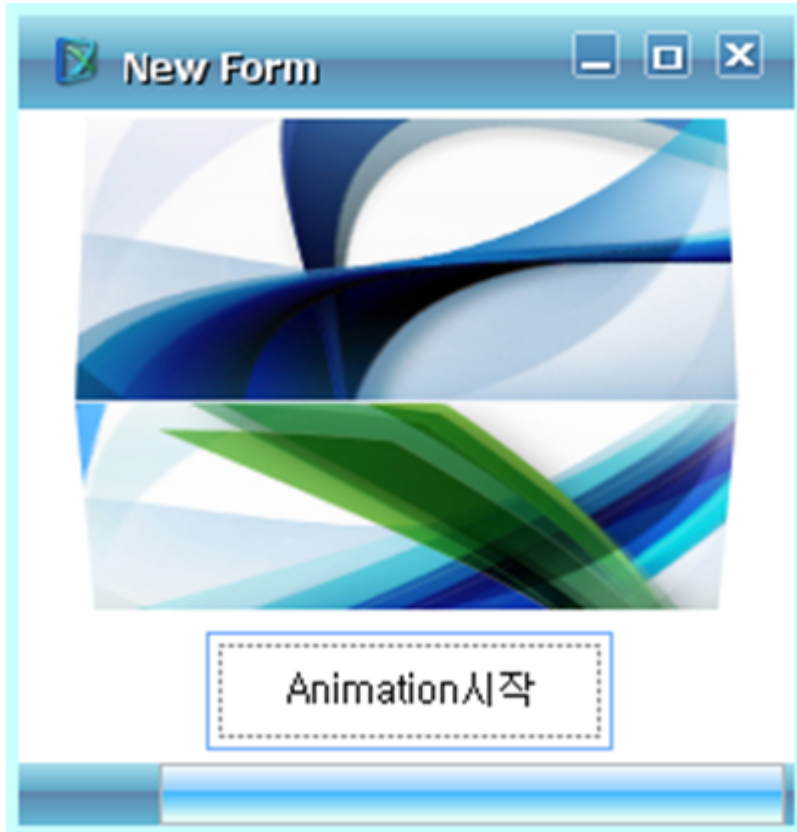
이제 Animation에 대한 모든 정보입력을 끝냈고, 작동 스위치만 만들면 됩니다. 여기서는 “Animation시작”이라는 버튼을 클릭하면 작동하도록 하겠습니다.

“Animation시작”버튼의 click event에 다음의 coding을 추가합니다.

```
var imageFlag = 0;
function Button00_onclick(obj:Button, e:ClickEventInfo)
{
    TransitionAnimation00.beginTransition();
    if (imageFlag == 0)
    {
        ImageViewer00.image = "URL('Image3')";
        imageFlag = 1;
    }
    else
    {
        ImageViewer00.image = "URL('Image2')";
        imageFlag = 0;
    }
    TransitionAnimation00.endTransition();
    TransitionAnimation00.run();
}
```

이렇게 coding을 하면 버튼을 클릭할 때마다 imageFlag값이 0>1>0로 토글하면서 ImageViewer00.image값을 계속 바꿔주게 됩니다. 그러면, 클릭할 때마다 3d cube Animation 기능이 작동하면서 두 개의 이미지가 계속 바뀌게 됩니다.

다음은 UX-Studio에서 Quick View를 실행한 후, “Animation시작”버튼을 클릭했을 때 이미지가 회전하는 모습입니다.



3.3.3 Transition Animation의 Property

다음은 Transition Animation의 Property 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요.

Name	Description
direction	Transition Animation 이 동작하는 방향의 기준을 나타내는 Property 입니다.
duration	Animation 의 지속 시간을 지정하는 Property 입니다.
interpolation	Animation 의 진행에 따른 변화 값을 구하기 위한 interpolation 함수를 지정하는 Property 입니다. Interpolation Object 에 미리 지정되어있는 predefined interpolation 함수를 사용할 수도 있고 user 가 직접 정의한 script 함수를 설정할 수도 있습니다.
items	TransitionPropItem 정보를 담고 있는 Property 입니다. (TransitionPropItem 은 TransitionAnimation 실행 시에 변경되어야 할 Property List 를 가지고 있는 Object 입니다.)
name	서로 구분하기 위한 고유의 id를 지정하는 Property입니다.
starttime	Animation 의 시작 시간을 지정하는 Property 입니다. Animation 이 Run 된 경우에 설정한 시간만큼 지연된 후에 Animation 이 진행됩니다.
targetcomp	Animation 이 적용될 target component 를 지정하는 Property 입니다.

Name	Description
type	TransitionAnimation 의 타입을 지정하는 Property 입니다.

3.3.4 Transition Animation의 Method

다음은 Transition Animation의 Method 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요.

Name	Description
beginTransition	TransitionAnimation 을 위해 변경전 상태 이미지를 만드는 method 입니다. 상태이미지를 만들 대상이 필요하므로 반드시 targetcomp property 가 설정되어 있어야만 합니다.
cancelTransition	TransitionAnimation 을 위한 변경 전, 후 이미지를 모두 삭제하는 method 입니다.
endTransition	TransitionAnimation 을 위해 변경 후 상태 이미지를 만드는 method 입니다. 상태이미지를 만들 대상이 필요하므로 반드시 targetcomp property 가 설정되어 있어야만 합니다.
run	Animation 을 시작하는 method 입니다.
stop	Animation 을 종료하는 method 입니다.

3.4 Property Animation

Property Animation이란 Animation Target의 Property값을 순차적으로 변화 시키면서 Animation효과를 갖는 것을 의미합니다. 만일, Property값의 x, y좌표 Property값에 Property Animation을 적용하면 해당 Animation Target이 움직이는 효과를 갖게 됩니다.

여기서 주위 할 점은 Animation을 적용하는 Property값이 시작 값, 중간 값과 끝 값을 갖는 특성을 가져야만 합니다. 즉, Text나 Boolean Type 처럼 시작, 중간과 끝 값의 개념이 모호한 Property는 Property Animation의 대상이 될 수 없습니다.

3.4.1 Transition Animation과 Property Animation의 차이점

Transition Animation은 두 이미지를 사진으로 보관한 상태에서 이 두 개의 이미지를 맞바꾸는 효과를 주는 것입니다.

Property Animation은 두 개의 이미지가 사용하지 않고 하나의 이미지만 사용합니다. Property Animation Target의 Property값을 변경하면서 화면에 실시간으로 반영하여 Animation 효과를 주는 것입니다.

주의할 점은 Transition Animation과 Property Animation이 상반된 개념이 아니라는 점입니다. 두 개의 Animation은 서로 다른 관점에서 Animation효과를 제공합니다. 예를 들어, 어떤 이미지를 회전하고 싶을 때는 Transition Animation을 사용하고, 그 이미지를 이동시키고 싶을 때는 Property Animation을 사용합니다. 만일 이미지를 회전하면서 이동시키고 싶을 때는 두 개의 Animation을 조합한 Composite Animation을 사용합니다.

3.4.2 Property Animation의 예

여기서는 button을 두 개, shape 한 개를 이용하여 Property Animation의 예제를 작성해 보겠습니다.

이 예제는 아래의 기능을 갖습니다.

- 버튼들 위에 마우스를 올려 놓으면 버튼의 길이가 늘어난다.
- 버튼들 위의 마우스가 다른 곳으로 이동하면 버튼의 길이가 원상태가 됩니다.
- 위의 버튼을 클릭하면 오른 쪽의 구형태의 모양이 떨어지는 효과를 발생한다.
- 아래의 버튼을 클릭하면 오른 쪽의 구형태의 모양이 제자리로 돌아간다.

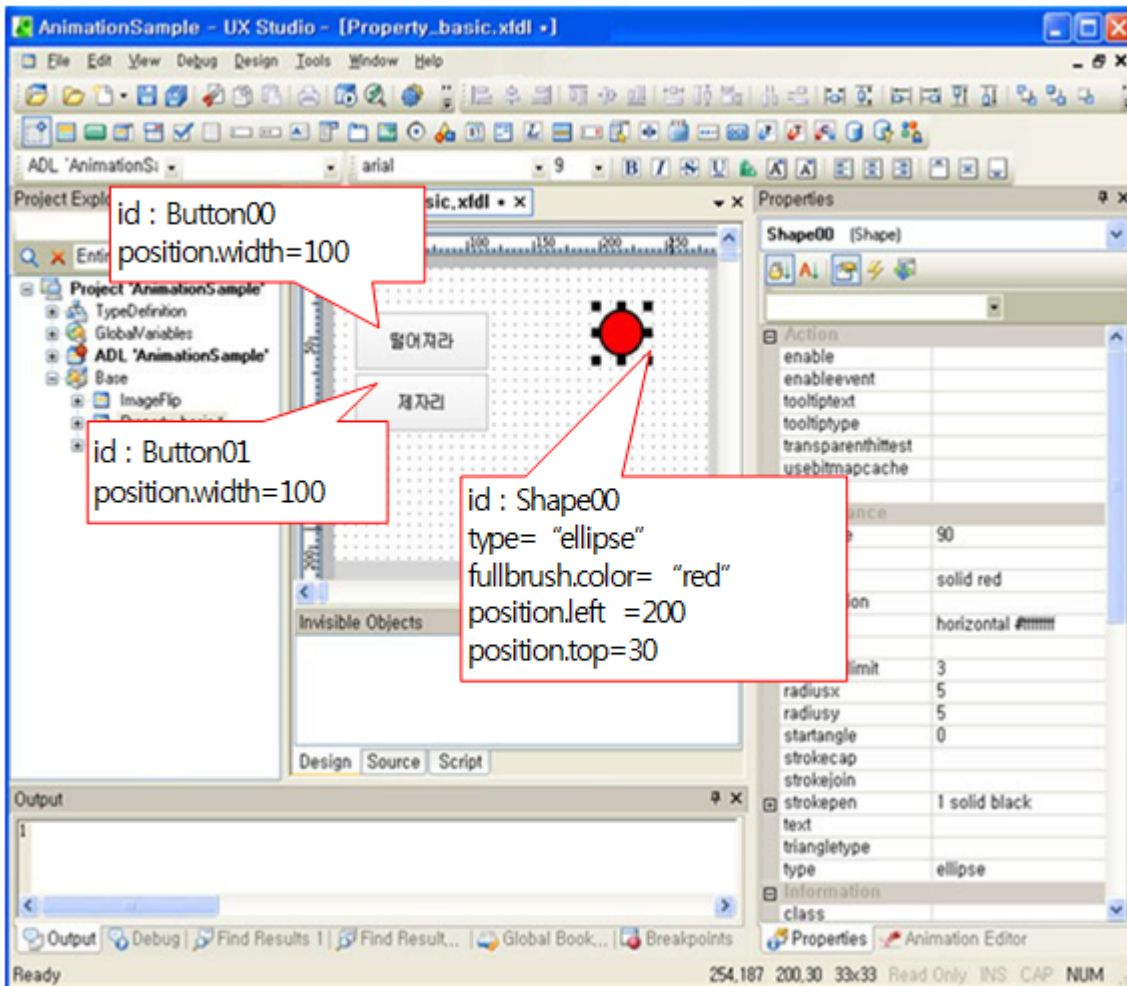
이 예제를 작성하기 위해서는 아래의 사항을 고려해야 합니다.

- Animation Target이 3개입니다.
button 컴포넌트 2개, shape 컴포넌트 1개로 총 3개 입니다.
- 총 6개의 Property Animation Object가 필요합니다.
2개의 button 컴포넌트의 길이가 늘어나는 효과에 대한 Animation 2개.
2개의 button 컴포넌트의 길이가 줄어드는 효과에 대한 Animation 2개.
shape의 떨어지는 이동효과에 대한 Animation 1개.
shape의 제자리로 돌아가는 이동효과에 대한 Animation 1개.
- 그러나, 실제 Animation Object는 3개만 사용합니다.
button 컴포넌트의 길이가 늘어나고 줄어드는 효과에 대한 Animation 1개.
shape의 떨어지는 이동효과에 대한 Animation 1개.
shape의 제자리로 돌아가는 이동효과에 대한 Animation 1개.
즉, 하나의 Animation Object를 여러 개의 Animation Target에 사용할 수 있습니다.

1단계: Animation Target을 선정합니다.

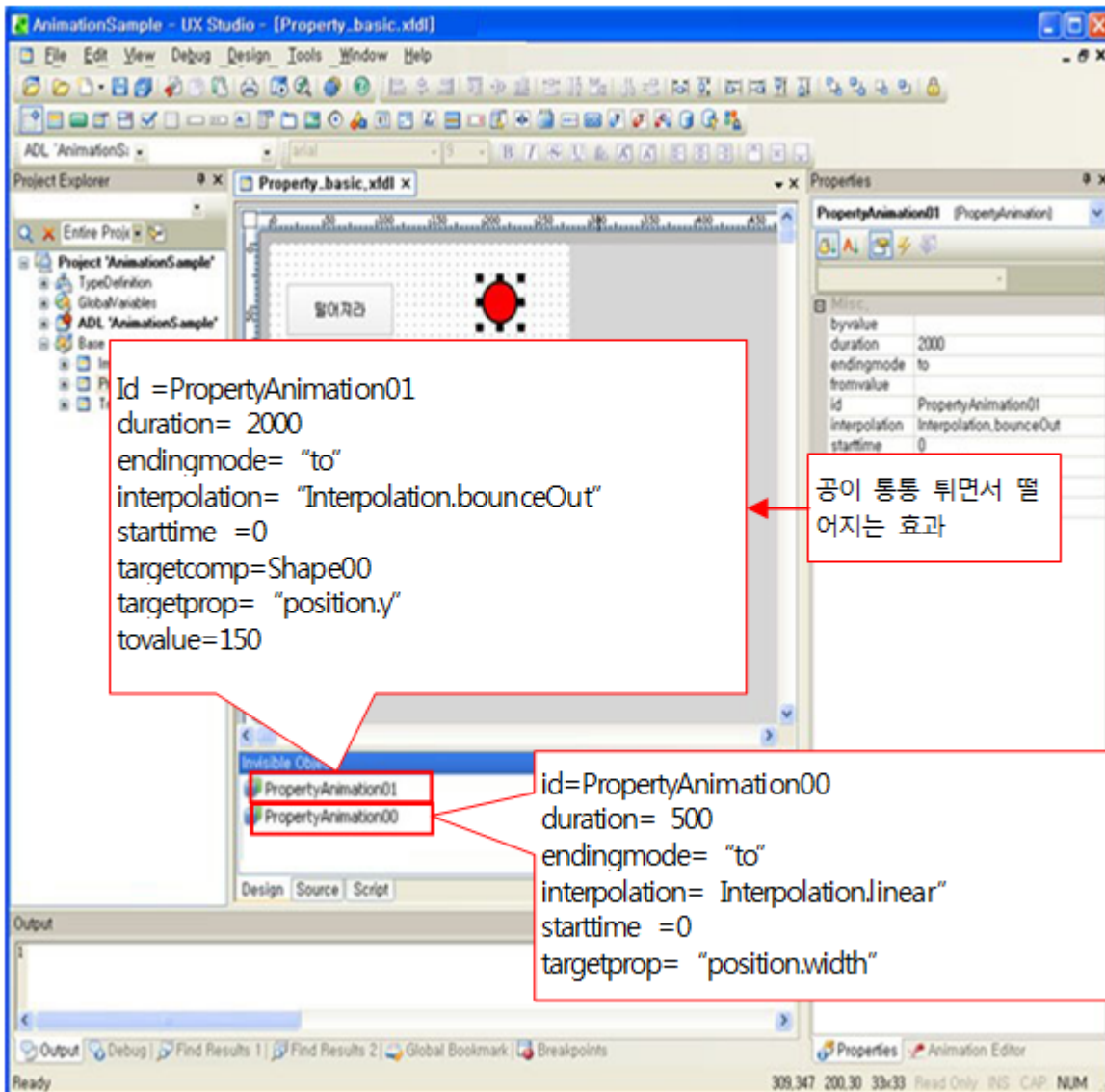
button컴포넌트 2개, shape 컴포넌트 1개를 준비합니다.

그리고 설명박스에서처럼 컴포넌트들의 Property값을 설정했습니다.



2단계 (어떻게): Property Animation Object를 생성합니다.

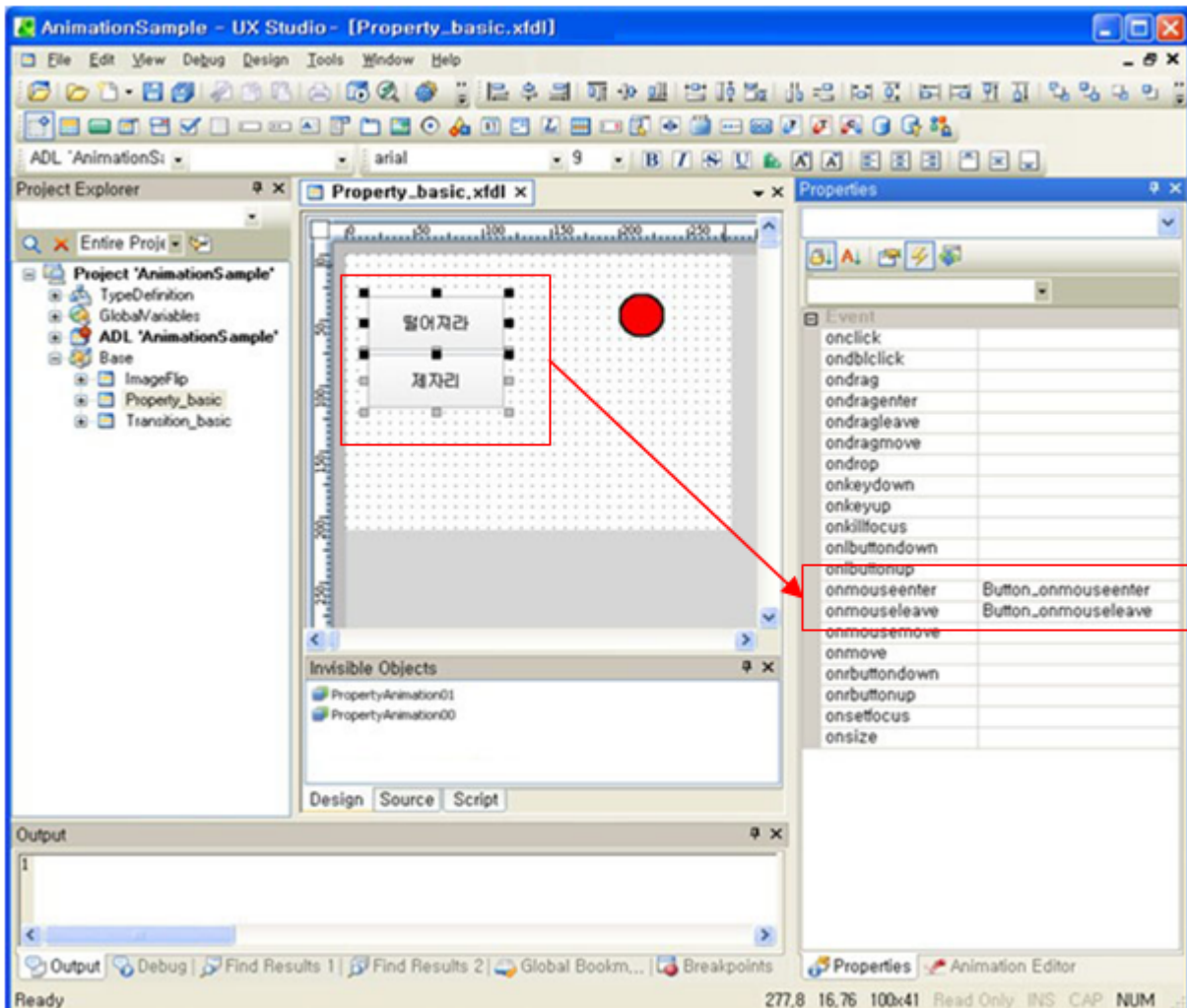
Property Animation Object 두 개를 생성합니다. 총 3개중 두 개는 정적으로 만들고 나머지 한 개는 동적으로 생성합니다. 그리고 설명박스에서처럼 Animation Object의 Property값을 설정했습니다.



- PropertyAnimation00은 Button의 길이를 조절하는데 사용합니다.
- PropertyAnimation01는 공모양의 Shape를 공처럼 떨어뜨리는데 사용합니다.
- PropertyAnimation02는 공모양의 Shape를 제자리로 위치시키는 데 사용하며, 5단계에서 동적으로 생성합니다.

3단계 (언제/어떻게) : 두 개의 버튼에 대한 Event를 코딩 합니다.

여기서는 두 개의 버튼 위에 마우스가 올라왔을 때, 버튼의 길이가 늘어나도록 효과를 구현합니다. 두 개의 button 컴포넌트를 drag로 group지은 후에 onmouseenter와 onmouseleave event를 등록합니다.



위와 같이 컴포넌트를 같이 선택한 후 event를 등록하면, 선택된 컴포넌트에서 발생한 event들을 한 event function에서 처리할 수 있습니다.

event function의 coding은 아래와 같이 등록합니다.

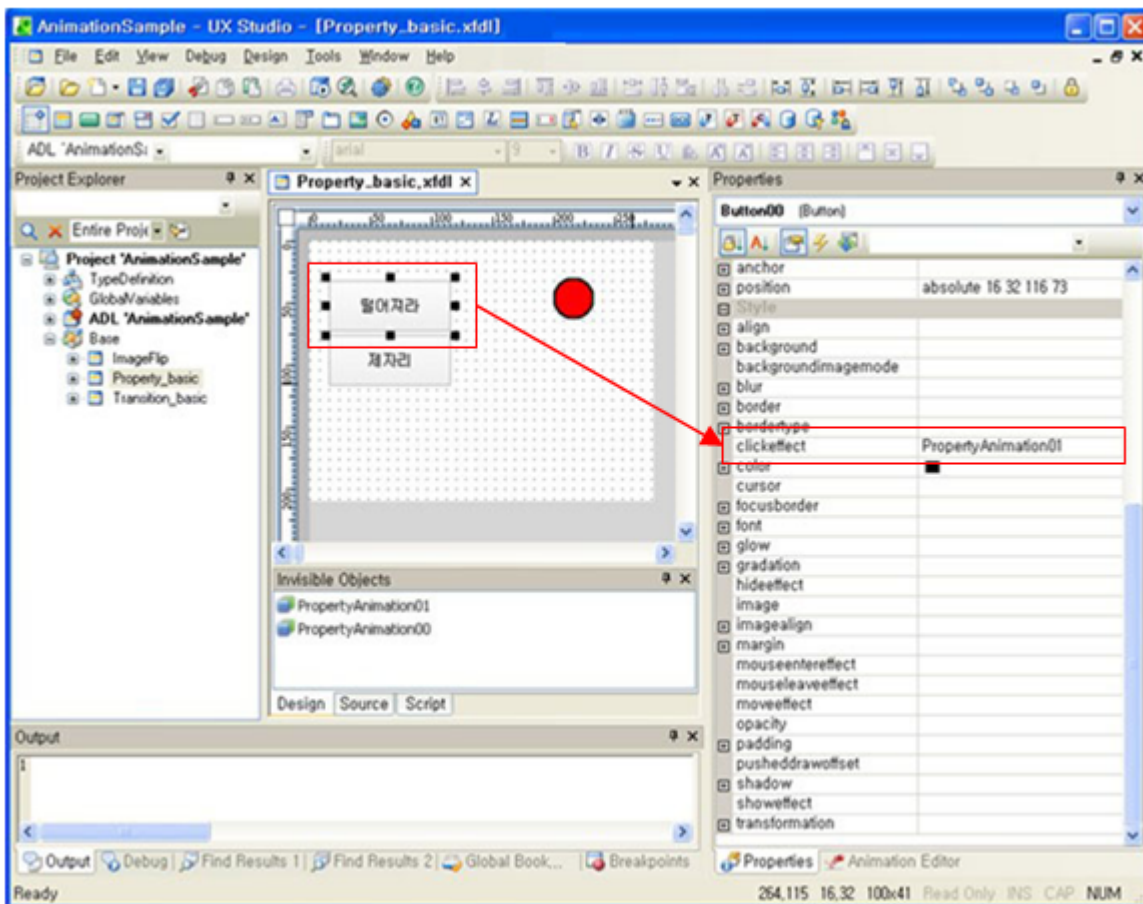
```
// *****
// 마우스가 해당 컴포넌트 위로 올라왔을 때, Property Animation을 이용하여
// 해당 컴포넌트의 width를 늘려줍니다.
// *****
function Button_onmouseenter(obj:Button, e:MouseEventInfo)
{
    PropertyAnimation00.targetcomp = obj.name
    PropertyAnimation00.tovalue = 150;
    PropertyAnimation00.run();
}

// *****
```

```
// 마우스가 해당 컴포넌트를 벗어 났을 때, Property Animation을 이용하여
// 해당 컴포넌트의 width를 줄여줍니다.
// *****
function Button_onmouseleave(obj:Button, e:MouseEventInfo)
{
    PropertyAnimation00.targetcomp = obj.name;
    PropertyAnimation00.tovalue = 100;
    PropertyAnimation00.run();
}
```

4단계 (언제): 버튼을 클릭했을 때 공이 떨어지는 효과를 연결합니다.

Button00(공을 떨어뜨리는 버튼)의 Event trigger Property인 clickeffect Property에 PropertyAnimation01을 연결하여 Button00을 클릭할 경우, PropertyAnimation01이 작동하도록 합니다.



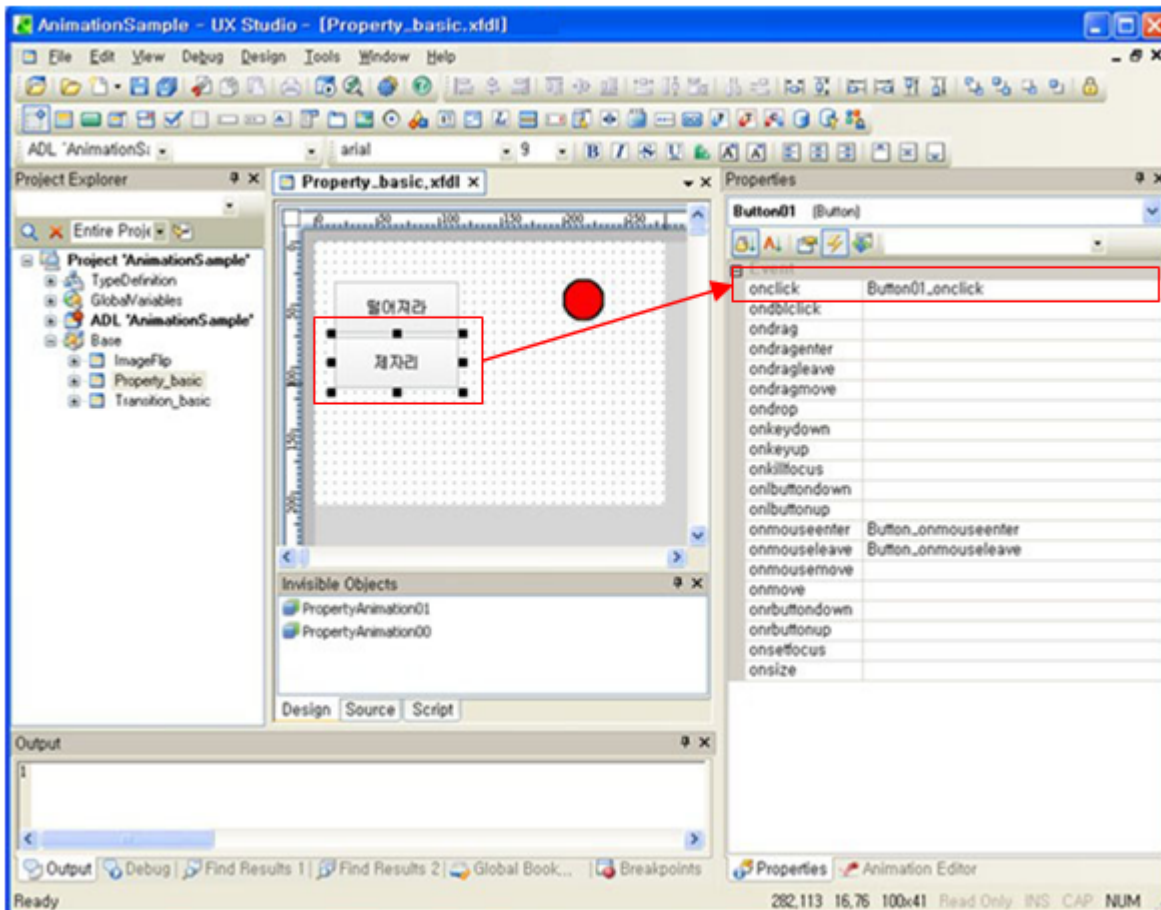
Button01(공을 제자리로 위치시키는 버튼)도 Event trigger Property를 이용할 수 있지만 여기서는 좀 더 다양한 사용 예를 보여주기 위하여 일반 event처리 합니다. 상세한 예는 5단계를 참조하세요.

5단계 (언제/어떻게): 버튼을 클릭했을 때 공을 제자리로 위치시키도록 Animation을 연결하고 실행하는 코딩을 합니다.

Button01(공을 제자리로 위치시키는 버튼)도 Event trigger Property를 이용할 수 있지만 여기서는 좀 더 다양한 사

용 예를 보여주기 위하여 일반 event처리 합니다.

우선, Button01의 onclick event를 event function에 연결합니다.



event function의 coding은 아래와 같이 등록합니다.

```
// *****
// Button01을 클릭했을 때, 동적으로 Property Animation Object를 생성하고
// 해당 Animation Object에 상세한 설정을 한 후 작동시킵니다.
// PropertyAnimation01에 의하여 다른 위치로 옮겨진 공을
// 다시 제자리로 되돌아 가도록 코딩 합니다.
// *****

function Button01_onclick(obj:Button, e:ClickEventInfo)
{
    PropertyAnimation02 = new PropertyAnimation();// Property Animation의 생성
    PropertyAnimation02.duration = 500; // 0.5동안 작동.
    PropertyAnimation02.interpolation = Interpolation.linear; // linear효과.
    PropertyAnimation02.targetcomp = Shape00; // Animation Target을 공모양의 shape로 함
    PropertyAnimation02.targetprop = "position.y"; // y축의 이동
    PropertyAnimation02.endingmode = "to";
}
```

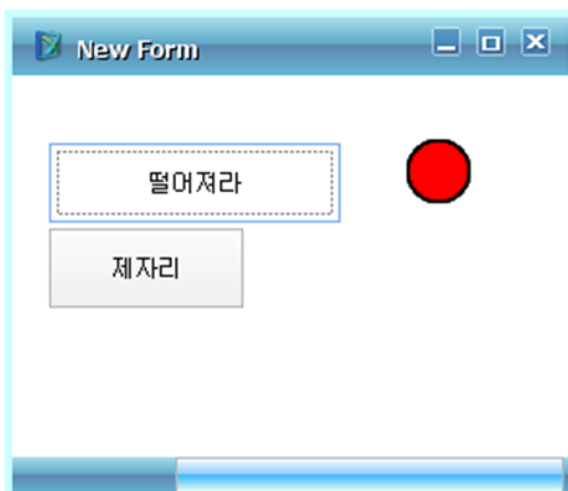
```
PropertyAnimation02.starttime = 0;
PropertyAnimation02.tovalue = 30; // y축의 값이 30이면 원래 있던 위치임.
PropertyAnimation02.run(); // Animation 실행.
}
```

다음은 UX-Studio에서 Quick View를 실행한 후의 화면입니다.



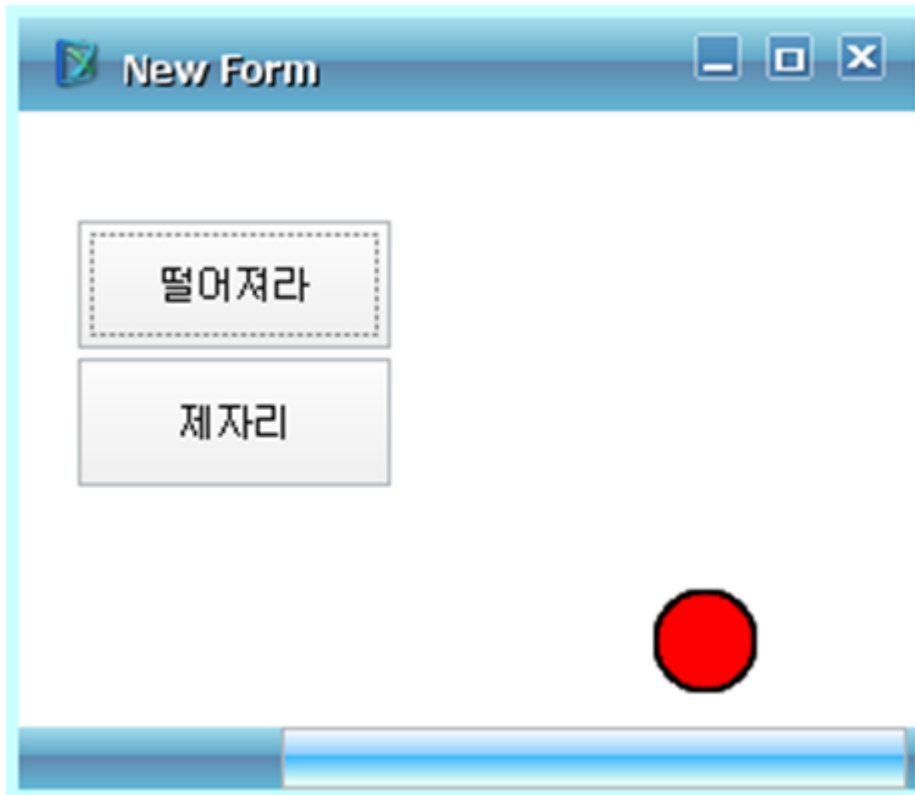
여기서 Mouse를 버튼 위에 올려 놓으면 버튼의 길이가 길어지고, 버튼 위에서 벗어나면 버튼의 길이가 원래 크기로 돌아옵니다.

실행 화면은 아래와 같습니다.



여기서 “떨어져라”버튼을 클릭하면 오른쪽의 빨간색 공이 아래로 떨어집니다.

실행화면은 아래와 같습니다.



그 다음으로 “제자리” 버튼을 클릭하면 오른쪽의 빨간색 공이 제자리로 돌아갑니다.

실행화면은 아래와 같습니다.



3.4.3 Property Animation의 Property

다음은 Property Animation의 Property 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요

Name	Description
byvalue	PropertyAnimation의 offset 값을 설정하는 property 입니다.
duration	Animation 의 지속 시간을 지정하는 Property 입니다.
endingmode	PropetyAnimation이 종료된 이후에 Property 값에 어떤 값을 적용할지 지정하는 property 입니다.
fromvalue	PropertyAnimation 의 시작값을 설정하는 property 입니다.
interpolation	Animation의 진행에 따른 변화값을 구하기 위한 interpolation 함수를 지정하는 Property 입니다. Interpolation Object 에 미리 지정되어있는 predefined interpolation 함수를 사용할 수도 있고 user 가 직접 정의한 script 함수를 설정할 수도 있습니다.
id	서로 구분하기 위한 고유의 id 지정하는 Property입니다.
starttime	Animation의 시작 시간을 지정하는 Property 입니다. Animation 이 Run 된 경우에 설정한 시간만큼 지연된 후에 Animation 이 진행됩니다.
targetcomp	Animation이 적용될 target component 를 지정하는 Property 입니다.
targetprop	PropertyAnimation이 적용될 target property 를 지정하는 Property 입니다.
tovalue	PropertyAnimation의 끝값을 설정하는 property 입니다

3.4.4 Property Animation의 Method

다음은 Property Animation의 Method 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요.

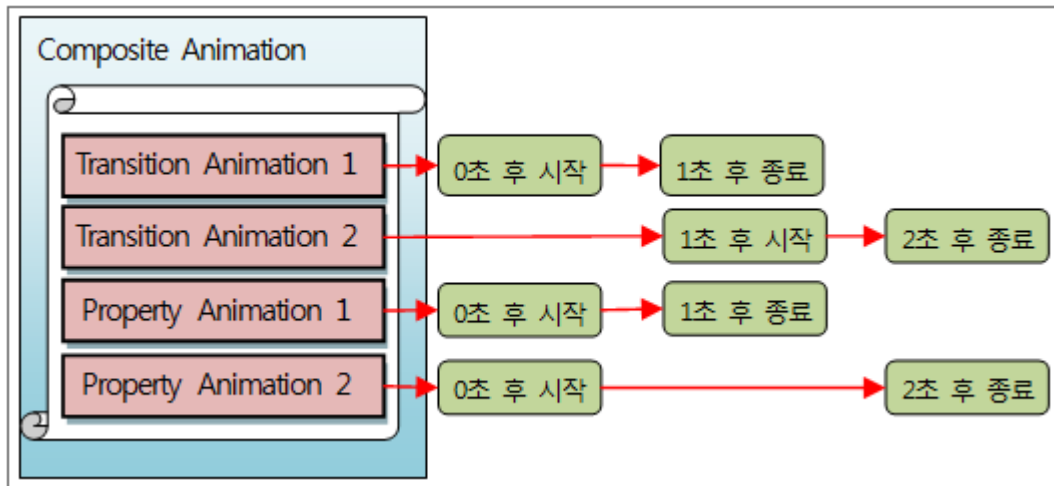
Name	Description
run	Animation 을 시작하는 method 입니다.
stop	Animation 을 종료하는 method 입니다.

3.5 Composite Animation

Composite Animation이란 Property Animation들과 Transition Animation들의 조합된 형태입니다. 그러므로, Property Animation과 Transition Animation들을 별도로 실행해도 유사한 효과를 얻을 수 있습니다. 그러나, 별도로 실행할 경우는 동시에 Trigger를 걸어줄 수 없으므로 미세하게 조금씩 시작시간의 차이가 나게 되어, 복합된 Ani

animation효과를 동기화 시키지 못합니다.

아래의 그림은 4개의 Animation들을 1개의 Composite Animation에 등록한 것을 도식화한 것입니다.



위 그림의 Composite Animation을 실행할 경우, 4개의 Animation이 작동하고 다음의 사항을 예측할 수 있습니다.

- Transition Animation1이 종료된 후에 Transition Animation 2가 시작됩니다.
- Property Animation1, Property Animation2와 Transition Animation 1은 동시에 시작합니다.
- Property Animation1이 종료된 이후에도 Property Animation2는 1초간 계속 실행합니다.
- Composite Animation은 시작 후 2초간 작동합니다.

3.5.1 Composite Animation의 예

여기서는 대포를 쏘아서 목표물에 떨어지면 폭발하는 Composite Animation을 예제로 작성해 보겠습니다.

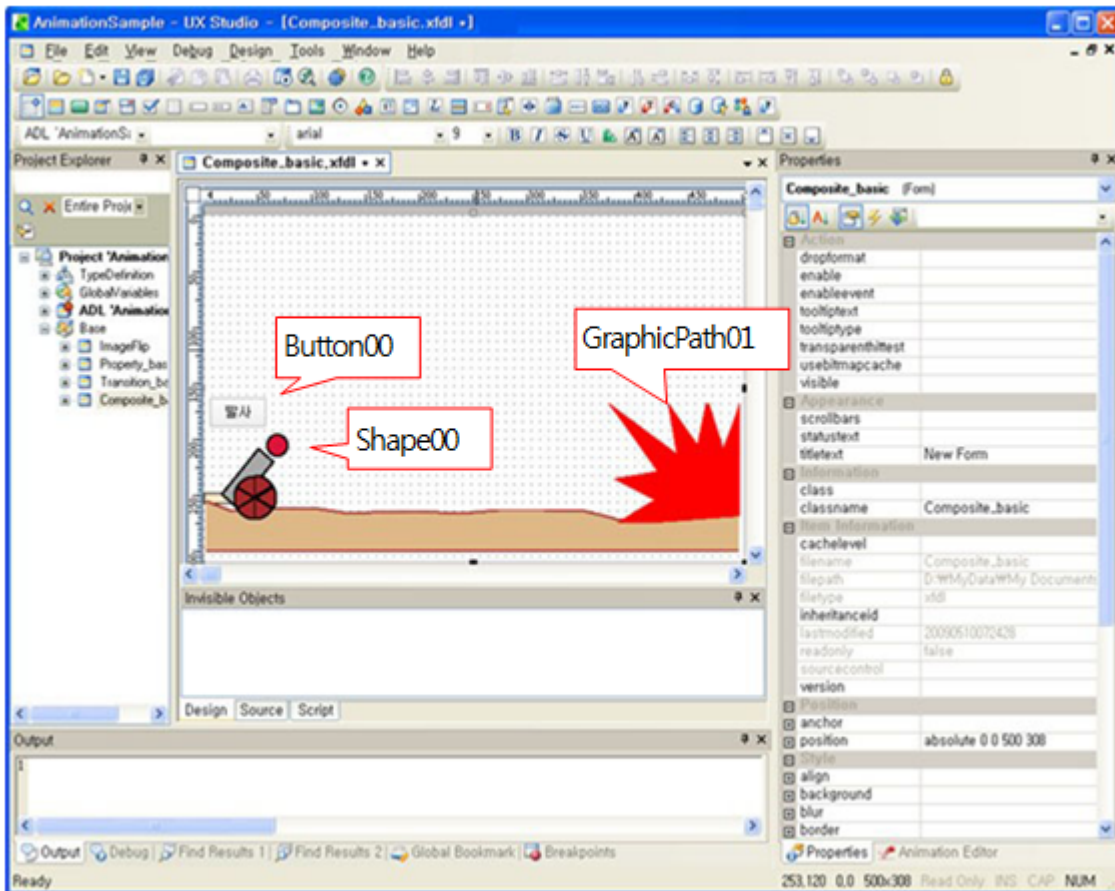
이 예제는 아래의 기능을 갖습니다.

- 발사버튼을 클릭하면 대포가 포탄을 발사합니다.
- 포탄은 3초간의 탄도 곡선으로 날아가서 목표에 떨어집니다.
- 포탄이 목표에 떨어지면 폭발합니다.

1단계: Animation Target을 선정합니다.

button컴포넌트 1개, shape 컴포넌트들로 대포와 포탄을 그려줍니다.

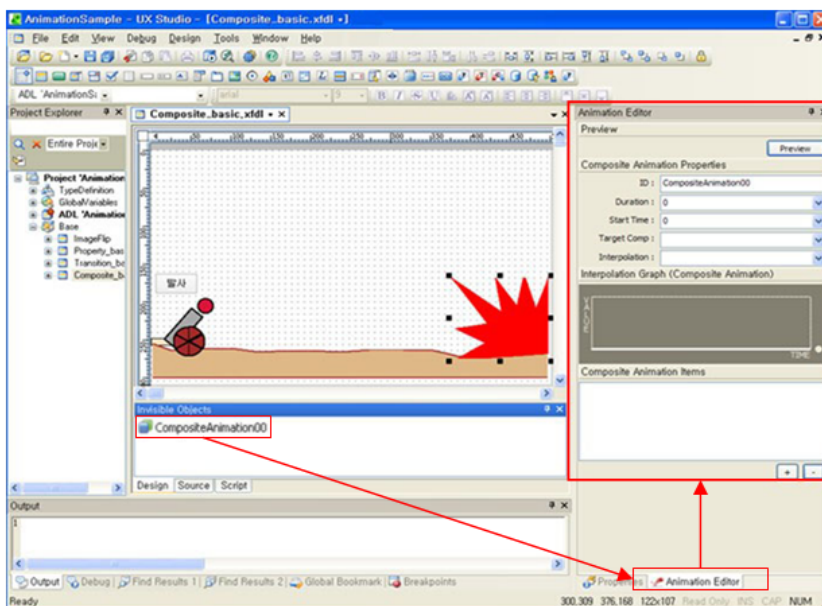
GraphicPath 컴포넌트로 바닥과 폭발모양을 그려줍니다.



위의 그림에서 빨간색 설명박스로 표시한 컴포넌트들이 Animation과 연관 있는 컴포넌트 입니다. 여기서는 이미지가 아닌 GraphicPath나 Shape 컴포넌트로 모든 화면 요소를 구성했으나, 필요에 따라서 이미지 처리해도 무방합니다.

2단계 (어떻게): Composite Animation Object를 생성합니다.

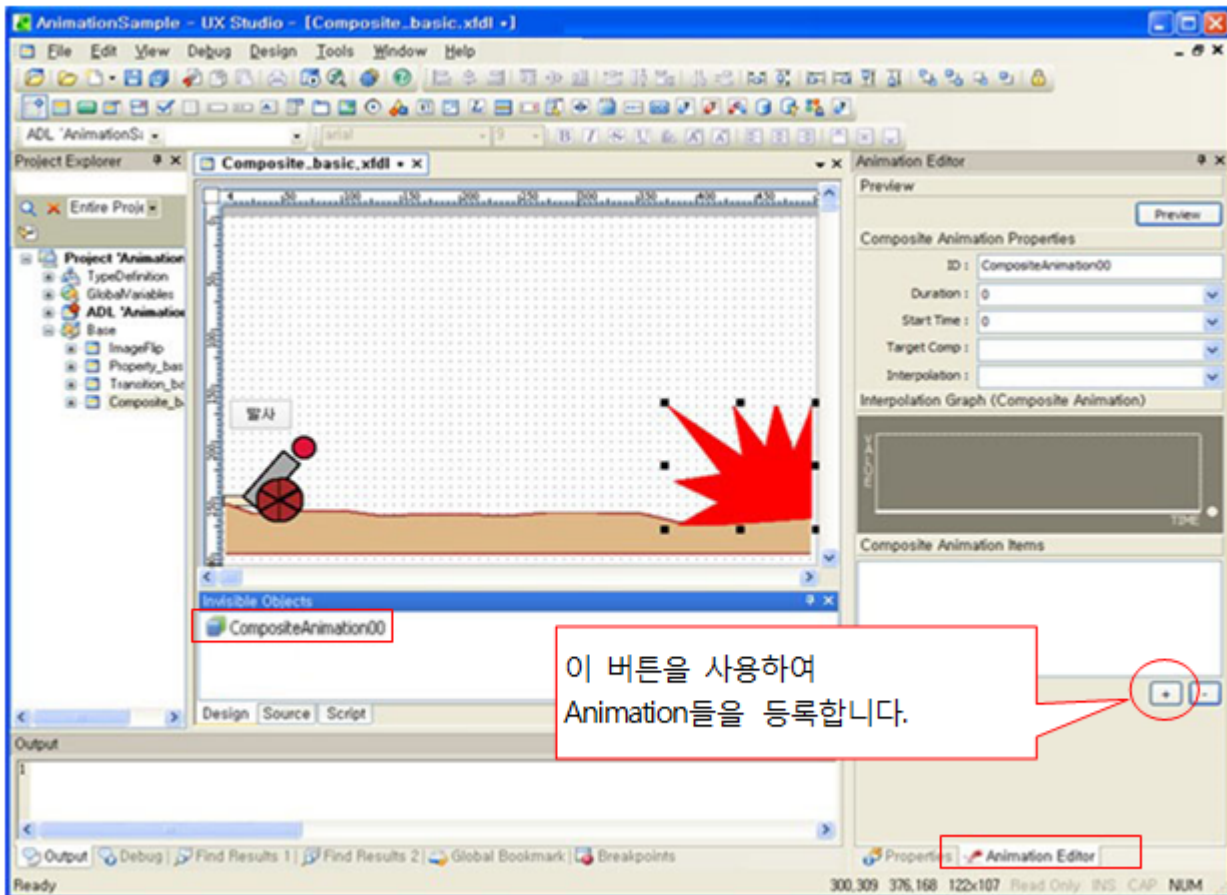
Composite Animation Object 한 개를 생성합니다.



Composite Animation은 Animation Editor없이 편집이 어렵습니다. 그러므로 Composite Animation을 생성한 이후에 Animation Editor를 open하세요.

3단계 (어떻게): Composite Animation Object에 여러 개의 Animation들을 등록합니다.

여기서는 5개의 Animation들을 등록했습니다.

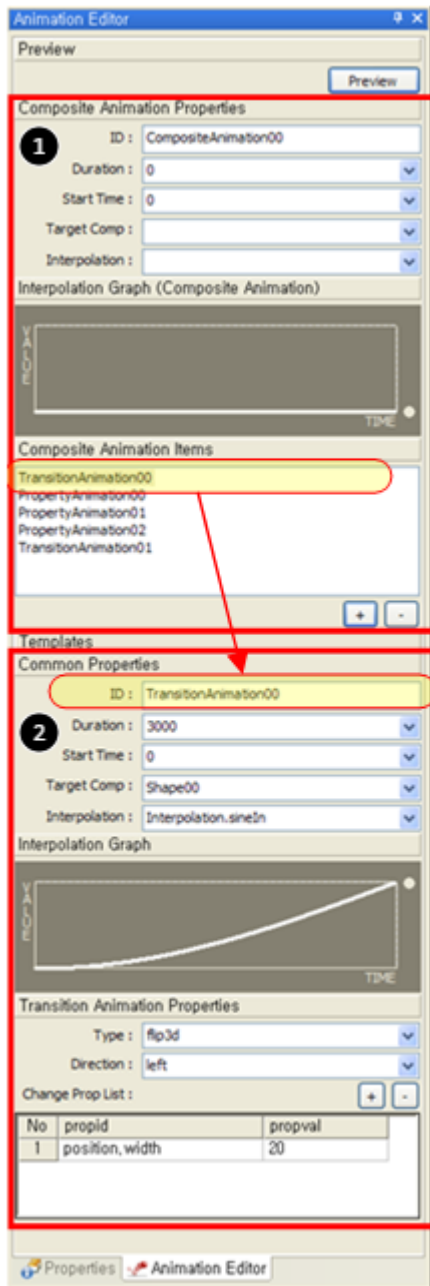


“+”버튼으로 등록할 때, “Transition Animation”과 “Property Animation”중 하나를 선택해야만 합니다. 여기서는 두 개의 Transition Animation과 세 개의 Property Animation을 등록합니다.

4단계 (어떻게): 등록된 5개의 Animation들의 상세 정보를 입력합니다.

여기서는 5개의 Animation들을 등록했습니다.

아래의 그림은 Animation Editor의 화면입니다.



1	Composite Animation을 편집하는 영역입니다.
2	Composite Animation에 등록된 Animation들을 편집하는 영역입니다. 여기서는 TransitionAnimation00의 Edit 화면을 보여주고 있습니다. 즉, 등록된 5개의 Animation을 편집하기 위해서 이런 화면을 5번 편집해야 합니다.

Composite Animation은 id외에 어떤 값도 설정하지 않습니다. 왜냐하면, Composite Animation의 Property값은 Animation효과에 영향을 미치지 않고, 등록된 Animation들의 Property값을 설정하지 않았을 때에 사용하는 Default값이기 때문입니다. 그러므로, 여기서는 Composite Animation Property값을 설정할 필요가 없습니다.

5개의 등록된 Animation들의 Property값들은 아래와 같습니다.

1. TransitionAnimation00의 설정 값 (포탄 회전)

포탄이 날아가면서 1회전 하는 효과를 줍니다.

Common Properties

ID : TransitionAnimation00

Duration : 3000

Start Time : 0

Target Comp : Shape00

Interpolation : Interpolation.sineIn

Interpolation Graph

Transition Animation Properties

Type : flip3d

Direction : left

Change Prop List :

No	propid	propval
1	position,width	20

Shape00이 포탄입니다.

Transition Animation은 이전이미지, 이후 이미지가 있어야만 작동하므로 무조건 propid를 1개 이상을 등록해야만 합니다. 여기서는 동일 값을 등록했습니다.

2. PropertyAnimation00의 설정 값 (포탄 x축 이동)

포탄을 x축 값을 400 pixel만큼 이동합니다. 이로 인해 포탄이 왼쪽에서 오른쪽으로 3초 동안 이동합니다.

Common Properties

ID : PropertyAnimation00

Duration : 3000

Start Time : 0

Target Comp : Shape00

Interpolation : Interpolation.linear

Interpolation Graph

Property Animation Properties

Target prop : position.x

From value :

To value :

By value : 400

Ending mode : to

Shape000이 포탄입니다.

x축의 이동은 linear하게 이동합니다.

3. PropertyAnimation01의 설정 값 (포탄 y축 이동-위로 올라감)

포탄을 y축 값을 -150 pixel만큼 이동합니다. 이로 인해 포탄이 1.5초 동안 최고점까지 올라갑니다.

Common Properties

ID : PropertyAnimation01

Duration : 1500

Start Time : 0

Target Comp : Shape00

Interpolation : Interpolation.sineOut

Interpolation Graph

Property Animation Properties

Target prop : position.y

From value :

To value :

By value : -150

Ending mode : to

Shape000이 포탄입니다.

y축의 sin곡선으로 처음에는 빠르게 올라가고 최고점에 도착할 때는 느리게 올라갑니다.

4. PropertyAnimation02의 설정 값 (포탄 y축 이동-아래로 떨어짐)

포탄을 y축 값을 150 pixel만큼 이동합니다. 이로 인해 포탄이 1.5초 동안 최고점에서 최저점으로 떨어집니다.

Common Properties

ID : PropertyAnimation02

Duration : 1500

Start Time : 1500

Target Comp : Shape00

Interpolation : Interpolation.sineIn

Interpolation Graph

VALUE

TIME

Property Animation Properties

Target prop : position.y

From value :

To value :

By value : 150

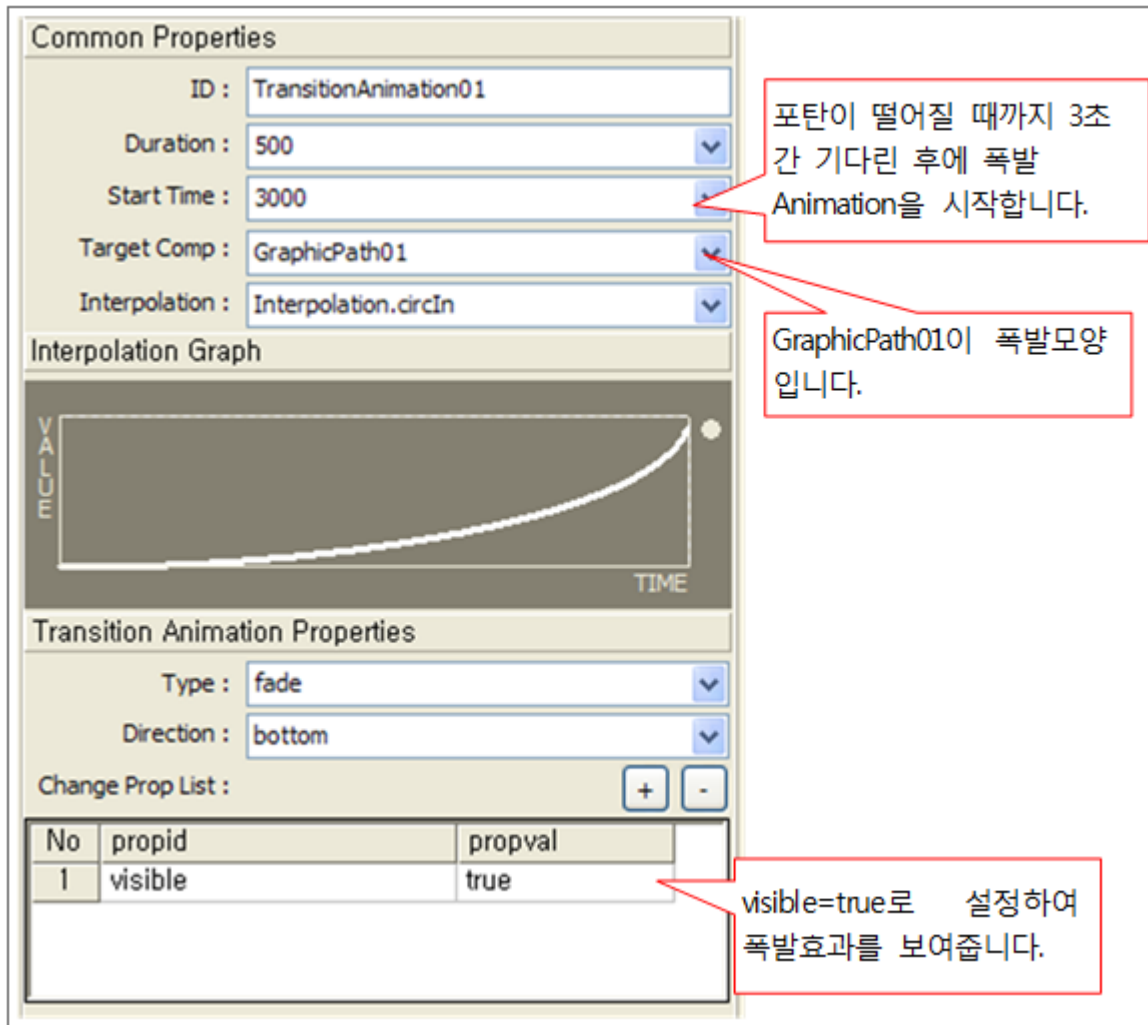
Ending mode : to

포탄이 최고점에 도달할 때까지 1.5초 지연 후에 떨어지는 Animation을 시작합니다.

y축의 sin곡선을 뒤집어서 적용하여, 처음에는 느리게 떨어지다가 최저점에서 빠르게 떨어집니다.

5. TransitionAnimation01의 설정 값 (폭발하는 모양을 보여줌)

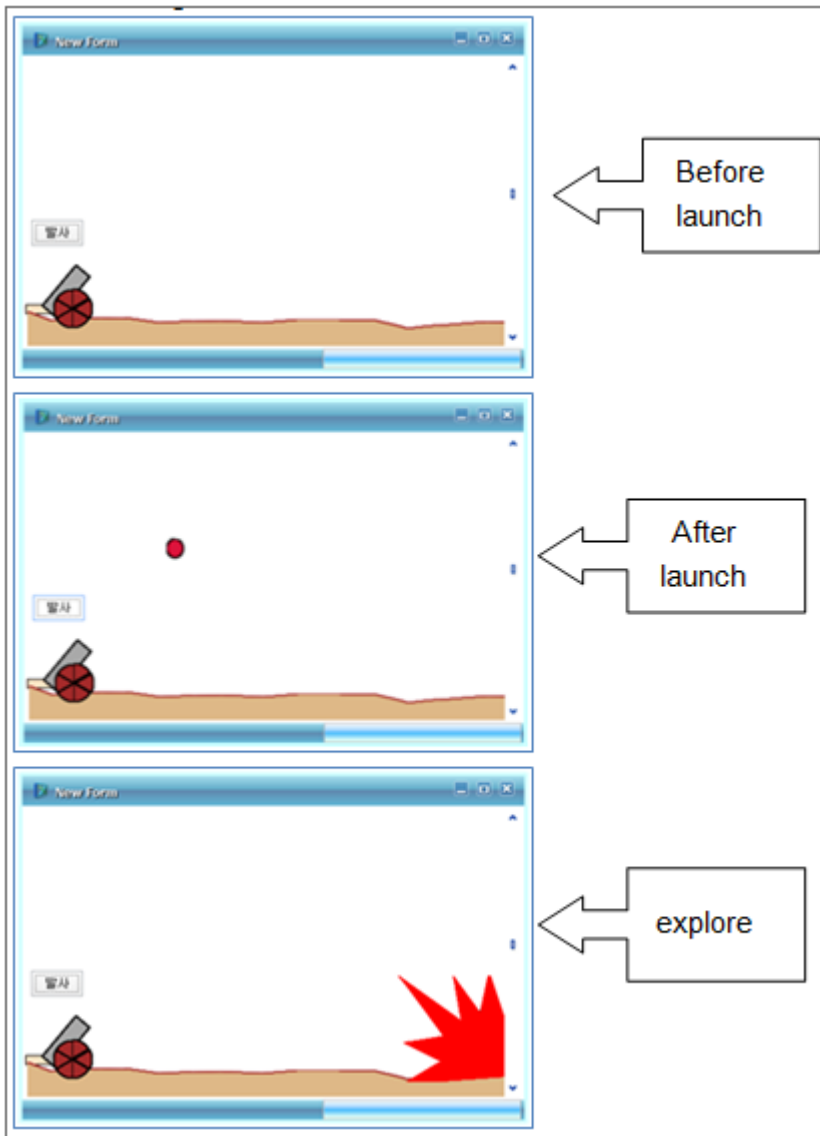
폭발하는 모양인 GraphicPath01 컴포넌트를 visible=false에서 visible=true로 상태를 바꾸어 fade효과로 보여줍니다.



5단계 (언제): 발사 버튼을 클릭하면 대포를 발사하도록 합니다.
발사 버튼의 onclick event function에 아래와 같이 입력합니다.

```
function Button00_onclick(obj:Button, e:ClickEventInfo)
{
    Shape00.position.x = 59; // 포탄을 발사 위치로 옮기기
    Shape00.position.y = 196; // 포탄을 발사 위치로 옮기기
    Shape00.visible = true; // 포탄을 보이게 함
    CompositeAnimation00.run(); // 포탄 발사
}
```

다음은 UX-Studio에서 Quick View를 실행한 후의 화면입니다.



3.5.2 Composite Animation의 Property

다음은 Composite Animation의 Property 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요.

Name	Description
duration	Animation의 지속 시간을 지정하는 Property 입니다.
interpolation	Animation의 진행에 따른 변화 값을 구하기 위한 interpolation 함수를 지정하는 Property 입니다. Interpolation Object에 미리 지정되어있는 predefined interpolation 함수를 사용할 수도 있고 user 가 직접 정의한 script 함수를 설정할 수도 있습니다.
items	CompositeAnimation의 Item 정보를 담고 있는 Property 입니다. CompositeAnimation의 item 은 PropertyAnimation, TransitionAnimation으로 구성됩니다.
id	서로 구분하기 위한 고유의 id를 지정하는 Property입니다.

Name	Description
starttime	Animation의 시작 시간을 지정하는 Property입니다. Animation 이 Run 된 경우에 설정한 시간만큼 지연된 후에 Animation 이 진행됩니다.
targetcomp	Animation이 적용될 target component를 지정하는 Property 입니다.

3.5.3 Composite Animation의 Method

다음은 Composite Animation의 Method 리스트입니다. 상세한 내용은 XPLATFORM Reference Guide를 참조하세요.

Name	Description
run	Animation 을 시작하는 method 입니다.
stop	Animation 을 종료하는 method 입니다.

3.6 향상된 Animation 기능의 사용

Animation 기능은 그 응용방법에 따라서 다양하게 복잡하게 사용할 수 있습니다. 여기서는 몇 가지 추가적인 사용방법을 설명합니다.

3.6.1 자동 Trigger사용하기

script에서 run()을 호출하여 수동으로 Animation을 시작한 것을 수동 Trigger라고 합니다. 자동 Trigger는 이러한 script coding을 하지 않고, 자동으로 Animation을 시작하게 하는 Trigger입니다.

다음은 자동 Trigger들의 리스트입니다.

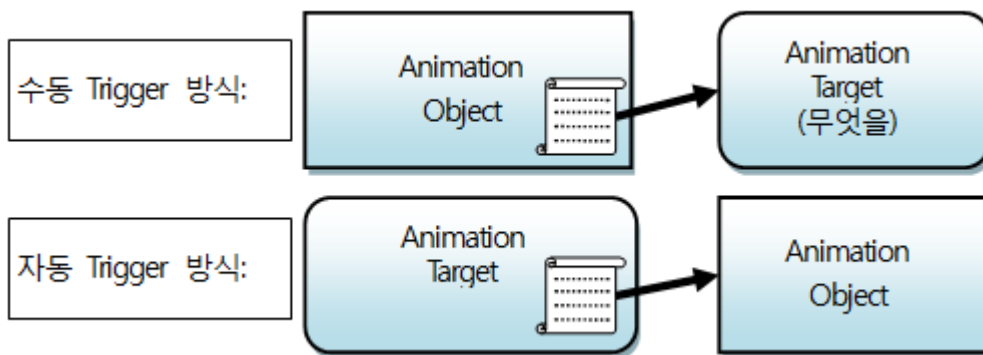
Trigger종류	정의	대응 Trigger Property
MouseEnter	마우스 포인터가 진입할 때	MouseEnterEffect
MouseLeave	마우스 포인터가 빠져 나갈 때	MouseLeaveEffect
Click	마우스 Click 발생시	ClickEffect
Show	Visible 상태가 될 때	ShowEffect
Hide	Invisible 상태가 될 때	HideEffect
UrlChange	Url이 바뀔 때	UrlChangeEffect
TapChange	Tab index가 바뀔 때	TabIndexChangeEffect

여기서는 Trigger들 중에서 MouseEnter를 예로 설명하겠습니다. MouseEnter Trigger는 Mouse가 특정 컴포넌트의 영역 안으로 들어왔을 때 발생하는 Trigger입니다.

1단계 (어떻게): Animation Object를 생성하여 Animation종류를 선정합니다.

Animation종류를 선택하는 방법은 [Transition Animation의 예](#)의 2단계와 동일합니다. 수동 Trigger는 Animation Target을 먼저 생성하지만, 자동 Trigger는 Animation Object를 먼저 생성합니다.

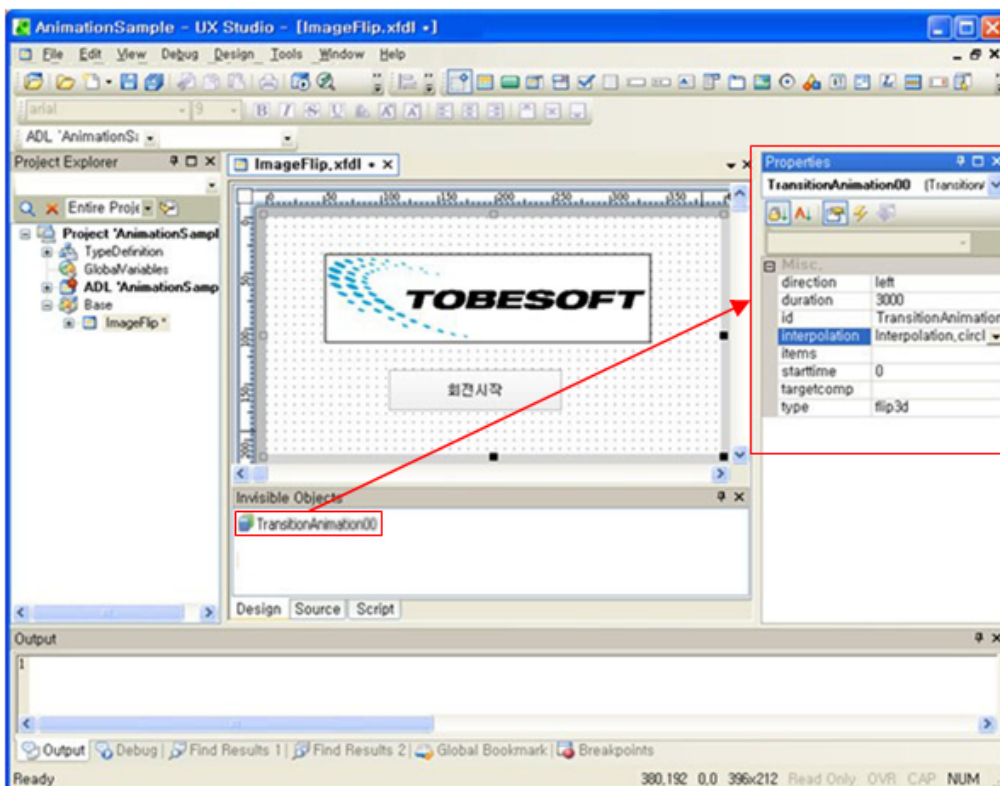
아래의 그림을 보면, 수동 Trigger와 다르게 자동 Trigger는 Animation Target이 Animation Object 정보를 가져야 하기 때문에 Animation Object를 먼저 생성해야만 합니다.



자세한 설명을 단계적으로 설명하겠습니다.

다음은 생성하는 예입니다.

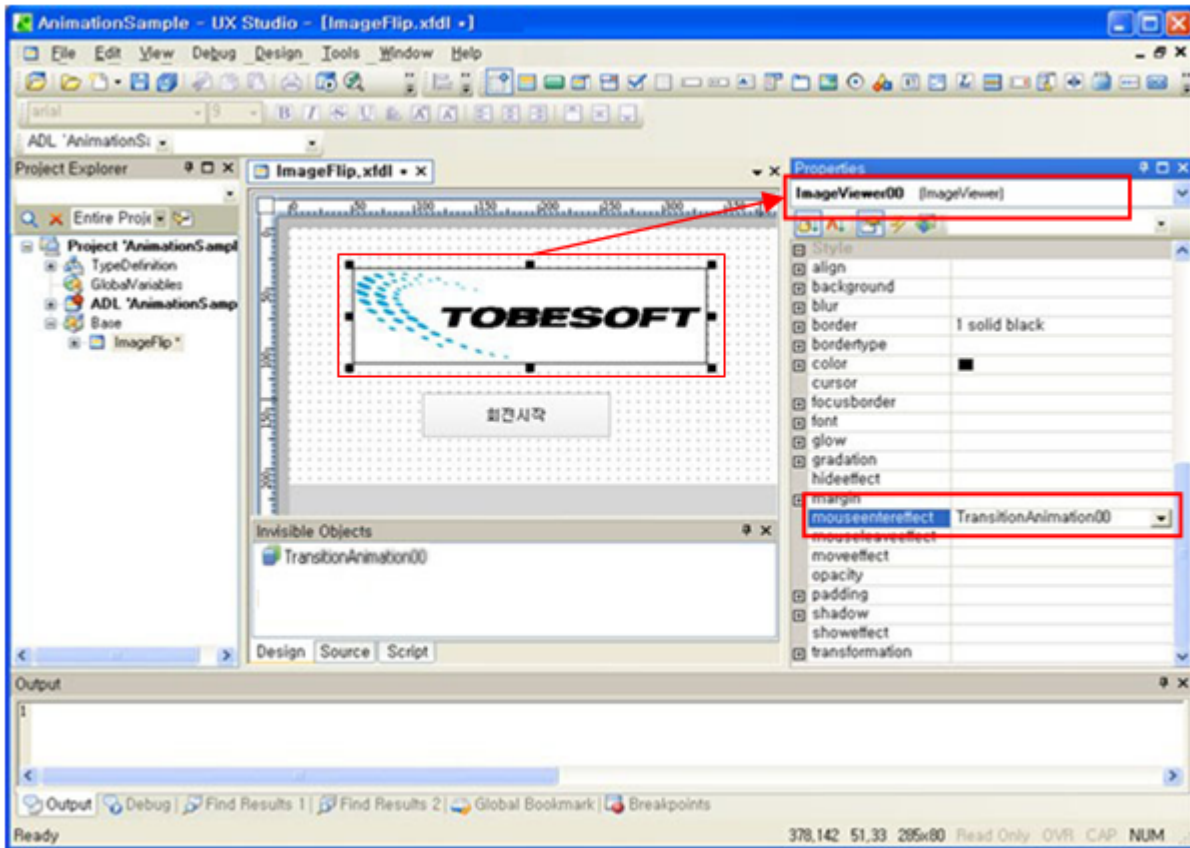
1단계 (어떻게/얼마나): Animation Object를 생성하고 상세 값을 설정합니다.



targetcomp값을 설정하지 않는 이유는 Animation효과를 적용 받을 Animation Target이 아직 결정되지 않았기 때문입니다.

2단계 (무엇을): 회전 대상이 선정하고, 어떤 경우에 회전할지를 결정합니다.

“TOBESOFT” 로고를 보여주는 ImageViewer00 컴포넌트의 영역 안에 Mouse가 들어왔을 때 회전하도록 Animation Object를 연결합니다.



위의 그림을 보면, Animation Target인 ImageViewer00의 mouseentereffect property값에 TransitionAnimation00을 연결하는 것을 알 수 있습니다.

3.6.2 User Interpolation만들기

XPLATFORM이 제공하는 interpolation외에 개발자는 직접 interpolation을 개발할 수 있습니다. interpolation함수를 작성하고 Animation의 interpolation Property에 등록하기만 하면 됩니다.

다음은 interpolation함수의 형태입니다.

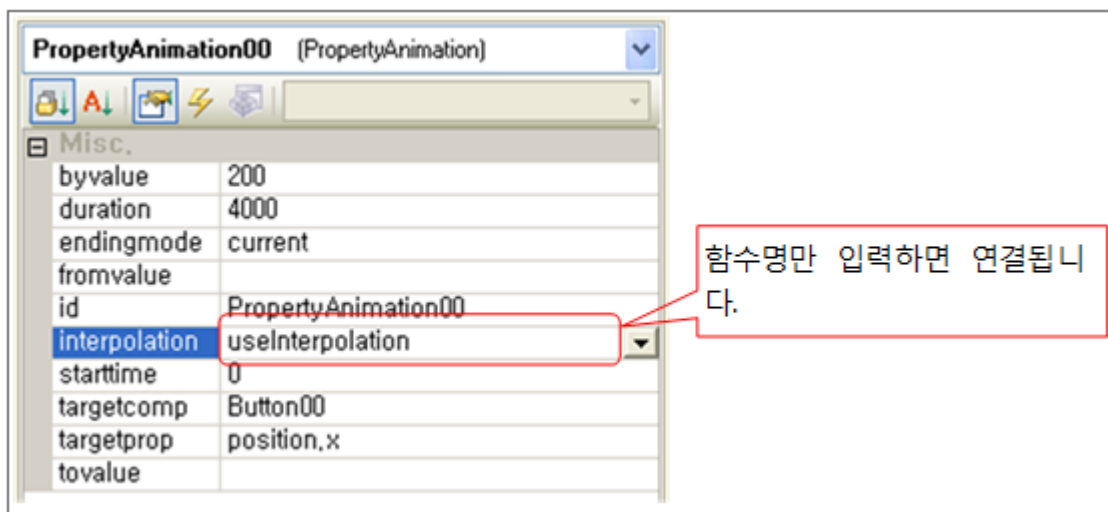
```
function useInterpolation(info)
{
    var d = info.duration;
```

```
var t = info.progresstime;
info.progressvalue = t/d;
}
```

- 함수명은 자유롭게 입력합니다.
 - 함수의 argument인 info object는 3개의 property를 갖습니다.
 - 계산된 결과 값은 info.progressvalue에 할당합니다.
- 참고로, 여기서 사용한 식은 "Interpolation.linear"과 같은 효과를 갖습니다.

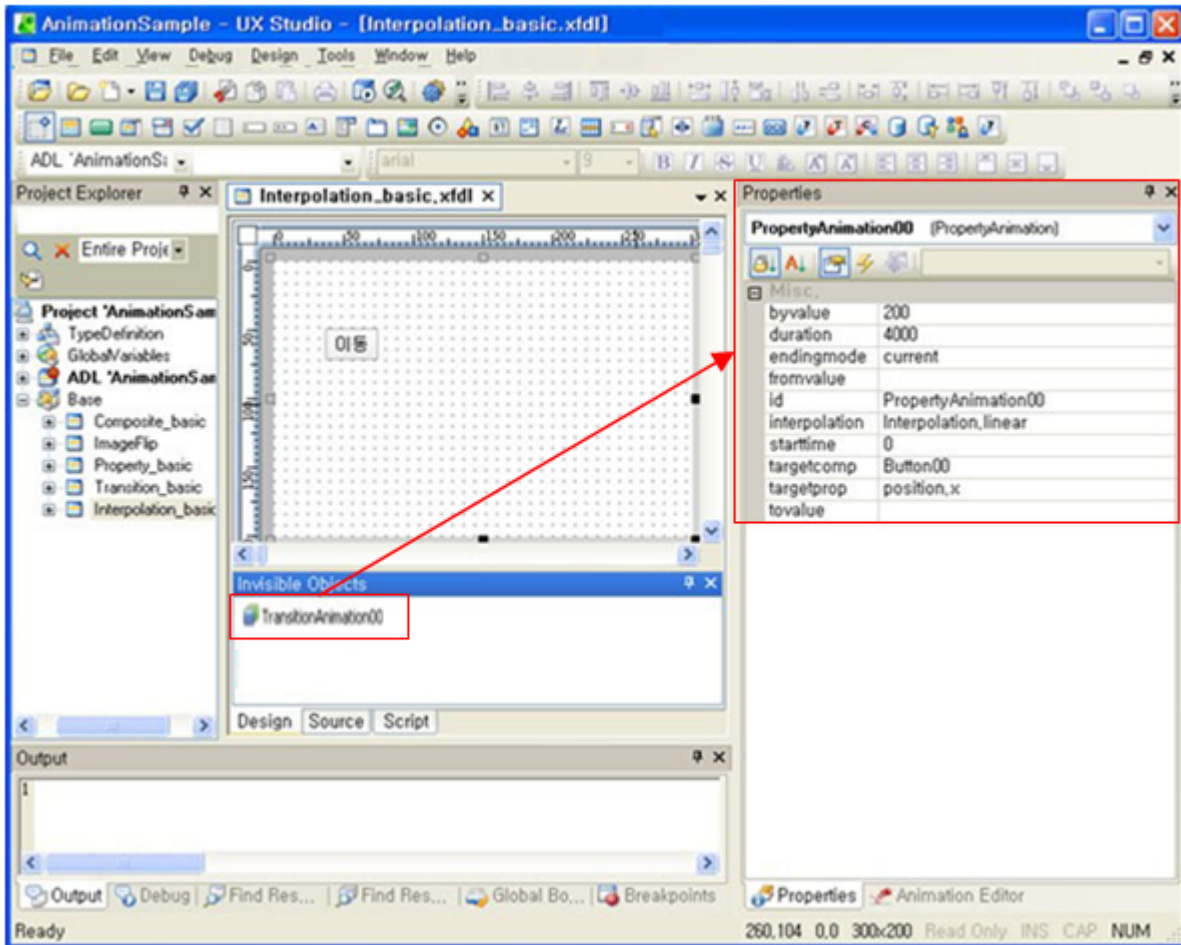
속성	설명
info.duration	Animation 총 수행시간 (readonly)
info.progresstime	Animation의 실행경과 시간 (readonly)
info.progressvalue	0~1 사이의 값으로, interpolation의 결과값으로 이 property에 값을 할당합니다.

개발자가 직접 개발한 User Interpolation은 해당 Animation의 Interpolation에 등록하면 됩니다. 다음은 Animation Property창에서 User Interpolation을 입력한 화면입니다.



이제 Interpolation을 사용하여 Animation효과를 바꾸어 보겠습니다.

아래의 화면은 버튼을 클릭했을 때 버튼이 왼쪽에서 오른쪽으로 이동하게 하는 Animation을 설정한 화면입니다. 여기서는 Animation Trigger에 대해서는 언급하지 않겠습니다.



Property Animation Object의 주요 설정 값은 아래와 같습니다.

Property명	값	의미
byvalue	200	200 pixel만큼 버튼을 이동
duration	4000	4초간 이동
endingmode	current	마지막 값이 최종값임
interpolation	Interpolation.linear	가속 없이 일정속도로 이동
targetcomp	Button00	“이동”버튼에 Animation효과를 줌
id	PropertyAnimation00	Animation효과를 구분하는 id입니다.
targetprop	position.x	“이동”버튼의 x좌표 값에 Animation효과를 줌

이 예제를 실행시키면 버튼을 클릭할 때, 버튼이 왼쪽에서 오른쪽으로 한 번만 이동합니다. 이 것을 두 번 왕복이동을 하여 최종에는 제자리에 오도록 하겠습니다.

이 경우, 2번 왕복을 위한 interpolation은 XPLATFORM이 제공하지 않습니다. 아래와 같이 interpolation 함수를 정의합니다.

```
function useInterpolation(info)
{
    var d = info.duration;
    var t = info.progresstime;
    var kk = 4*t/d;
    if (kk<1)
        info.progressvalue = kk; // 0~1초 사이에 0에서 1로 증가 시킵니다.
    else if (kk<2)
        info.progressvalue = 2-kk; // 1~2초 사이에 1에서 0으로 감소 시킵니다.
    else if (kk<3)
        info.progressvalue = kk-2; // 2~3초 사이에 0에서 1로 증가 시킵니다.
    else
        info.progressvalue = 4-kk; // 3~4초 사이에 1에서 0으로 감소 시킵니다.
}
```

이 함수는 “0>1>0>1>0”으로 info.progressvalue값을 변화시킵니다. 이 함수를 위의 Property Animation의 interpolation property에 적용시키면 버튼이 4초간 2번 왕복이동을 하게 됩니다.

3.6.3 그 밖에 Animation적용

그 밖에도 Frame, Form, Widget에 Animation을 적용할 수 있습니다.

여기서는 간단하게 Animation적용 방법만 설명하겠습니다.

- Frame Animation 적용 방법

단계	설명
Animation생성	GlobalVariable의 Objects에 Animation을 등록합니다.
Frame에 Animation적용	원하는 Frame을 등록한 후, Frame의 openstatuseffect Property에 등록된 Animation명을 설정합니다.
실행	Frame을 Minimize/Maximize할 때 해당 효과가 나타납니다.

- Form Animation 적용 방법

단계	설명
Animation생성	해당 Form의 Animation을 사용하므로, 컴포넌트에 Animation을 적용할 때와 동일한 방법으로 Animation을 등록합니다.
Form에 Animation적용	Form에 Animation을 적용하려면 Property창에서 등록할 수 없고, script로만 등록할 수 있습니다. “AnimationName.targetcomp=this;”라고 script 입력하면 됩니다.

단계	설명
실행	Form Animation Trigger는 컴포넌트의 Animation Trigger와 동일하게 작동합니다.

4.

Frame Tree

이 장에서는 Form들의 상호관계를 좀 더 구체적으로 도식화하여 설명합니다.

4.1 표기법

Frame의 상호관계를 도식화 할 때 사용하는 표기법을 설명합니다.

4.1.1 box

	collection상의 Item으로 실제 이름을 의미하지는 않습니다
---	---------------------------------------

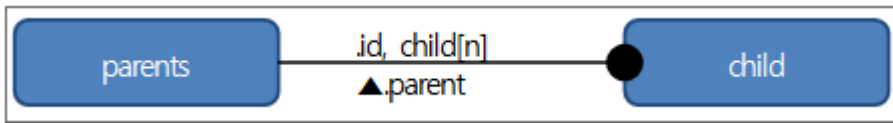
4.1.2 연결선

	1:1 연결, 마름모 쪽이 Child가 됩니다.
	1:N 연결, 원 쪽이 Child가 됩니다.

4.1.3 설명

- Item간의 Script문법은 연결선 위에 표기합니다.
- Parents에서 Child로의 Script는 Prefix없이 표기합니다.

- Child에서 Parents로의 Script는 "▲" Prefix를 사용합니다.



위 표기는 다음을 의미합니다.

- parents와 child는 1:N의 관계입니다.
- Script상에서 "parents.id"로 child에 접근할 수 있습니다.
- Script상에서 parents.child[index]로 child에 접근할 수 있습니다.
- Script상에서 child.parent로 parents에 접근할 수 있습니다.

4.1.4 용어정의

용어	설명
element	component의 property, method, event를 통칭하여 component의 element라고 합니다.

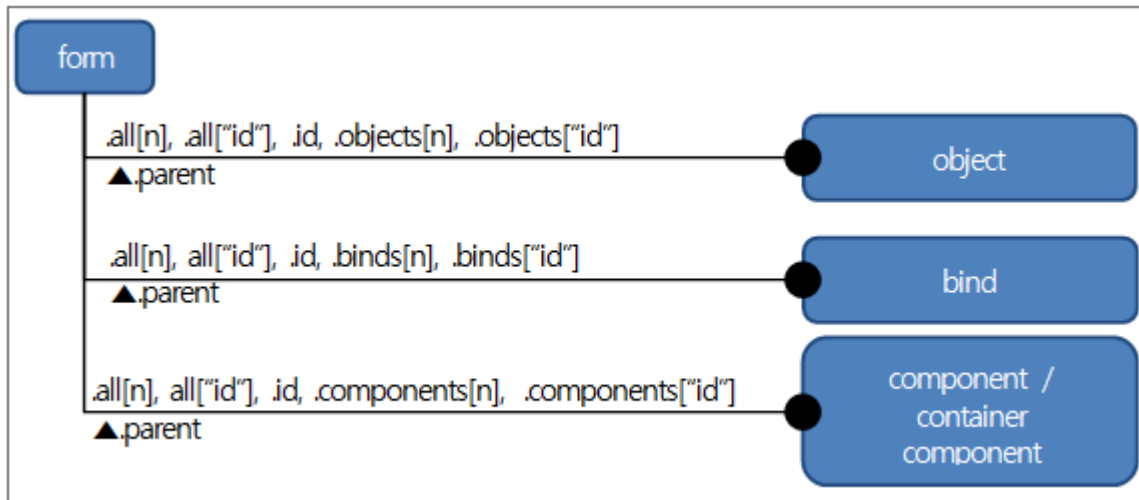
4.1.5 공통으로 사용하는 Syntax

- 배열표시 "[" , "]"는 "(" , ")"로 대체할 수 있습니다.
즉, this.all[0]은 this.all(0)과 동일합니다.
- "[" , "]"와 "(" , ")"내에는 index 또는 "id"가 올 수 있습니다.
즉, this.all[0]과 this.all["btn0"] 둘 다 동일 component를 가리킬 수 있습니다.

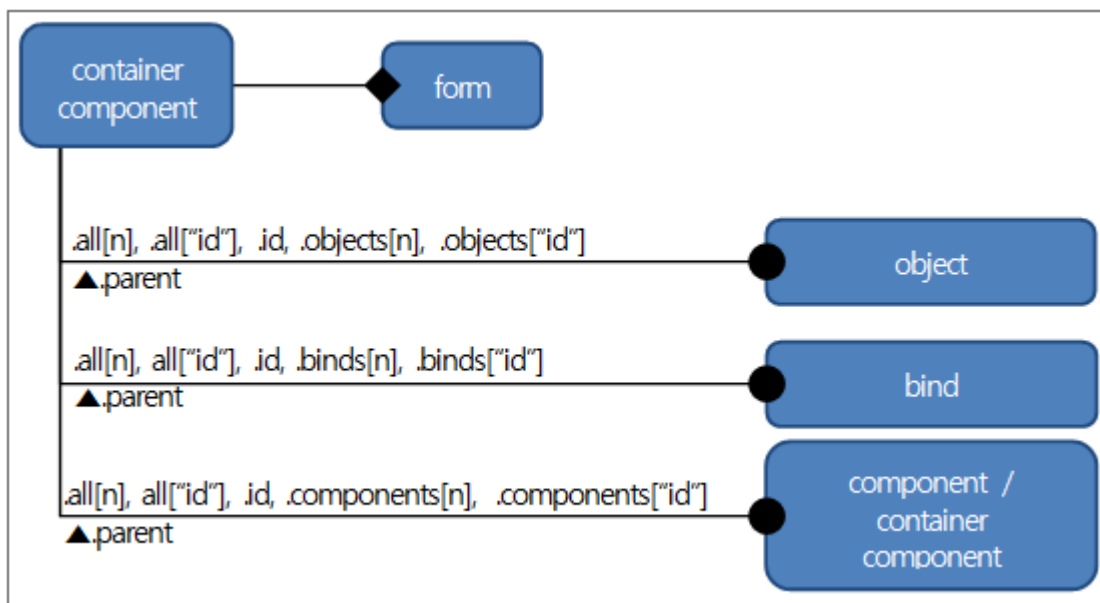
4.2 Form

4.2.1 관계도

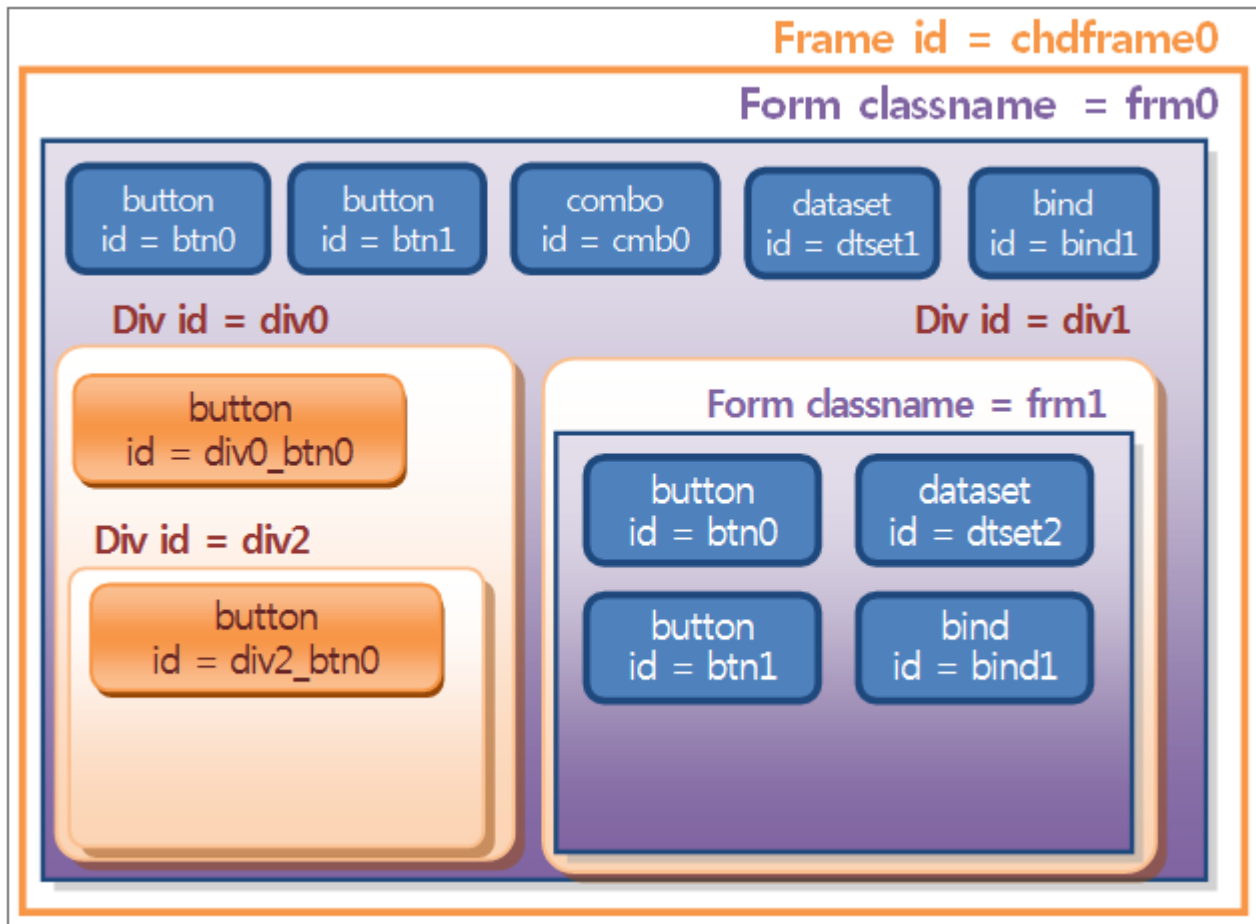
- 일반 Form인 경우



- container component인 경우



4.2.2 Script 예시 화면



index순서를 btn0, btn1, cmb0, dtset1, bind1, div0, div1으로 가정한다.



한 form에서 같은 level에서만 id가 중복될 수 없다. 즉, btn0과 div0_btn0과 div2_btn0은 동일 Level 이 아니므로, 같은 id로 지정하여도 무방합니다.

4.2.3 component / invisible object / bind 의 Script 접근

```
this.classname == "frm0"
this.name == "frm0"
```

- btn0으로의 접근

```

this.all["btn0"].name == "btn0"
this.all[0].name == "btn0"
this.btn0.name == "btn0"
this.components[0].name == "btn0"
this.components["btn0"].name == "btn0"

```

- dtset1으로의 접근

```

this.all["dtset1"].name == "dtset1"
this.all[3].name == "dtset1"
this.dtset1.name == "dtset1"
this.objects[0].name == "dtset1"
this.objects["dtset1"].name == "dtset1"

```

- component / invisible object / bind의 개수

```

this.all.length == 7
this.components.length == 5 //btn0, btn1, comb0, div0, div1
this.binds.length == 1

```

4.2.4 container component에 Script 접근

form을 연결하지 않은 container component는 invisible object / bind를 가질 수 없고 component만 가질 수 있다.

- div0내의 div0_btn0으로의 접근

```

this.div0.btn0.name == "div0_btn0"
this.div0.all[0].name == "div0_btn0"
this.components[3].components[0].name == "div0_btn0"

```

- div2내의 div2_btn0으로의 접근

```

this.div0.div2.div2_btn0.name == "div2_btn0"
this.all["div0"].all["div2"].all["div2_btn0"].name == "div2_btn0"
this.components["div0"].components["div2"].components["div2_btn0"].name == "div2_btn0"

```

4.2.5 form을 연결한 container component에 Script 접근

- frm1에 있는 dtset1의 접근 (frm1안의 script인 경우)

```
this.name== "div1" // 이 값은 form을 연결한 container component의 id이므로 값이 가변적입니다.
this.btn0.name== this.all["btn0"].name == "btn0"
```

- frm1에 있는 dtset1의 접근 (frm0안의 script에서 접근할 경우)

```
this.name == "frm0"
this.div1.btn0.name == "btn0" //"frm1"을 흡수한 "div1"으로만 접근할 수 있습니다.
```

4.2.6 parent의 Script 사용

```
this.div1.dtset2.parent.parent.classname == "frm0"
this.div1.dtset2.parent.name == "div1"// frm1은 div1에 연결되면서 그 특성을 잃게 됩니다.
this.div1.dtset2.parent.classname == undefined// frm1은 div1에 연결되면서 그 특성을 잃게 되어
classname이 존재하지 않게 됩니다.
this.div1.dtset2.parent.name == "div1"
```

4.2.7 container component의 element사용

form을 연결한 container component는 form처럼 components, invisible object, bind (이하 element)를 가질 수 있습니다.

form을 연결하지 않은 container component는 invisible object, bind를 가질 수 없고 component만 가질 수 있습니다.

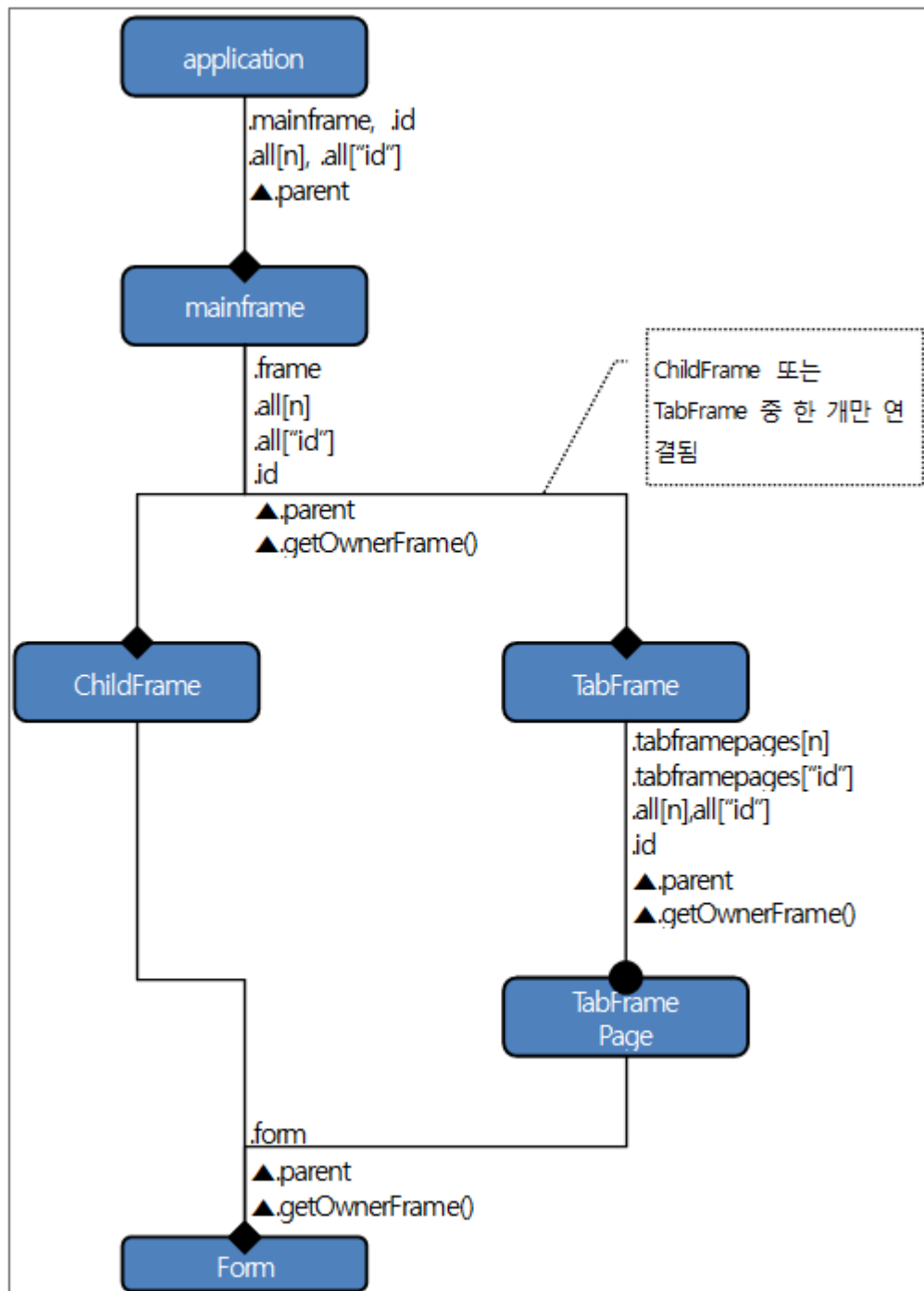
container기능이 없는 component와 invisible object, bind는 element를 가질 수 없습니다.

container component는 연결된 form의 특성을 갖게 됩니다.

- form script에서 this로 접근할 경우, 연결된 form 내부의 element뿐만 아니라 container component의 element에도 접근이 가능합니다.
- container component가 속한 form에 this.component_name으로 접근할 경우, 연결된 form의 모든 elements에 접근이 가능합니다.

4.3 Single Frame 형식의 Application

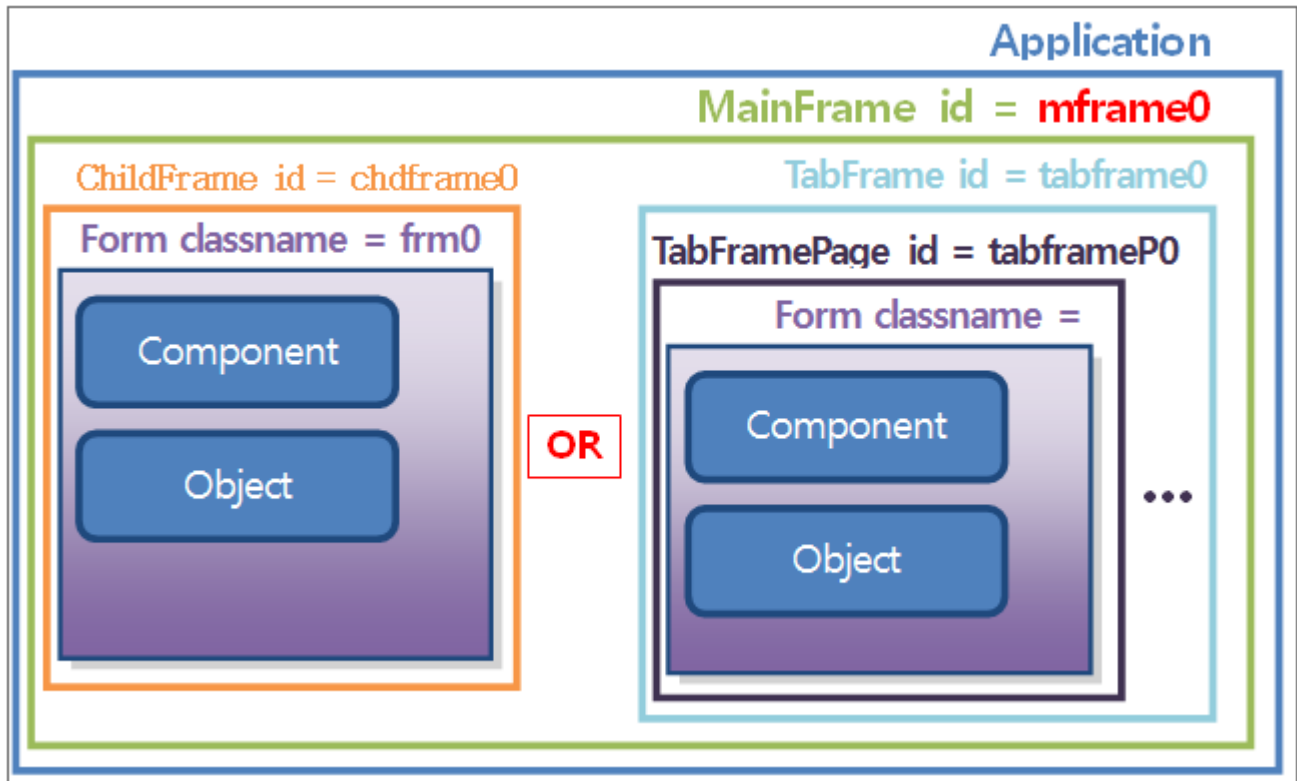
4.3.1 관계도





form내의 component / bind / invisible object는 .getOwnerFrame()이나 .getOwnerForm()이 없습니다.

4.3.2 Script 예시 화면



single Frame 의 경우 MainFrame은 ChildFrame, TabFrame 중 한 개만 가질 수 있습니다.

4.3.3 application에서 form의 script 접근

- frm0로의 접근 (ChildFrame인 경우)

```
application.mainframe.frame.form.name == "frm0"
application.mframe0.frame.form.name == "frm0"
application.mframe0.chdframe0.form.name == "frm0"
application.all[0].all[0].form.name == "frm0"
application.all["mframe0"].all["chdframe0"].form.name == "frm0"
mainframe.frame.form.name == "frm0"
```


- frm1로의 접근 (TabFrame인 경우)

```
application.mainframe.frame.tabframepages[0].form.name == "frm1"
application.mainframe.frame.tabframepages["tabframeP0"].form.name == "frm1"
application.all[0].all[0].all[0].form.name == "frm1"
application.all["mframe0"].all["tabframe0"].all["tabframeP0"].form.name == "frm1"
```

4.3.4 form에서 application의 script 접근

- frm0에서 application으로 접근 (ChildFrame인 경우)

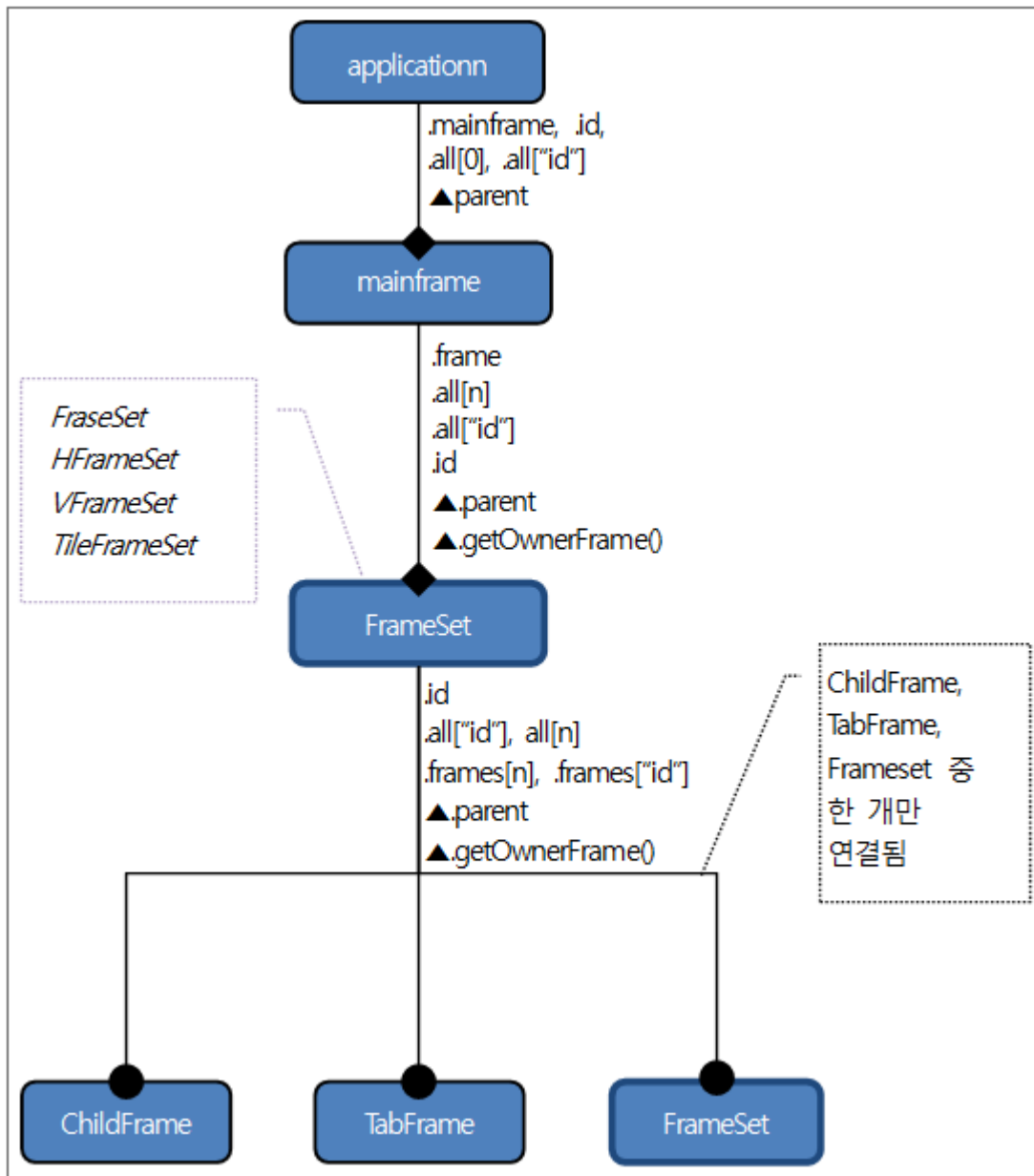
```
this.parent.name == "chdframe0"
this.parent.parent.name == "mframe0"
this.getOwnerFrame().getOwnerFrame().name == "mframe0"
this.getOwnerFrame().getOwnerFrame().parent.mainframe.name == "mframe0"
```

- frm1 에서 mainframe으로 접근 (TabFrame인 경우)

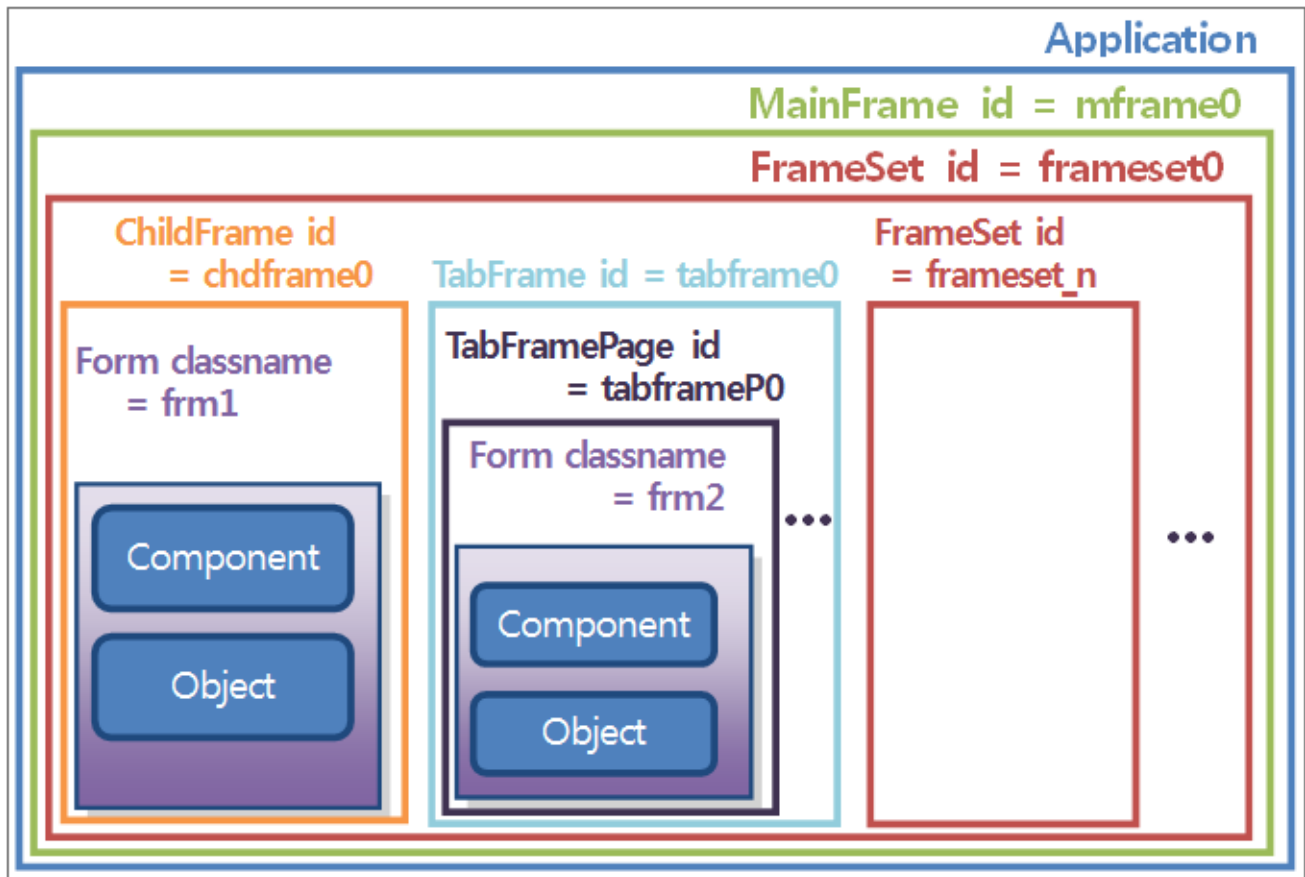
```
this.parent.name == "tabframeP0"
this.parent.parent.name == "tabframe0"
this.parent.parent.parent.name == "mframe0"
this.getOwnerFrame().getOwnerFrame().getOwnerFrame().name == "mframe0"
```

4.4 Multi Frame형식의 Application

4.4.1 관계도



4.4.2 Script 예시 화면



frameset_n의 각 FrameSet은 "frameset0"의 구조를 가질 수 있습니다.

frames[]를 제외하고는 single frame과 동일한 방법으로 접근합니다.

4.4.3 application에서 form의 script 접근

```
application.mainframe.frame == application.mainframe.frameset0
```

- frm1로의 접근

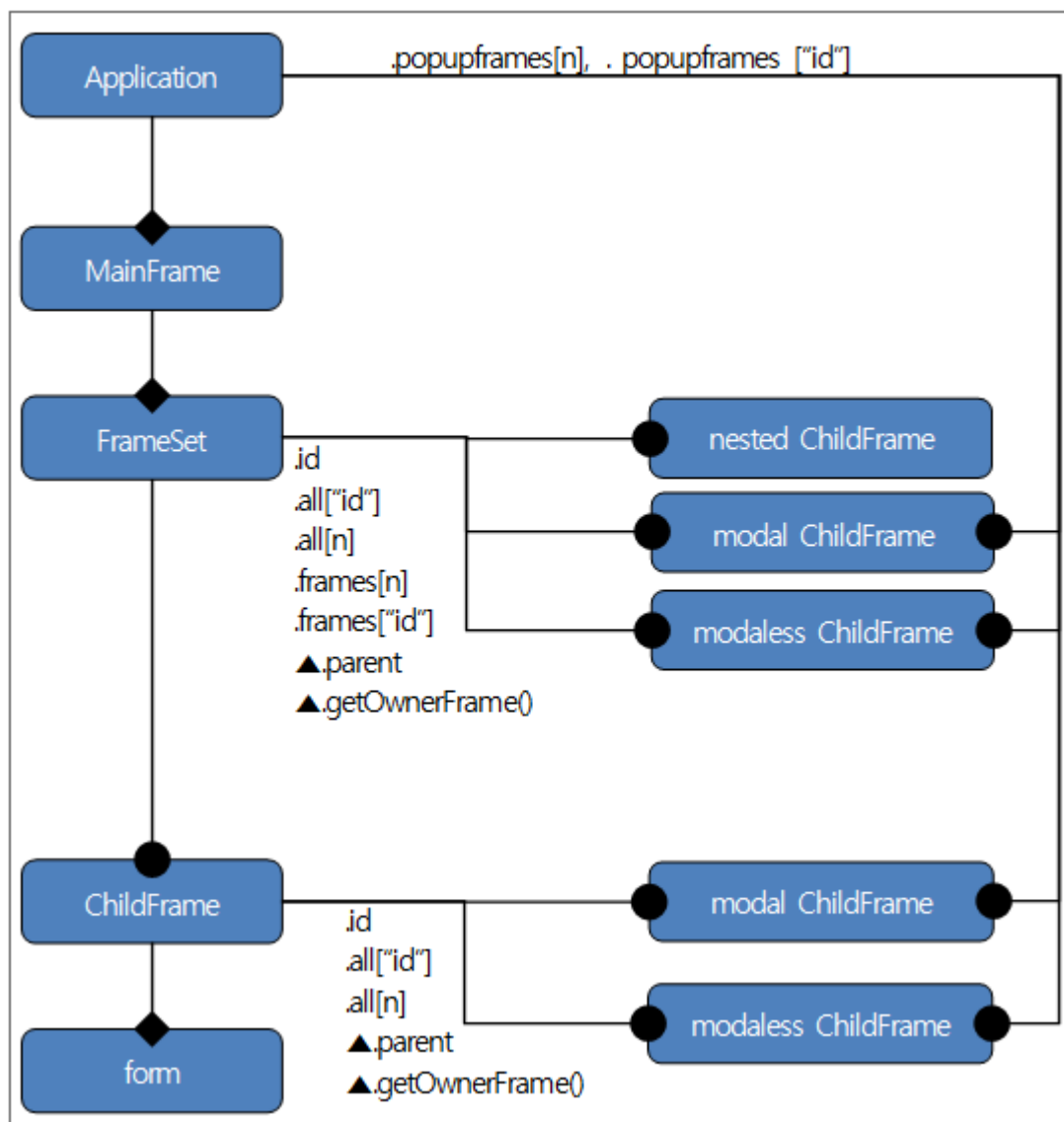
```
application.mainframe.frame.chdframe0.form.name== "frm1" //
application.mainframe.frame.frames[0].form.name == "frm1"
application.mainframe.frame.frames["chdframe0"].form.name == "frm1"
```

- frm2로의 접근

```
application.mainframe.frame.frames[1].tabframepages[0].form.name == "frm1"
```

4.5 Modal, Modalesse 형식의 Form

4.5.1 관계도



4.5.2 nested ChildFrame에서 Script 사용법

- nested ChildFrame 생성 및 삭제 방법

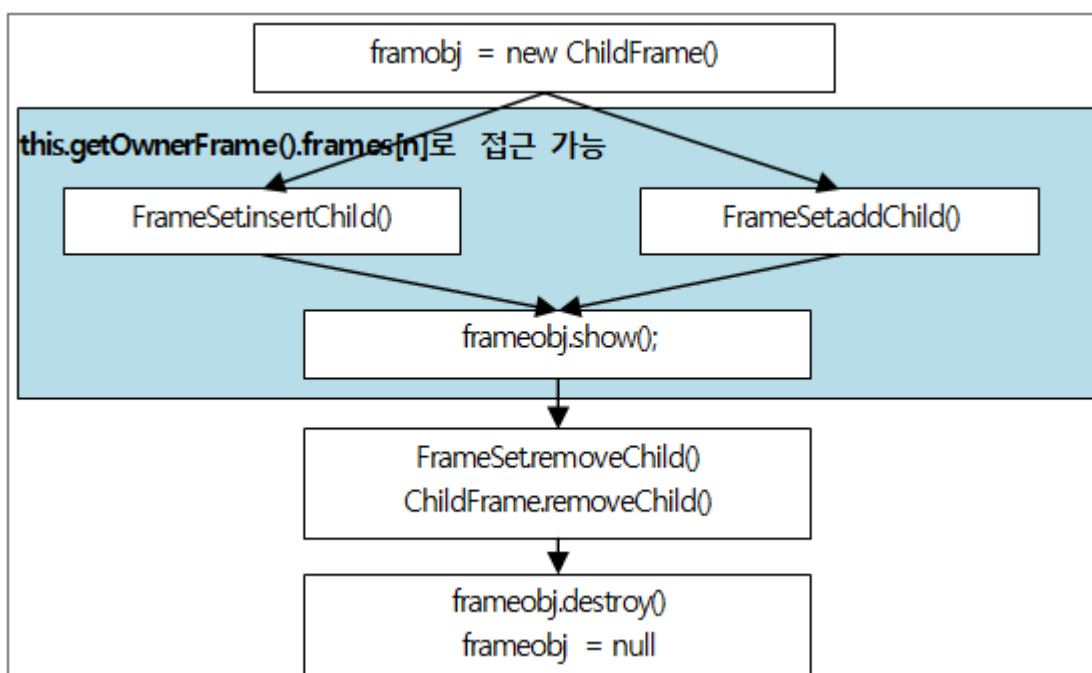
```
// ***** ChildFrame을 생성합니다.*****
var frameobj = new ChildFrame("childfrm0", 10, 10, 100, 100, "url");
or
var frameobj = new ChildFrame("childframe");
frameobj.init ("childframe", 10, 10, 100, 100);

// ***** ChildFrame을 Frameset에 연결합니다. *****
mainframe.frameset.addChild("chd0", frameobj);
or
frameset.inertChild(1, "chd0", frameobj);

// ***** ChildFrame을 화면에 보여줍니다. *****
frameobj.show();

// ***** ChildFrame을 화면에 보여줍니다. *****
mainframe.frameset.removeChild("chd0");
frameobj.destroy();
frameobj = null;
```

- Script flow

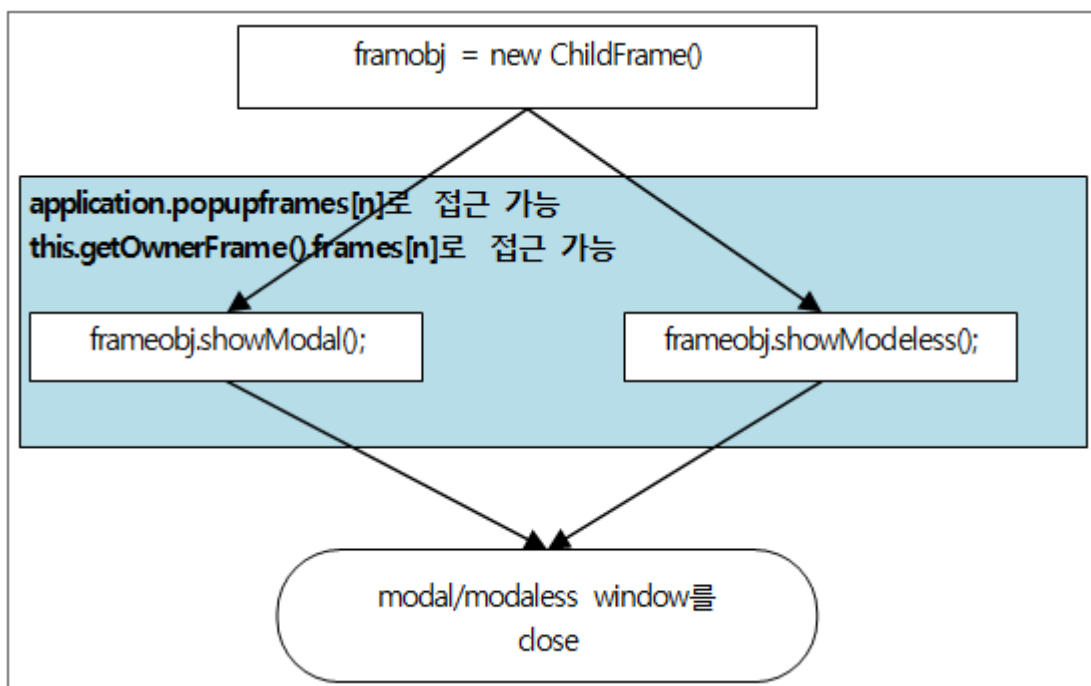


4.5.3 modal/modaless ChildFrame에서 Script 사용법

- modal / modaless ChildFrame 생성방법

```
var frameobj = new ChildFrame("childfrm0", 10, 10, 100, 100, "url");
or
var frameobj = new ChildFrame("childframe");
frameobj.init ("childframe", 10, 10, 100, 100);
var parentframeobj = mainframe.frameset.ChildFrame0;
frameobj.showModal(parentframeobj);
or
frameobj.showModeless(parentframeobj);
```

- Script flow



5.

Style(CSS)과 Theme의 관리

Style(CSS: Cascading Style Sheets)과 Theme는 XLATFORM 화면을 구성하는 화면요소들을 디자인하는 기능을 의미합니다.

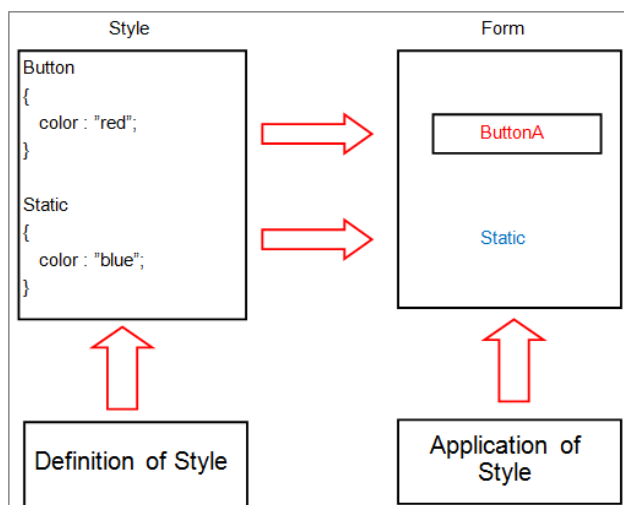
Style과 Theme를 적용할 수 있는 화면 요소들로는 컴포넌트들, 화면 Form, Frame, Widget Form, Alert, Confirm, TitleBar, StatusBar, ScrollBar등이 있습니다.

5.1 Style / Theme 개요

XPLATFORM에서 Style과Theme란 화면상에 보이는 UI요소들의 디자인을 정의하는 것을 의미합니다. 버튼을 예로 들면, 투명도, 폰트, 칼라, 그림자, 번짐과 기울임 등의 효과를 버튼 컴포넌트에게 주는 것을 의미합니다.

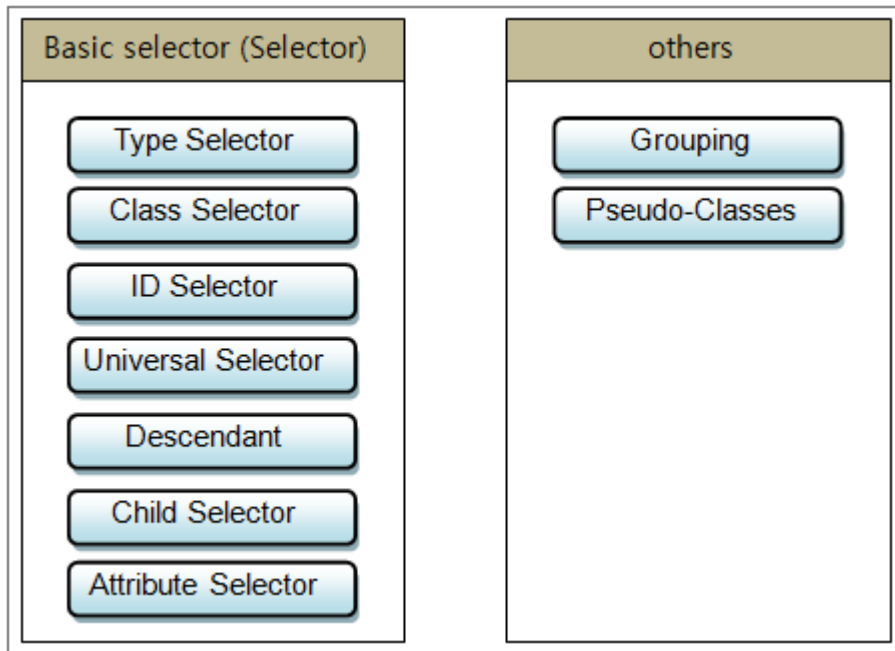
5.1.1 Style의 정의 및 적용

Style은 컴포넌트별로 정의합니다. 각각의 컴포넌트는 Style적용이 가능한 Property가 있고, 해당 Property의 Style 값을 설정함으로써 Style을 정의합니다.



Style의 정의는 Selector를 이용합니다. 여기서는 CSS에 대한 사전지식이 있다는 가정하에 설명합니다.

XPLATFORM이 지원하는 Selector는 다음과 같습니다.



- Type Selector : “Button”과 같이 컴포넌트명을 지정합니다.

Syntax	Name (Element)
예	Button { color : green; }
의미	모든 Button의 color는 green으로 정의합니다.

- Class Selector : 컴포넌트별로 특정 Style을 지정합니다.

Syntax	.(Period)
예	Button.BlueButton { color : blue; }
의미	특정 Button에 class를 지정하여 해당 Button의 color는 blue로 정의합니다.

- ID Selector : “Button00”과 같이 컴포넌트의 ID를 지정합니다.

Syntax	#
예	Button#Button00 { color : yellow; }
의미	모든 Button중에 Button00이라는 ID를 가진 Button의 color는 yellow로 정의합니다.

- Universal Selector : 모든 컴포넌트를 지정합니다.

Syntax	*(Asterisk)
예	*>#contextmenu { background : gray; }

Syntax	*(Asterisk)
의미	Edit가 가능한 컴포넌트(Edit, MaskEdit, TextArea 등)들의 PopupMenu의 background는 gray로 정의합니다.

- Descendant Selector : 특정 컴포넌트의 하위 속성을 지정합니다.

Syntax	Name SubName
예	Combo background { color : red; }
의미	모든 Combo의 background의 color는 red로 정의합니다.

- Child Selector : 모든 컴포넌트의 하위를 지정합니다.

Syntax	>
예	Div>Button { color : blue; }
의미	모든 Div안의 Button의 color는 blue로 정의합니다.

- Attribute Selector중 일부 : 모든 컴포넌트의 하위 속성값의 비교로 지정합니다. 추가적으로 해당 속성값의 비교는 "[property=value]" 형태의 적용을 지원합니다.

Ex. Combo[text] { color : black; }

Combo[text="CmbSearch"] { color : red; }

Syntax	[]
예	Button[text="Search"] { color : red; }
의미	특정 Button의 text값이 "Search"이면 해당 Button의 color는 red로 정의합니다.

- Pseudo-classes중 일부 : Selector들을 사용하여 표현될 수 없는 정보(focused, disabled, mouseover, pushed 등)들의 Selection 처리를 지정합니다.

Syntax	:
예	Button:focus { color : black; }
의미	모든 Button에 focus가 되면 해당 Button의 color는 black으로 정의합니다.



UI element states pseudo-classes: enabled(default), disabled, focused, mouseover, pushed, selected

Structural pseudo-classes: root, nth-child(), nth-last-child(), first-child, last-child, only-child, empty

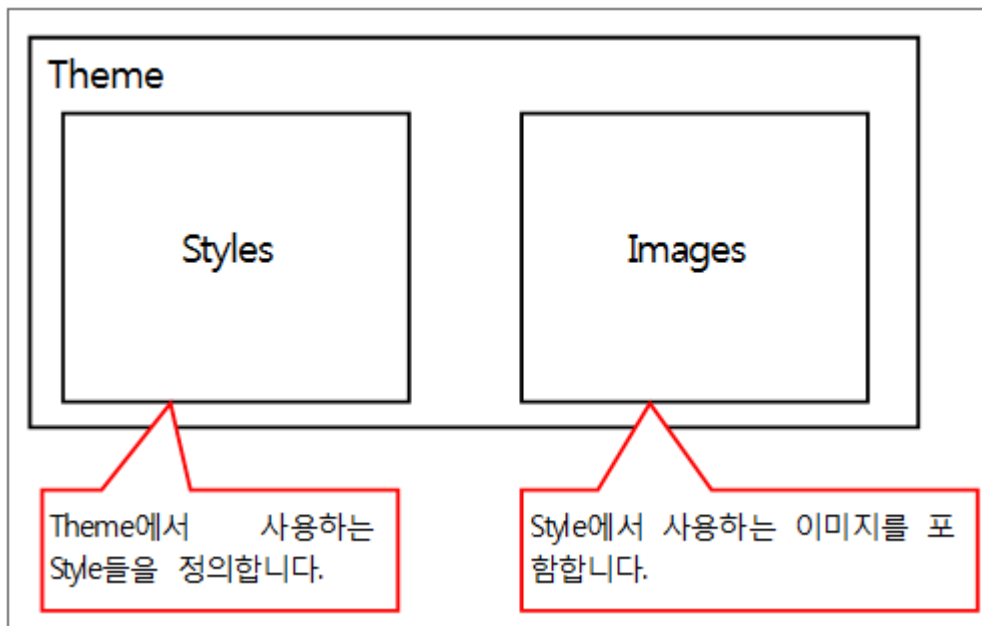
- Grouping : 같은 값(declarations)을 갖는 여러 개의 Selector들을 컴포넌트명으로 지정합니다.

Ex. Button, Combo, Calendar { font : Arial,9; }

Syntax	,(Comma)
예	Button, Combo, Calendar { font : Arial,9; }
의미	모든 Button에 Button, Combo, Calendar의 font는 “Arial 9”으로 정의합니다.

5.1.2 Theme의 정의

Theme는 Style과 Image를 조합한 형태입니다.



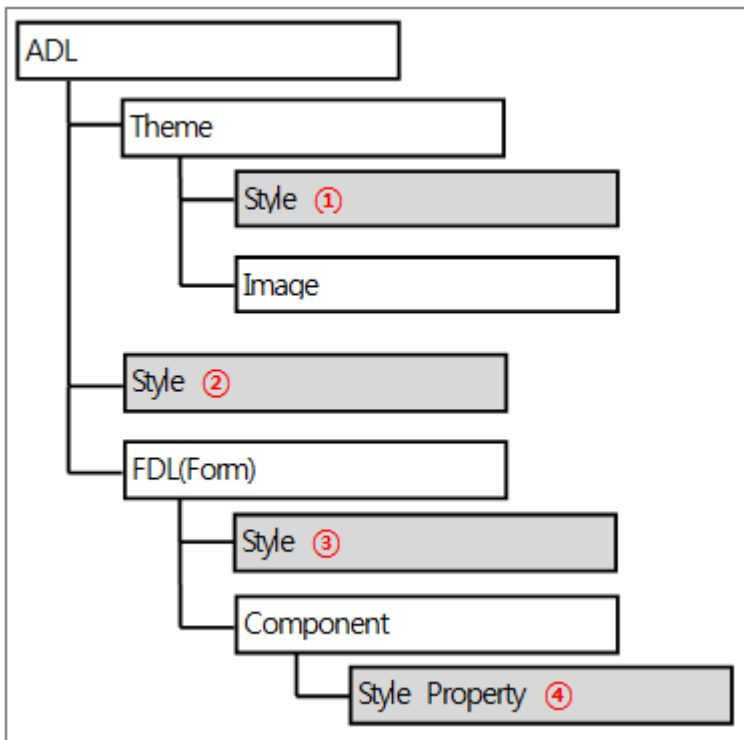
응용프로그램은 하나의 Theme만을 사용할 수 있습니다.

XPLATFORM은 default, black, gray, blue의 4가지 Theme파일을 제공합니다. 해당 Theme는 UX-Studio의 Theme, Style Editor창을 통해 필요한 부분을 수정할 수 있습니다. Theme는 XPLATFORM에서 꼭 필요한 기본정보를 가지고 있으므로 사용자는 필요한 부분만을 선정해서 추가, 수정작업을 하여야 합니다.

기본정보가 삭제되거나 변경할 경우, 그로 인해 화면 출력에 문제가 생길 수 있습니다. 예를 들면 “#contextmenu”와 같이 Edit형식의 컴포넌트(Edit, MaskEdit, TextArea등)들에서 발생하는 PopupMenu의 설정이 없을 경우 PopupMenu가 나타나지 않는 경우가 발생합니다.

따라서 가능한 Theme는 새로 생성하는 것보다는 기본으로 제공되는 Theme를 수정해서 다른 이름으로 저장하거나 복사해서 사용하도록 합니다.

5.1.3 Style/Theme의 등록 및 적용



- UI요소들에 디자인 적용의 순서

	설명
1	ADL에 속한 Theme내의 Style
2	ADL에 속한 Style
3	FDL에 속한 Style
4	FDL내의 Component의 Style

컴포넌트에 Style이 적용되는 순서는 ①>②>③>④ 입니다. 그러므로 중복된 style값 정의한 경우, 마지막 Style인 ④번 Style이 화면에 적용되어 보여집니다.

5.1.4 Style과 Theme의 적용 대상

Style은 하나의 Form에 선언해서 사용할 수 있으며, Global하게 사용하기 위해 Project(ADL)에 등록하거나 Theme에 포함해 사용할 수도 있습니다.

Style의 적용대상은 단순한 컴포넌트뿐만 아니라, 다양한 화면요소에 적용됩니다. 다음은 그 적용 대상들을 표로 나타낸 것입니다.

다음은 Style적용이 가능한 대상들입니다.

종류	적용 대상
컴포넌트 (Visible Object)	Button, Calendar, CheckBox, Combo, Div, Edit, GraphicPath, Grid GroupBox, ImageViewer, ListBox, MaskEdit, Menu, Progressbar Radio, Shape, Spin, Splitter, Static, Tab, TextArea
화면 Frame	Form, WidgetForm, ChildFrame, FrameSet, HFrameSet TabFrame, TileFrameSet, VFrameSet
기타	Alert, ApplicationMenu, Confirm, ScrollBar, StatusBarForm TitleBarForm, ToolTip

5.2 Style의 적용

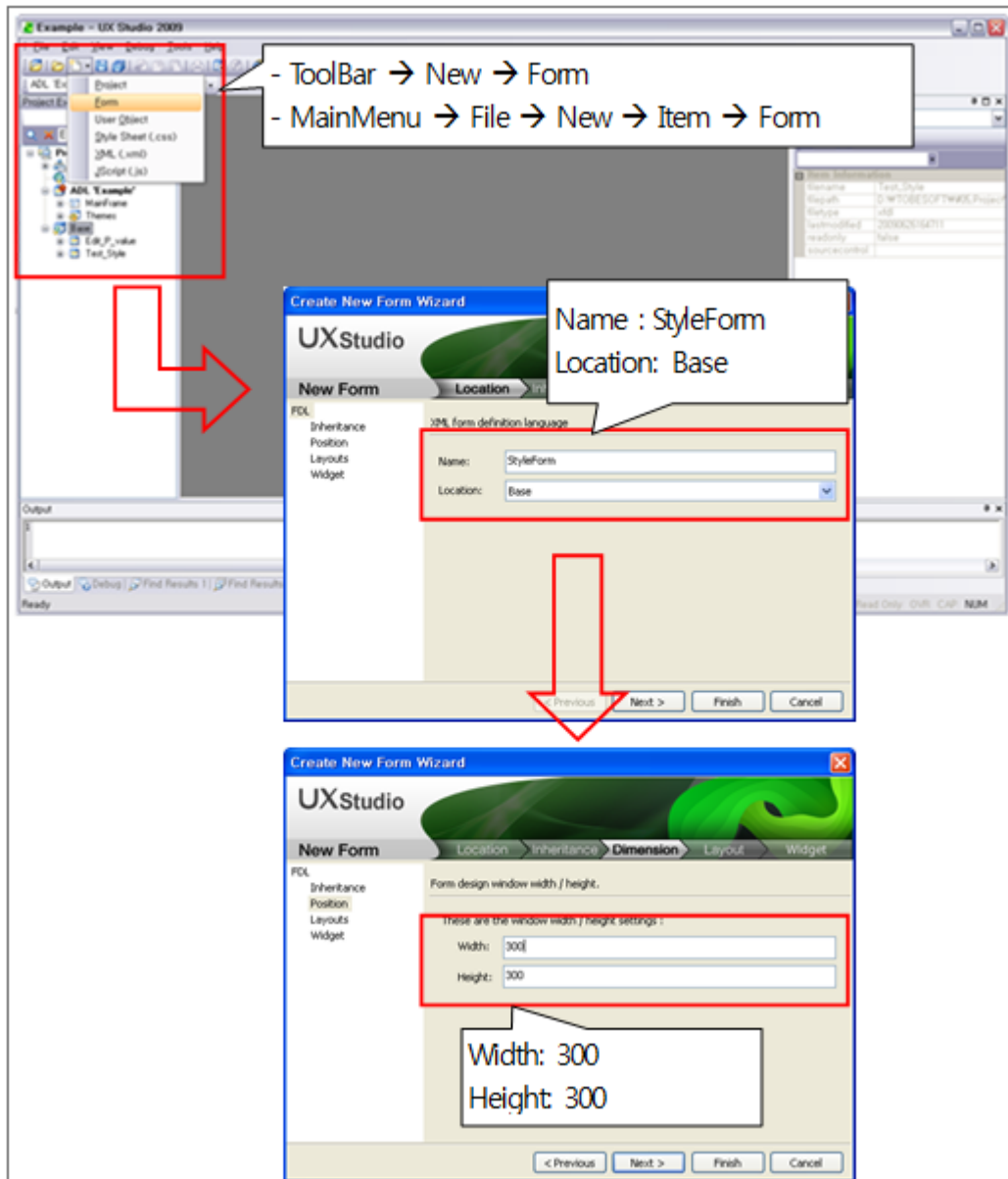
Style의 개요에 대해 앞에서 간단하게 언급했습니다. 이제부터 Style의 작성방법 및 적용, 활용하는 방법에 대하여 설명합니다.

5.2.1 Style의 생성

ADL에서 Theme를 활용하지 않을 경우나 Form에 Style을 주고 싶을 때 새로 생성하거나 기존에 작성된 Style파일을 등록합니다.

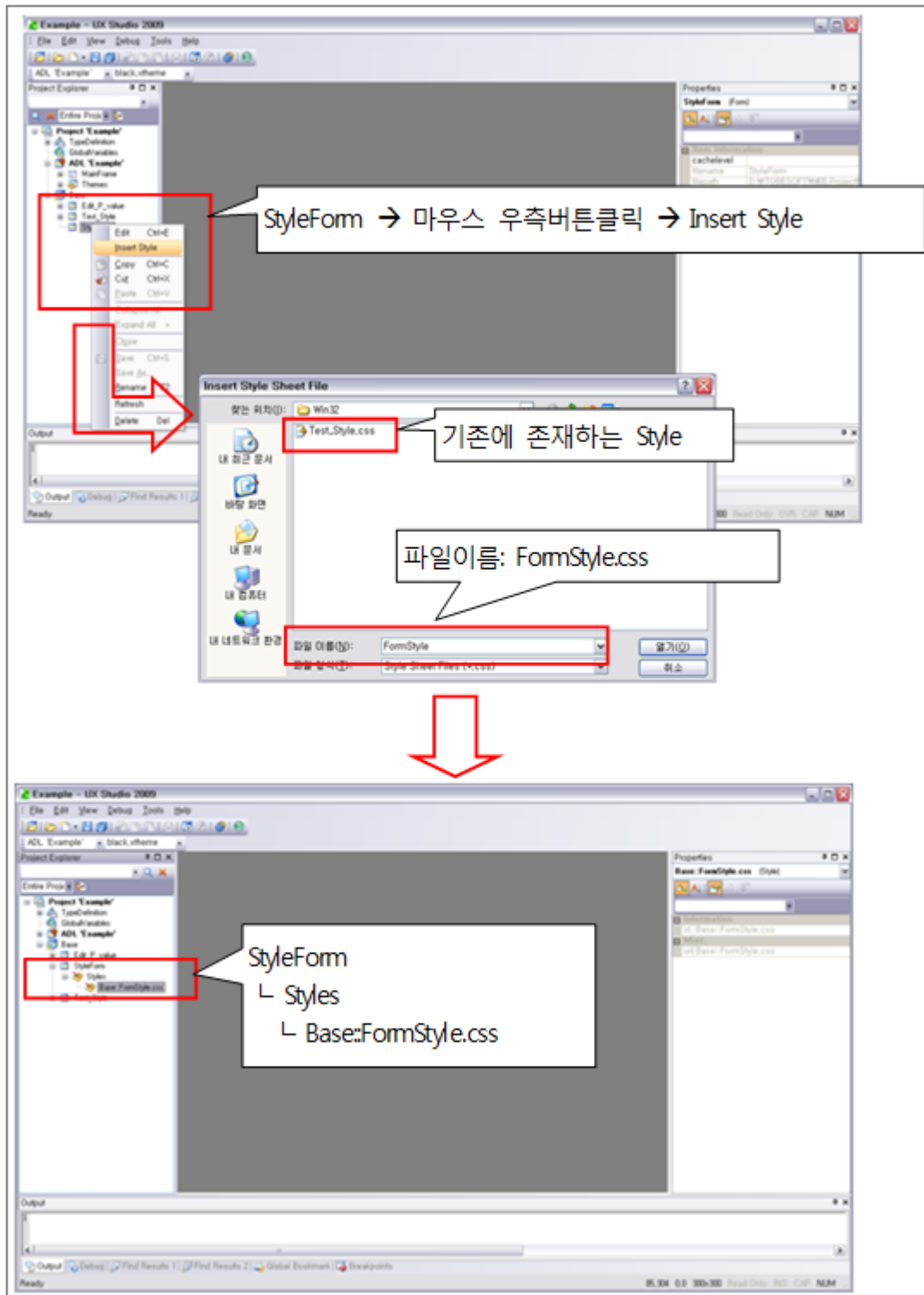
1단계: 새로운 Form을 만듭니다.

New Form으로 Form을 만들어 Form명을 StyleForm으로 합니다.

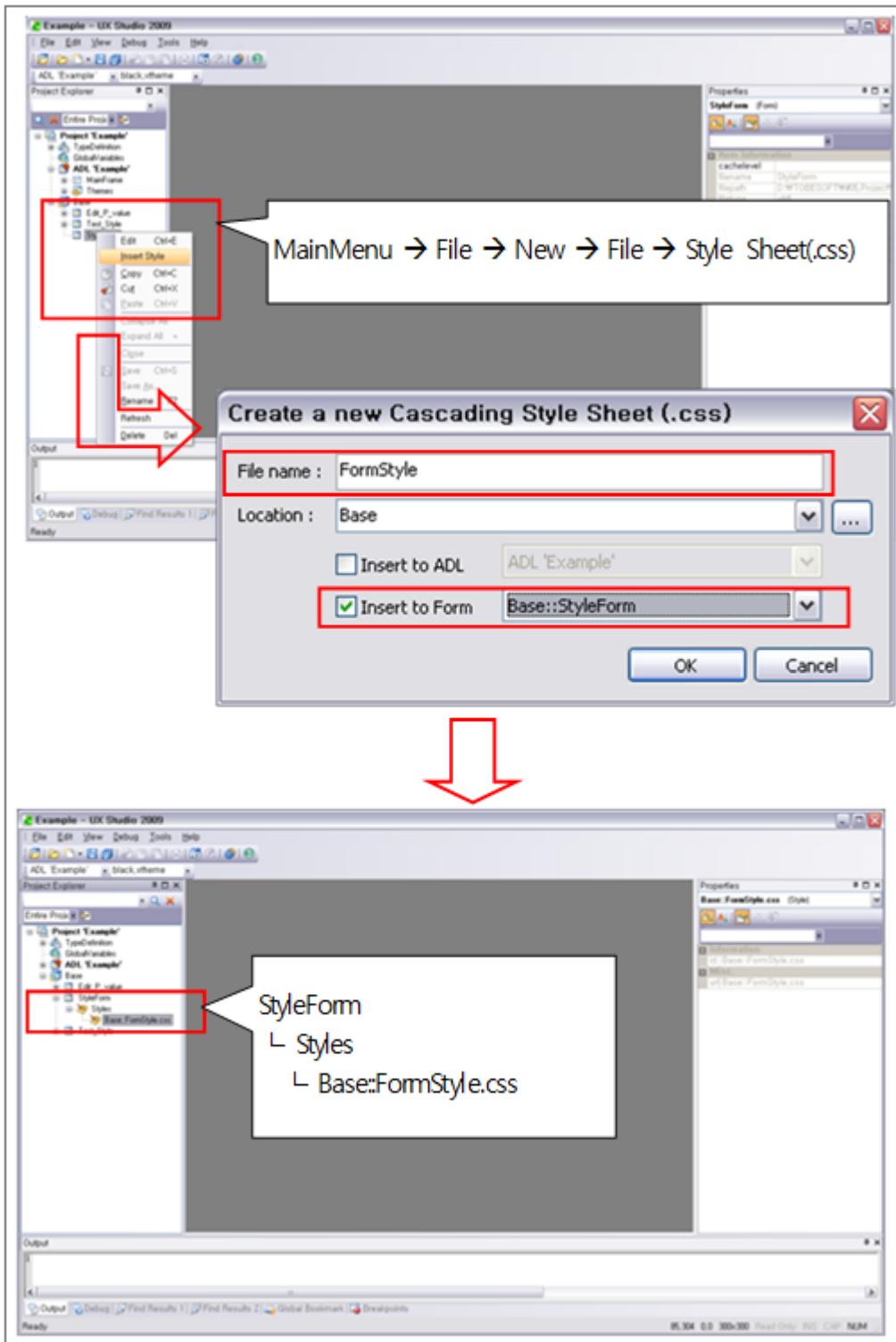


2단계: 만들어진 Form에 새로운 Style 파일을 생성합니다.

만들어진 StyleForm에 Insert Style로 하나의 Style을 생성해 Style명을 FormStyle.css로 추가합니다. Form이 열려 있으면 Style을 생성할 수 없습니다.

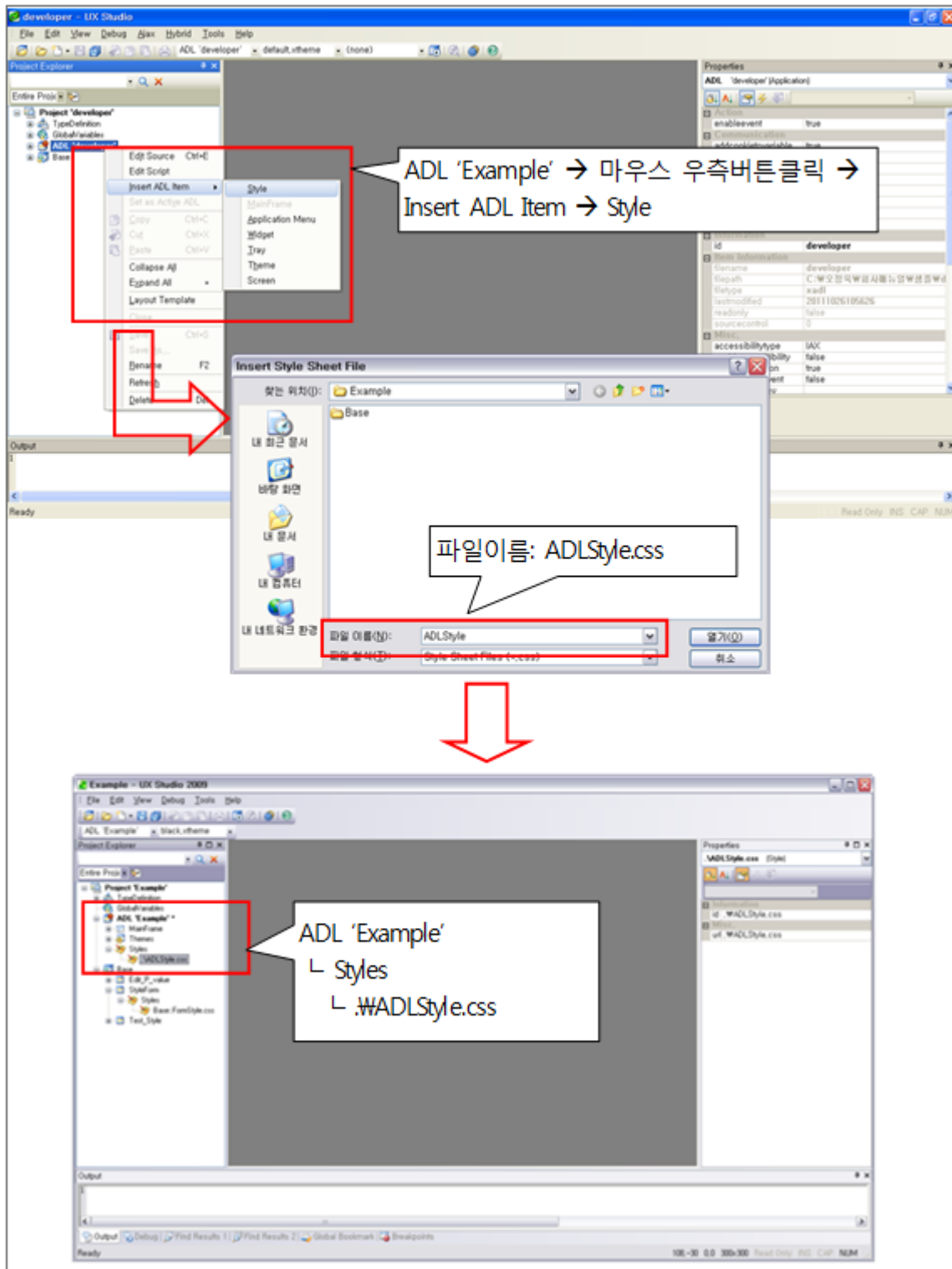


MainMenu를 통해서 생성하는 방법입니다.



3단계: ADL에 Style 파일을 생성합니다.

ADL에 Insert ADL Item으로 하나의 Style을 생성해 Style명을 ADLStyle.css로 추가합니다.



5.2.2 Style의 편집

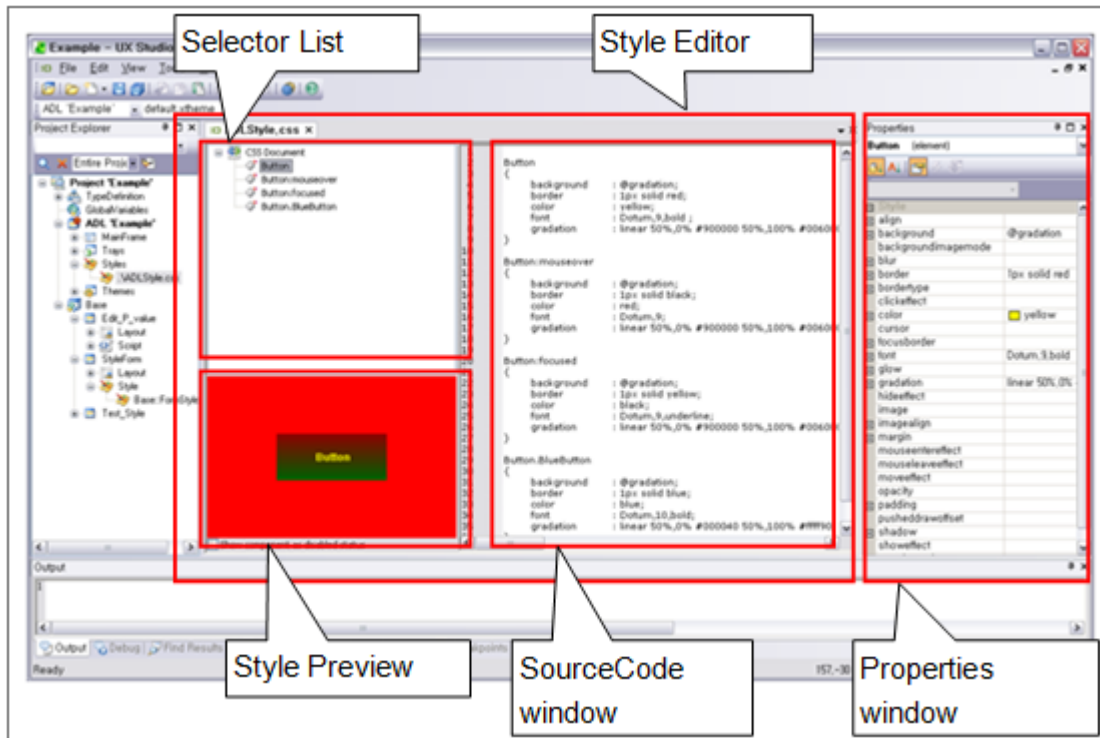
Style의 편집방법은 Theme내의 Style, ADL의 Style, Form의 Style이 모두 동일합니다.

Style의 편집은 Style편집기를 통해서 할 수 있습니다.

Style편집기는 Selector List를 Tree형태로 제공해 추가/삭제가 가능하게 하였으며, SourceCode창에서 내용편집이 가능하도록 해주고 있습니다.

Style편집기에서는 Properties창을 제공해 편리하게 입력/수정을 할 수 있도록 해주고 있습니다.

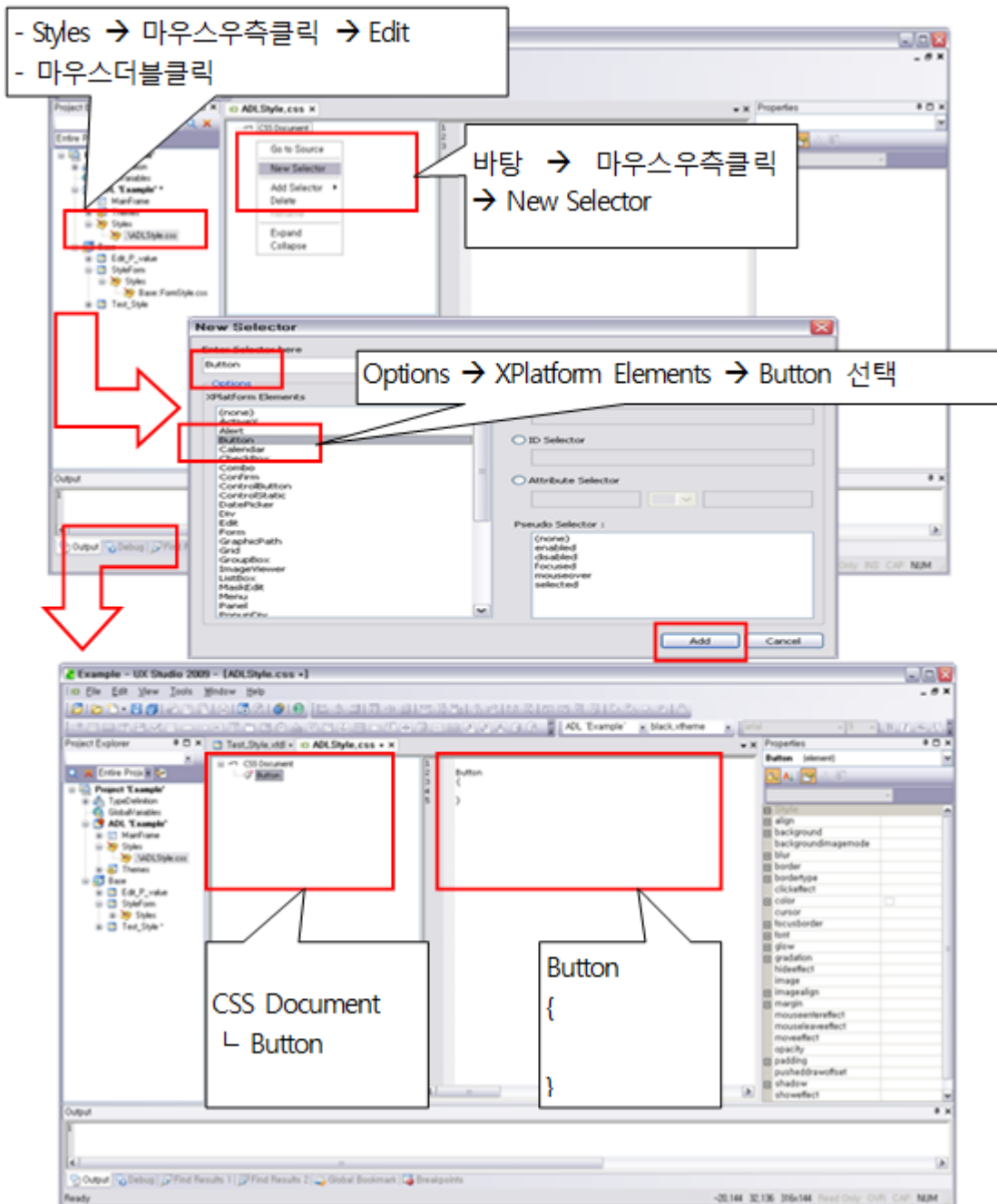
이때 Properties창에 입력된 내용은 SourceCode창에 바로 반영되어 확인할 수 있습니다.



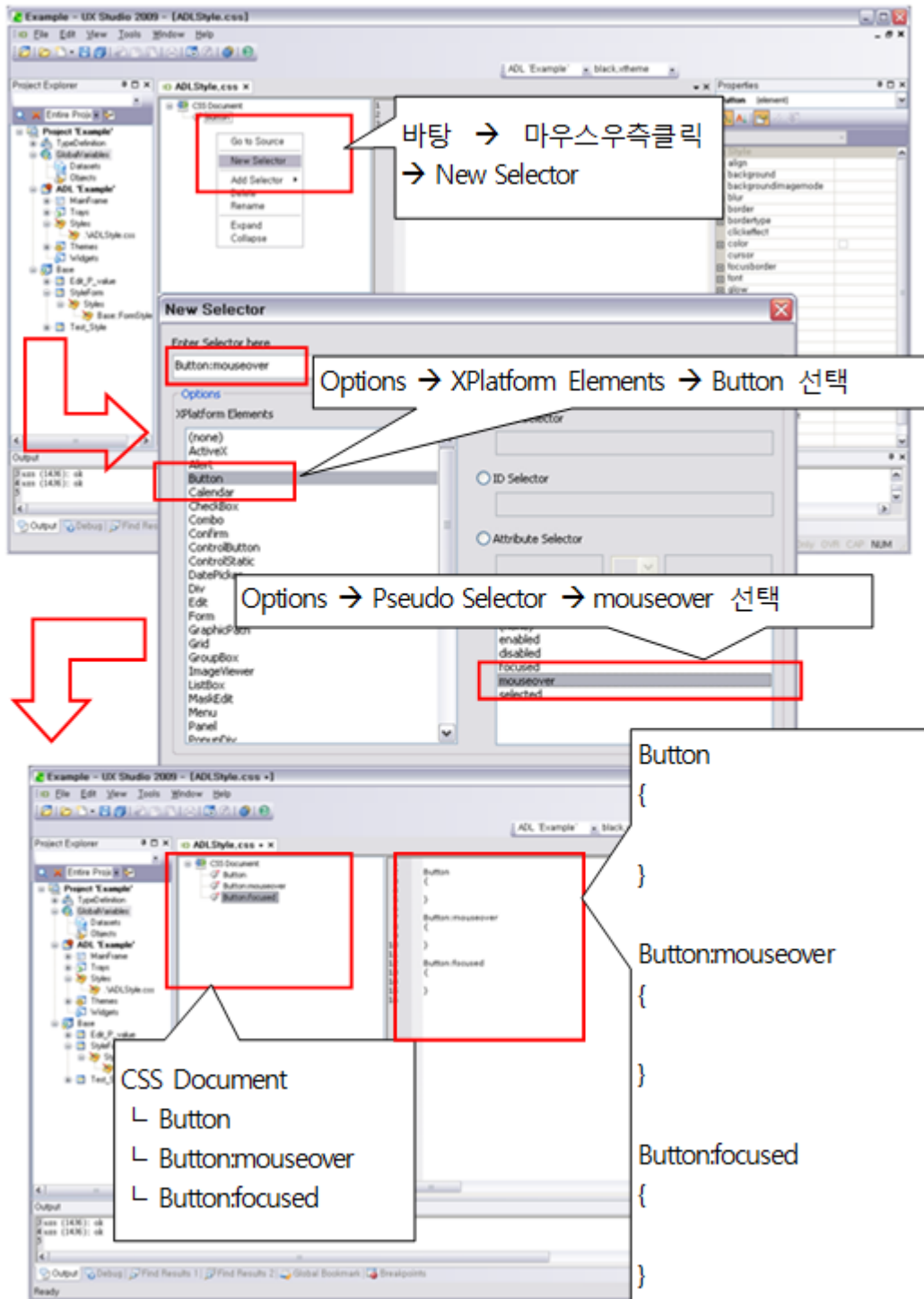
1단계: Style Editor를 통해 Selector를 등록합니다.

생성한 ADLStyle.css를 편집창을 통해 Button Selector들(Type Selector, Pseudo Selector, Class Selector)을 등록합니다.

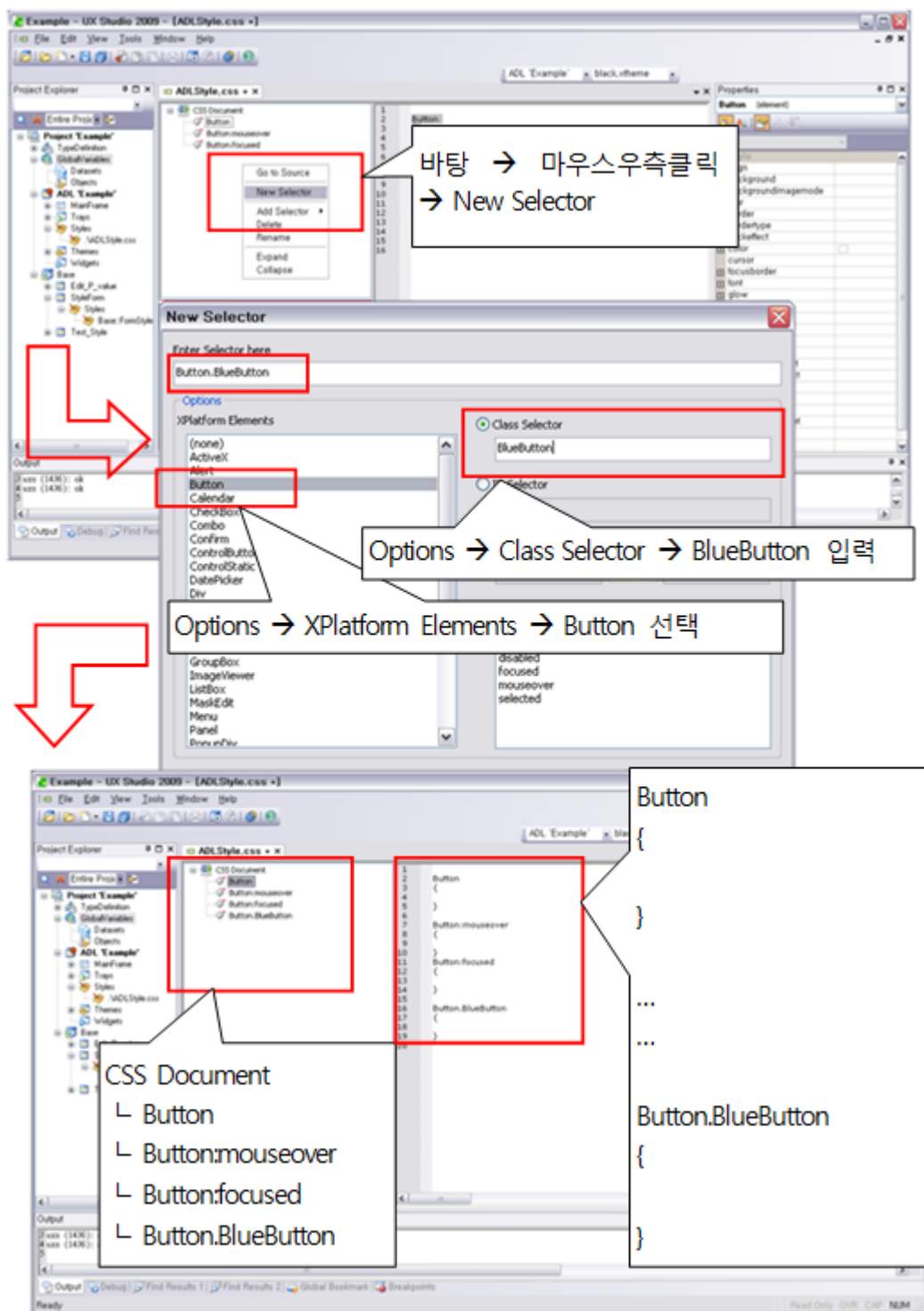
먼저 Type Selector를 등록합니다



Pseudo Selector를 등록합니다. (mouseover, focused)

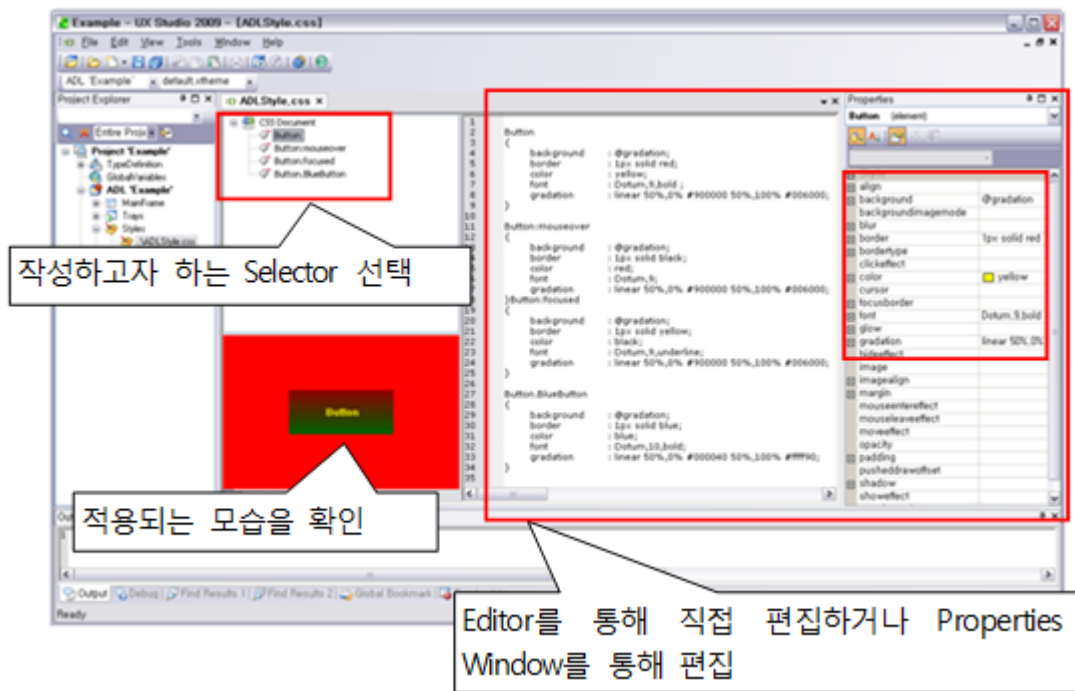


Class Selector를 등록합니다.



2단계: Style Editor를 통해 등록된 Selector의 내용을 작성합니다.

생성한 Button Selector들(Type Selector, Pseudo Selector, Class Selector)의 정보를 작성합니다.



Button

```
{
    background    : @gradation;
    border        : 1px solid red;
    color         : yellow;
    font          : Arial,9,bold ;
    gradation     : linear 50%,0% #900000 50%,100% #006000;
}
```

...

Button.BlueButton

```
{
    background    : @gradation;
    border        : 1px solid blue;
    color         : blue;
    font          : Arial,10,bold;
    gradation     : linear 50%,0% #000040 50%,100% #ffff90;
}
```

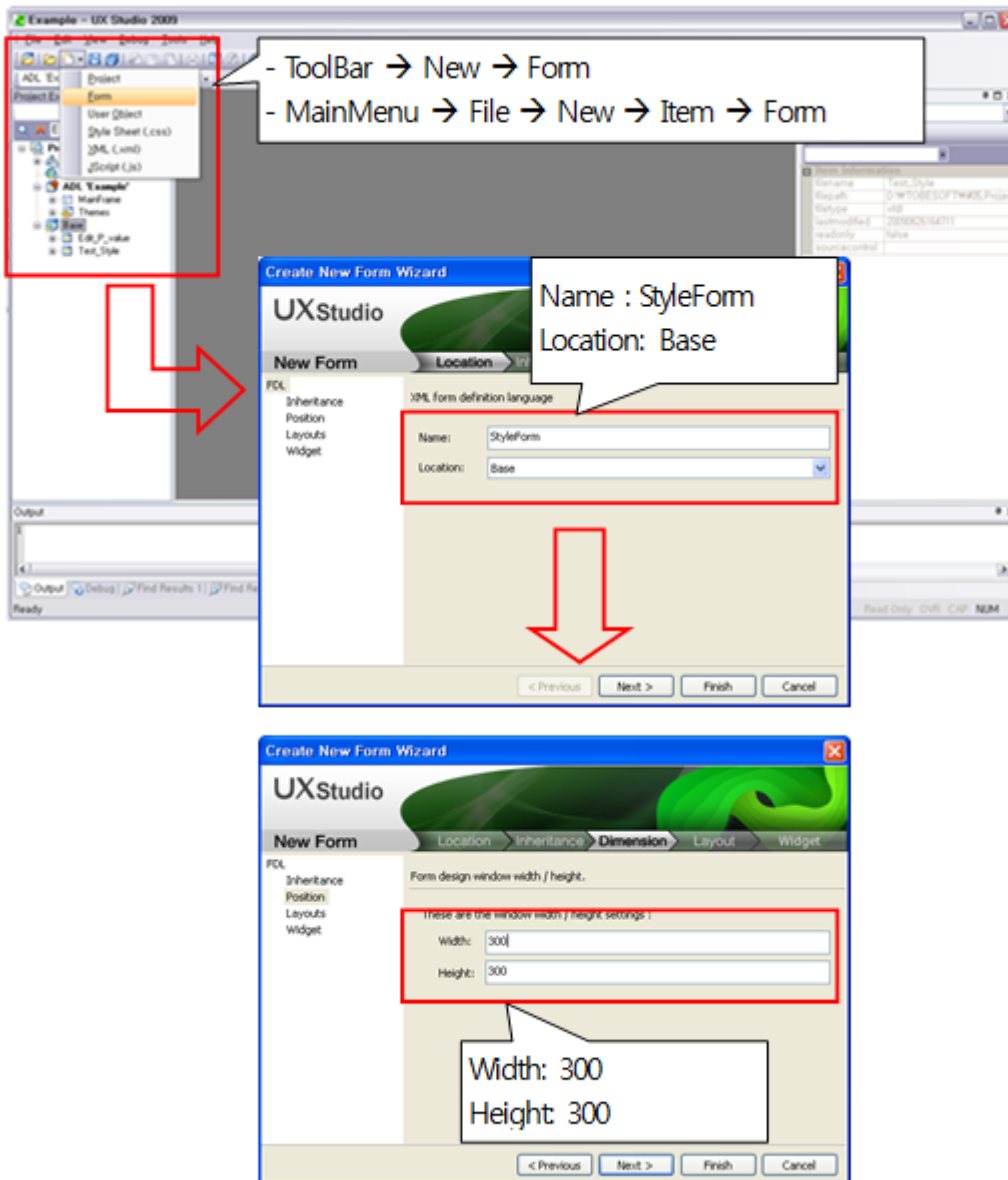
5.2.3 Style의 적용

Form에 Button을 생성해 ADL에 있는 Style과 Form에 있는 Style에 따라 Design에 반영되도록 합니다.

1단계: 새로운 Form을 만듭니다.

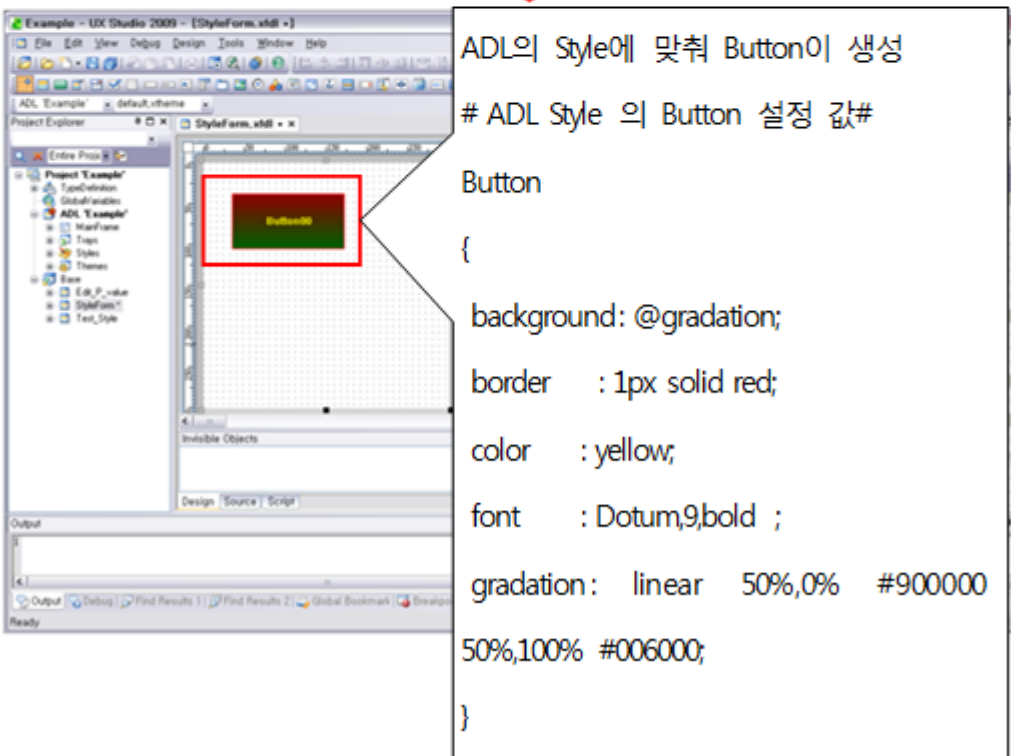
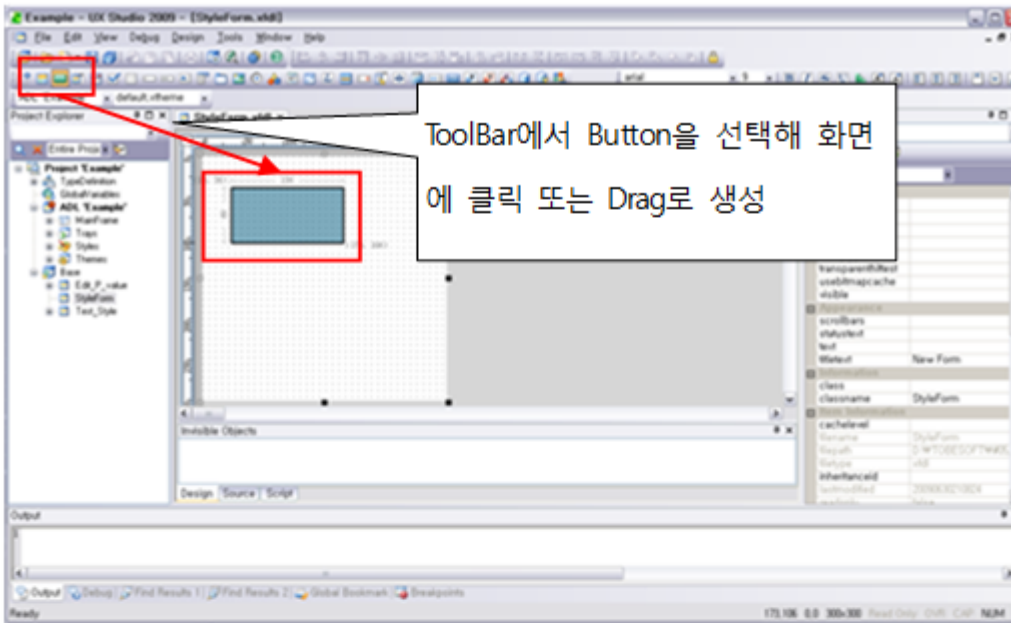
New Form으로 Form을 만들어 Form명을 StyleForm으로 합니다.

(“Style의 생성” 단계에서 만든 Form을 활용합니다.)



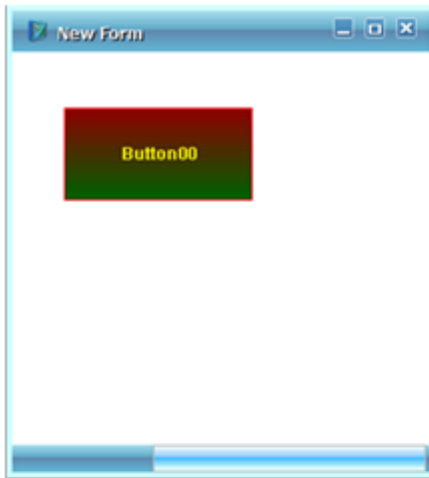
2단계: Button을 생성해 Style의 적용을 확인합니다.

StyleForm에 Button을 생성해 ADL에 작성된 Style에 맞추어 Style이 적용되는지 확인합니다.

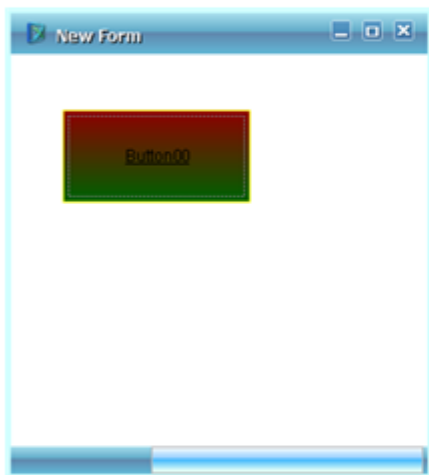


3단계: Form을 실행해 Button의 Pseudo Selector의 적용을 확인합니다.

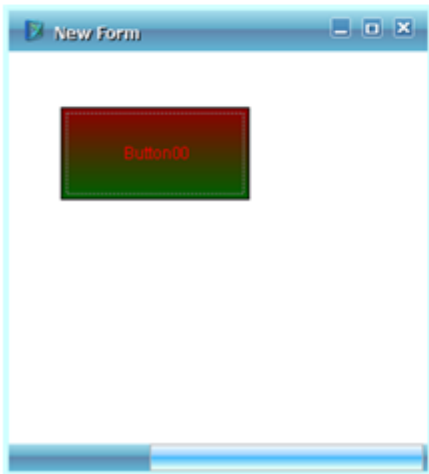
ADLStyle에 설정한 Pseudo Selector(mouseover, focused)의 Form에 적용되는 것을 QuickView로 실행해서 확인합니다.



```
Button
{
  background : @gradation;
  border    : 1px solid red;
  color     : yellow;
  font      : Dotum,9,bold ;
  gradation : linear 50%,0%    #900000
              50%,100% #006000;
}
```



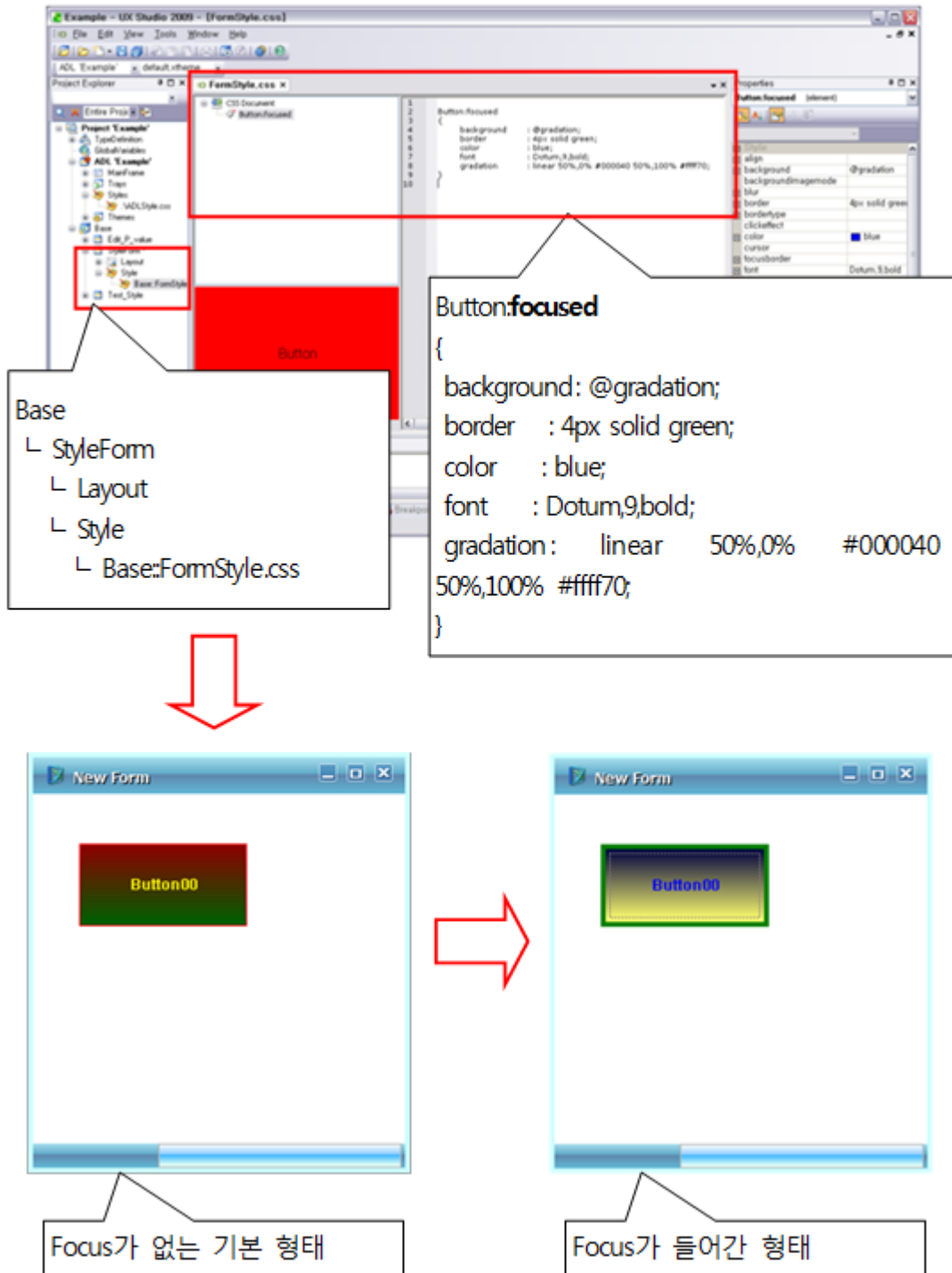
```
Button:focused
{
  background : @gradation;
  border     : 1px solid yellow;
  color      : black;
  font       : Dotum,9,underline;
  gradation  : linear 50%,0%    #900000
                  50%,100% #006000;
}
```



```
Button:mouseover
{
  background : @gradation;
  border     : 1px solid black;
  color      : red;
  font       : Dotum,9;
  gradation  : linear 50%,0%    #900000
                  50%,100% #006000;
}
```

4단계: 앞서 Form에 생성한 FormStyle을 편집하고, 적용을 확인합니다.

FormStyle에 작성된 Pseudo Selector 중 focused 정보를 ADLStyle과 다르게 설정해 다르게 적용되는 것을 Quick View로 실행해서 확인합니다.

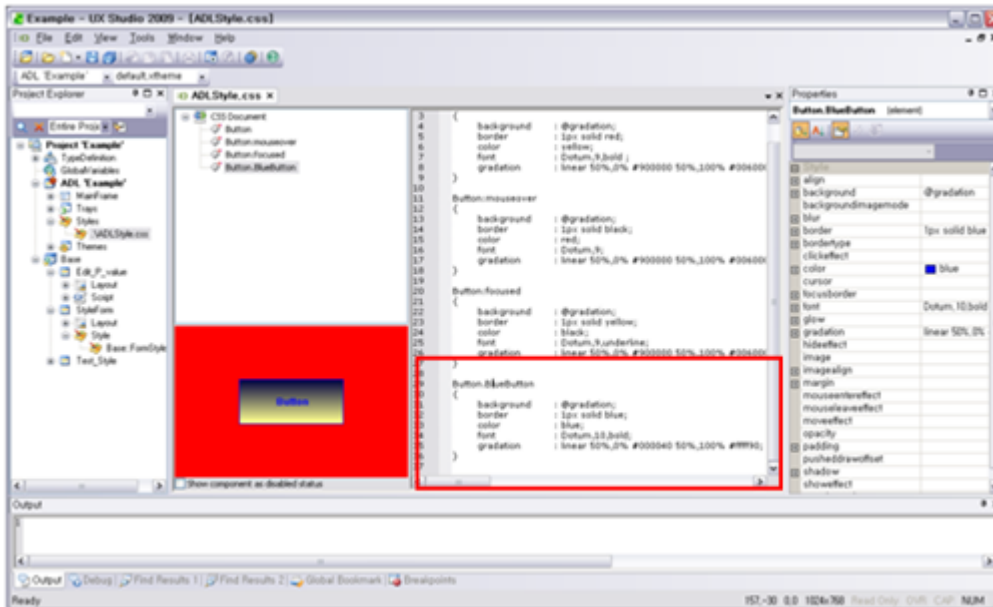


5.2.4 Style Class의 적용

동일한 Form에서 동일한 컴포넌트에 서로 다른 Style을 적용하기 위해 등록된 Class Selector를 사용하여 표현을 다르게 할 수 있습니다.

1단계: ADLStyle에 생성한 Class Selector를 편집하고, 적용을 확인합니다.

ADLStyle에 Class Selector(BlueButton)를 편집합니다.

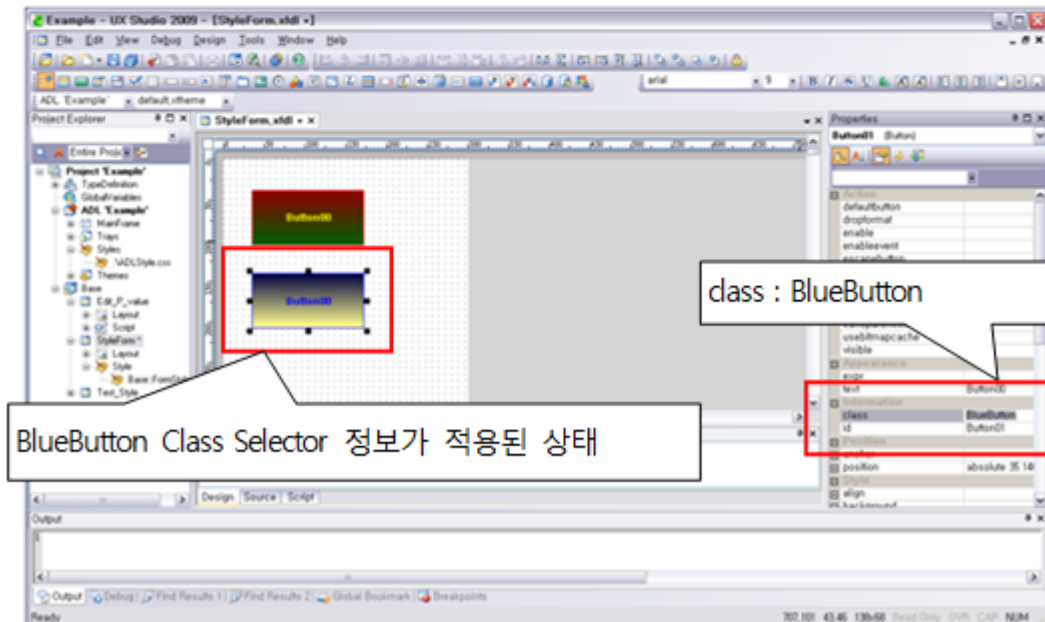
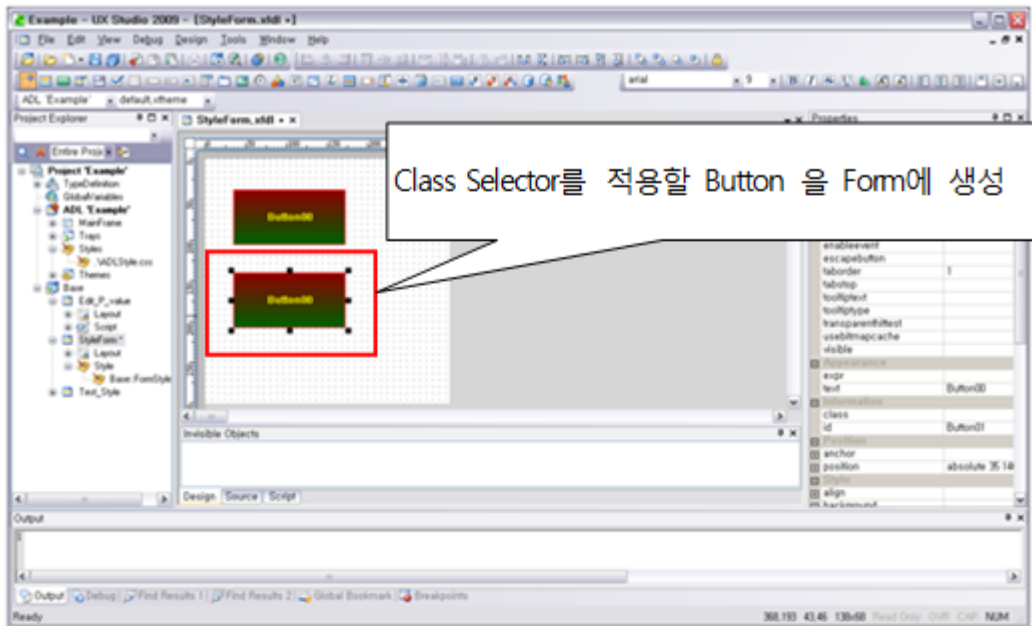


Button.BlueButton

```
{
    background    : @gradation;
    border        : 1px solid blue;
    color         : blue;
    font          : Arial,10,bold;
    gradation     : linear 50%,0% #000040 50%,100% #ffff90;
}
```

2단계: Form의 Button에 class Property를 활용해 Style의 적용을 확인합니다.

Form의 class Property에 ADLStyle에 작성된 Class Selector인 BlueButton을 설정하고, Style의 적용을 확인합니다.



5.3 Theme의 적용

Theme의 개요에 대해 앞에서 간단하게 언급했습니다. 이제부터 Theme의 작성방법 및 적용, 활용하는 방법에 대하여 설명합니다.

5.3.1 Theme의 생성

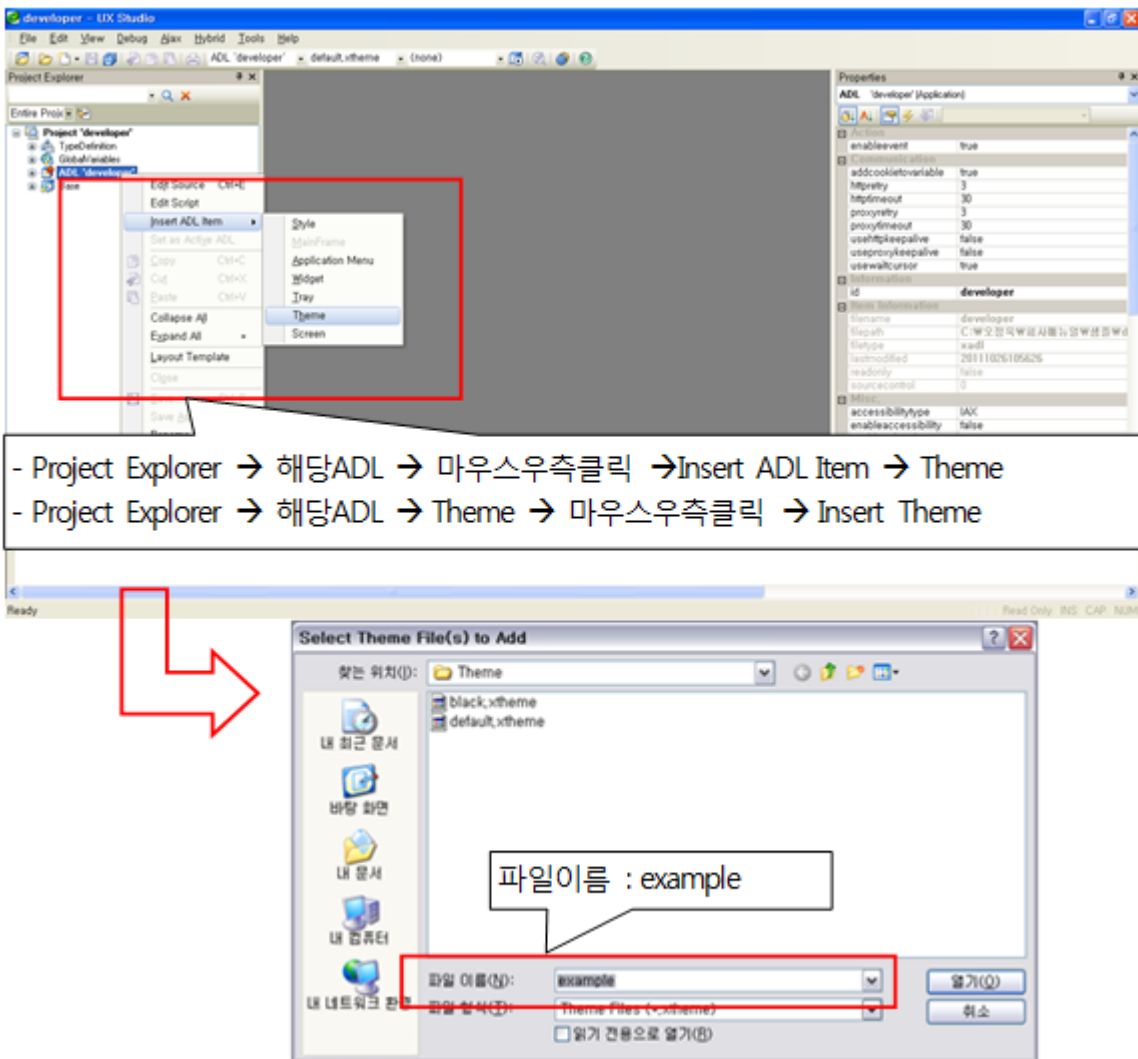
Theme의 생성방법은 Insert Theme(Insert ADL Item)을 통해서 미리 작성된 Theme를 선택해 등록할 수 있습니다.

Theme편집기는 Image, Style파일들을 추가, 삭제할 수 있으며 Style의 경우는 Style편집기를 통한 관리가 가능합니다.

Style이 변경되는 경우에는 반드시 Theme도 저장해 주어야 합니다.

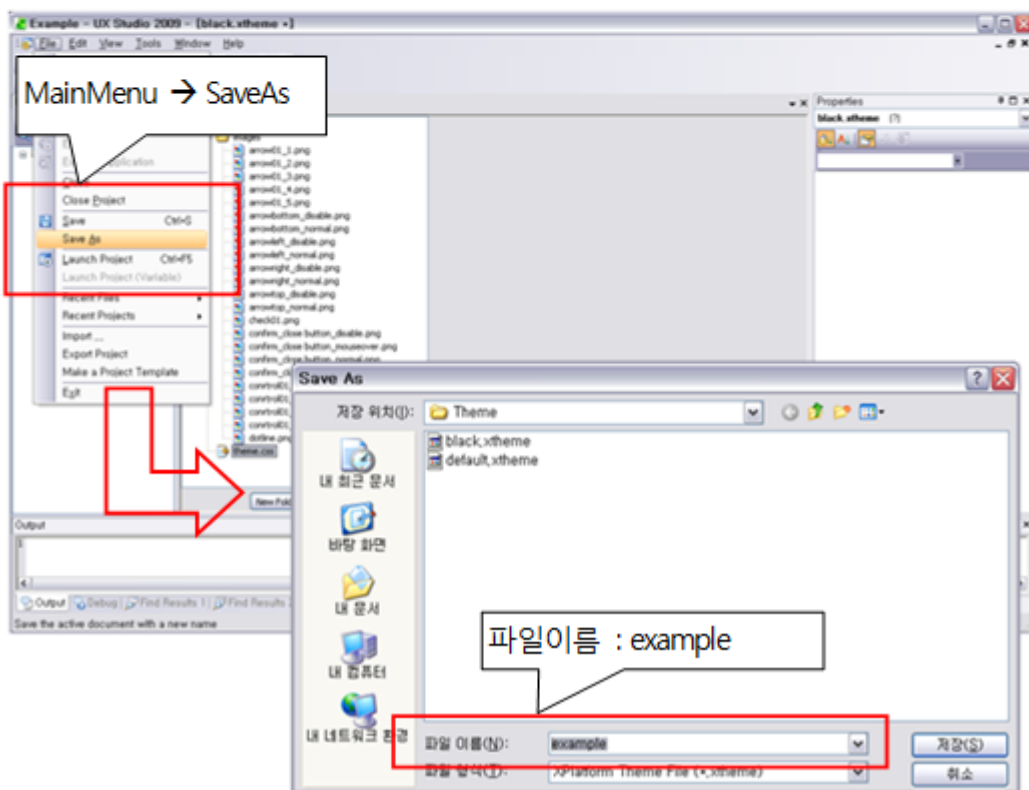
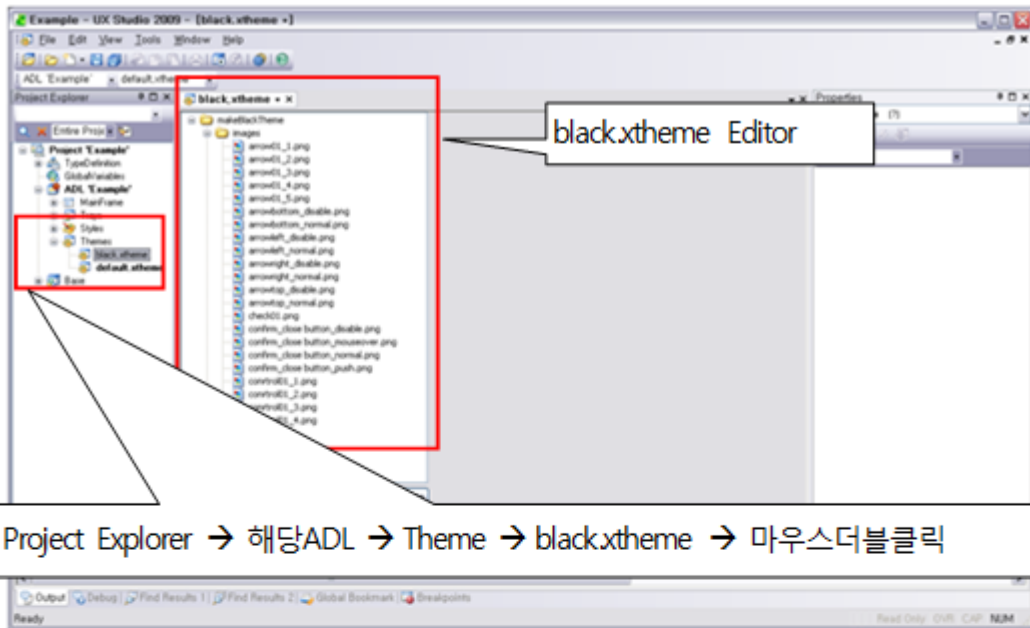
1단계: 새로운 Theme를 만듭니다.

ADL에 Item추가로 Theme를 추가로 생성합니다. (권장하지 않습니다.)



2단계: 기본으로 Theme를 편집해 새로운 이름으로 저장합니다.

black.xtheme 파일을 Open해 Theme편집기를 통해 example.xtheme로 저장합니다.



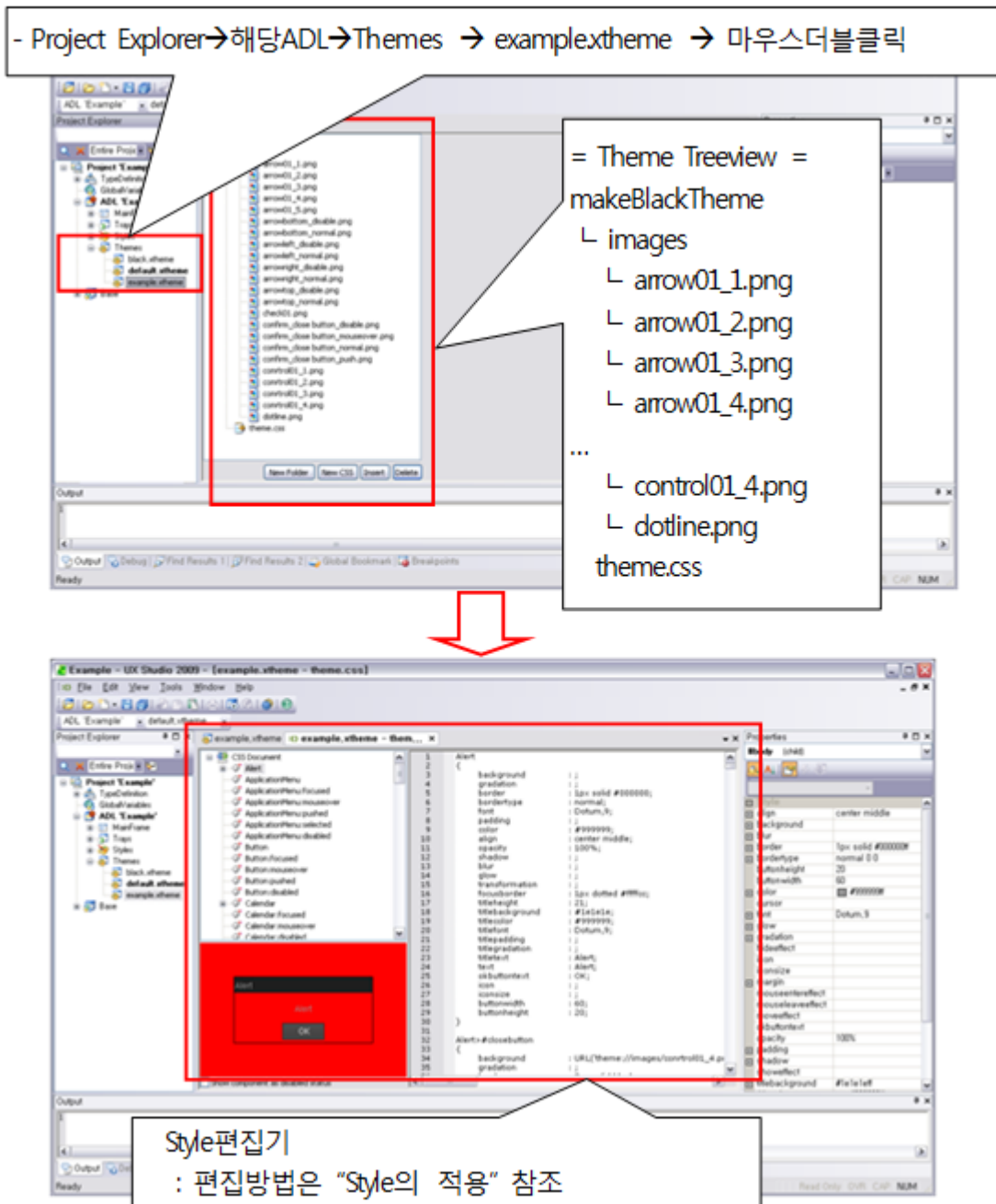
5.3.2 Theme의 편집

Theme파일은 Image와 Style은 서로 분리되어있어 별도로 편집합니다.

폴더의 생성, Style의 생성, Image와 Style의 추가/삭제는 Theme편집기의 Theme Treeview 하단의 버튼을 활용합니다.

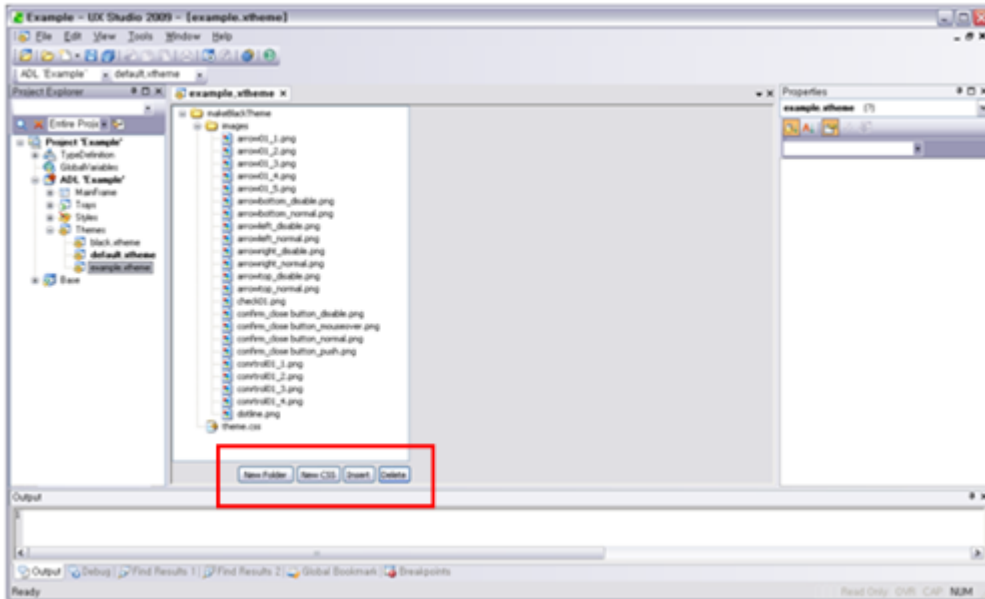
1단계: Theme Treeview의 내용을 확인하고 Style파일을 편집합니다.

Theme파일인 example.xtheme를 열어 Theme Treeview를 통해 Image List와 Style파일(theme.css)을 확인하고, Style편집기를 통해 Style파일을 편집합니다.



2단계: Theme편집기를 통해 Image, Folder, Style의 생성/추가/삭제 작업을 합니다.

Theme Treeview 하단의 버튼들을 통해 Theme 내용을 편집합니다.



메뉴	기능
New Folder	Theme 내에 새로운 폴더를 생성
New CSS	Theme 내에 새로운 Style 파일을 생성
Insert	Image 또는 Style파일 추가
Delete	Treeview에서 선택된 파일 삭제

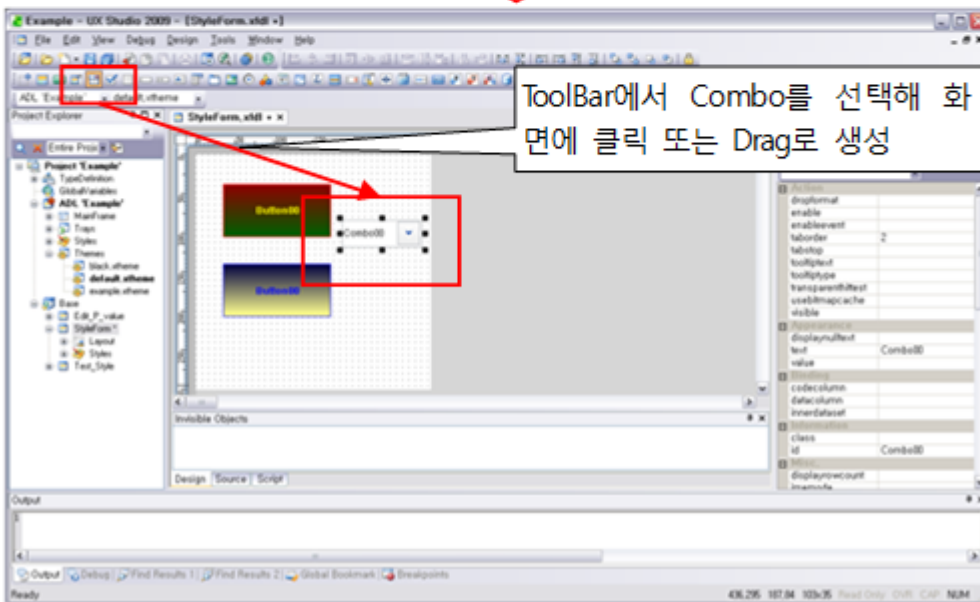
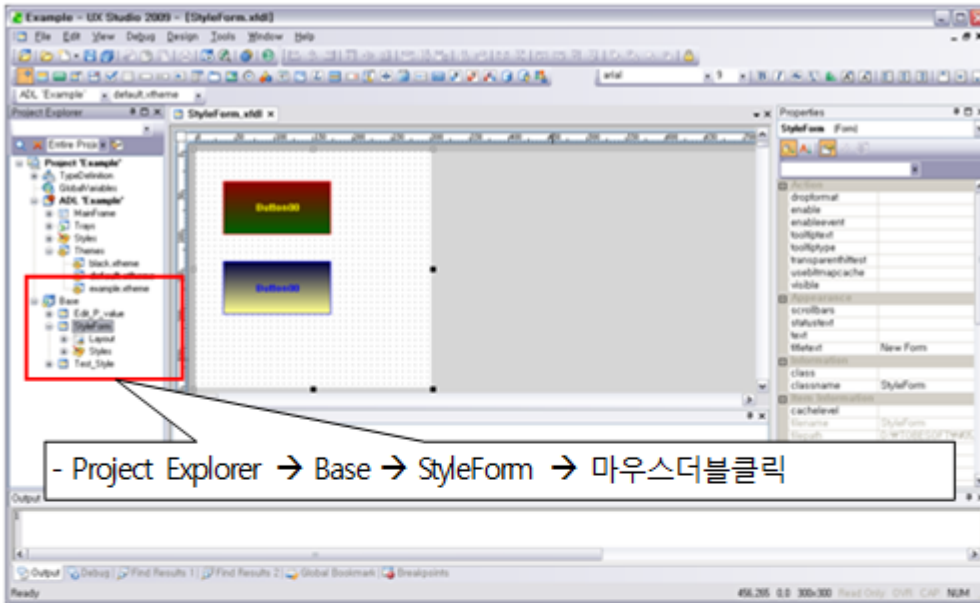
5.3.3 Theme의 적용

기본으로 제공되는 default Theme의 Image와 Style이 반영되는 모습과 Set Active로 바꾸면서 컴포넌트에 반영되는 것을 확인할 수 있습니다.

작성은 Combo 컴포넌트를 기준으로 작성하도록 하겠습니다.

1단계: default Theme의 Combo Style을 확인합니다.

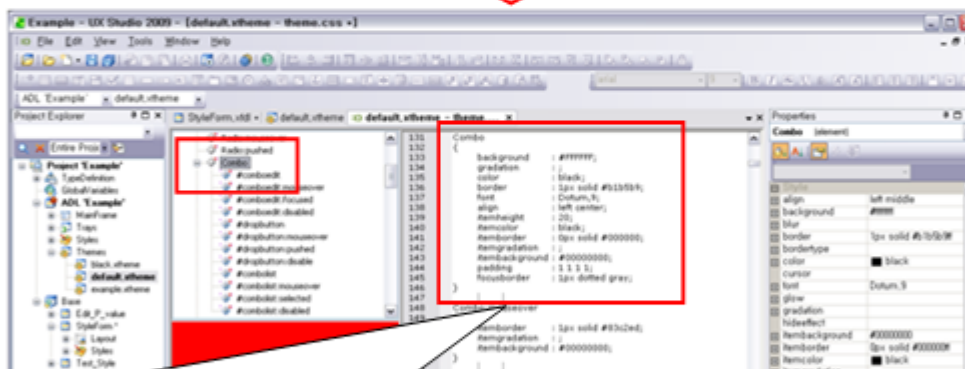
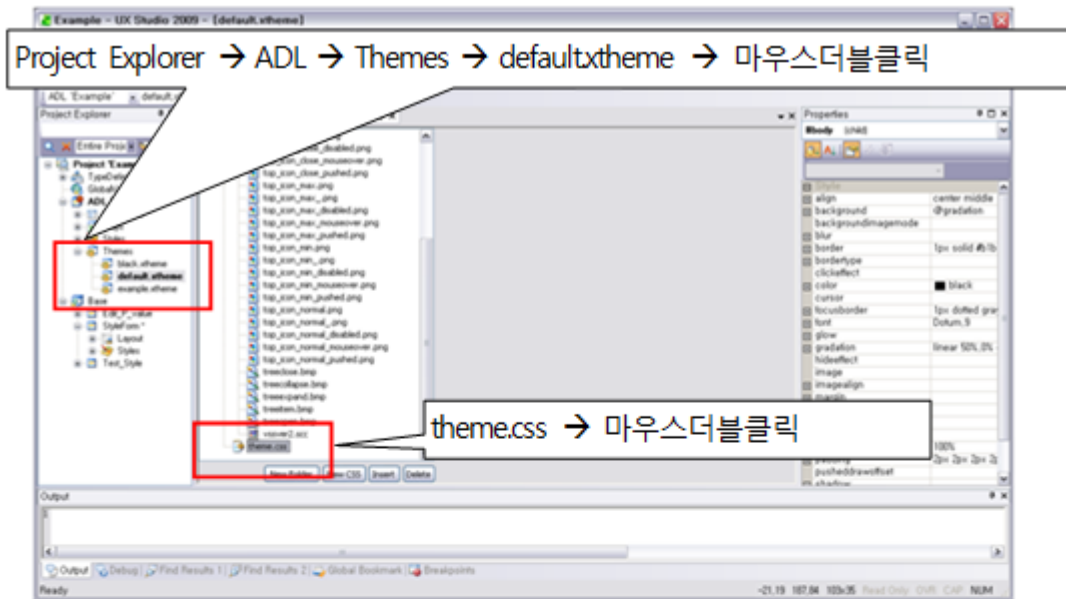
StyleForm을 열어 변경되기 전의 Combo를 생성해 Style을 확인합니다.



2단계: default Theme의 Style정보를 수정합니다.

Active 되어 있는 default.xtheme를 Theme Editor를 통해 수정해 줍니다.

theme.css를 열어 Combo 컴포넌트의 정보를 수정합니다.



```

Combo
{
  background : #FFFFFF;
  gradation : ;
  color      : black;
  ...
  focusborder : 1px dotted gray;
}

```



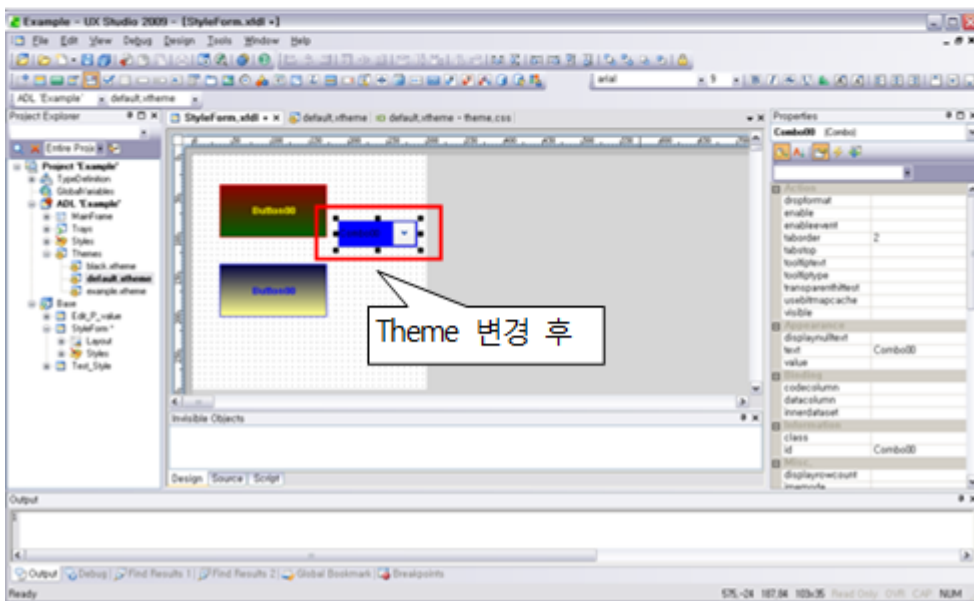
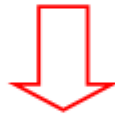
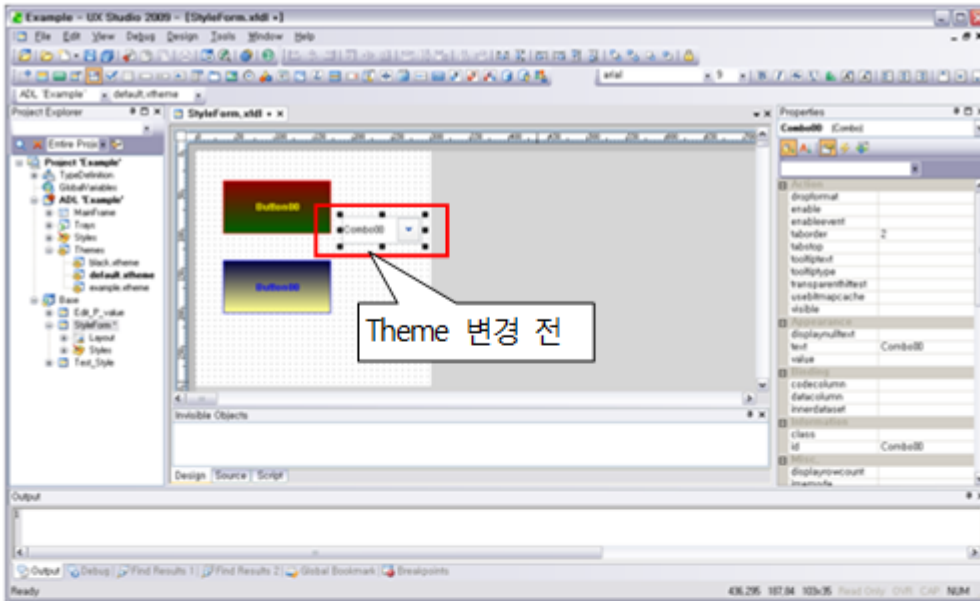
```

Combo
{
  background : blue;
  gradation : ;
  color      : black;
  ...
  focusborder : 1px dotted gray;
}

```

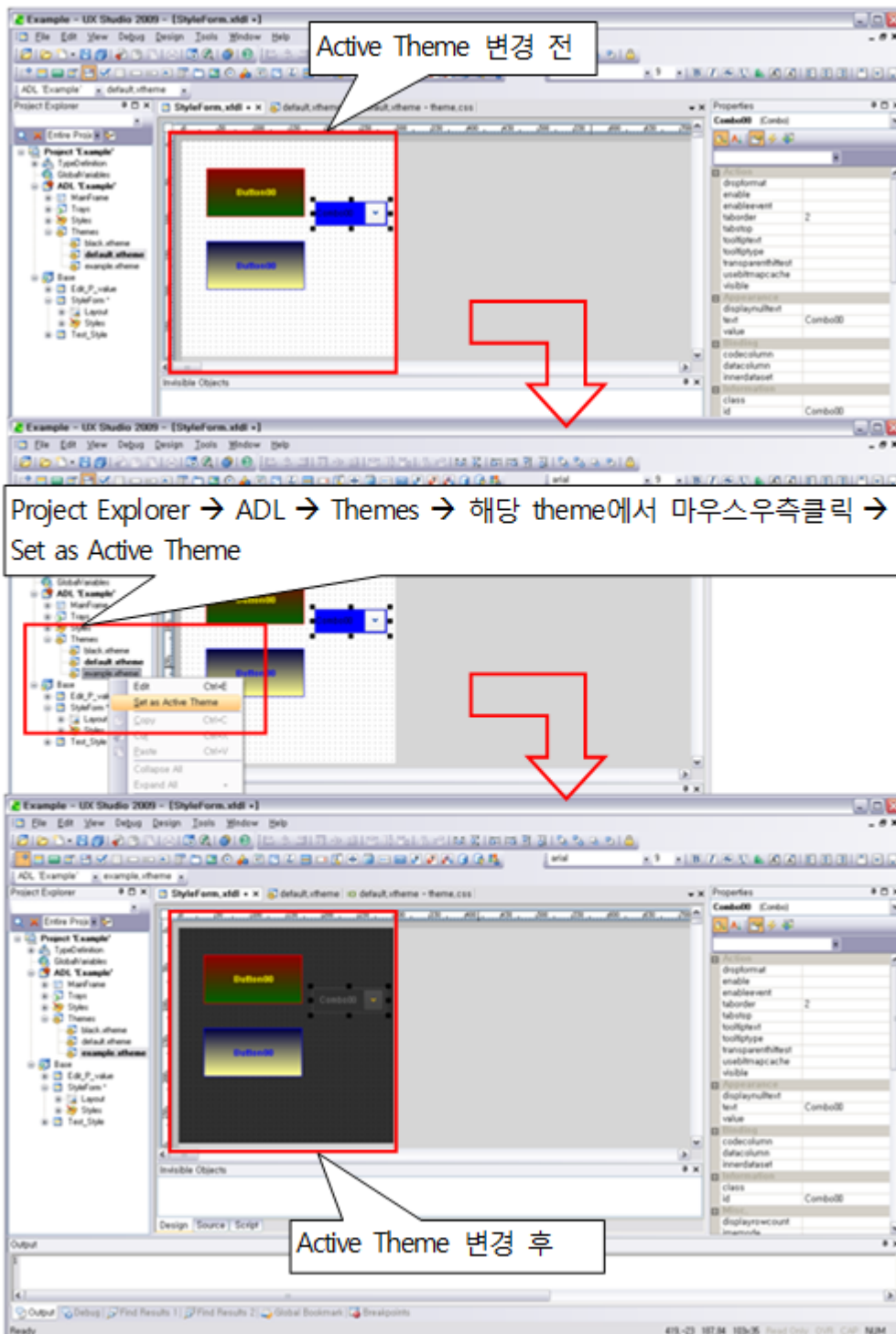
3단계: default Theme의 Style정보를 수정을 Form에서 확인합니다.

StyleForm의 Combo 컴포넌트의 Style 변화를 확인합니다.



4단계: Active Theme를 example Theme로 변경해 변화를 확인합니다.

새로 생성한 example Theme를 Set Active Theme로 Active하고, StyleForm의 Style의 변화를 확인합니다.



5.3.4 Deploy Theme

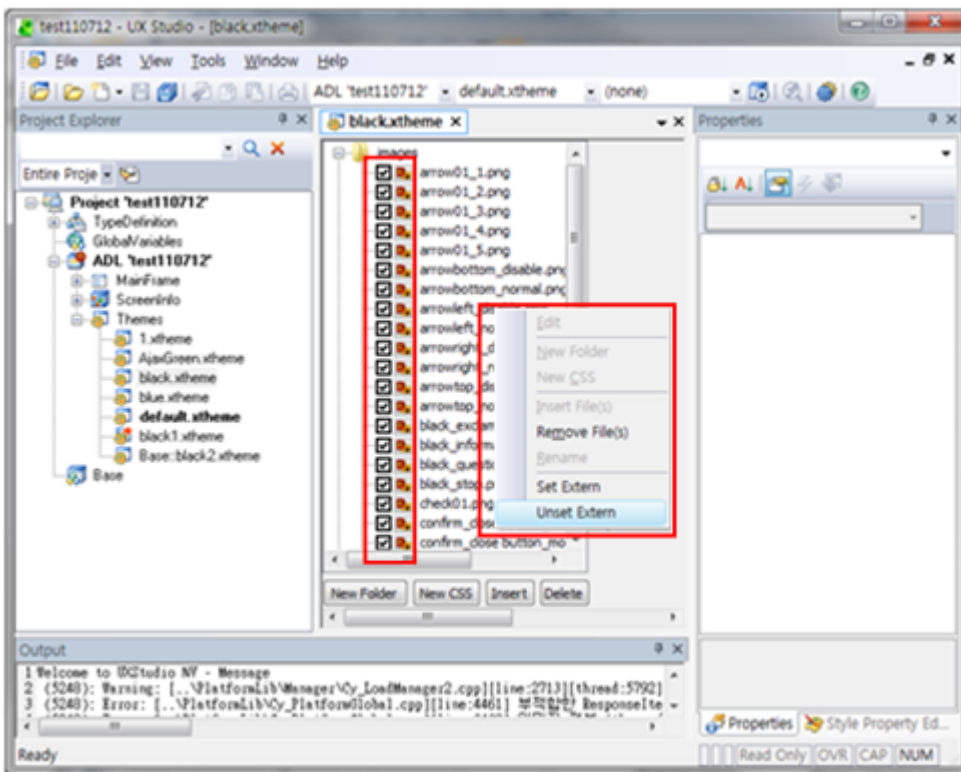
기존의 Theme는 Image와 Style이 결합되어 있어 그 크기가 크다는 단점이 있었습니다. Deploy Theme란 기존의 xt heme에서 Image를 분리하여 Theme의 크기를 줄이기 위해 만든 기능입니다. Image는 선택하여 분리할 수 있으며 이렇게 분리된 xtheme 파일을 “Deploy Theme”라 하고 분리된 Image를 “Extern File”이라 합니다. 또한, 이 기능은

XPLATFORM Runtime, HTML5, Hybrid모두에서 지원됩니다.

- Extern File로 분리할 Image선택

CheckBox를 이용하여 Extern File로 분리할 Image를 선택할 수 있습니다.

- CheckBox에서 UnCheck상태에 있는 Image만 extern file로 분리되며 Check상태에 있는 Image는 기존 방식처럼 Theme에 포함됩니다. 또한, UnCheck상태에 있는 Image 각각을 Extern Item이라 합니다.
- default.xtheme와 같이 기본적으로 제공되는 Theme는 CheckBox가 나타나지 않습니다. 즉, 기본제공 Theme는 Image를 분리할 수 없습니다.
- Space Bar를 이용해서 CheckBox를 선택할 수도 있습니다. Multi Select도 가능하며 선택한 첫 번째 아이템의 상태를 기준으로 변경됩니다.
- 마우스 우측 버튼을 클릭 후 Popup Menu를 이용해서 CheckBox를 선택할 수도 있습니다. Multi Select도 가능합니다. Set Extern메뉴가 UnCheck상태이고 Unset Extern메뉴가 Check상태입니다.



- 저장

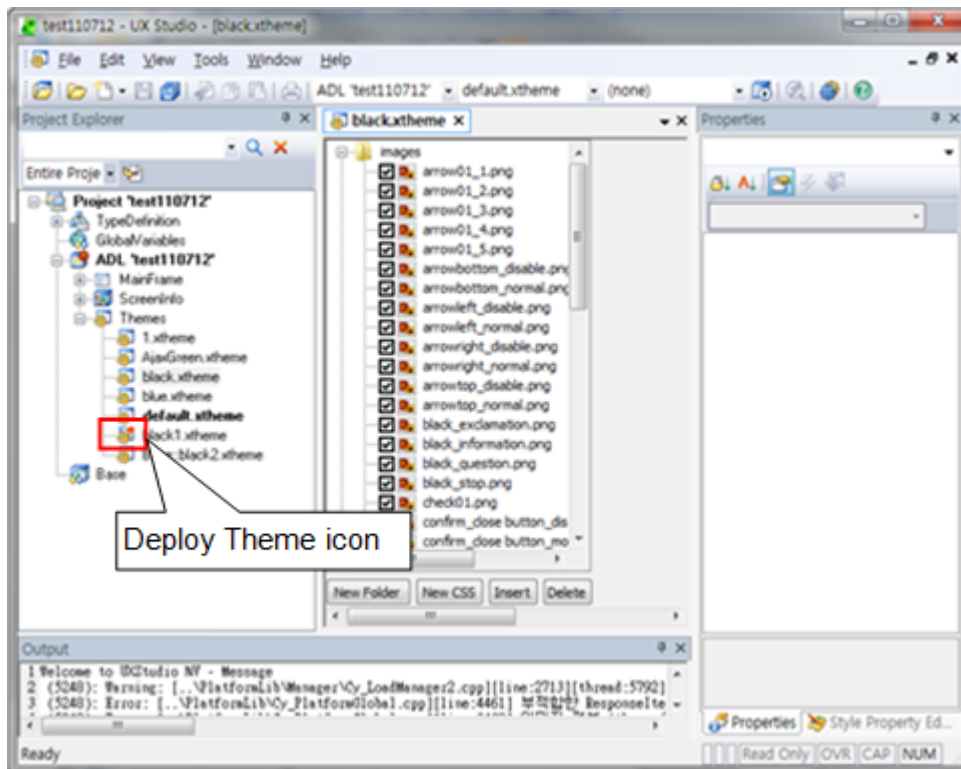
Deploy Theme의 저장은 기존 저장법과 동일하게 Save버튼을 누르면 됩니다. 저장 시 Deploy Theme와 Extern File의 저장내용 및 경로는 다음과 같습니다.

구분	저장내용	저장경로
Deploy Theme	Extern File을 제외한 Theme	기존 theme경로 및 파일명과 동일 예) 기존 theme를 C:\Wa.xtheme라 하면 C:\Wa.xtheme로 저장됨
Extern File	분리된 Image	기존 theme경로/images/분리된 Image들(파일명동일) 예) 기존 theme를 C:\WthemeWa.xtheme라 하고 분리할 Image가 b.png라 하면

구분	저장내용	저장경로
		C:\Wawimages\wb.png로 저장됨

- Deploy Theme표시

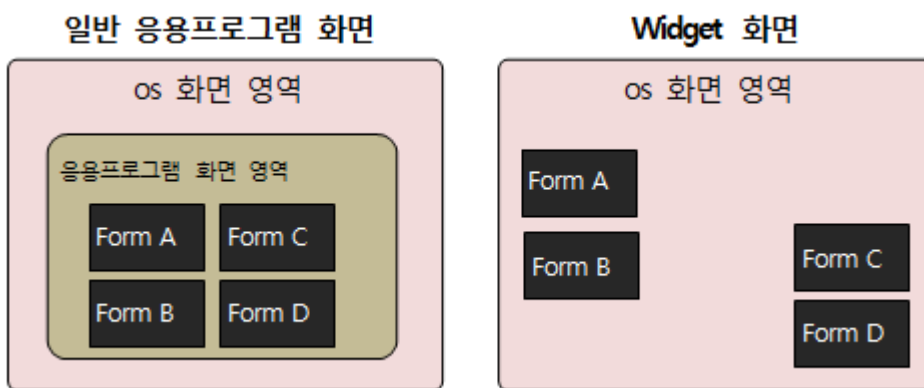
Extern Item이 있는 상태로 저장을 하면 Project Explorer에 Deploy Theme표시가 생기며 Extern Item을 제거하고 저장을 하면 Deploy Theme표시도 사라집니다.



6.

Widget

Widget은 소형 응용프로그램으로써 사용자의 구미에 맞게 재 배치가 가능하다는 장점을 갖고 있습니다. 일반 응용 프로그램의 경우, 그 UI영역을 큰 틀 안에 모든 화면 Form을 배치하는 Interface를 제공합니다. 반면에 Widget인 경우, 각각의 화면 Form들을 독립적으로 실행시켜서 그 배치를 사용자가 원하는 위치에 마음대로 배치할 수 있습니다.



Widget은 다음의 용도에 주로 사용됩니다.

- 기업내의 업무를 외부의 지시 또는 변화되는 정보에 의해 작업자에게 자동으로 업무진행을 요청하는 용도
- 개인스케줄에 의한 업무와 해당 업무화면간의 자연스러운 연동이 되는 용도
- 자주 사용하는 화면을 Widget에 등록하거나 삭제 하는 등의 용도
- 차세대 Enterprise Portal로서의 Widget의 용도
- 증권, 상담원, 날씨, 뉴스, 생활정보, 일정관리, 시계 등 지속적으로 갱신되는 정보를 팝업 형태로 보여주거나 알림창을 띄워주는 용도

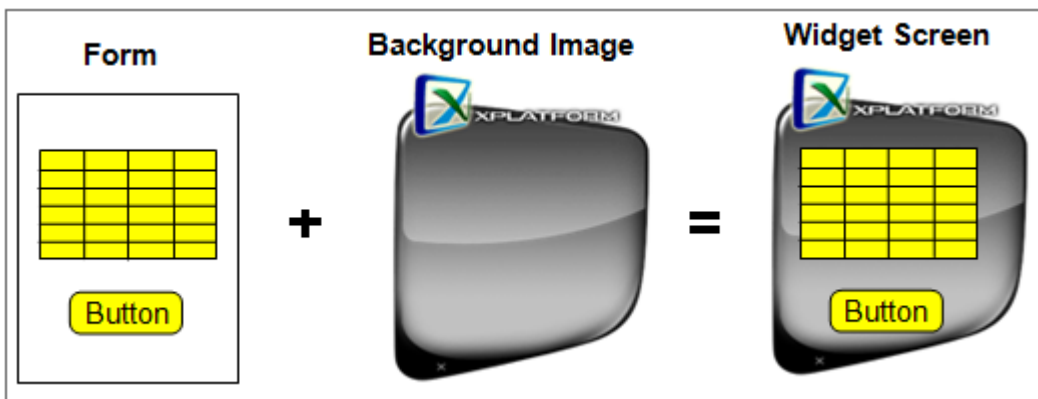
6.1 Widget 개요

Widget은 MainFrame 없이 Form만으로 화면UI를 구현할 수 있습니다.

6.1.1 Widget의 구성

일반 응용프로그램의 UI화면은 Frame과 Form의 조합으로 만들어지는 반면, Widget은 Form과 Background이미지의 조합에 의하여 만들어집니다.

아래의 그림을 보면 이해가 쉬울 것입니다.



6.1.2 Widget Form의 제약사항

Widget에서 사용하는 Form은 background이미지를 반드시 가져야 하므로, Form자체의 `style.background.color` 값은 "transparent"로 설정해야만 background이미지가 정상적으로 출력됩니다.

6.1.3 Widget와 Layered기능

Layered기능이란, UI화면의 창이 이미지만큼의 영역만 갖게 하는 기능입니다.

다음의 그림을 보면 쉽게 이해하실 수 있습니다.

Layered기능이 없는 화면

이 부분을 Click하면 Widget에서 event가 발생합니다. 즉, 이 부분도 Widget화면의 일부분입니다.

Layered기능이 있는 화면

이 부분을 Click하면 Widget에서 event가 발생하지 않습니다. 즉, 이 부분은 Widget화면이 아닙니다.

특히, Widget의 경우는 그 화면의 모양이 사각형이 아닌 다양한 모양을 갖는 경우가 많습니다. 그러므로 이 Layered 기능을 사용하는 경우가 많습니다.

Layered기능을 위해서는 background이미지가 투명이미지여야만 합니다. 그렇지 않을 경우, 이미지의 바탕 사각형 영역도 이미지로 간주되어 Layerd기능을 올바르게 사용할 수 없게 됩니다.

6.1.4 Widget 만드는 방법

XPLATFORM은 Widget 만드는 방법을 3가지를 제시하고 있습니다.

- Widget Project로 만들기

Widget만 사용할 경우의 개발 방법입니다.

MainFrame없이 Widget만 사용합니다. 그러므로, 일반 응용프로그램 화면 없이 Widget으로만 사용자 화면을 구성할 때 사용합니다.

- 일반 Project에서 Widget 만들기
일반 응용프로그램 화면이 있고, 부가적으로 Widget를 병행하여 사용할 경우의 개발 방법입니다.
MainFrame과 Widget이 공존하여 다양한 UI를 제공하고 할 때 사용합니다.
- script로 Widget만들기
동적으로 Widget를 만들 때 사용합니다.

이 3가지 Widget만드는 방법은 다음 장에서 상세히 다룹니다.

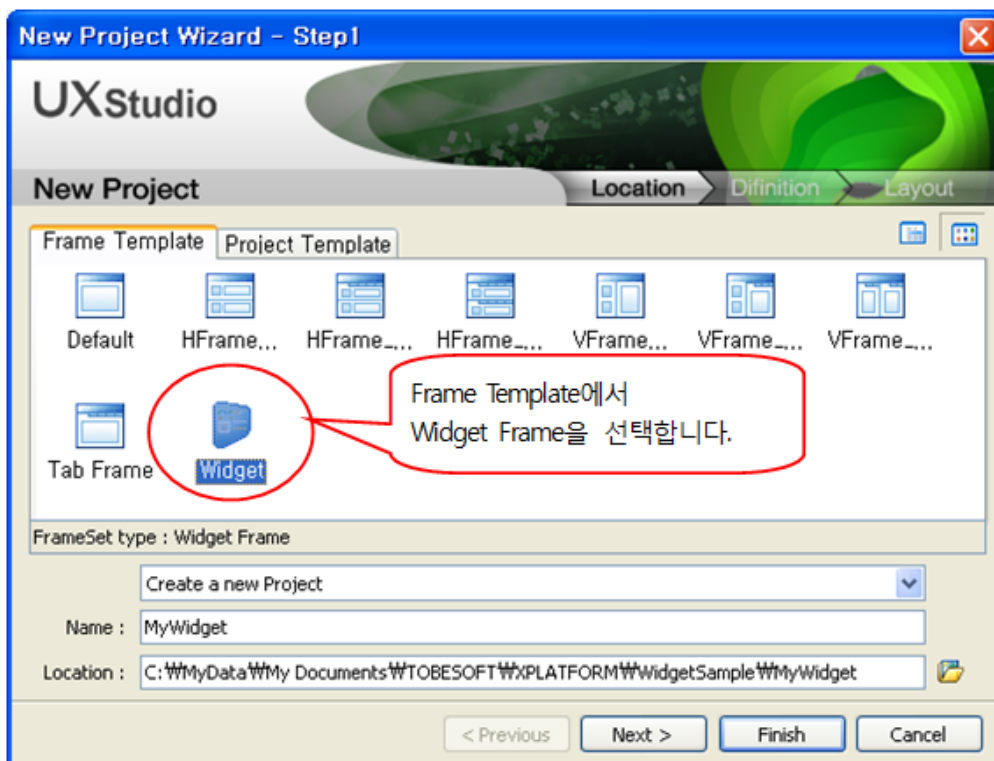
6.2 Widget Project로 만들기

일반 응용프로그램 화면 없이 Widget만으로 화면을 구성할 때 사용합니다.

6.2.1 Widget 프로젝트의 생성

UX-Studio상에서 Project를 맨 처음 생성할 때부터 Widget Project로 생성해야만 합니다.

1 단계:Widget Project선택





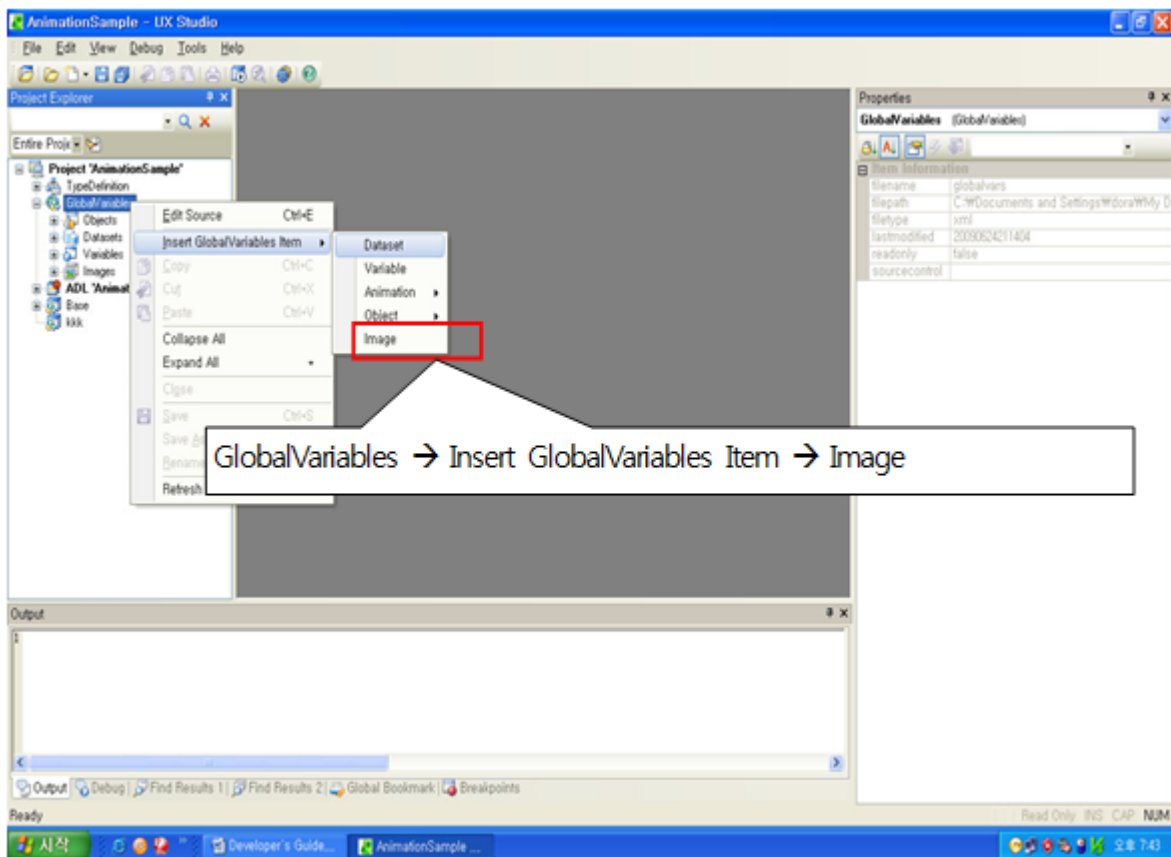
HTML5는 widget을 지원하지 않습니다.

6.2.2 background 이미지 등록

Widget Project를 생성한 후에 Widget이 사용할 이미지를 먼저 등록합니다. Form을 생성할 때 연결할 background 이미지를 물어보므로 먼저 이미지를 등록하는 것이 좋습니다.

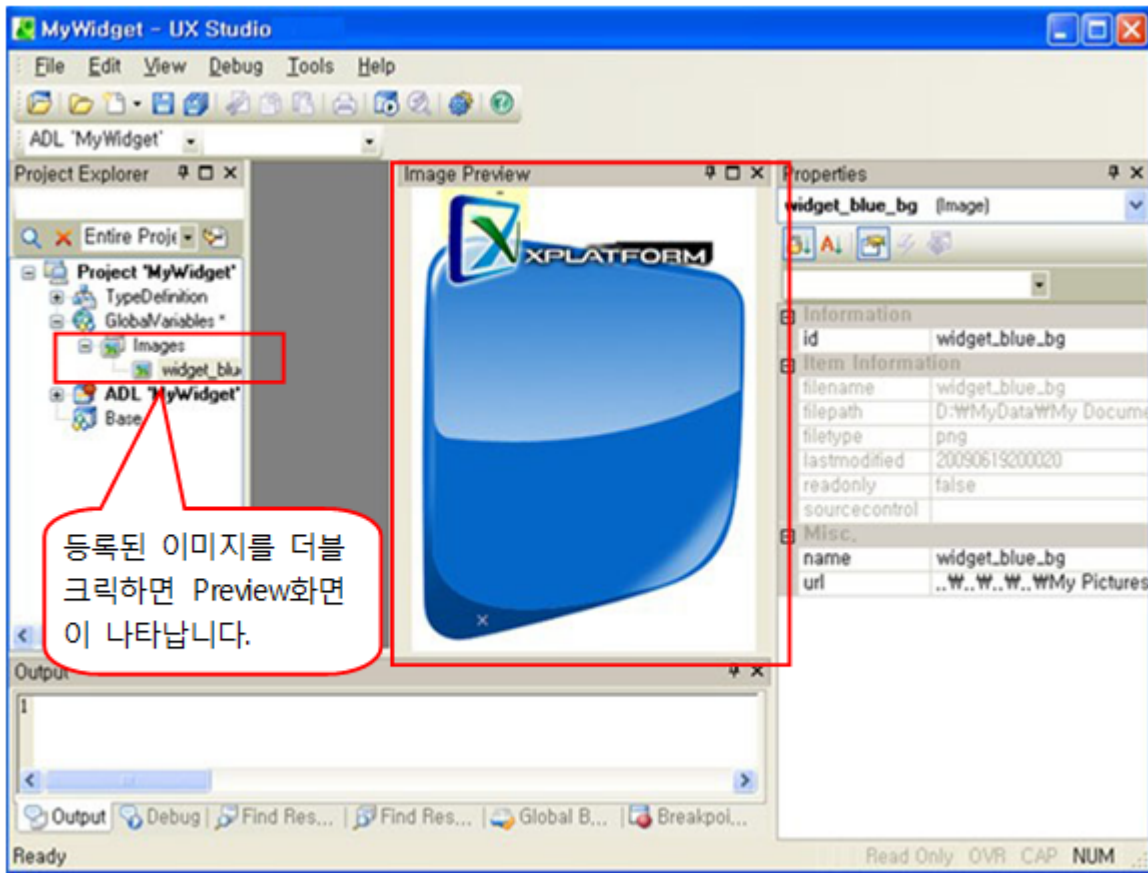
1단계 : background 이미지 등록

Project Explorer에서 GlobalVariables에 아래와 화면처럼 이미지를 등록합니다. 여기서 등록한 이미지는 Widget 생성시에 background이미지로 사용합니다.



2단계 : 등록된 이미지 확인

등록한 이미지를 Project Explorer창의 GlobalVariable에서 확인합니다.



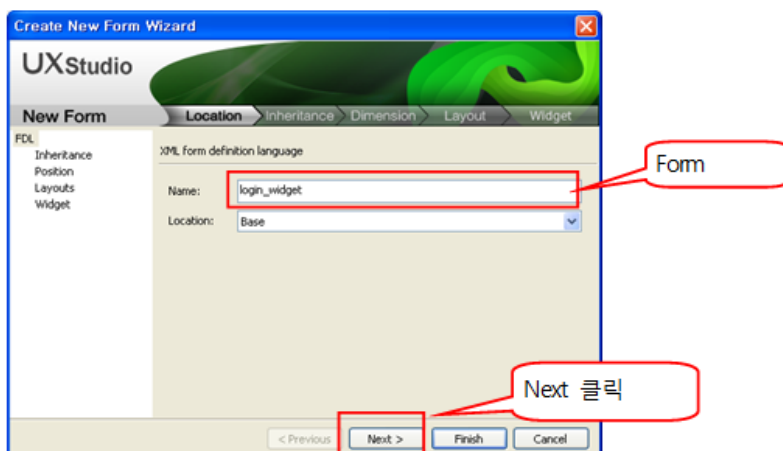
이 이미지는 배경이 투명 처리된 이미지 입니다.

6.2.3 Widget Form의 등록

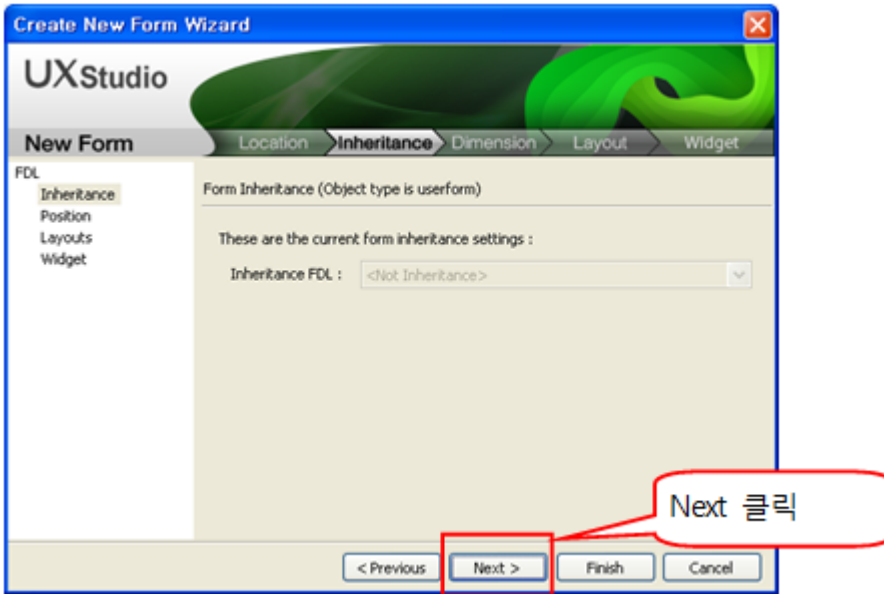
이제 Form을 생성합니다. Widget Form을 생성하면 자동으로 ADL에 Widget이 등록됩니다.

1단계 : Widget Form생성

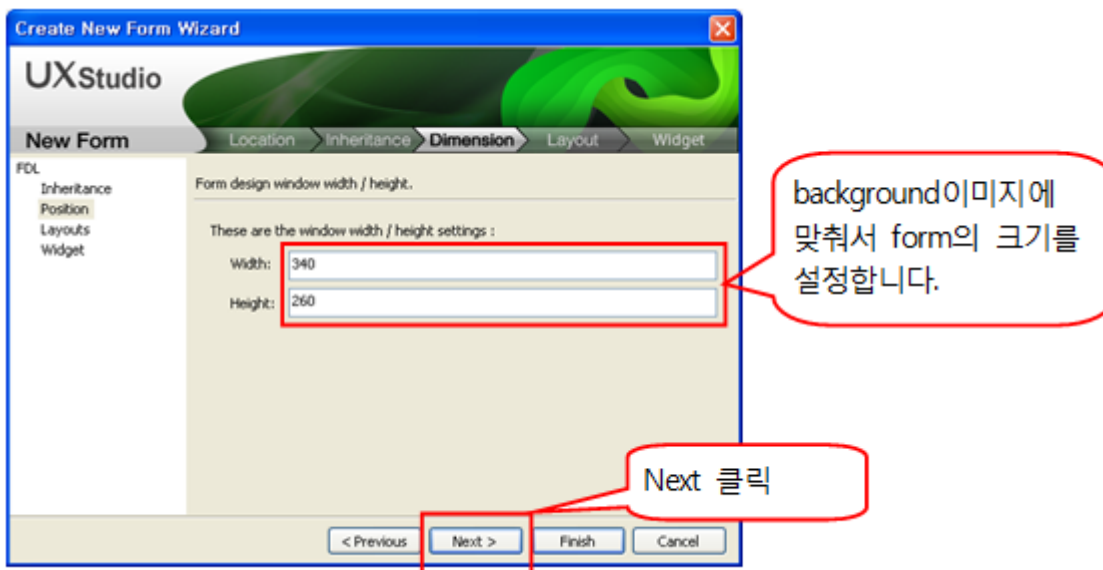
다음의 화면은 Form을 생성하는 화면으로 첫 번째 화면은 일반 form생성과 동일합니다.



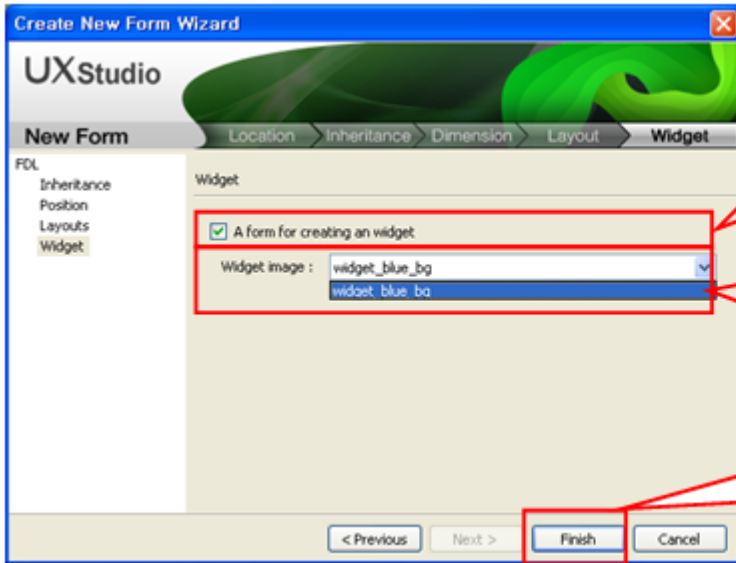
그 다음 화면도 일반 form생성과 동일합니다.



그 다음 화면도 일반 form생성과 동일합니다. 단지 Form의 크기를 background 이미지 크기를 고려하여 설정합니다.



그 다음 화면은 일반 form생성과 다릅니다. 다음과 같이 설정합니다.



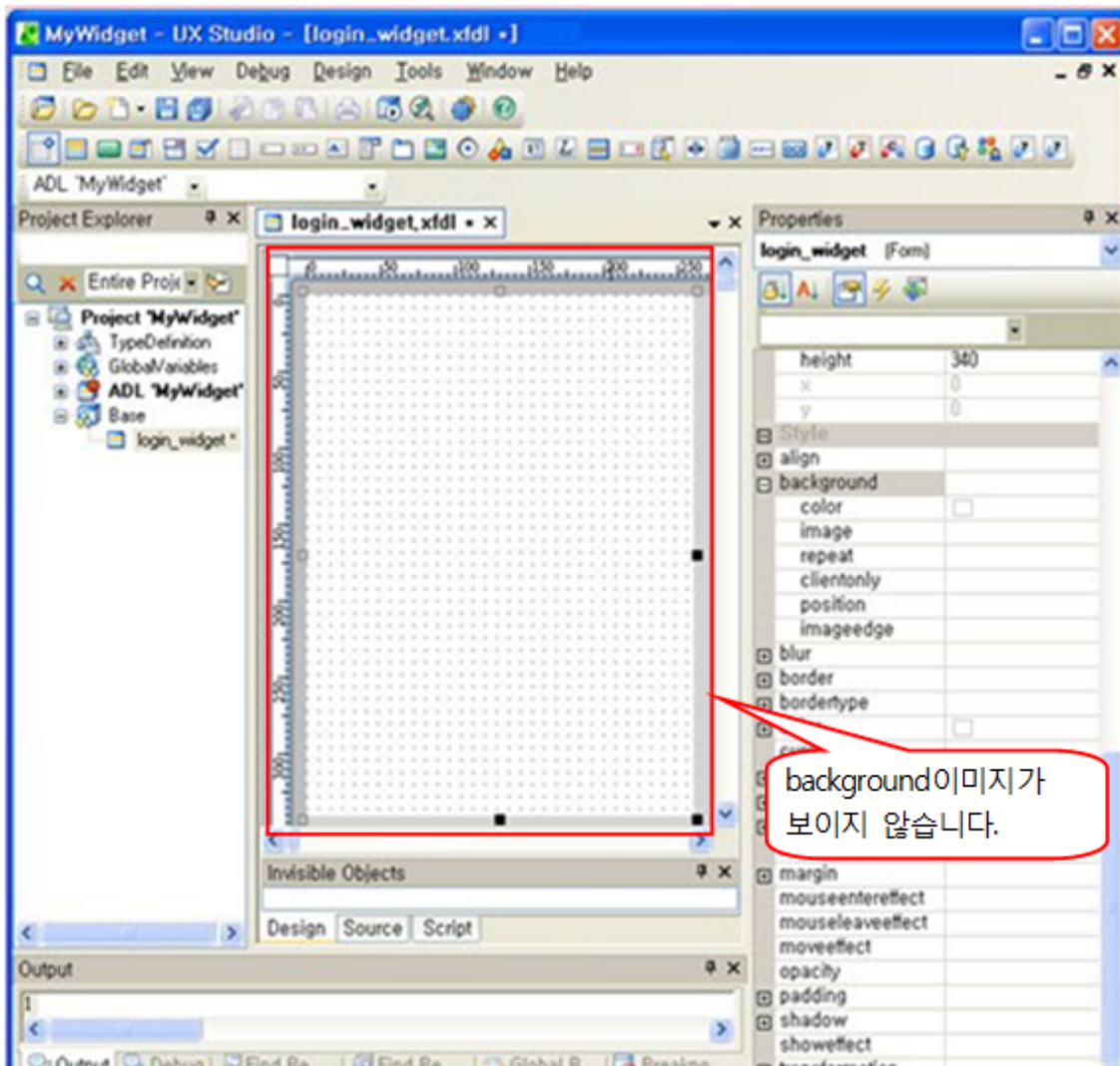
이 체크박스를 체크하여 Widget 폼을 생성합니다.

여기서 8.2.1장에서 등록한 이미지를 선택합니다.

OK를 클릭하여 폼을 생성합니다.

2단계 : Widget Form설정

다음은 1단계에서 생성한 Form의 Editor화면입니다. 이 화면을 보면, Background 이미지가 보이지 않고 있습니다.

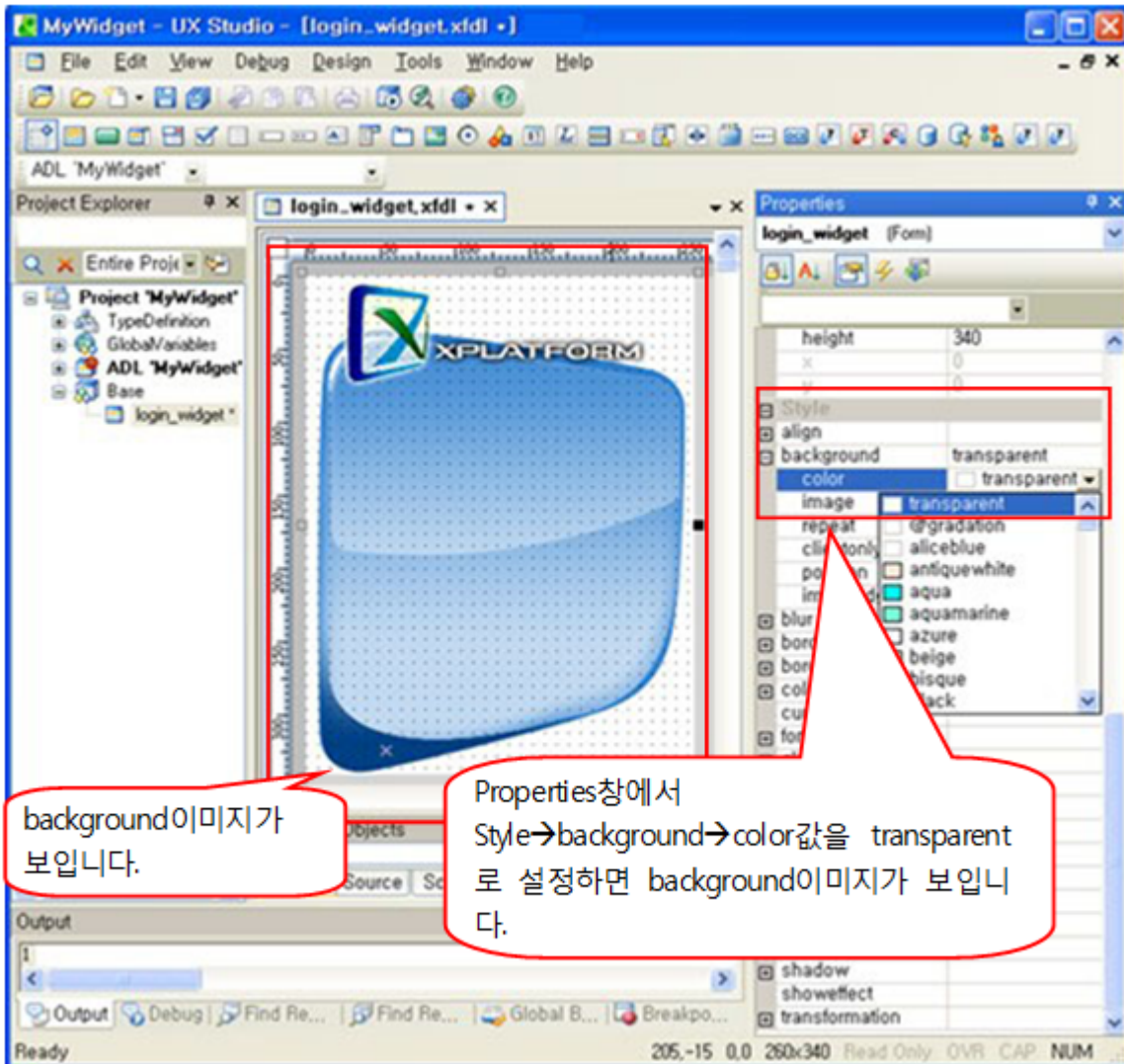


background이미지가 보이지 않습니다.

background 이미지가 보이지 않는 이유는 Form의 바탕색이 background 이미지를 가려서 보이지 않는 것 입니다.

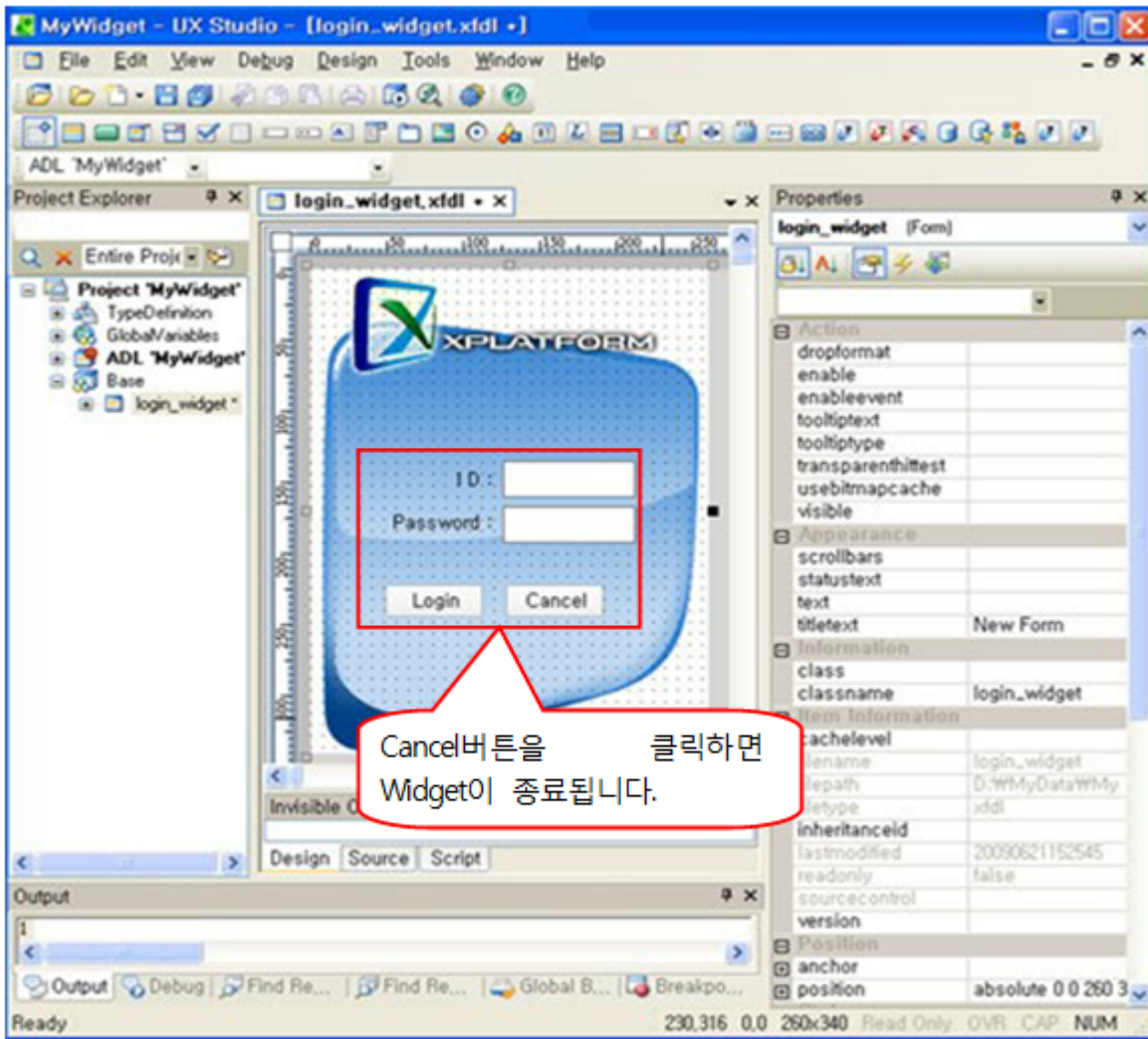
background 이미지가 보이게 하려면, Form의 style을 투명 처리해야만 합니다.

다음과 같이 Form의 Property값을 설정합니다.



3단계 : Widget Form의 개발

다음은 간단하게 Form에 컴포넌트들을 사용하여 Login 화면을 만든 것입니다. 여기서는 Cancel버튼을 클릭할 때, Widget이 종료되는 Event Function을 구현하겠습니다.



“Cancel”버튼의 Click Event Function에 다음과 같이 코딩 합니다.

```
function Button01_onclick(obj:Button, e:ClickEventInfo)
{
    exit(); // close()를 사용하지 않고 exit()를 사용했습니다.
}
```

여기서 exit()와 close()의 차이점은 exit()는 Widget 응용프로그램 자체를 종료하는 것이고, close()는 Widget Form만 종료하는 것입니다. 만일, close()를 사용하면 응용프로그램 Process가 메모리에 남아있게 됩니다.

6.2.4 Widget의 실행

UX-Studio에서 Widget을 실행하려면, “Quick View”로는 실행할 수 없습니다. 반드시 “Launch Project”로 실행해야 합니다.

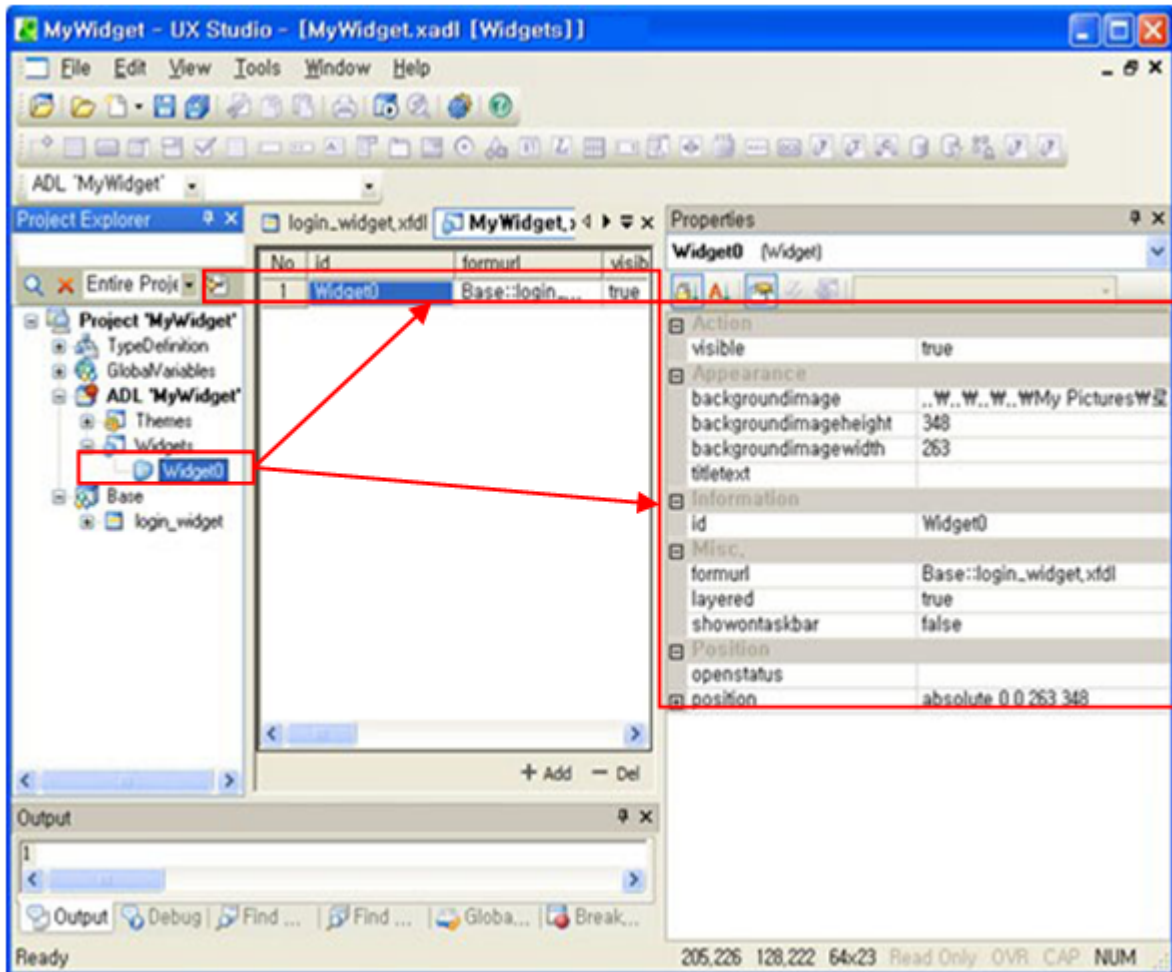


이 화면은 Widget을 Web Browser위에서 실행시킨 화면입니다. 보이는 바와 같이 투명 처리된 이미지들 사이로 뒤의 Web Browser의 화면이 보입니다.

6.2.5 응용프로그램에 등록된 Widget 확인

개발을 완료하면 ADL에 자동으로 Widget이 등록됩니다.

다음은 Project Explorer창에서 ADL에 등록된 Widget정보입니다.



[Project > ADL > Widgets > Widget0]에 자동으로 등록되어 있습니다. 물론, 이 화면에서 그 설정을 변경할 수 있습니다.

Widget의 주요 Property값을 보면 다음과 같습니다.

Name	Description
visible	Widget을 보여줄지를 결정합니다. 기본값은 true입니다.
id	ADL에 등록된 Widget의 ID입니다. Script로 접근할 때 주로 사용합니다.
background image	등록한 Image의 Path값이 설정되어 있습니다.
formurl	생성한 form의 url값이 설정되어 있습니다.
layered	layered기능 사용여부입니다. 기본값은 true입니다.

6.3 일반 Project에서 Widget 만들기

일반 Project에서 Widget를 생성하는 방법은 “[Widget 프로젝트의 생성](#)”을 제외하고, “[Widget Project로 만들기](#)”의 내용과 동일합니다.

즉, Project 자체는 일반 응용프로그램으로 만들고, 이후에 Widget을 등록하는 것으로 “[background 이미지 등록](#)”부터 “[응용프로그램에 등록된 Widget 확인](#)”까지 수행하면 Widget을 등록할 수 있습니다.

6.4 Script로 Widget 만들기

대부분의 Widget은 UX-Studio의 Wizard기능을 이용하여 생성 및 등록했습니다. 실제로 “[Widget 개요](#)”의 모든 예제화면이 Wizard화면입니다.

그러나, 필요에 따라서 동적으로 Widget을 생성할 필요가 있습니다. 여기서는 Script를 이용하여 동적으로 Widget을 만드는 방법을 설명합니다.

6.4.1 동적 Widget생성을 위한 준비

동적 Widget의 생성 이전에 아래의 두 가지 준비를 해야만 합니다.

- Widget용 background Image를 준비합니다.
“[background 이미지 등록](#)”에서와 같은 방법으로 Image를 등록합니다.
- Widget용 Form을 생성합니다.
Widget에서 사용할 Form을 생성합니다. Widget용으로 생성한 것이 아니므로, ADL에 Widget이 등록되지 않은 상태입니다.

6.4.2 Script로 동적 Widget 생성

Script로 Widget을 생성하는 것은 실제로는 Widget Object를 생성하고, 그 생성된 Object에 Form과 background 이미지를 등록하는 행위를 Script로 처리하는 것을 의미합니다.

1단계 : Widget Object를 생성하고 이미지와 Form을 연결합니다.

```
newWidget = new Widget(); // widget Object의 생성

// 생성한 Widget Object에 초기값을 설정합니다.
// Widget Object명은 "Widget1" 입니다.
// Widget이 보여질 Screen상의 Left 좌표값이 10입니다.
// Widget이 보여질 Screen상의 Top 좌표값이 100입니다.
// Widget이 사용하는 Form url이 "Base::Sample2.xfdl"입니다.
// Widget이 사용하는 background 이미지가 "image::widget.png"입니다.
newWidget.init("Widget1",10,100,"Base::Sample2.xfdl","image::widget.png");
```

상세한 Method의 사용법은 XPLATFORM Reference Guide를 참조하세요.

2단계 : Widget Object를 ADL에 등록합니다.

1단계에서 생성한 Widget Object를 ADL에 등록합니다.

```
application.addWidget("widgetID", newWidget);
```

UX-Studio에서 Wizard로 Widget용 Form을 생성하면, 자동으로 등록됩니다. 등록결과는 [“응용프로그램에 등록된 Widget 확인”](#)에서와 같이 확인할 수 있습니다. 그러나 이렇게 동적으로 등록한 경우에는 UX-Studio상에서는 확인할 수 없습니다.

6.4.3 Widget을 화면에 출력합니다.

생성한 Widget을 화면에 보여줍니다.

```
newWidget.show();
```

6.4.4 Widget Object를 삭제합니다.

Widget Object를 소멸시킵니다.

```
newWidget.destroy();
newWidget = null;
```

6.4.5 Script로 Widget에 접근 하기

동적으로 생성한 Widget Object는 Script상에서 다음과 같은 방법으로 접근 할 수 있습니다.

```
application.widgets[index];  
application.widgets(index);  
application.widgets["id"];  
application.widgets("id");
```

여기서 접근할 수 있는 Widget Object들은 동적으로 생성한 Object들 뿐만 아니라, Wizard로 생성한 Object들에게도 접근할 수 있습니다.

7.

Event Flow

XPLATFORM 응용프로그램은 EDA(Event-Driven Architecture)를 기본으로 하고 있습니다. 즉, XPLATFORM 응용프로그램에서 개발자가 개발하는 부분의 시작점은 모두 Event Function부터 시작합니다.

7.1 Event의 개요

7.1.1 Event 처리 대상

XPLATFORM 응용프로그램에서 모든 Object들이 Event처리 대상은 아닙니다. 주로 동적으로 상태가 변하는 Object들이 Event 처리 대상이 됩니다. Event처리를 위하여 개발자가 Script처리하는 부분을 Event Function이라고 합니다.

다음은 UX-Studio상에서 Event Function을 등록할 수 Object입니다.

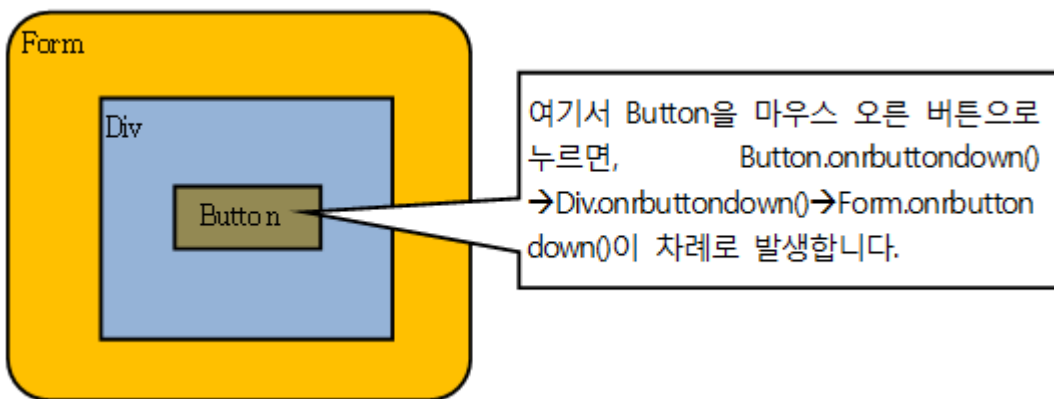
Project Explorer 분류	Event를 등록할 수 있는 Object	설명
ADL	ADL	- application의 로딩/시작/종료 등에 대한 Event를 처리하는 Function들을 등록합니다.
	Frame들	- MainFrame, Frameset, ChildFrame등 Frame Object들에서 발생하는 Event들을 처리하는 Function들을 등록합니다.
	Tray	- Tray에서 발생하는 Event들을 처리하는 Event들을 등록합니다.
	Application Menu	- Application Menu에서 발생하는 Event들을 처리하는 Event들을 등록합니다.
Global Variable	Datasets	- Global Variable에 등록된 Dataset의 Event들을 처리하는 Event들을 등록합니다.
Service	Form	- Form이 작동하면 발생하는 Event들을 처리하는 Event들을 등록합니다.
	컴포넌트들	- Form내의 컴포넌트들이 작동하면 발생하는 Event들을 처리

Project Explorer 분류	Event를 등록할 수 있는 Object	설명
		하는 Event들을 등록합니다.
	Invisible Object들	- Form내의 Invisible Object들이 작동하면 발생하는 Event들을 처리하는 Event들을 등록합니다.

7.1.2 Parent-Child간 Event 전이

parent-child과 관계를 갖는 object들은 event가 상위 parent로 전이되는 것을 기본으로 합니다.

즉, 어떤 Form에 속한 컴포넌트에서 Event가 발생하면, 그 컴포넌트가 포함된 Form에서도 동일한 Event가 발생합니다. 그러나 parent가 해당 Event를 취급하지 않는 경우는 발생하지 않습니다.



다음은 parent로 Event를 전이하는 Event들입니다.

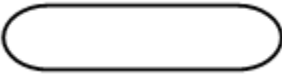
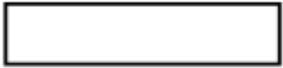
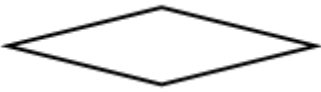
분류	Event들	Default가 parent발생?
키보드 Key	onkeyup()	O
마우스 Moving	onmousemove() onmouseleave() onmouseenter()	O O O
마우스 Button	onmousewheel() onmousedown() onmouseup() ondblclick() onmousedown() onmouseup() onmousedown() onmouseup()	O O O O O O O O
마우스 drag	ondrag() ondragenter() ondragleave()	X O O

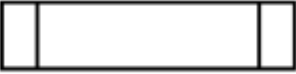
분류	Event들	Default가 parent발생?
	ondrop()	O

7.2 표기법

“Key event Flow”부터는 컴포넌트별 Event발생 순서를 흐름도로 나타냅니다. 여기서는 그 흐름도의 표기법을 설명합니다.

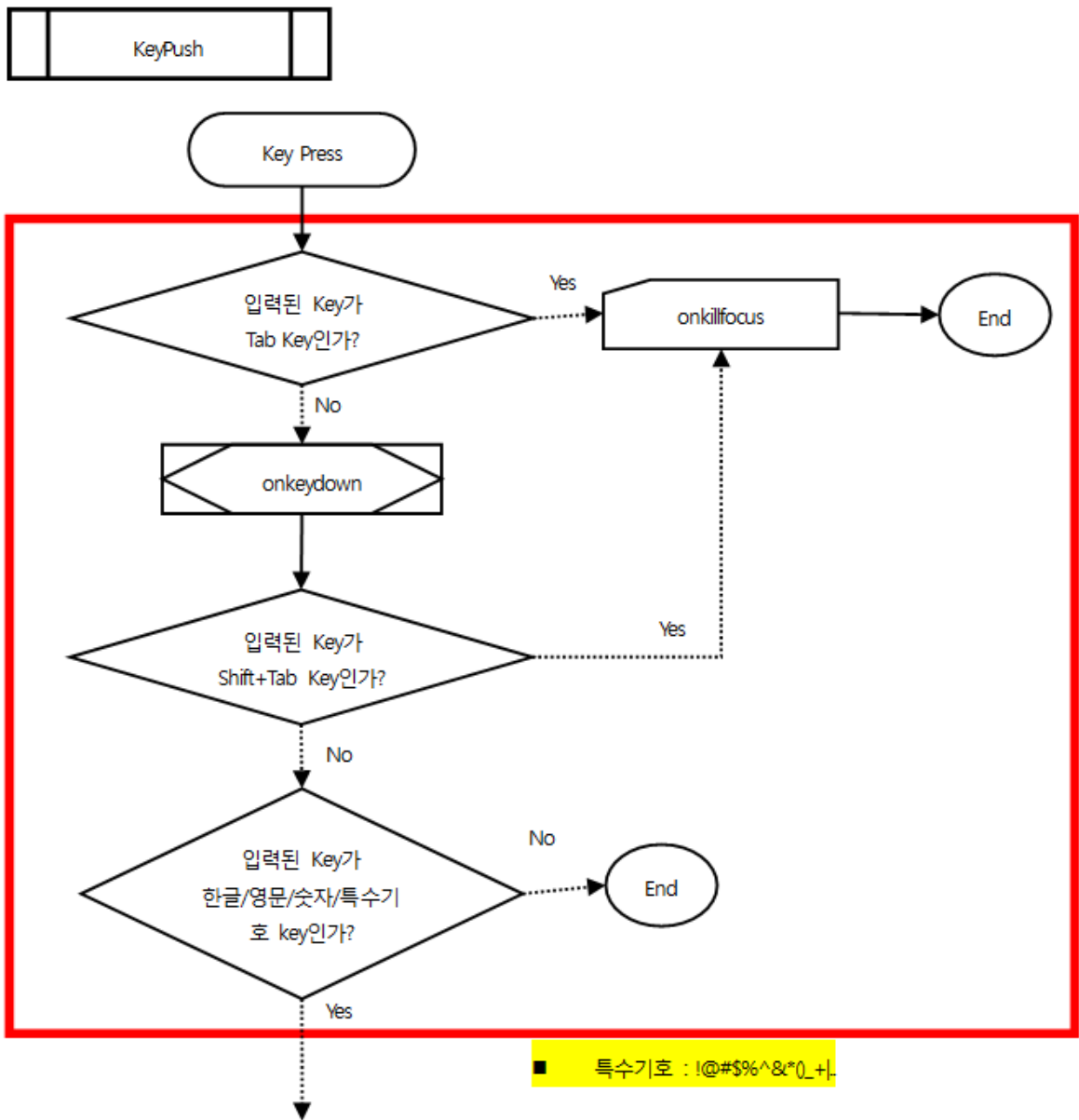
7.2.1 Event Flow에서 사용하는 도형

도형	정의
	사용자 입력
	Event
	Return value없는 event
	true return시, 상위 컴포넌트에서 해당 event가 발생하지 않는 Event default return값은 false이다.
	true return시, 상위 컴포넌트에서 해당 event가 발생하는 Event default return값은 true이다.
	조건/판단
	Return value
	Default return value
	조건 흐름
	이후 작업 없음

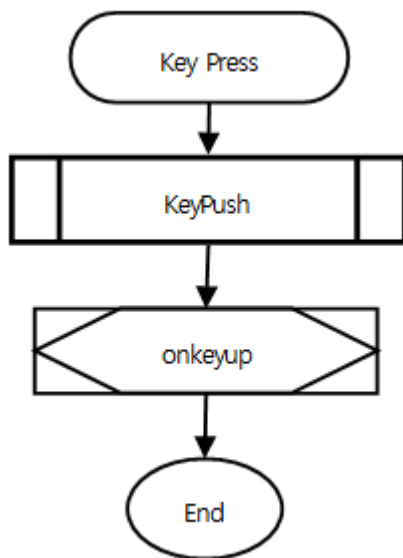
도형	정의
	Event Module

7.3 Key event Flow

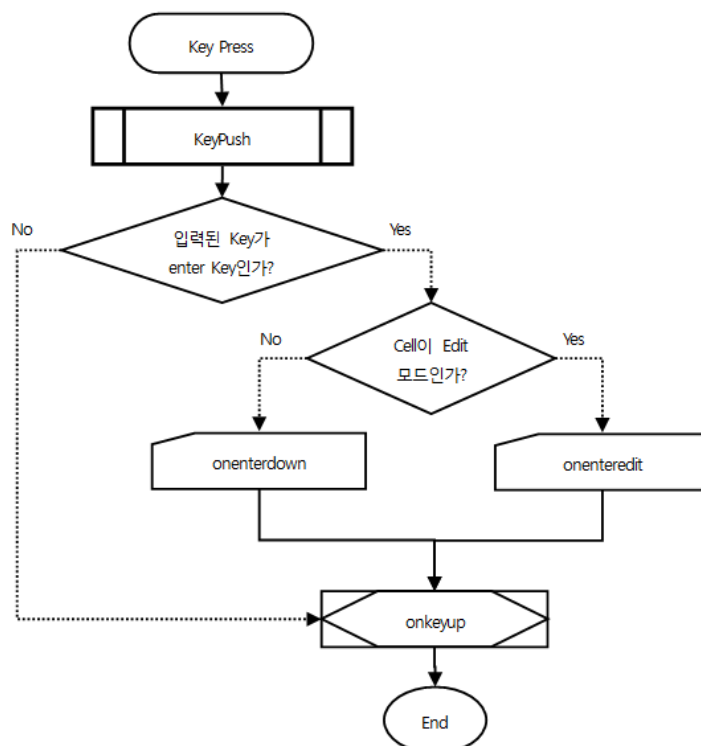
7.3.1 KeyPush module 정의



7.3.2 ApplicationMenu,Button,Calendar,CheckBox,Combo,Div,Edit,Form,ImageViewer,ListBox,MaskEdit,Menu,PopupDiv,PopupMenu,Radio,Spin,Tab,TabPage,TextArea

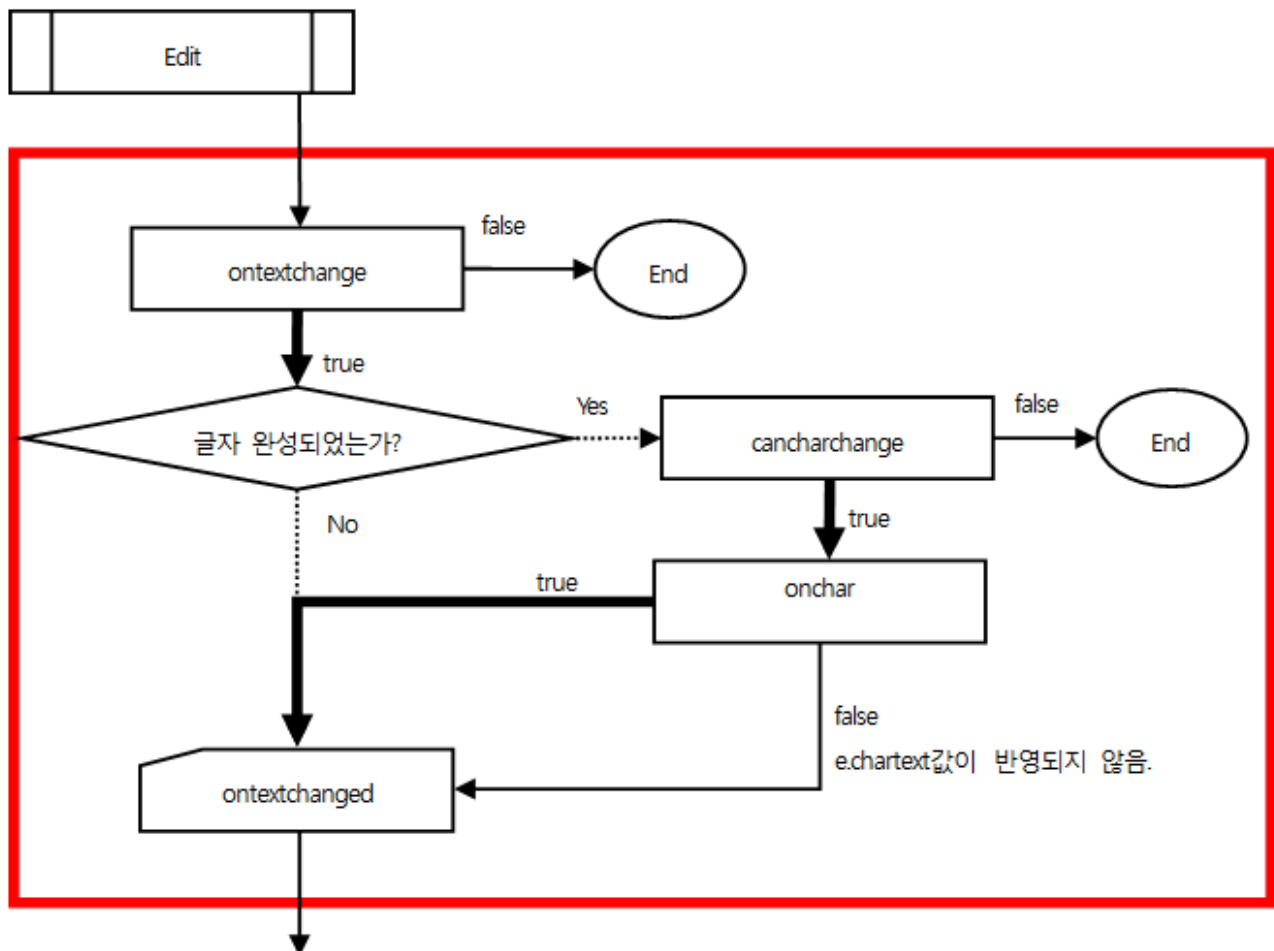


7.3.3 Grid



7.4 Edit event Flow

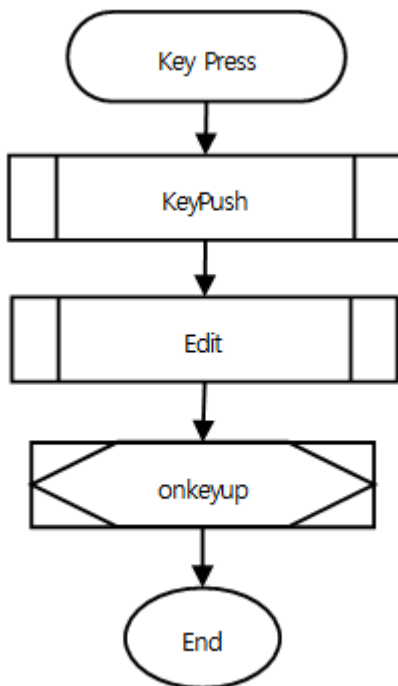
7.4.1 Edit module 정의



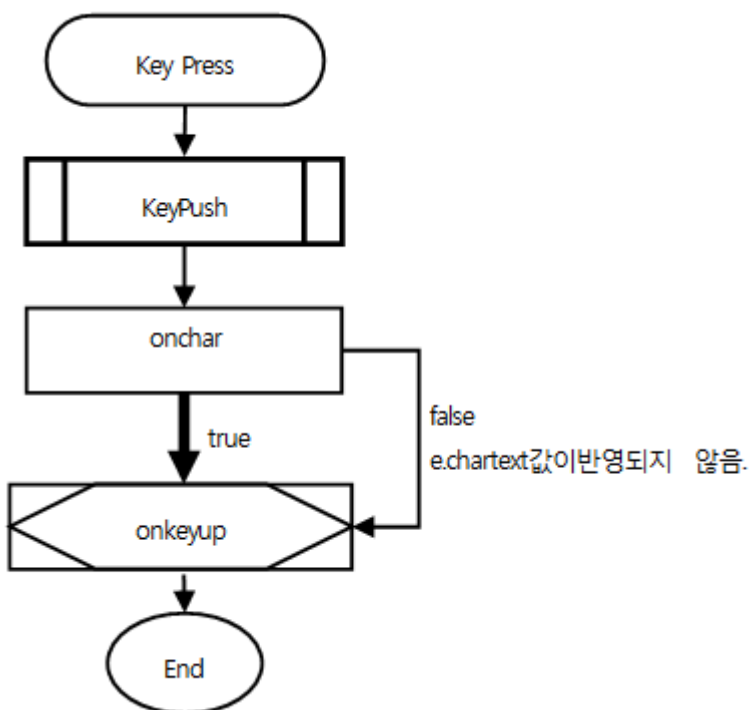
Edit기능이 있는 컴포넌트

Edit / MaskEdit / TextArea / Combo / Calendar / Spin
Grid

7.4.2 Edit, MaskEdit, TextArea, Combo, Calendar, Spin

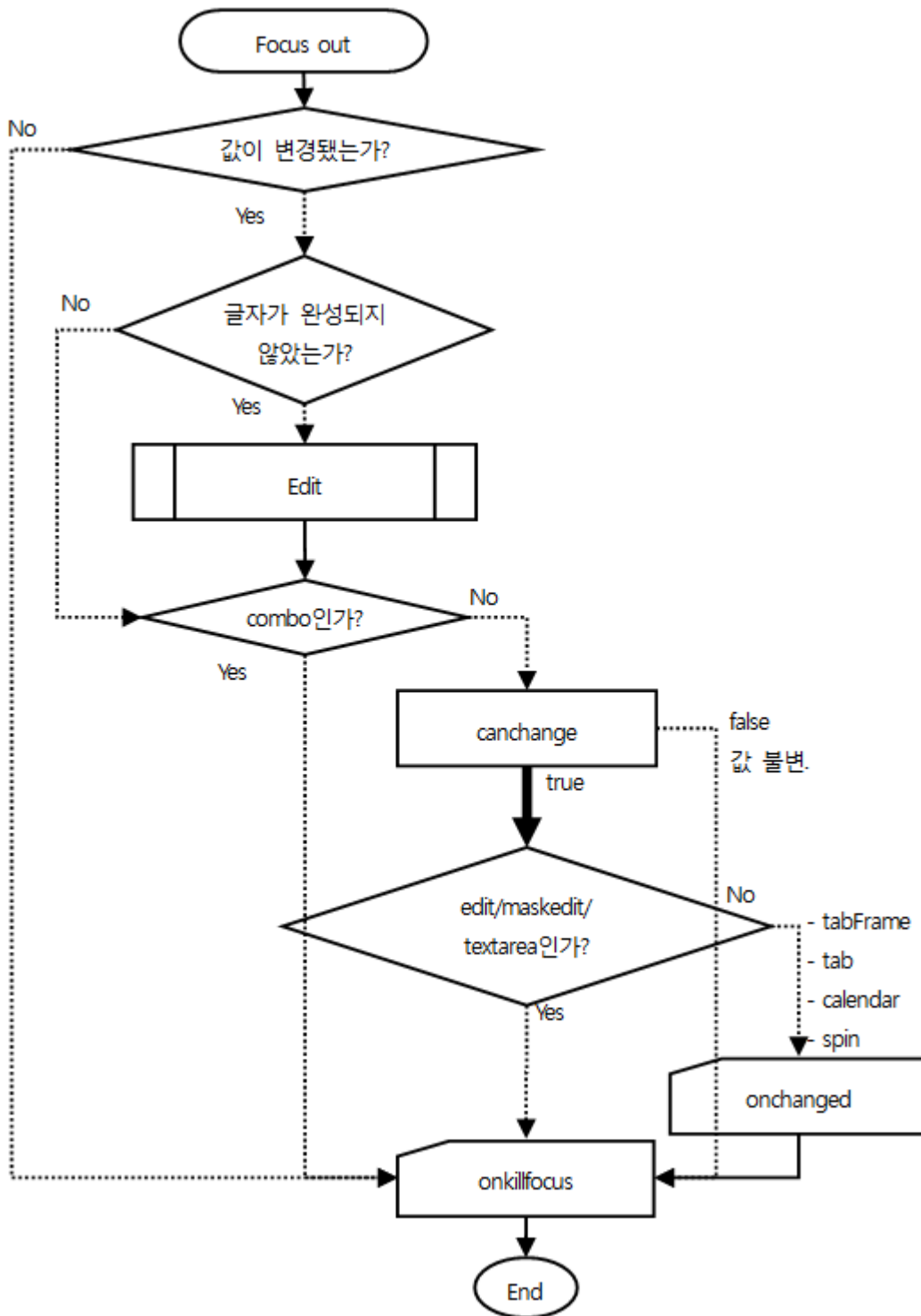


7.4.3 Grid



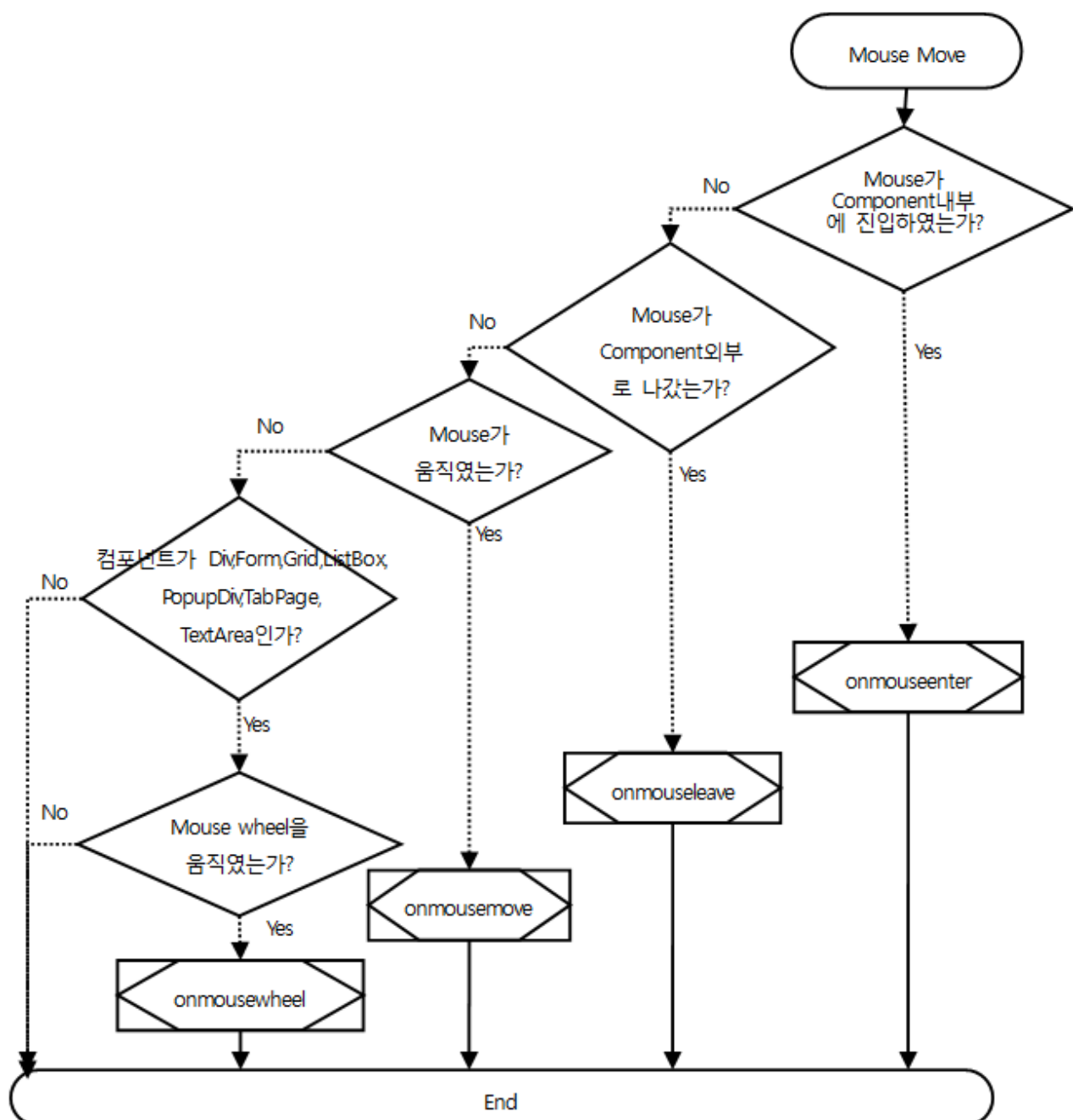
7.5 Edit중 Focus Out Flow

7.5.1 Calendar/Combo/Spin, Edit/MaskEdit/TextArea



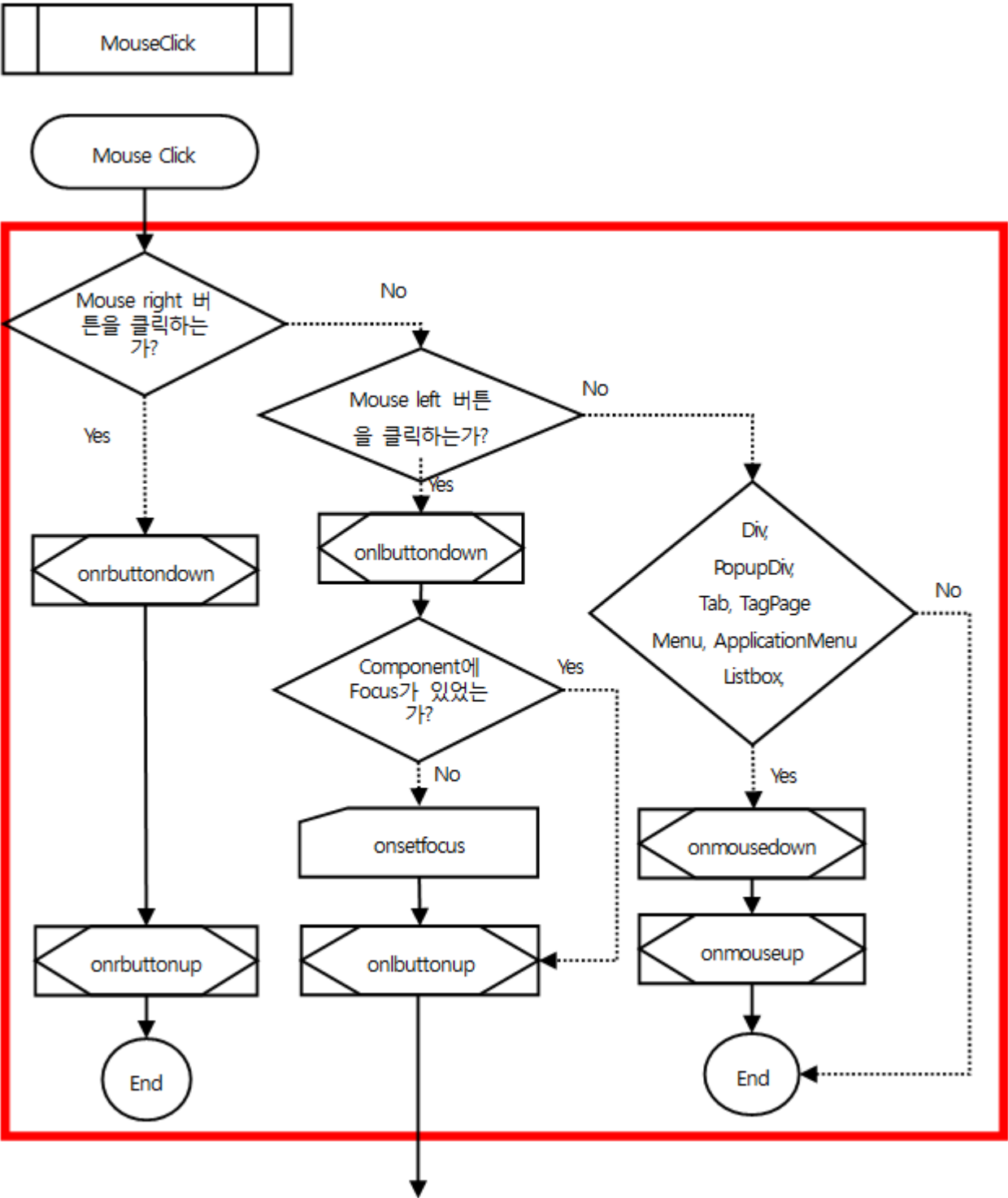
7.6 Mouse Move event Flow

7.6.1 Edit/MaskEdit/TextArea, Calendar/Combo/Spin, Tab/TabPage, ListBox/Radio, PopupMenu/ Menu, Button, CheckBox, GraphicPath, GroupBox, ImageViewer, Panel, ProgressBar, Shape, Splitter, Static, Div, Form, Grid, PopupDiv

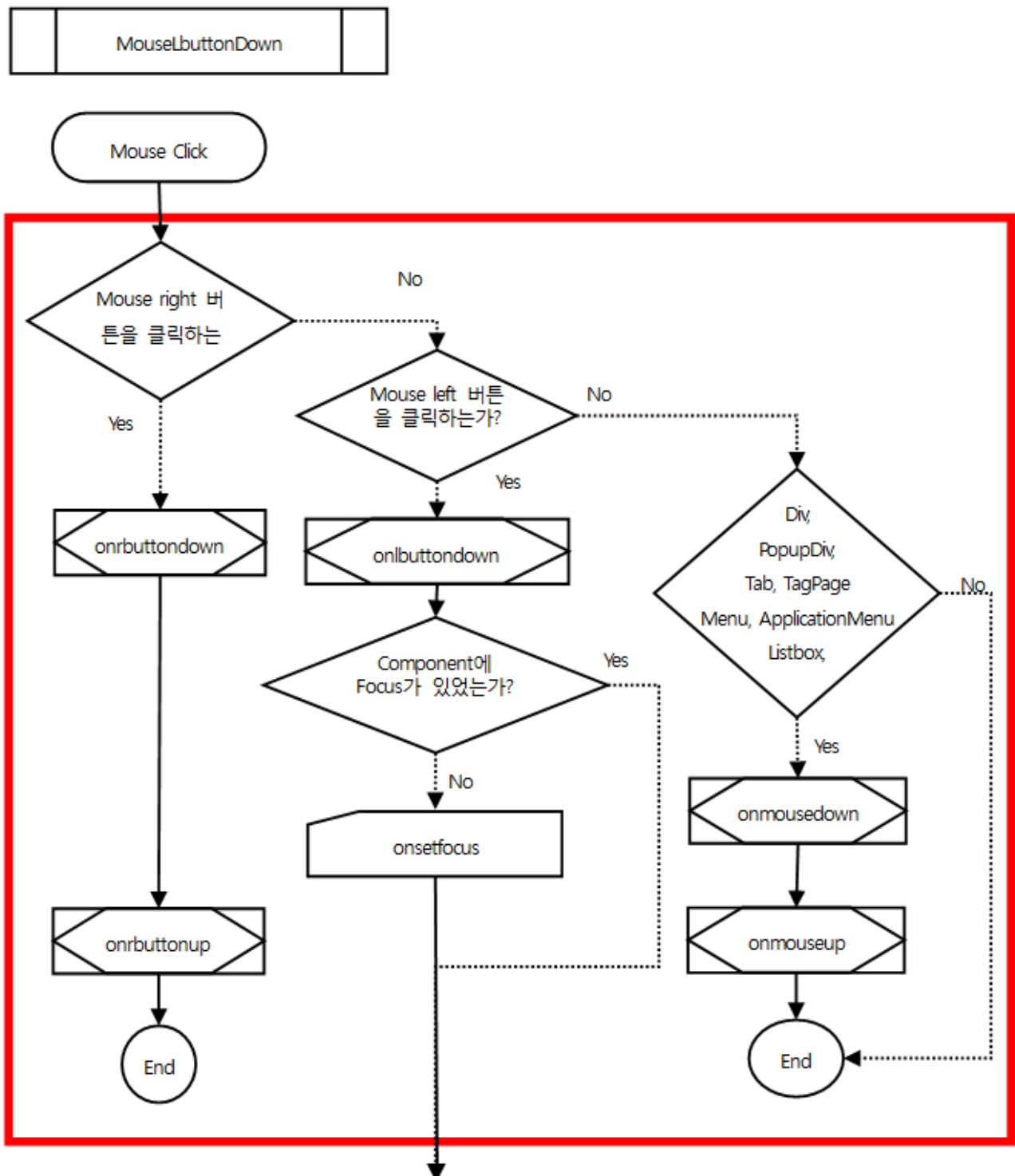


7.7 Mouse Click event Flow

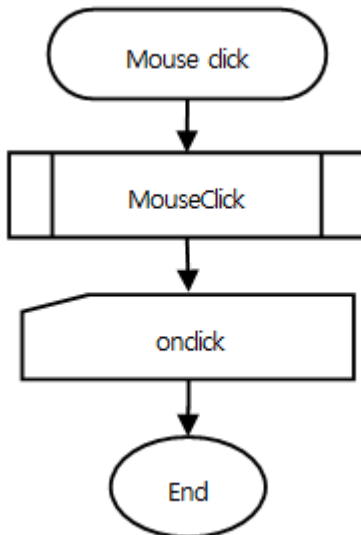
7.7.1 MouseClick Module



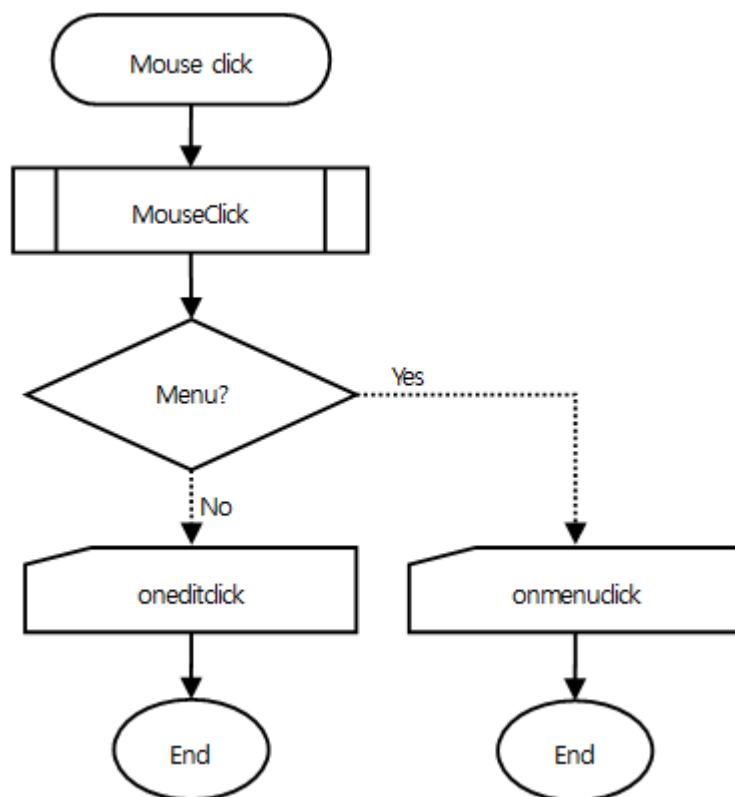
7.7.2 MouseButtonDown Module



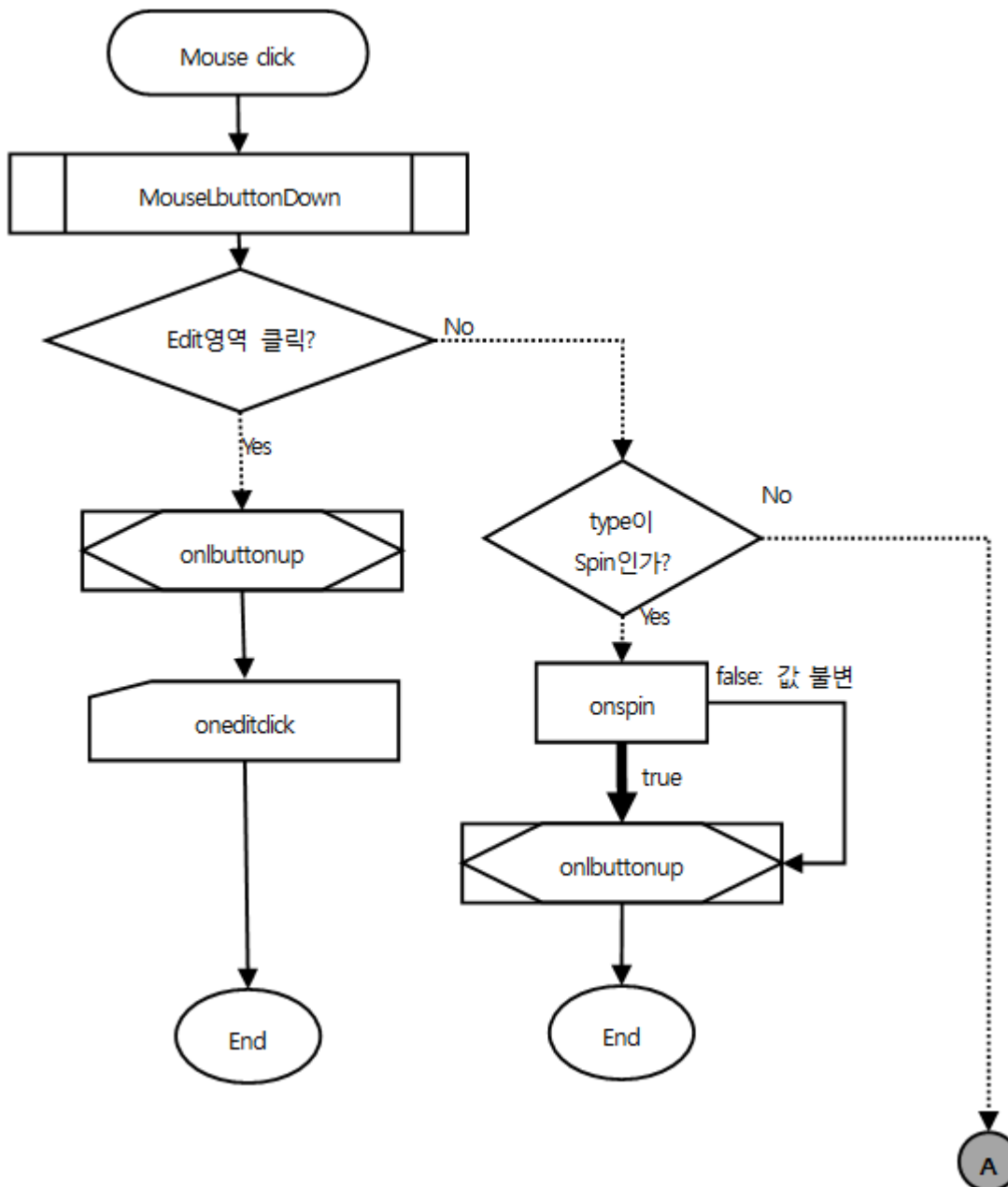
7.7.3 Button,CheckBox,Form,GraphicPath,ImageViewer,Panel,PopupDiv,ProgressBar,Shape,Static, Div/TabPage

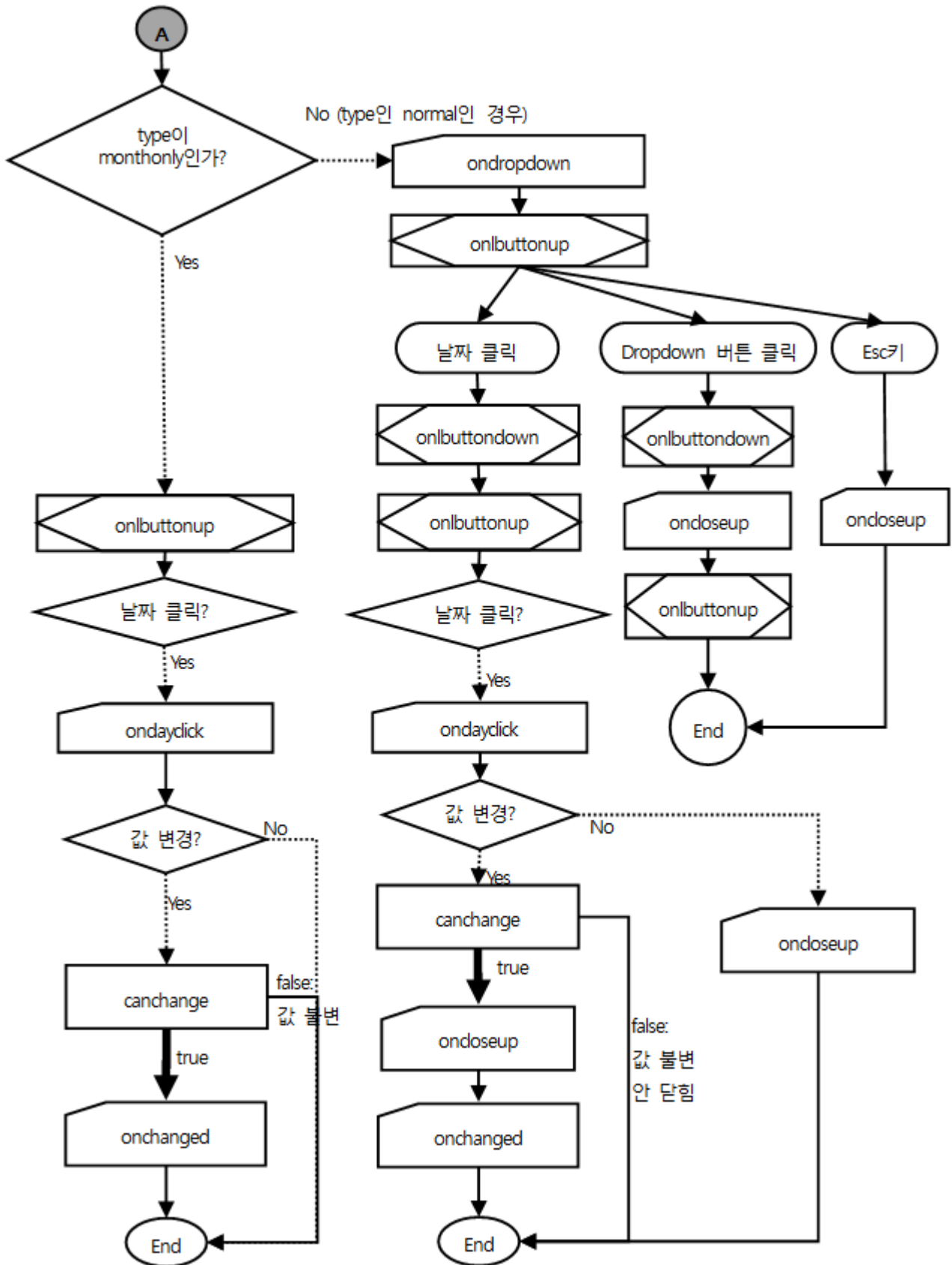


7.7.4 Edit/MaskEdit/TextArea, Menu

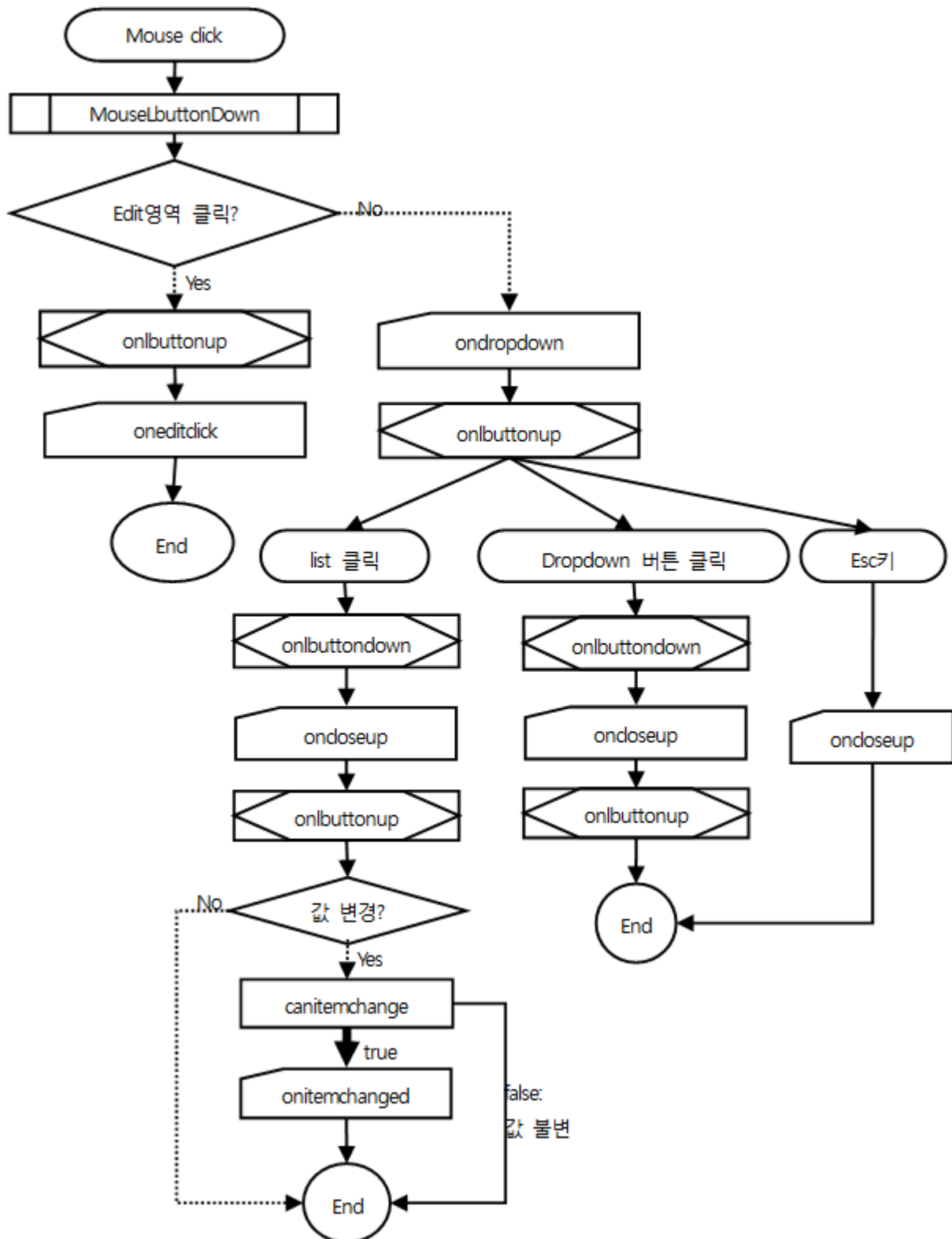


7.7.5 Calendar

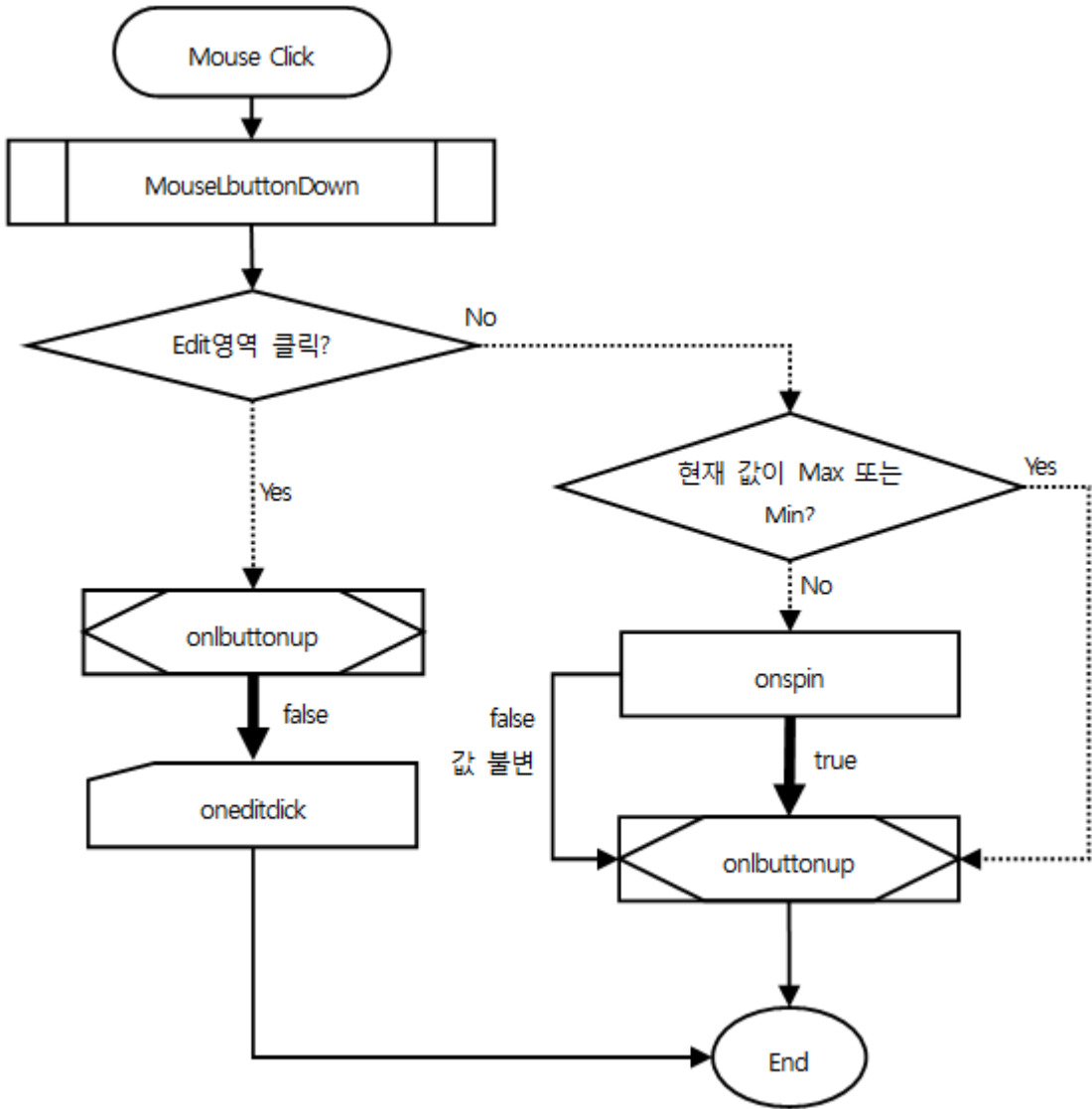




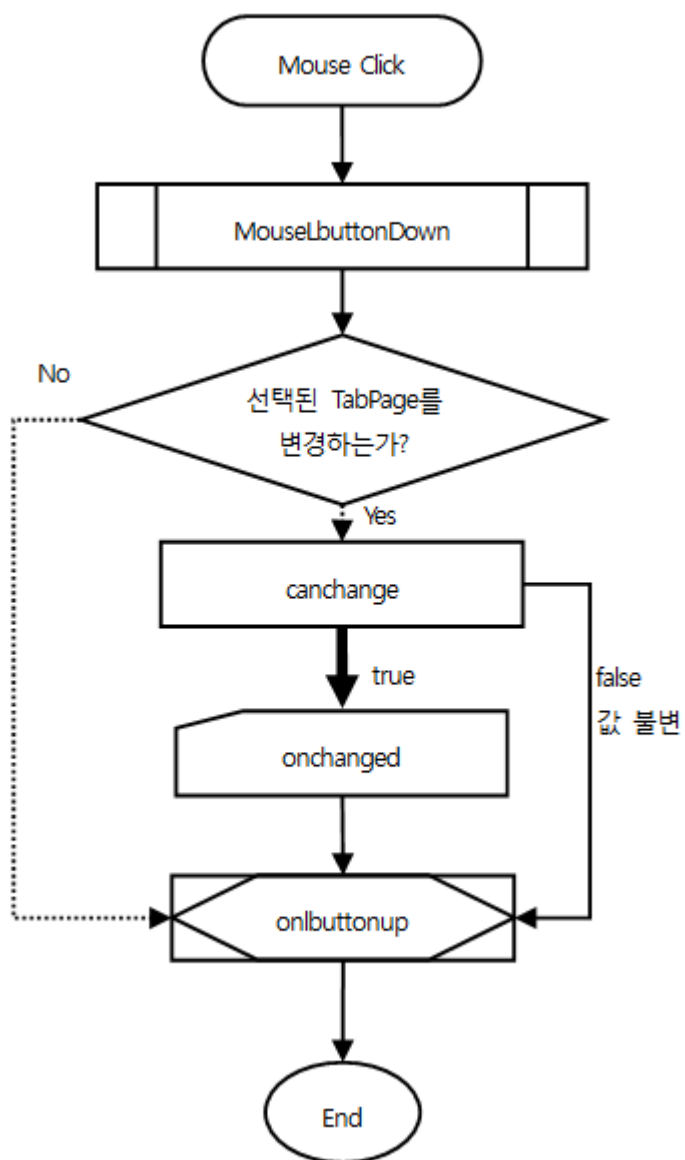
7.7.6 Combo



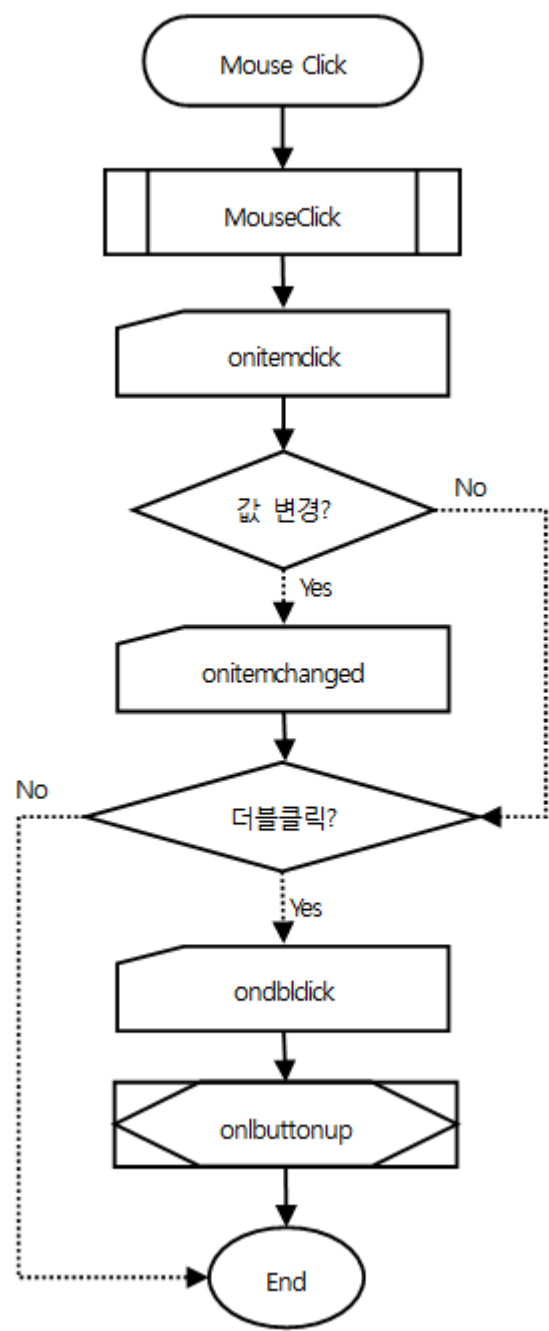
7.7.7 Spin



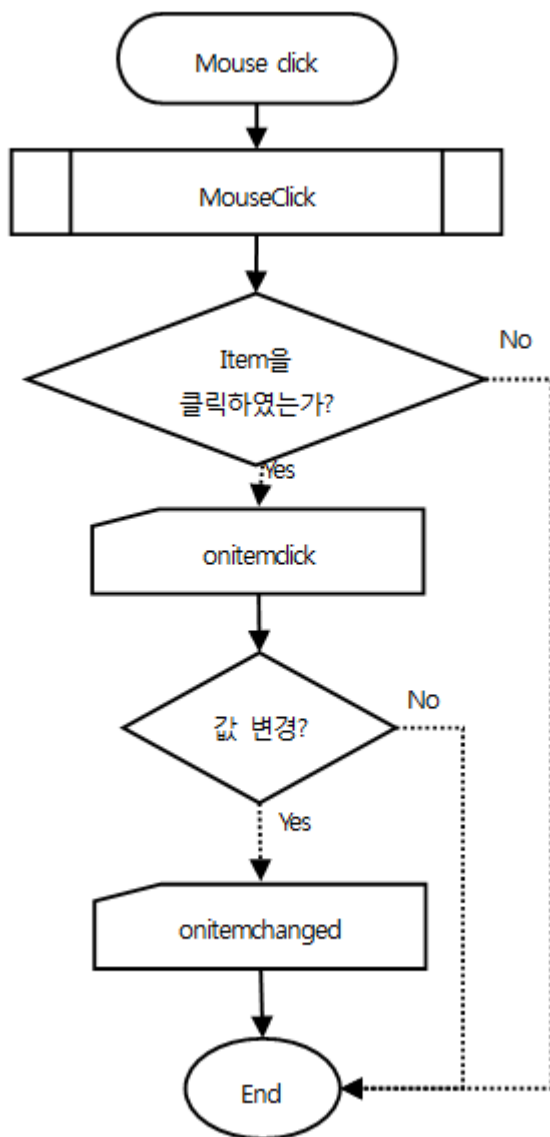
7.7.8 Tab



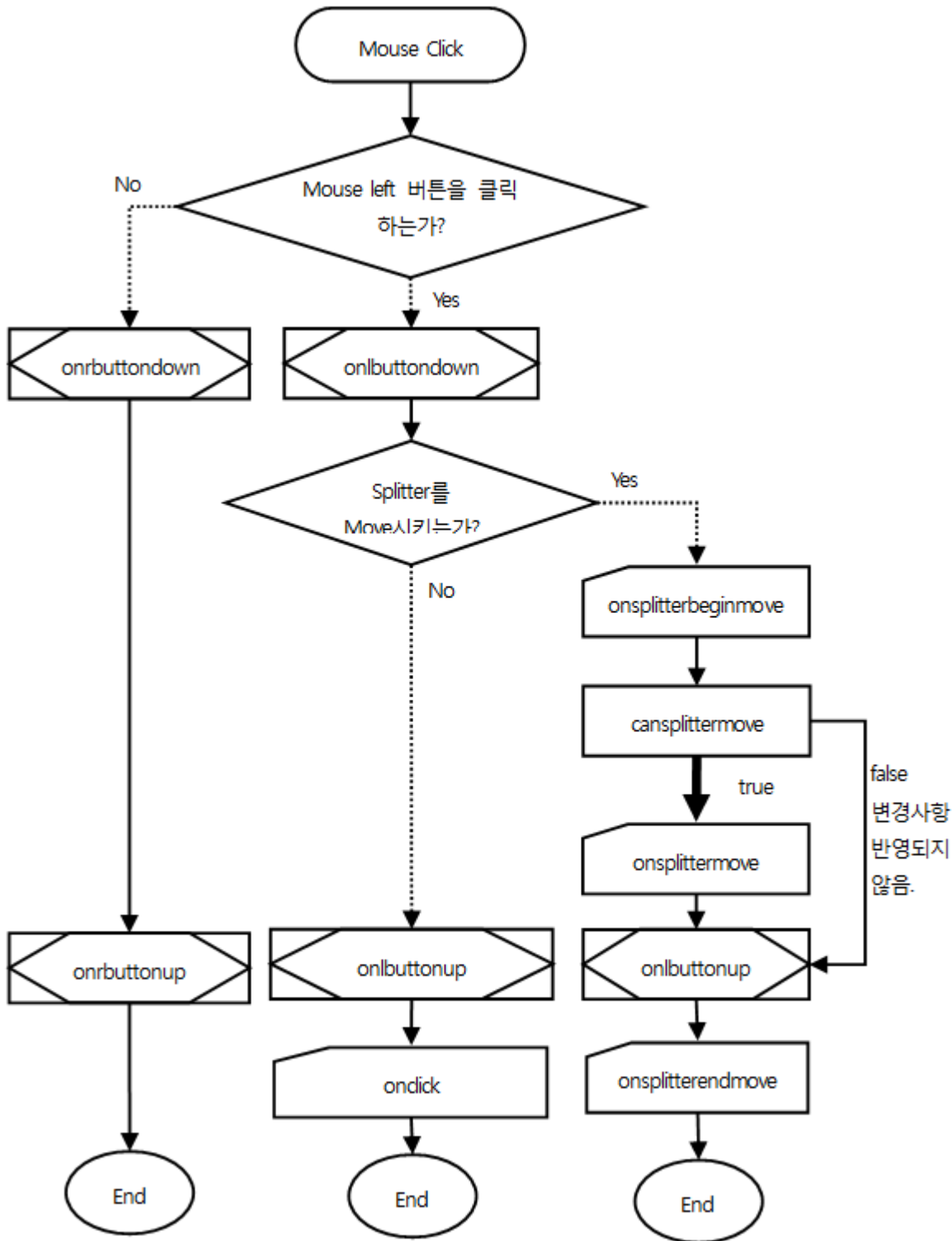
7.7.9 ListBox



7.7.10 Radio

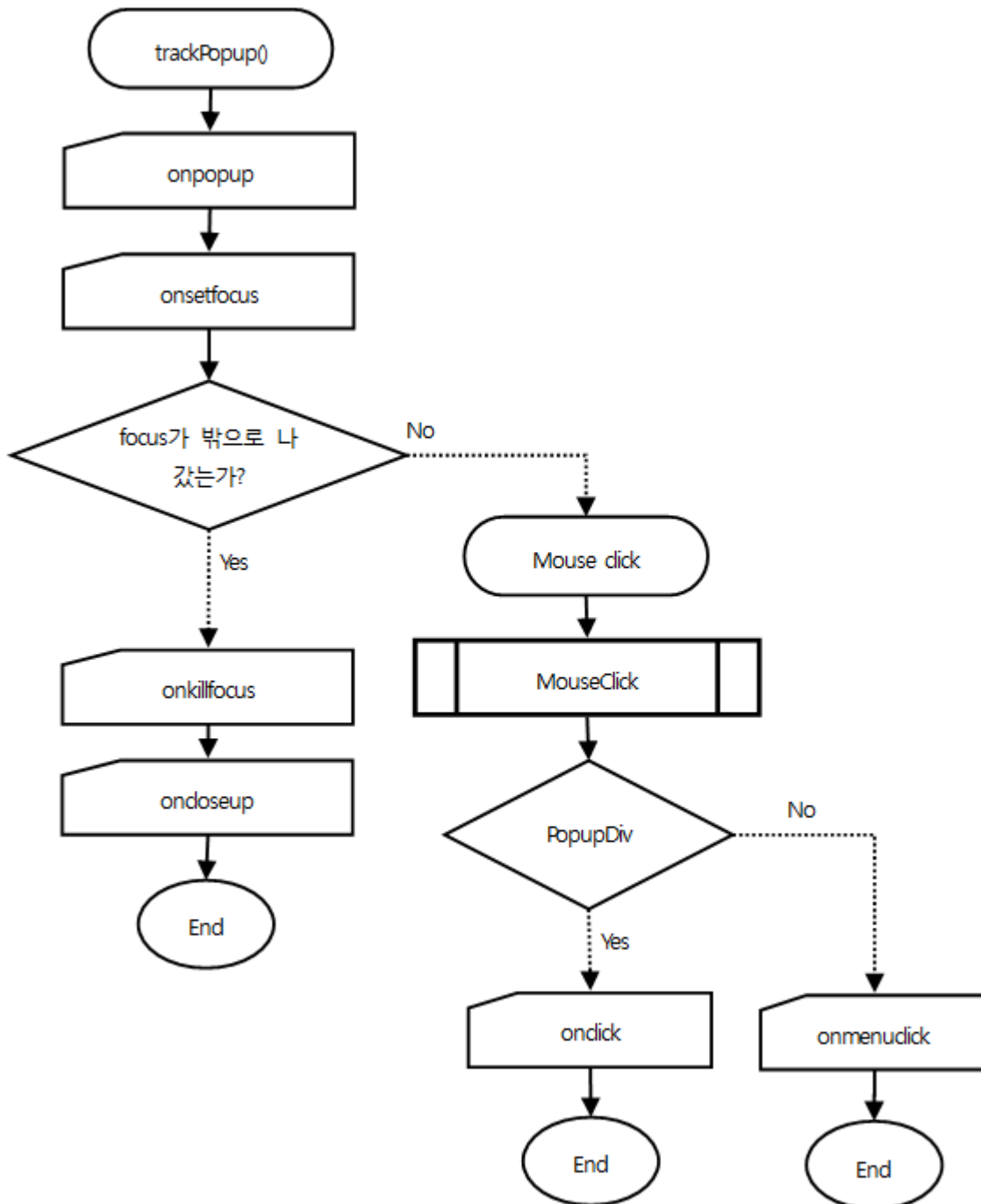


7.7.11 Splitter

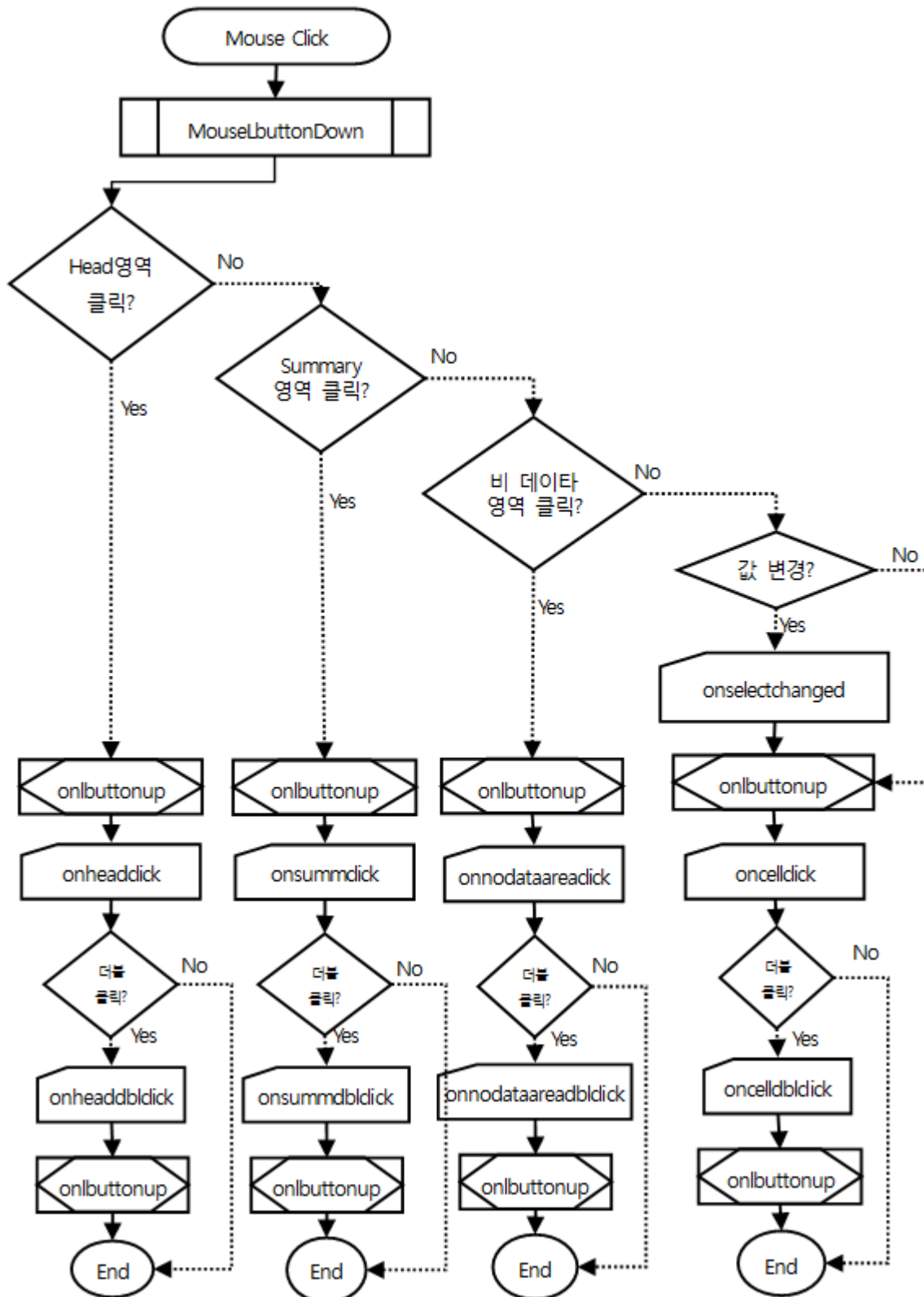


7.7.12 PopupDiv / PopupMenu

Focus가 밖으로 나가면 자동 close되는 component들

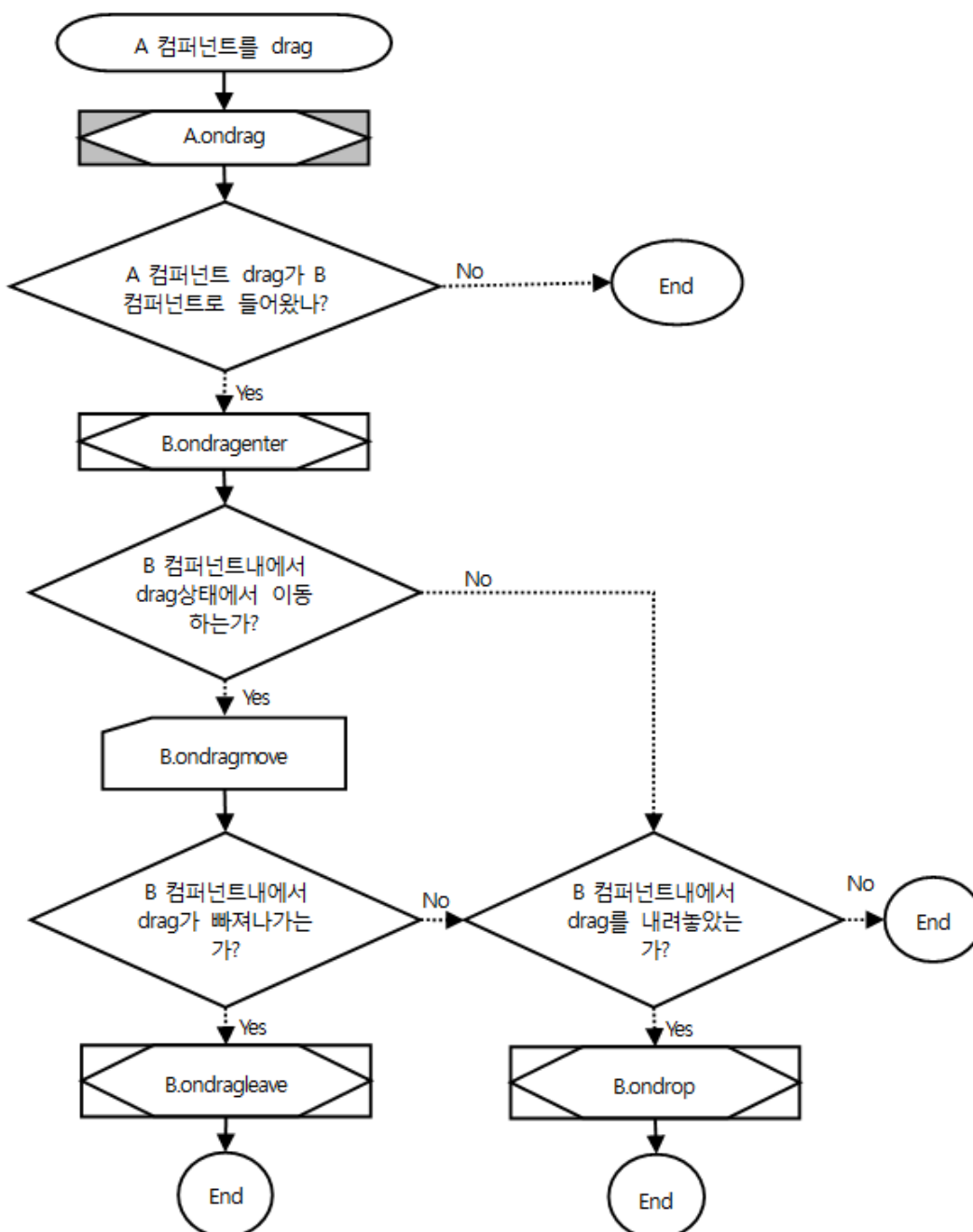


7.7.13 Grid



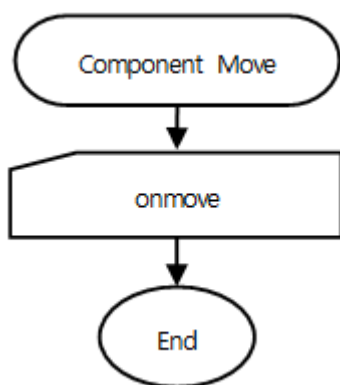
7.8 Drag event Flow

7.8.1 ApplicationMenu,Button,Calendar,CheckBox,Combo,Div,Edit,Form,Grid,ImageViewer,ListBox,MaskEdit,Menu,Pop upDiv,PopupMenu,ProgressBar,Radio,Sign,Spin,Tab,TabPage,TextArea



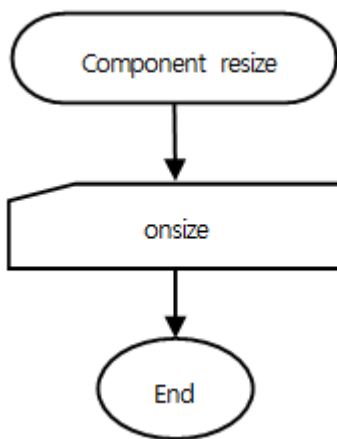
7.9 Component Move event Flow

7.9.1 ApplicationMenu,Button,Calendar,Chart,CheckBox,Combo,Div,Edit,Form,GraphicPath,Grid,GroupBox,ImageViewer,ListBox,MaskEdit,Menu,Panel,PopupDiv,PopupMenu,ProgressBar,Radio,ScrollBar,Shape,Sign,Spin,Splitter,Static,Tab,TabPage,TextArea

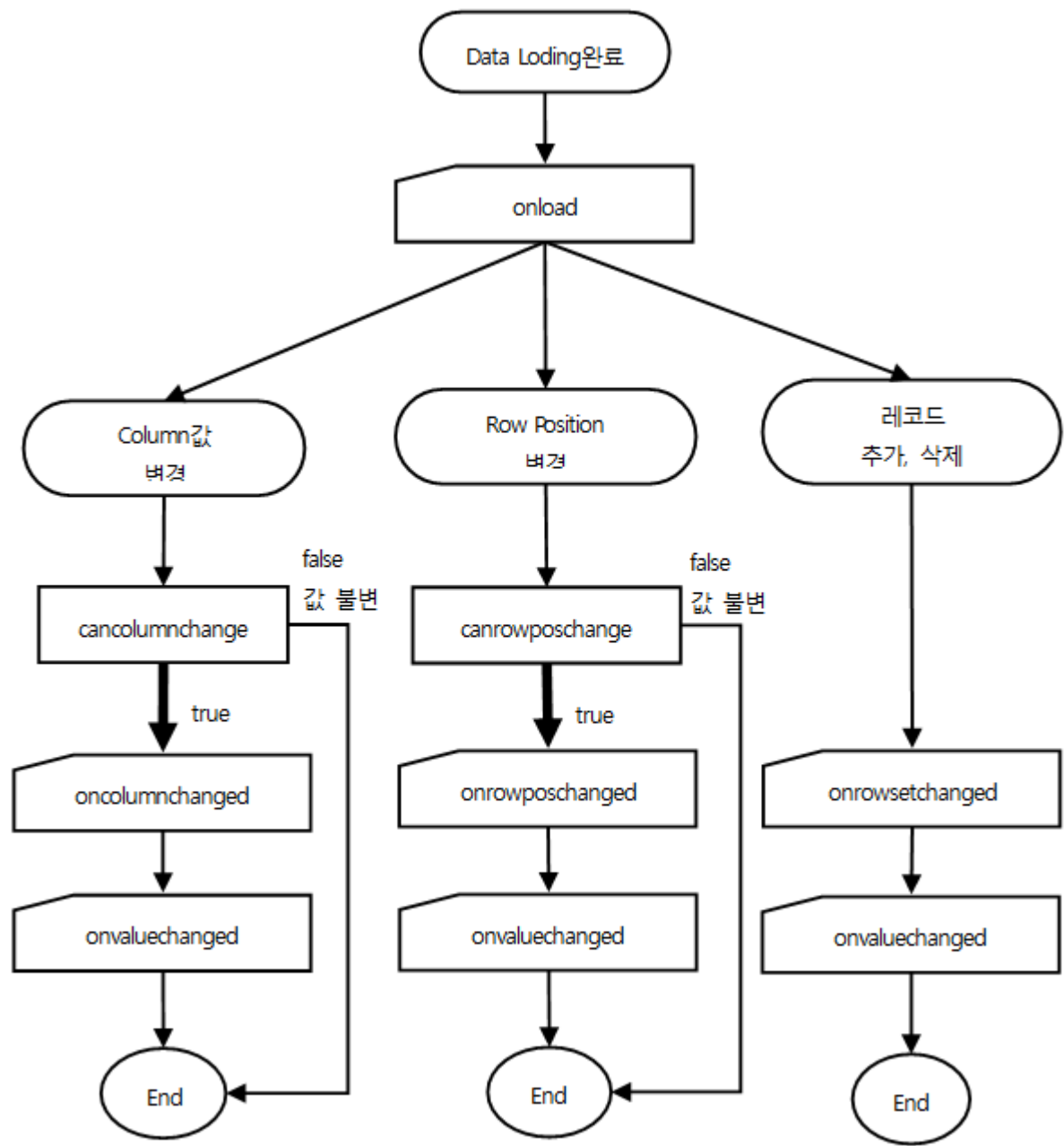


7.10 Component Resize event Flow

7.10.1 ApplicationMenu,Button,Calendar,CheckBox,Combo,Div,Edit,Form,GraphicPath,Grid,GroupBox,ImageViewer,ListBox,MaskEdit,Menu,Panel,PopupDiv,PopupMenu,ProgressBar,Radio,ScrollBar,Shape,Sign,Spin,Splitter,Static,Tab,TabPage,TextArea

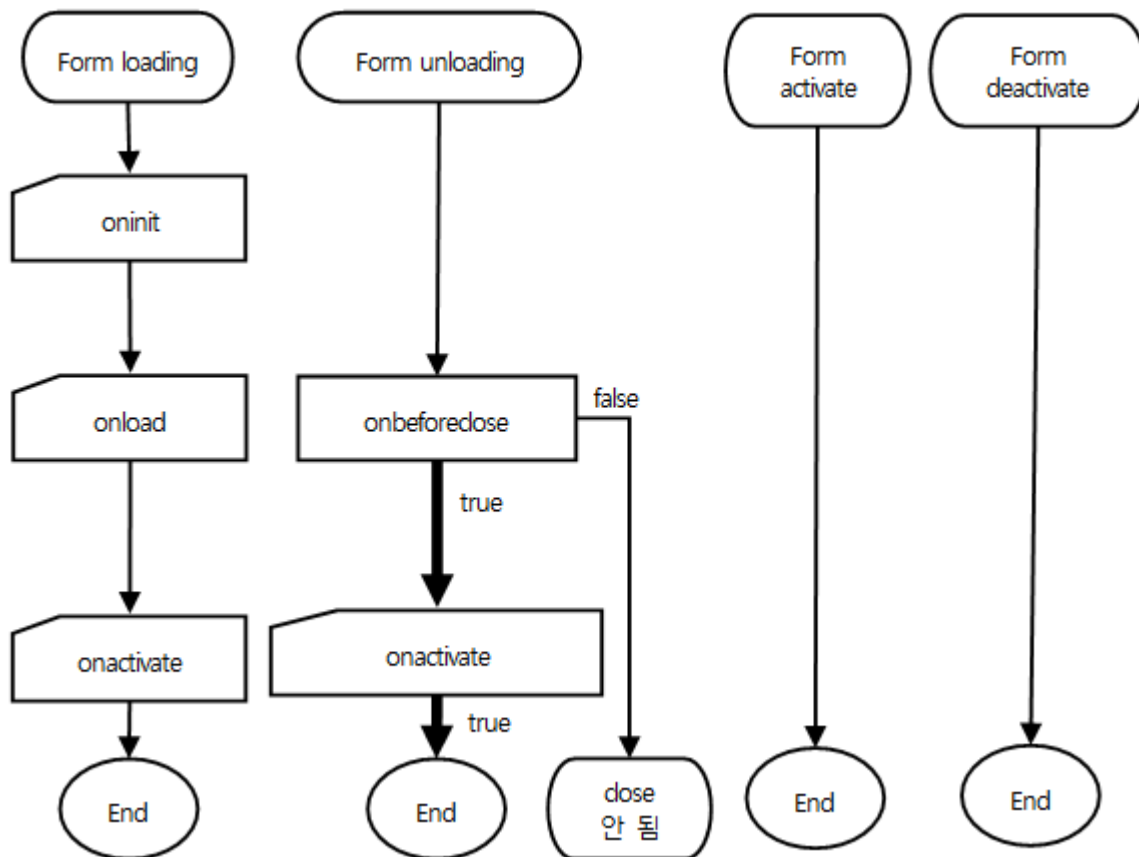


7.11 Dataset, Filtered Dataset Flow



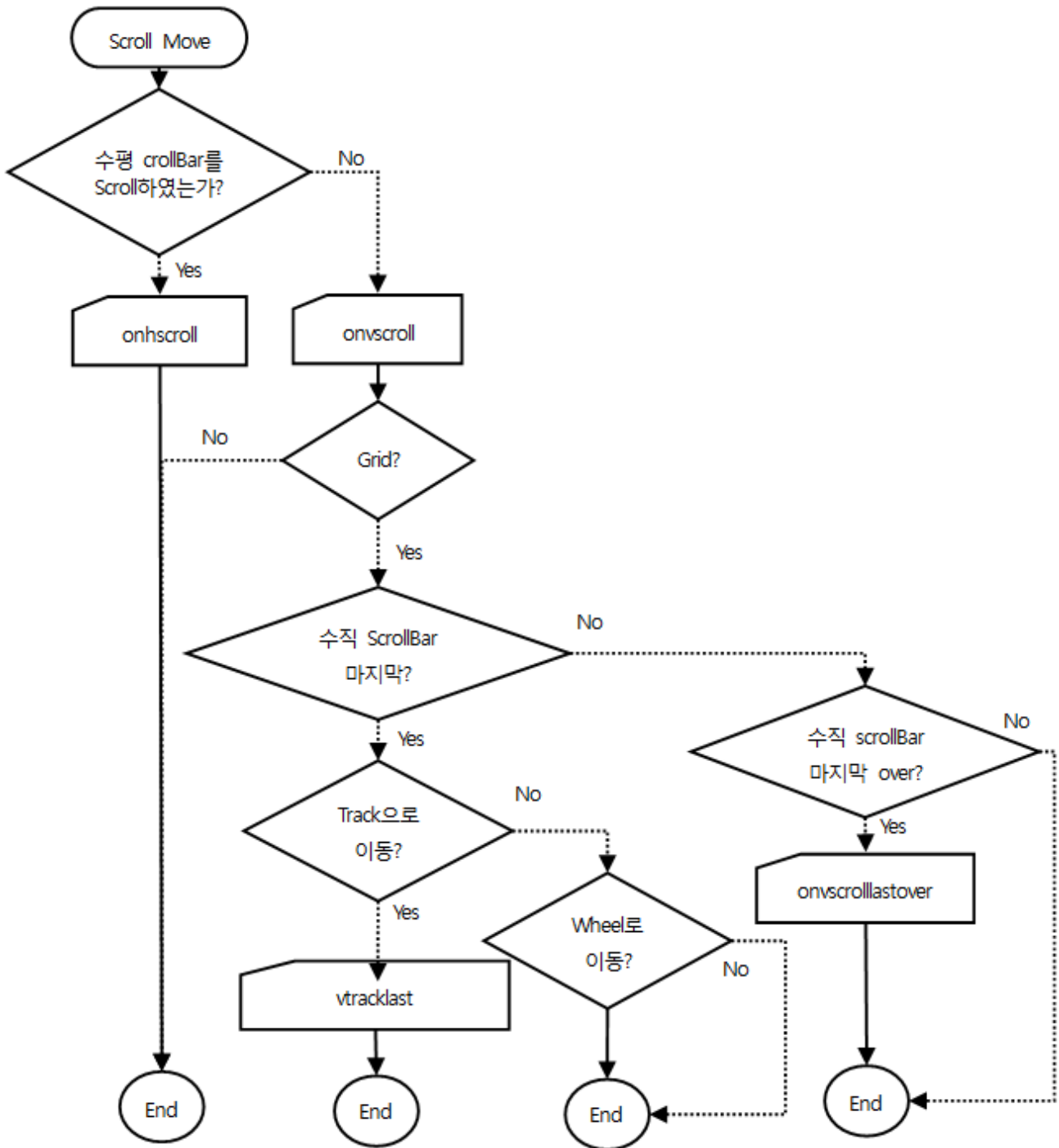
7.12 Form event Flow

7.12.1 Form load / unload



7.13 Scroll event Flow

7.13.1 Div,Form,Grid,ListBox,PopupDiv,TabPage,TextArea



8.

Customized Object

사용자가 원하는 component나 object를 만들어서 XLATFORM에서 사용할 수 있는 것을 Customized Object라고 합니다.

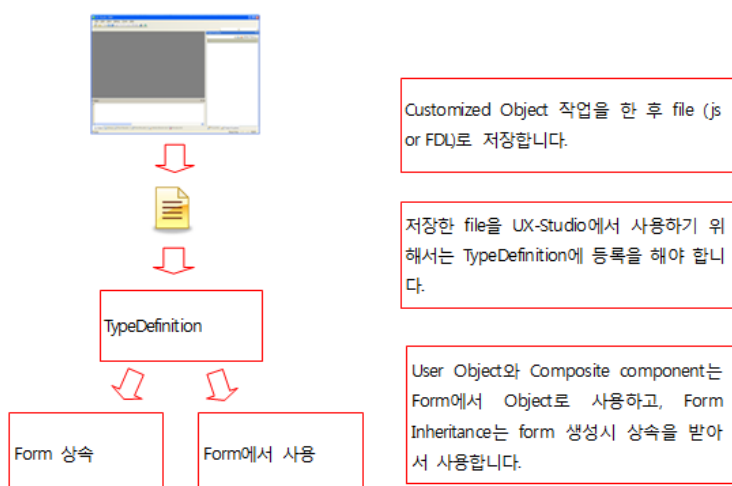
8.1 Customized Object 란?

Customized Object을 쉽게 설명하자면, XPLATFORM에서 제공하는 Binary Object를 이용해서 사용자가 원하는 기능을 사용자가 직접 개발을 해서, 기존 XPLATFORM에서 제공하는 Binary Object와 동일하게 사용 할 수는 것을 말합니다.

XPLATFORM에서 제공하는 Customized Object의 종류는 다음과 같습니다.

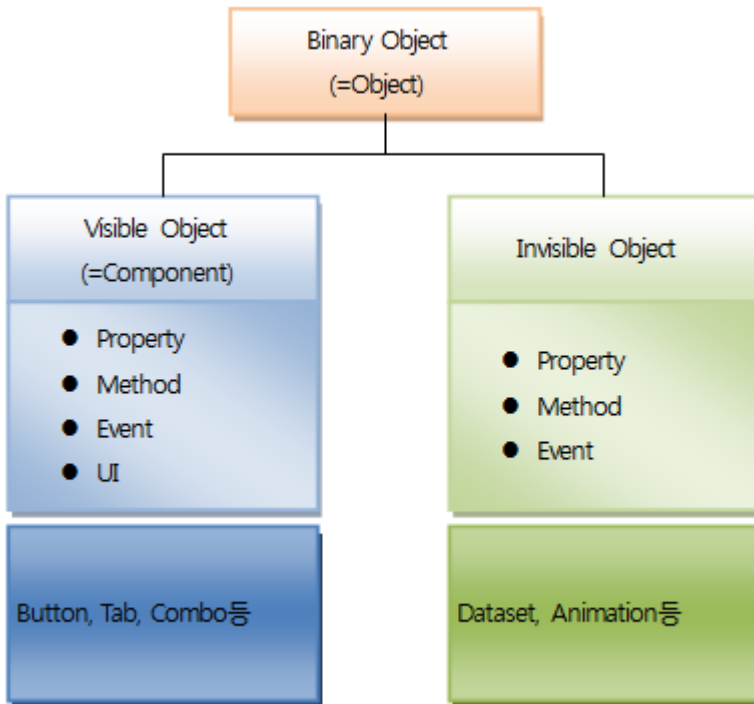
- User Object
- Composite 컴포넌트
- Form Inheritance

Customized Object을 XPLATFORM에서 개발하고 사용하는 것을 간략히 도식화 하면 다음과 같습니다. 즉, UX-Studio에서 직접 개발을 한 후 TypeDefinition에 등록해서 사용을 합니다.



8.1.1 Binary Object란?

Binary Object란 XPLATFORM에서 제공하는 모든 object입니다. 이 object는 다음과 같이 Visible Object와 Invisible Object로 구분을 합니다.



Visible Object를 XPLATFORM에서는 component라는 용어로 사용을 합니다. 그리고 object는 XPLATFORM에서 제공하는 모든 객체를 칭할 때 사용합니다.

Invisible Object는 Object중 visual 속성을 갖고 있지 않은 일부를 구분하기 위한 편의상의 명칭입니다. 통상적으로 object라고 하면 component를 포함한 모든 객체를 칭하는 것이고, 특별히 component와 구분을 해야 할 경우만 in visible object라고 사용합니다.

8.1.2 Customized Object와 Binary Object의 관계

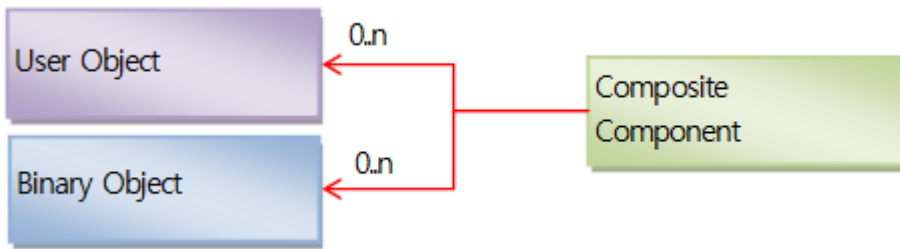
Customized Object와 Binary object의 관계를 간략히 설명하면 다음과 같습니다.

- User Object와 Binary object 관계



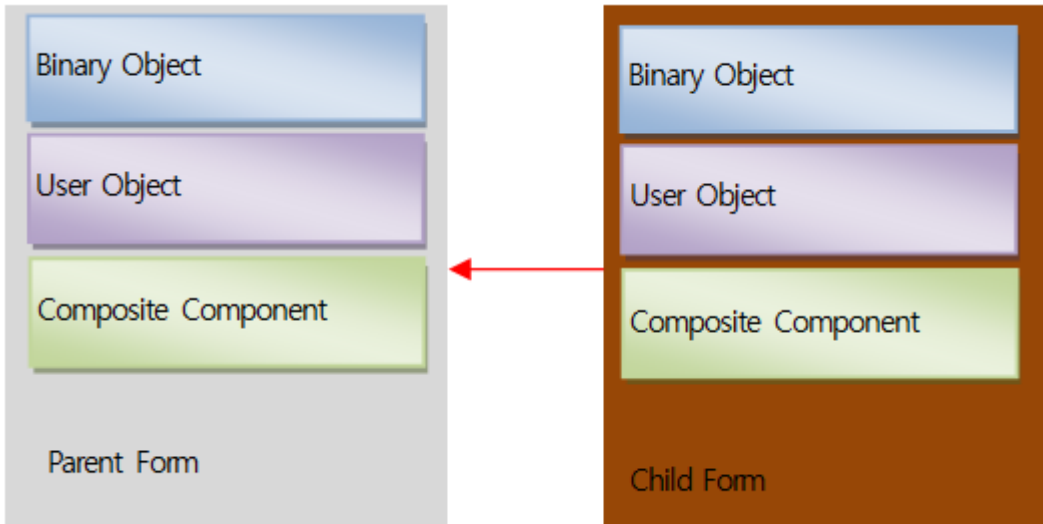
User Object는 Binary Object와 상속관계 입니다. 따라서 User Object는 Binary Object의 모든 기능을 기본적으로 가지고 있습니다. 물론 모든 Binary Object들을 상속 받을 수 있는 것은 아닙니다. Final로 선언해서 상속을 허용하지 않은 object를 제외한 모든 object은 상속을 받을 수 있습니다. (참고로 상속 가능한 object list는 UX-Studio의 wizard에서 제공을 합니다.) 그리고 XPLATFORM은 단일 상속만을 지원 합니다. 따라서 User Object도 단일 상속만 가능합니다.

- Composite 컴포넌트와 Binary object 관계



Composite 컴포넌트는 다수의 User Object들과 Binary Object들을 조합하여 만드는 Customized 컴포넌트입니다.

- Form Inheritance에서의 Binary Object



Form Inheritance는 Form을 상속 받아서 구성합니다. 상속 받을 수 있는 form에는 특별한 제약이 없습니다. 즉, Binary object, User Object, Composite 컴포넌트들로 구성된 form을 상속 받아서 Binary object, user object, composite component들을 추가해서 form을 구성할 수 있습니다.

8.1.3 Customized Object들의 비교

	User Object	Composite 컴포넌트	Form Inheritance
사용 대상	XPLATFORM에서 정의한 object, User Object, Composite 컴포넌트	TypeDefinition에 등록된 object중 type이 UserForm이 아닌 것	TypeDefinition에 등록된 object중 type이 UserForm인 것
TypeDefinition 등록 여부	O	O	O
Design 변경여부	O	X	X
Toolbar 등록가능 여부	O	O	X

- O는 가능, X는 불가능
- 사용 대상: User Object - object를 상속 받을 수 있는 대상
Composite 컴포넌트 - Composite 컴포넌트를 구성할 수 있는 object
Form Inheritance - form을 상속 받을 수 있는 대상
- TypeDefinition 등록 여부: TypeDefinition에 등록 여부
- Design 변경 여부: 대상을 design 시 변경 가능 여부 (자세한 내용은 각 개념을 참조하십시오.)
- Toolbar 등록가능 여부: UX-Studio Toolbar에 icon으로 등록 되는지 여부



멤버 변수와 new 연산자의 맞지 않는 사용 예와 해결책

new 연산자는 새로운 객체를 생성하고 이를 초기화 하기 위하여 생성자 함수를 호출합니다.

사용자 클래스에 선언된 멤버변수를 new 연산자로, 사용자 클래스의 멤버변수로서 객체 생성을 의도할 수 있습니다. 이때 new 연산자를 클래스 선언과 함께 사용하면, 의도와는 달리 해당 멤버변수가 global static으로 사용됩니다.

따라서 멤버변수를 선언하고, 이 멤버변수의 객체를 생성 할 때는, 사용자 클래스의 생성자 함수에서 해당 멤버 변수를 new가 되도록 사용해야 합니다.

8.2 Composite 컴포넌트

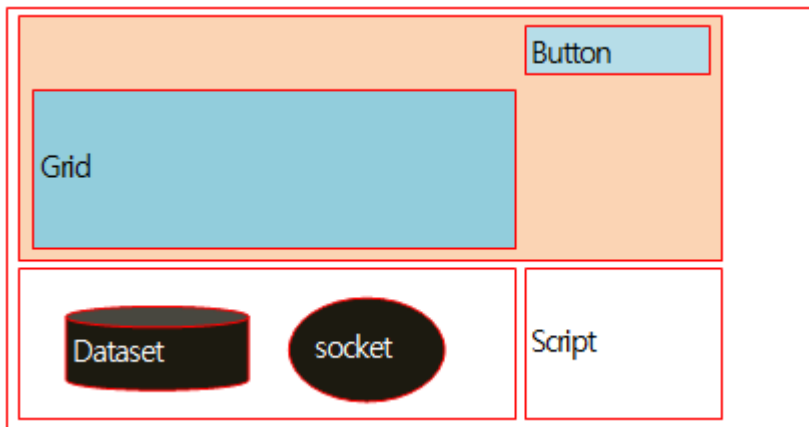
User Object는 다중 상속을 지원하지 않기 때문에 여러 Object들의 기능을 묶어서 원하는 Object를 만드는 것 자체가 불가능합니다. 이에 XPLATFORM에서는 여러 Object들의 기능을 묶어서 하나의 component로 만들 수 있는 기능을 제공하는 데, 그것이 바로 Composite 컴포넌트입니다.

8.2.1 Composite 컴포넌트 개념

Composite 컴포넌트는 XPLATFORM에서 사용 가능한 Object를 이용해서 새로운 component를 만드는 것입니다. Composite 컴포넌트가 가지고 있는 기본 개념은 다음과 같습니다.

- Form을 가지고 있습니다.

하나의 Form에 필요한 object들을 추가하여 Composite Component로 저장하는 방식을 사용합니다. 물론 UI를 배제한 invisible object(socket, dataset등)나 script로만 필요한 기능을 구현해서 Composite 컴포넌트를 만들 수도 있으나 반드시 Form위에 올려서 저장해야만 Composite 컴포넌트를 만들 수 있습니다.



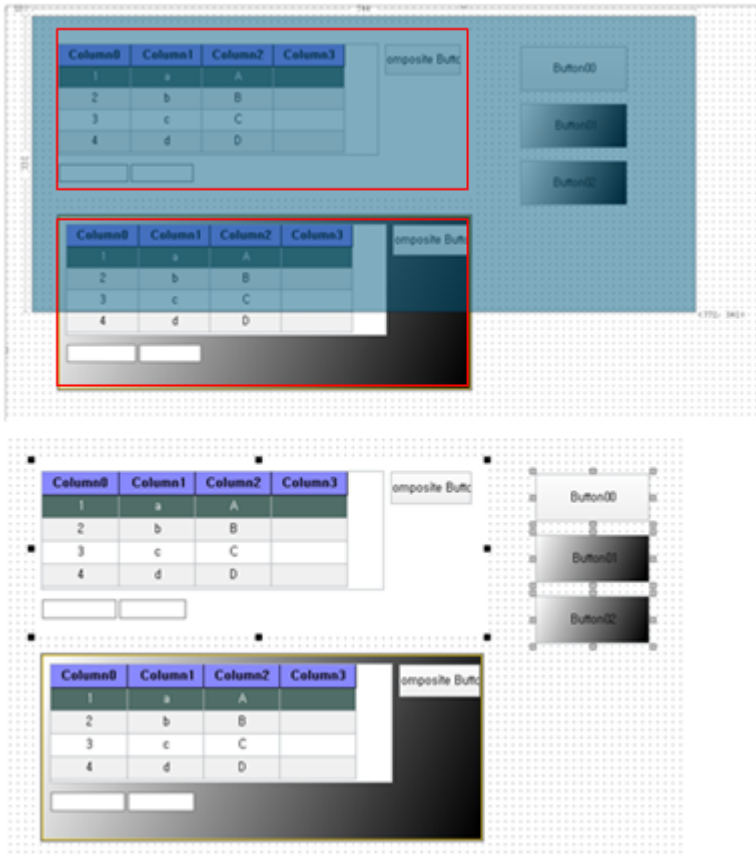
- UX-Studio Toolbar에 등록해서 사용합니다.

Composite 컴포넌트는 Binary component처럼 UX-Studio의 toolbar에 등록해서 사용합니다. 그러므로, 하나의 화면form에 여러 개의 Composite 컴포넌트가 올라 갈 수 있습니다. 각 component들은 ID로 구분해서 사용할 수 있습니다. 물론 Toolbar에 등록해서 사용하기 위해서는 TypeDefinition에 등록해야 사용이 가능합니다.

- Design 변경은 불가능 합니다.

Composite 컴포넌트를 등록한 후, Binary 컴포넌트처럼 사용할 때 Composite 컴포넌트 안의 object와 script는 수정할 수 없습니다. 즉 Design 시 Composite 컴포넌트안의 Button의 position을 변경한다거나 style Property값을 변경하는 것은 불가능합니다. 하지만, Runtime시 script로는 변경 가능합니다.

다음과 같이 form에 같은 Composite 컴포넌트를 2개 올린 다음 마우스를 drag한 경우 composite 컴포넌트의 내부 component들은 선택이 되지 않음을 알 수 있습니다. 즉, 내부 component들이 선택되지 않으므로 해당 property를 변경할 수 없게 되는 것입니다.



8.2.2 Composite 컴포넌트 파일 만들기

Composite 컴포넌트는 Form을 만드는 것과 동일하게 만들면 됩니다. 단, 만든 form을 TypeDefinition에 등록해야만 Composite 컴포넌트로 사용할 수 있습니다.

8.2.3 Composite 컴포넌트 TypeDefinition에 등록하기

개발자가 만든 Composite 컴포넌트를 XPLATFORM에서 Binary 컴포넌트처럼 사용하기 위해서는 TypeDefinition에 등록을 해야 합니다. 그 방법은 다음과 같습니다.

단계 1. Edit TypeDefinition windows 띄우기

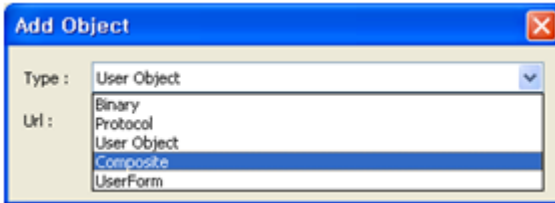
UX-Studio의 Project Explorer에서 TypeDefinition을 double click 하거나, 오른쪽 마우스 클릭 후 edit를 선택합니다.

단계 2. Add Object windows 띄우기

Edit TypeDefinition windows의 Objects에서 Add button을 클릭합니다.

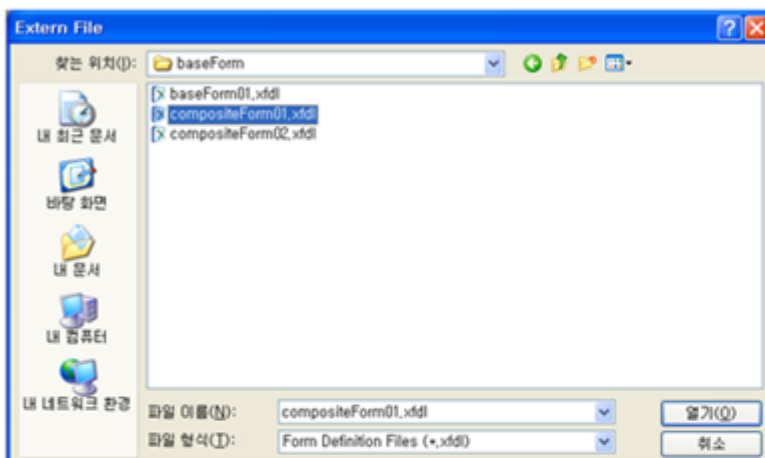
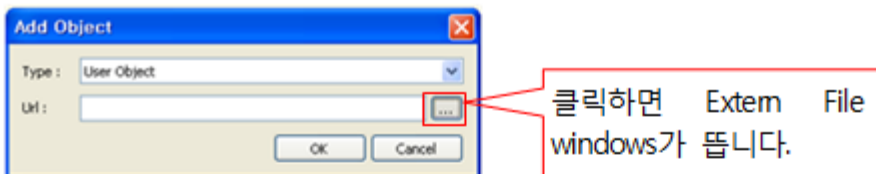
단계 3. Object Type 선택하기

Add Object windows에서 Type중에 Composite를 선택합니다.



단계 4. Composite 컴포넌트 파일 선택

Add Object windows에서 url 찾기 버튼을 통해서 Extern File windows를 띄워서 추가하려는 Composite 컴포넌트 file을 선택하고, OK 버튼을 누르면 Object에 추가 됩니다.



Extern File에서 선택 후 열기 버튼을 누르고, Add Object에서 OK 버튼을 누르면 Objects에 추가됩니다.

No	Type	ID	ClassName	Module	Version	Default W
28	Bin	Dataset	Dataset	XPlatformLib	1000	
29	Bin	FilteredDataset	FilteredDataset	XClassLib	1000	
30	Bin	FileDialog	FileDialog	XPlatformLib	1000	
31	Script	myButton	myButton	UC::myButton...	1000	
32	UserForm	baseForm01	baseForm01	baseForm::ba...	1000	
33	Composite	compositeFor...	compositeForm02	baseForm::co...	1000	
34	Script	usrCalendar	usrCalendar	UC::usrCalen...	1000	
35	Composite	compositeForm01	compositeForm01	baseForm::co...	1000	



UX-Studio의 Toolbar에 등록됩니다.

8.2.4 Composite 컴포넌트 사용하기

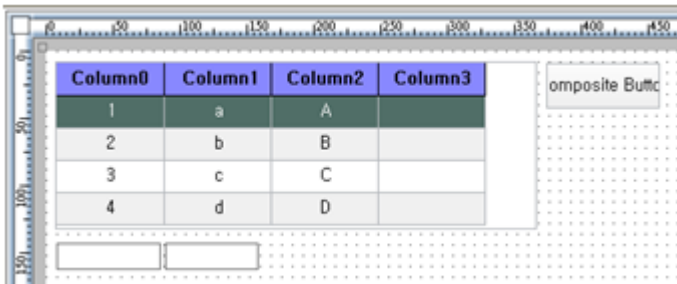
Form에서 Composite 컴포넌트는 binary Object와 User Object를 사용하는 것처럼 Composite 컴포넌트 id로 사용할 수 있습니다. 즉 Composite 컴포넌트 id가 compositeForm00이라고 했을 경우, script로 compositeForm00으로 접근을 해서 해당 Composite 컴포넌트에서 만든 함수, 변수를 사용할 수 있습니다. 또한 Composite 컴포넌트에 있는 Object들도 접근해서 사용할 수 있습니다.

다음은 Composite 컴포넌트의 id가 compositeForm00인 경우 사용하는 예들입니다.

```
// Composite 컴포넌트에서 선언한 변수 및 함수 접근
compositeForm00.myTrace(compositeForm00.strCompositeForm);
// Composite 컴포넌트안에 있는 Button에 onclick event를 발생
compositeForm00.Button00.onclick.fireEvent(obj, e);
// Composite 컴포넌트안에 있는 Button 속성 변경
compositeForm00.Button00.text = "oh~";
compositeForm00.Button00.style.gradation = "linear 0,0 white 100,100 black";
// Composite 컴포넌트안에 있는 Button move method 실행
compositeForm00.Button00.move(374, 20);
```

8.2.5 Composite 컴포넌트 예제

Dataset에 bind한 Grid와 Edit 2로 화면을 다음과 같이 구성했습니다.



Script는 다음과 같이 구현했습니다.

```
var strCompositeForm = "CompositeForm";

function Button00_onclick(obj:Button, e:ClickEventInfo)
{
    alert(Dataset00.rowcount);
    myTrace("Button00_onclick");
}
```



```

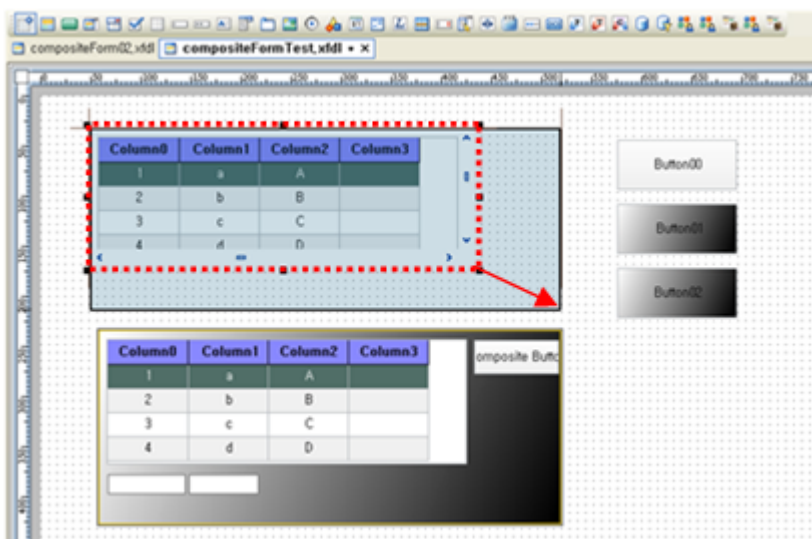
}

function myTrace(val)
{
    trace("my trace : " + val);
}

```

즉, form에 변수(strCompositeForm) 하나와 함수(myTrace), 그리고 button에 handler를 등록해서 해당 handler를 구현했습니다.

만든 form을 TypeDefinition에 composite로 등록해서 form에 올린 다음 다음과 같이 디자인을 할 수 있습니다.



위 화면은 composite component를 2개 올려서 하나는 background에 gradation을 지정했습니다.

Script는 다음과 같이 만들었습니다.

```

function Button00_onclick(obj:Button, e:ClickEventInfo)
{
    compositeForm00.myTrace(compositeForm00.Button00.text);
    compositeForm00.Button00.onclick.fireEvent(obj, e);
    alert("^^");
}

function compositeForm00_onclick(obj:compositeForm02, e:ClickEventInfo)
{
    trace("compositeForm00_onclick");
    for (x in e)
        trace(x + " : " + e[x]);
}

```

```

    trace("object ");
    for (y in obj)
        trace(y + " : " + obj[y]);
}

function Button01_onclick(obj:Button, e:ClickEventInfo)
{
    compositeForm00.Button00.text = "oh~";
    compositeForm00.Button00.style.gradation = "linear 0,0 white 100,100 black";
}

function Button02_onclick(obj:Button, e:ClickEventInfo)
{
    compositeForm00.Button00.move(374, 20);
    trace(compositeForm00.strCompositeForm);
}

```

Script의 설명은 “10.2.4.장” Composite 컴포넌트 사용하기를 참조하십시오.

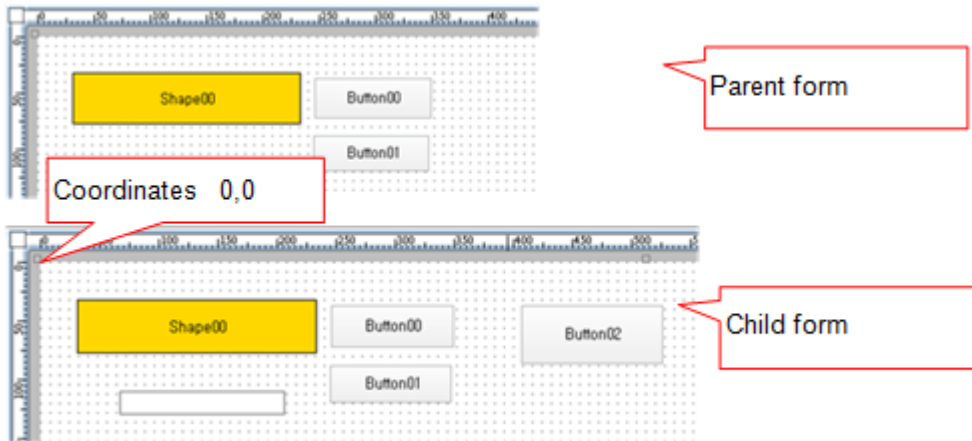
8.3 Form Inheritance

이미 만들어진 Form을 상속받아서 다른 Object들을 추가하여 새로운 Form을 만들 수 있는 것이 Form Inheritance 입니다.

8.3.1 Form Inheritance 개념

Form Inheritance 개념을 설명하기 위해서 상속 받을 form을 parent form이라고 하고, 상속 받은 form을 child form이라고 하고 설명을 하겠습니다.

1. Parent form은 child form의 좌표 0,0을 기준으로 그림니다.
Parent form에 UI가 있는 경우 parent form의 component들은 child form의 좌표0,0 (Top, left 기준)를 기준으로 그림니다.



Child form의 Source에는 parent form에 대한 정보는 다음과 같이 parent form명만 명시되어 있을 뿐 어떠한 정보도 가지고 있지 않습니다.

```
<?xml version="1.0" encoding="utf-8"?>
<FDL version="1.0">
  <TypeDefinition url="..\..\default_typedef.xml"/>
  <Form id="childForm03" classname="childForm03" inheritanceid="baseForm01"
    cachelevel="" position="absolute 0 0 1024 768" version=""
    titletext="New Form">
    <Layout>
      <Button id="Button02" position="absolute 407 37 527 87"
        taborder="0" text="Button02"/>
    </Layout>
  </Form>
</FDL>
```

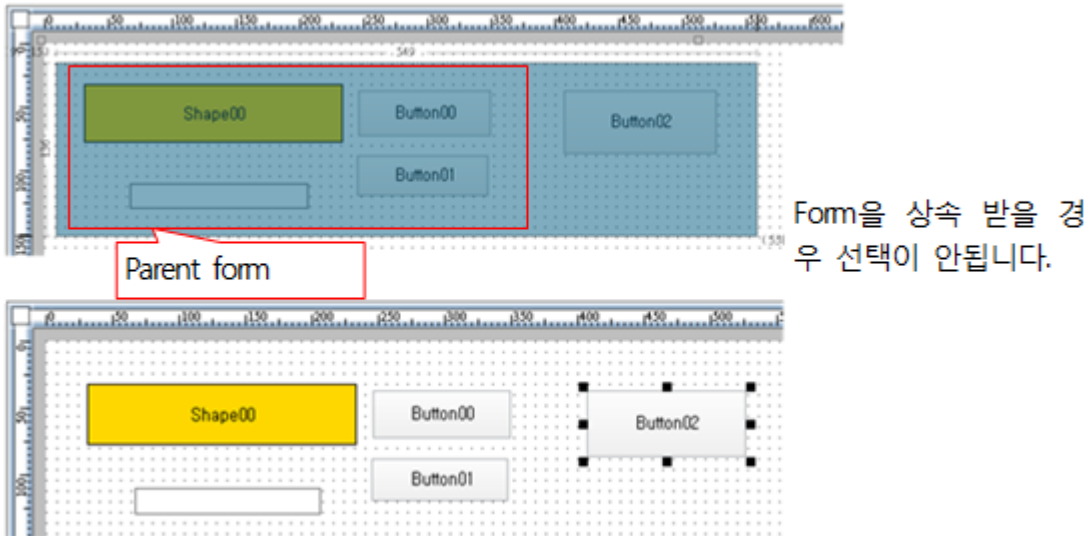
2. TypeDefinition 등록

Form을 상속 받기 위해서는 TypeDefinition에 등록 돼있어야 합니다. User Object와 Composite 컴포넌트처럼 Toolbar에 등록되지는 않습니다. 단지 form을 만들 때 상속 받을 form을 지정만 하면 됩니다.

3. Parent Form 수정 불가

Parent form의 object, script는 수정할 수 없습니다. 즉 Design 시 parent form안의 Button의 position을 변경한다거나 style을 변경하는 것은 불가능합니다. 하지만, Runtime시 script로는 변경 가능합니다.

다음과 같이 form을 상속 받은 다음 마우스를 drag한 경우 parent form의 내부 component들은 선택이 되지 않음을 알 수 있습니다. 즉, 내부 component들이 선택되지 않으므로 해당 property를 변경할 수 없게 되는 것입니다.



8.3.2 Form을 TypeDefinition에 등록하기

개발자가 만든 form을 상속해서 사용하기 위해서는 TypeDefinition에 등록을 해야 합니다. User Object와 Composite 컴포넌트는 TypeDefinition에 등록을 하면 UX-Studio의 toolbar에 해당 component id로 icon이 생기지만 Form은 Object가 아니기 때문에 toolbar에 icon이 생기지는 않습니다.

TypeDefinition에 등록하는 방법은 다음과 같습니다.

단계 1. Edit TypeDefinition windows 띄우기

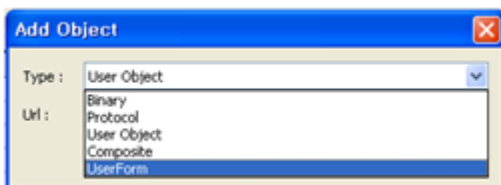
UX-Studio의 Project Explorer에서 TypeDefinition을 double click 하거나, 오른쪽 마우스 클릭 후 edit를 선택합니다.

단계 2. Add Object windows 띄우기

Edit TypeDefinition windows의 Objects에서 Add button을 클릭합니다.

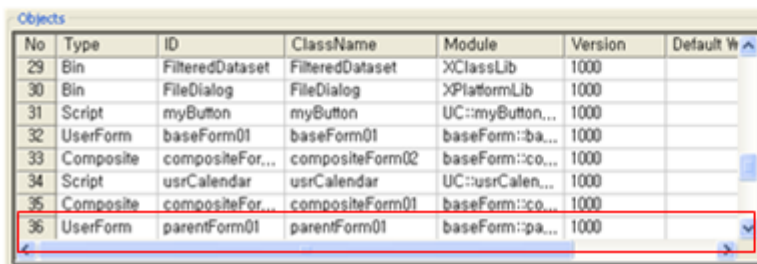
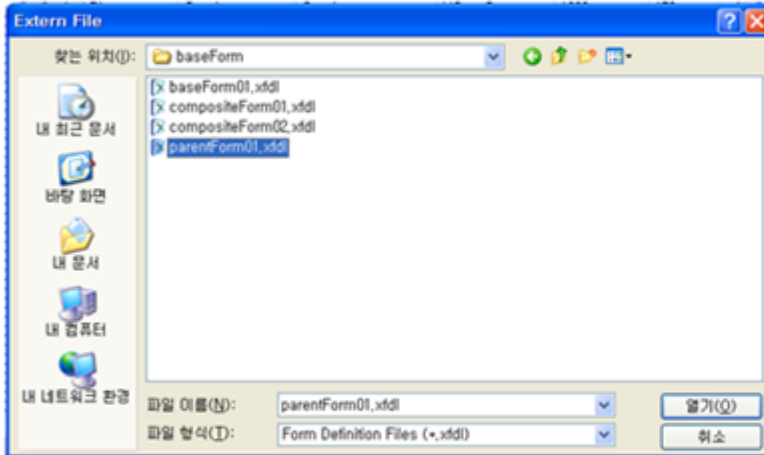
단계 3. Type 선택하기

Add Object windows에서 Type중에 UserForm을 선택합니다.



단계 4. Form 파일 선택

Add Object windows에서 url 찾기 버튼을 통해서 Extern File windows를 띄워서 추가하려는 form을 선택하고, OK 버튼을 누르면 Objects에 추가 됩니다.



Extern File에서 선택후 열기 버튼을 누르고, Add Object에서 OK 버튼을 누르면 Objects에 추가됩니다.

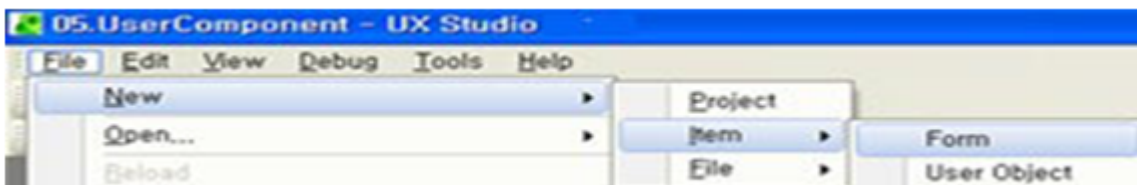
8.3.3 상속 받아서 Form 파일 만들기

Form을 만들 때 상속 받을 form을 지정해서 만들면 됩니다. 자세한 방법은 다음과 같습니다.

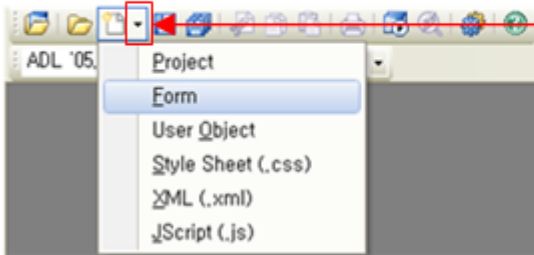
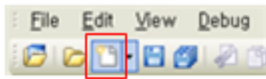
단계 1. Create New Form Wizard windows 띄우기

UX-Studio의 Toolbar의 New를 선택하거나, 메뉴(File > New > Item > Form)에서 선택합니다.

- 메뉴에서 선택



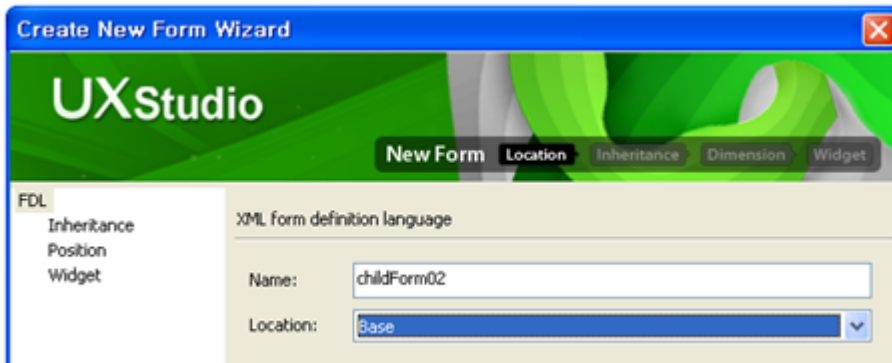
- Toolbar의 New 선택



New를 누르거나, New의 확장기를 통해서 Form을 선택합니다.

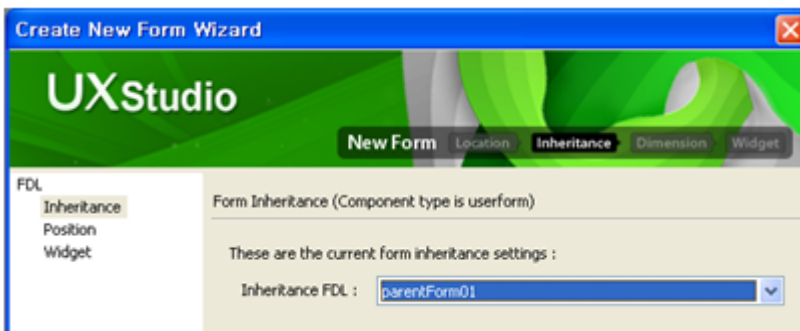
단계 2. Form 이름 및 경로 지정하기

Create New Form Wizard windows에서 Name에 Form의 이름을 Location에 경로를 지정하고 Next 버튼을 누릅니다.



단계 3. 상속 받을 form 지정하기

Inheritance FDL에서 상속 받을 form을 지정하고 Next를 누릅니다.

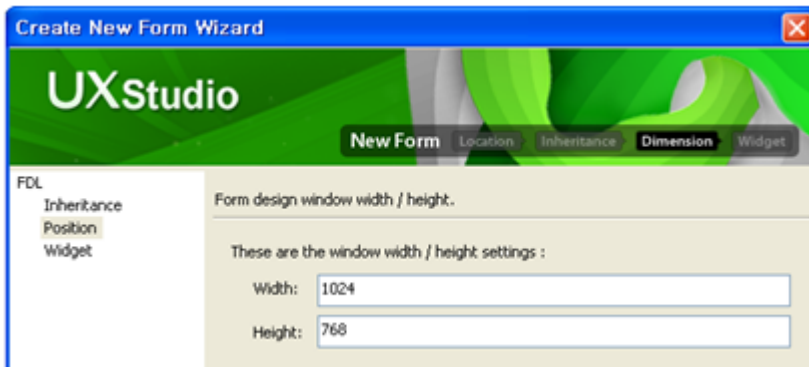


Inheritance FDL의 목록은 TypeDefinition의 Objects의 type이 UserForm인 것만 나옵니다.

단계 4. form size 지정하기

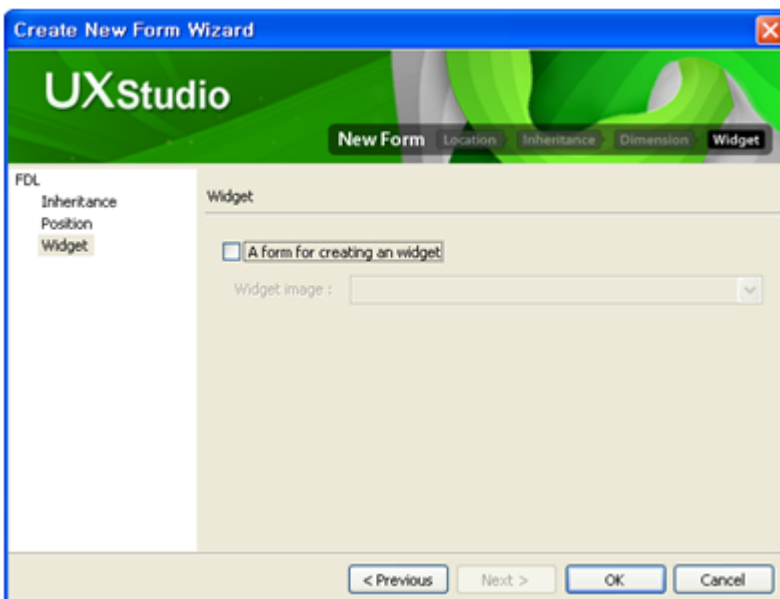
만들고자 하는 form의 크기를 지정합니다.

Width에 form의 가로 길이를 height에 form의 세로 길이를 입력합니다.



단계 5. Widget form 지정여부 설정하기

Widget으로 개발할 때는 “A form for creating an widget”을 check 합니다. 그 외의 form들은 check하지 않고 OK 버튼을 누르면 됩니다.



8.3.4 상속받은 form 사용하기

Child form에서 parent form의 변수, 함수뿐만 아니라 parent form에 있는 object들도 접근할 수 있습니다. 접근하는 방법은 child form에 있는 object, 함수, 변수를 접근하는 것과 동일하게 접근하면 됩니다. 만약 child form에서 override한 함수가 있으면 child form에 있는 것을 호출하게 됩니다. Child form에서 강제로 parent form에 있는 것을 호출하려면 super라는 reserved word를 사용해서 접근하면 됩니다.

8.3.5 Function Override

Parent에 있는 function을 child에서 재 정의하고 싶으면 parent와 동일한 이름으로 function을 만들면 됩니다. ECMAScript spec3에서는 동일한 이름의 function이 여러 개 있을 경우 맨 마지막에 함수를 호출하는 원리와 비슷합니다. 또한 ECMAScript spec3에서는 function의 parameter가 다르다고 하더라도 동일한 function으로 인식하고 맨 마지막 function만 인식합니다. 즉, function overload는 지원하지 않습니다. 참고로 function overload를 사용하려면 arguments를 이용해서 사용해야 합니다.

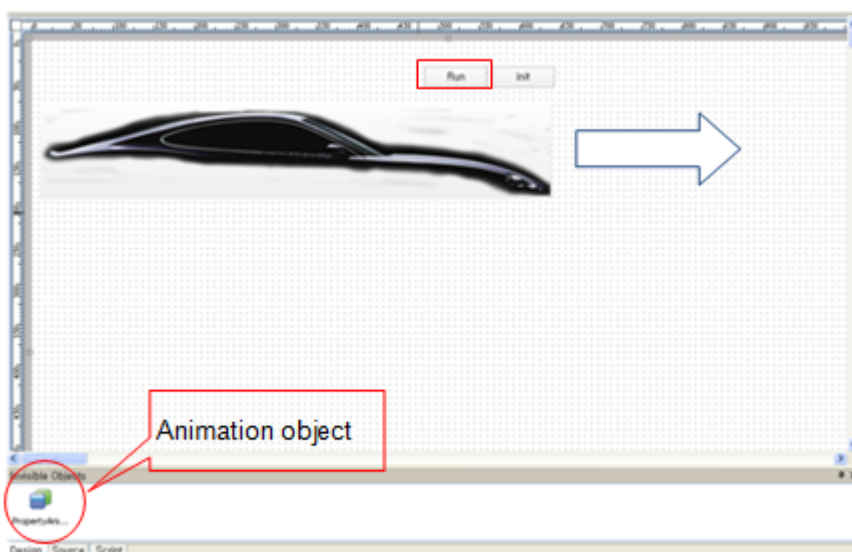
arguments.length를 이용한 overload 구현 예

```
function myOverload ()
{
    if(arguments.length == 1) {
        return(arguments[0] + 10);
    } else if (arguments.length == 2) {
        return(arguments[0] * arguments[1]);
    }
}

alert(myOverload(10)); // 20
alert(myOverload(10, 10)); // 100
```

8.3.6 Form Inheritance 예제

Image에 animation효과를 준 form을 다음과 같이 만들었습니다.



Run button을 누르면 자동차 모양의 이미지가 앞으로 움직이고, init button을 누르면 처음 위치로 되 돌아가는 간단한 form입니다. Animation 기능을 구현하기 위해서 PropertyAnimation 객체를 form에 올려서 position x값을 변경해 주었습니다.

Source는 다음과 같습니다.

```
<?xml version="1.0" encoding="utf-8"?>
<FDL version="1.0">
  <TypeDefinition url="..\default_typedef.xml"/>
  <Form id="parentForm" classname="parentForm" inheritanceid="" cachelevel=""
    position="absolute 0 0 1024 768" version="" titletext="New Form">
    <Layout>
      <ImageViewer id="ImageViewer00" taborder="0" text="ImageViewer00"
        image="URL('images::intro_image01.png')"
        position="absolute 10 80 640 196"/>
      <Button id="Button00" taborder="1" text="Run"
        position="absolute 483 32 560 58"
        style="clিকেffect:PropertyAnimation00;"/>
      <Button id="Button01" taborder="2"
        position="absolute 566 32 643 58" text="init"
        onclick="Button01_onclick"/>
    </Layout>
    <Objects>
      <PropertyAnimation id="PropertyAnimation00" starttime="0"
        targetcomp="ImageViewer00" interpolation="Interpolation.bounceOut"
        fromvalue="10" duration="3000" targetprop="position.x"
        tovalue="400" endingmode="current"/>
    </Objects>
  </Form>
</FDL>
```

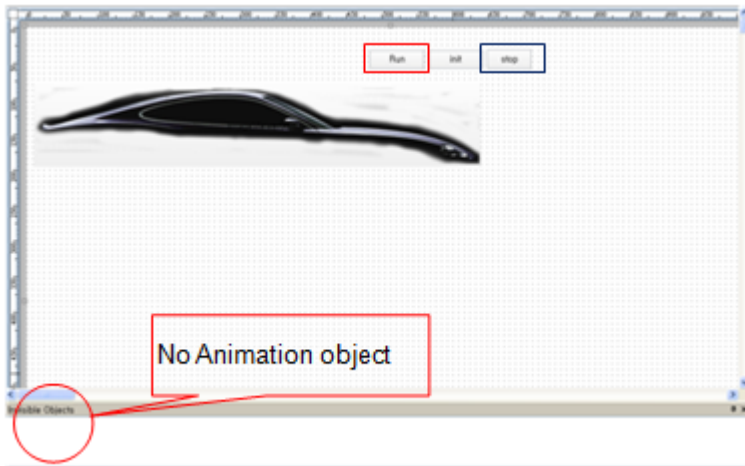
Script는 다음과 같습니다.

```
function Button01_onclick(obj:Button, e:ClickEventInfo)
{
  ImageViewer00.move(10, 80);
}
```

Animation에 대한 자세한 설명은 “5장 Animation 기능”을 참조하십시오.

위에서 구현한 form을 상속 받기 위해서 TypeDefinition에 등록합니다.

TypeDefinition에 등록한 form을 상속 받아서 다음과 같이 child form을 만들었습니다.



Child form은 parent를 상속 받았기 때문에 Animation객체가 없어도 parent form과 동일하게 Run button을 누르면 이미지가 앞으로 움직입니다. 그리고 Animation을 중간에 중단하는 기능을 추가하기 위해서 Stop button을 추가해서 Animation 중단 코드를 추가했습니다.

Source는 다음과 같습니다.

```
<?xml version="1.0" encoding="utf-8"?>
<FDL version="1.0">
  <TypeDefinition url="..\default_typedef.xml"/>
  <Form id="childForm" classname="childForm" inheritanceid="parentForm"
    cachelevel="" position="absolute 0 0 1024 768" version=""
    titletext="New Form">
    <Layout>
      <Button id="Button03" taborder="1" text="stop"
        onclick="Button03_onclick" position="absolute 648 31 711 58"/>
    </Layout>
    <Objects/>
  </Form>
</FDL>
```

그리고 script는 다음과 같습니다.

```
function Button03_onclick(obj:Button, e:ClickEventInfo)
{
  PropertyAnimation00.stop();
}
```

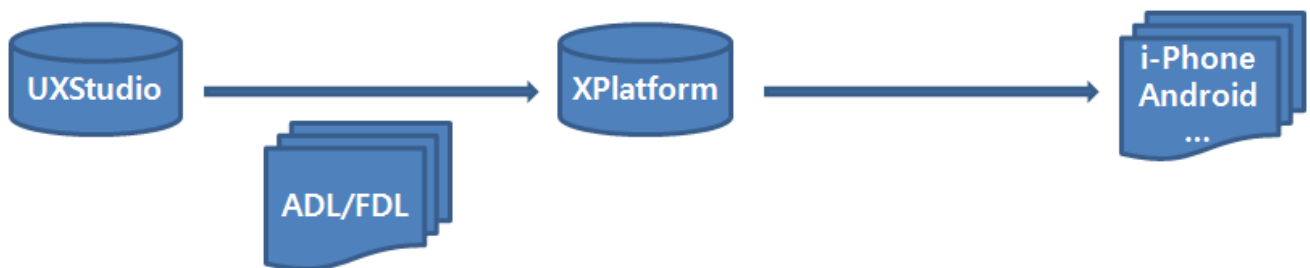
Child form의 source를 보면 상속 받은 form명을 inheritanceid에 명시하고 신규로 추가한 Button만 가지고 있음을 알 수 있습니다. 또한 child form에는 없는 object인 PropertyAnimation00을 script로 접근을 해서 Animation을 중단하는 기능을 추가했음을 알 수 있습니다.

9.

MLM(V9.2추가)

현존하는 다양한 매체들은 서로 다른 표현영역을 가지고 있기 때문에 매체에 맞는 Design작업 및 Source관리를 해야 하는 문제가 생깁니다. 따라서 One Source로 다양한 상황에 맞는 Design을 표현할 수 있도록 해줄 수 있는 Multi Layout Manager기능이 추가되었습니다.

- MLM구조



UX-Studio에서 생성된 ADL/FDL을 XPLATFORM이 Device에 맞는 Layout을 선택하여 출력합니다.

이하 사용된 용어 설명

- Default Layout
기본이 되는 Layout. Layout정보를 가지고 있지 않으며 기존 V9.1의 Form과 같은 기능
- 추가 Layout
Default Layout을 제외한 모든 Layout
- Sub Layout
Div, PopupDiv, TabPage등 하위 contents를 가지고 있는 component들의 Layout
- Target 컴포넌트
Sub Layout Editor를 실행한 컴포넌트

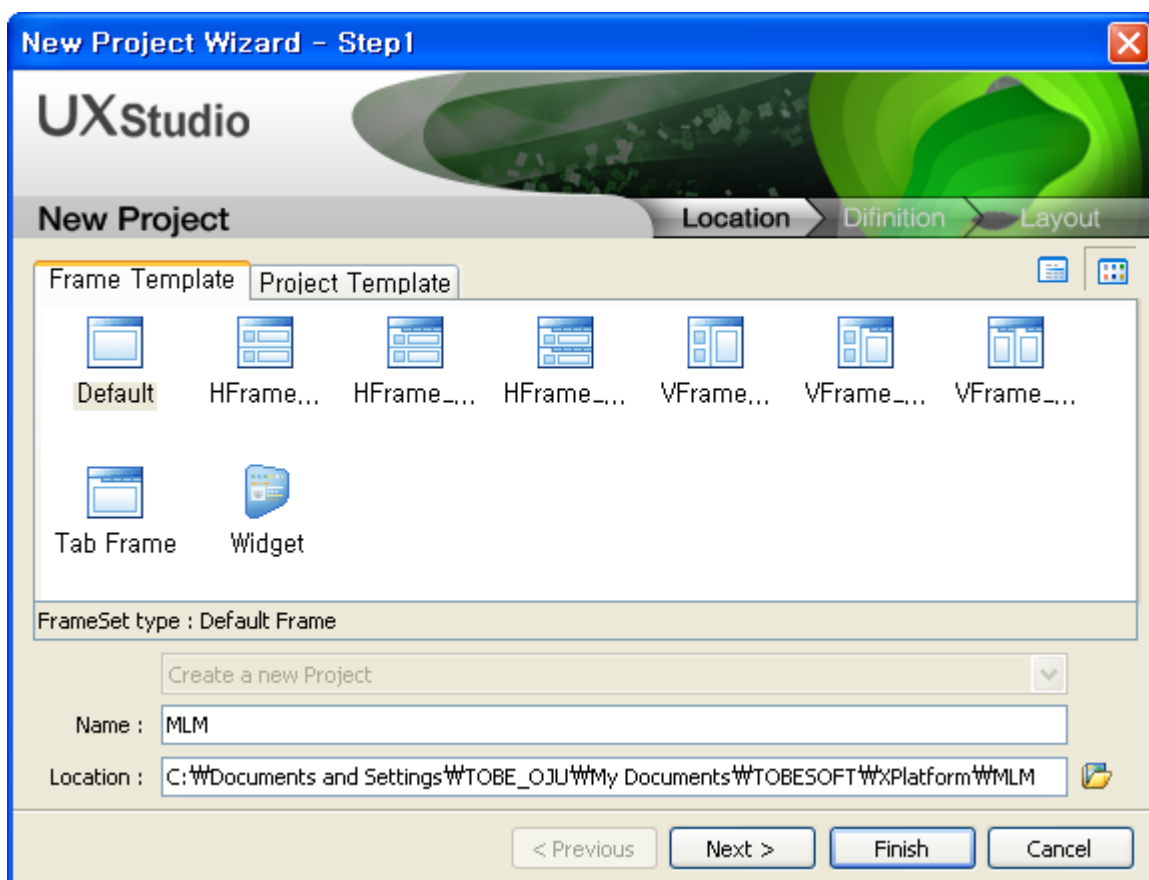
9.1 Project 생성 및 수정

9.1.1 Project생성

‘I-Phone’, ‘I-Pad’, ‘GalaxyTab’의 Layout정보를 가진 Project를 생성하는 방법을 예로 들어 설명합니다. UX-Studio 에서 File > New > Project 메뉴를 호출할 경우 아래와 같이 Project를 생성하는 Wizard가 표시되며, 각 단계별로 정보를 입력하여 생성할 수 있습니다.

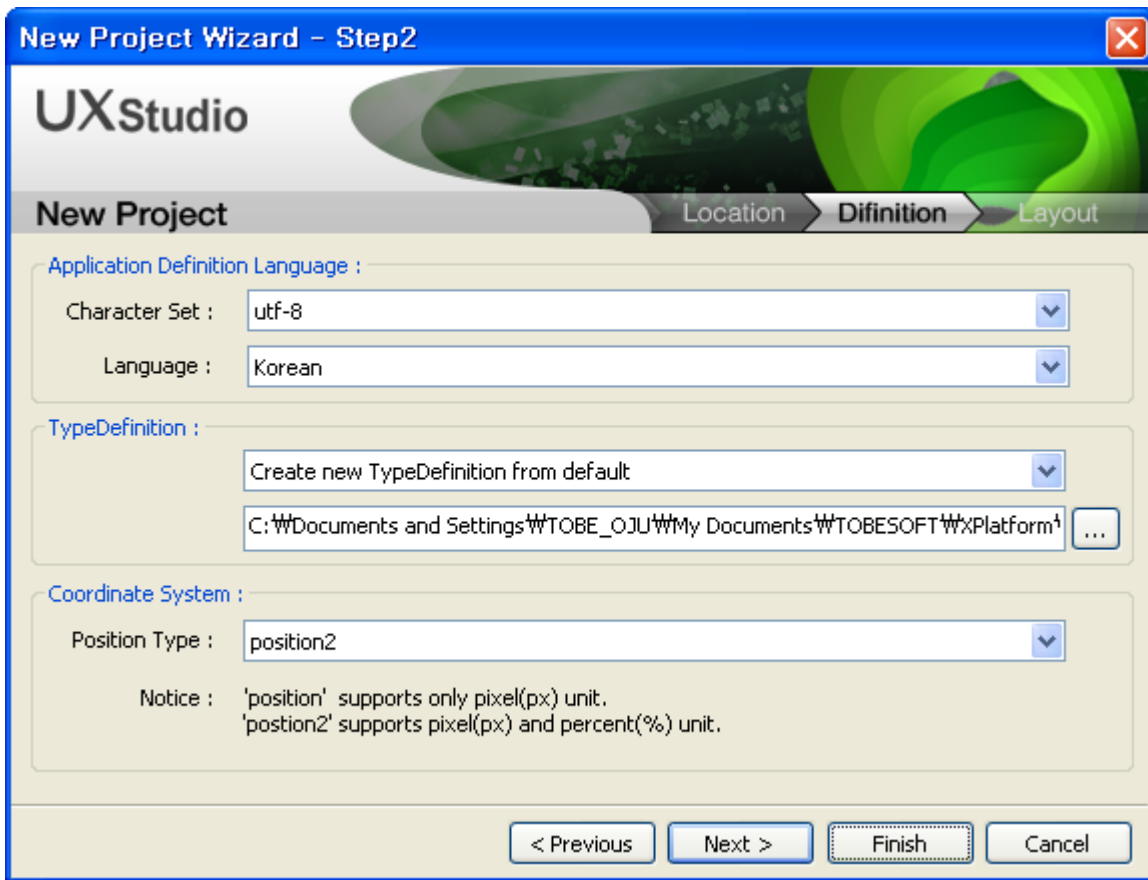
1. New Project Wizard - Step1

생성될 Project의 Frame Template와 생성될 경로 및 Project명을 입력하는 단계입니다. Project명은 반드시 입력해야 하는 필수 항목이며, 생성될 경로에 동일한 Project명이 존재할 경우에는 생성할 수 없습니다.



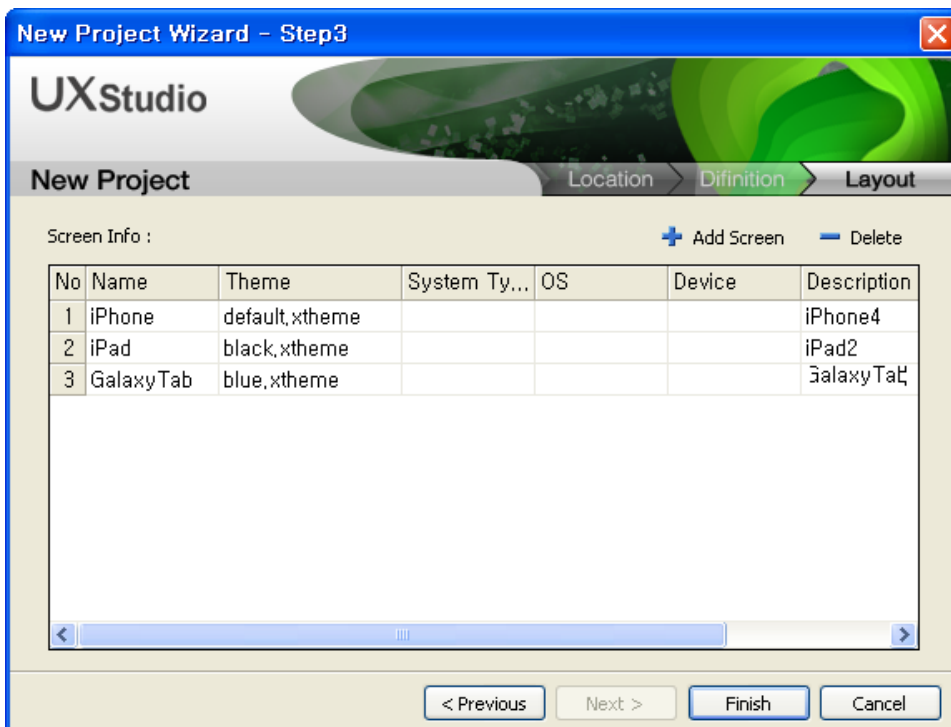
2. New Project Wizard - Step2

Character Set정보 및 TypeDefinition정보 등을 설정하는 단계입니다.



3. New Project Wizard - Step3

Project에서 사용할 Screen정보를 입력하는 단계입니다. I-Phone, I-Pad, GalaxyTab을 개발할 목적이기 때문에 각각의 항목을 입력하고, 해당 Screen에서 사용할 Theme를 지정합니다. 'Finish'를 클릭하여 'I-Phone', 'I-Pad', 'GalaxyTab' Screen정보를 가진 Project를 생성합니다.

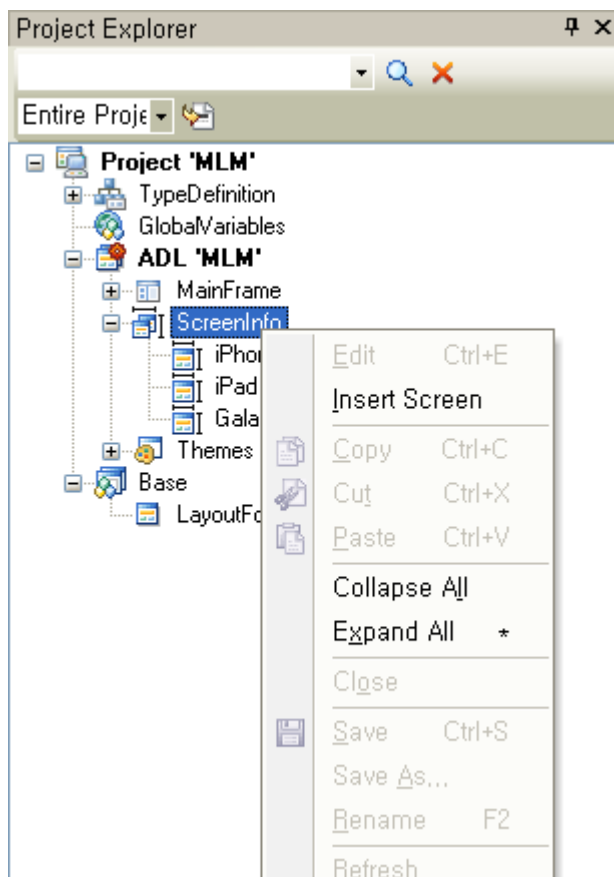


- Name : Screen의 이름
- Theme : Screen에 사용할 Theme
- System Type
 - win32 : 일반 Desktop
 - Android : Android Mobile
 - iPhone : Apple Mobile
- OS : 해당 장비에서 사용되는 운영체제
- Device : 장비의 종류
- Description : Screen에 대한 설명(Description은 기능 동작에 영향을 주지 않습니다.)

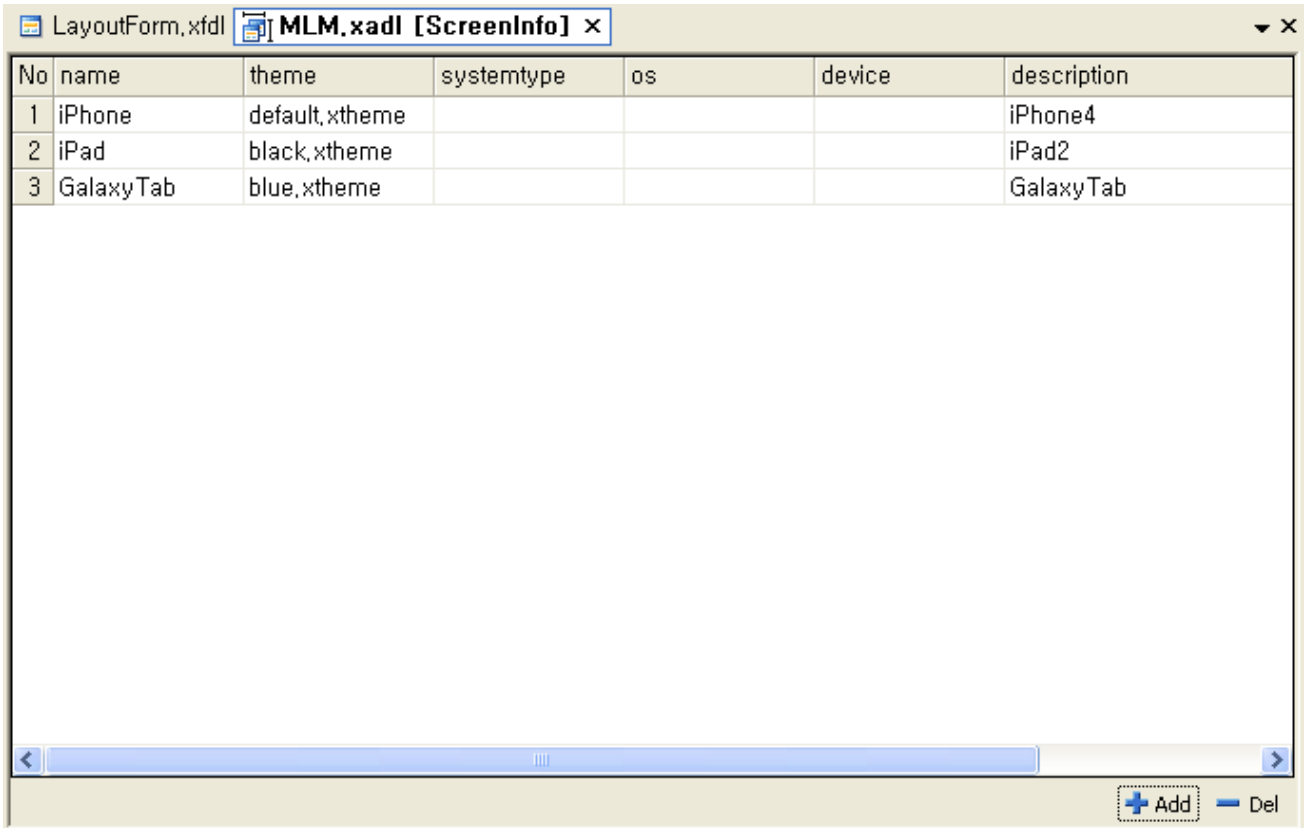
9.1.2 Screen정보의 수정

‘New Project Wizard’에서 입력한 Screen정보를 수정하거나 새로운 Screen정보를 입력할 수 있는 Editor기능이 추가되었습니다.

‘Project Explorer’에서 ADL Item에서 제공되는 Popup메뉴를 사용하여 새로운 Screen정보를 추가하거나 ADL Item의 하위 정보로 표시되는 ‘ScreenInfo’등을 선택하여 편집할 수 있습니다.



‘ScreenInfo Editor’는 기존에 제공되던 ‘Widget Editor’, ‘Variable Editor’와 동일한 방식으로 편집기능을 제공합니다.

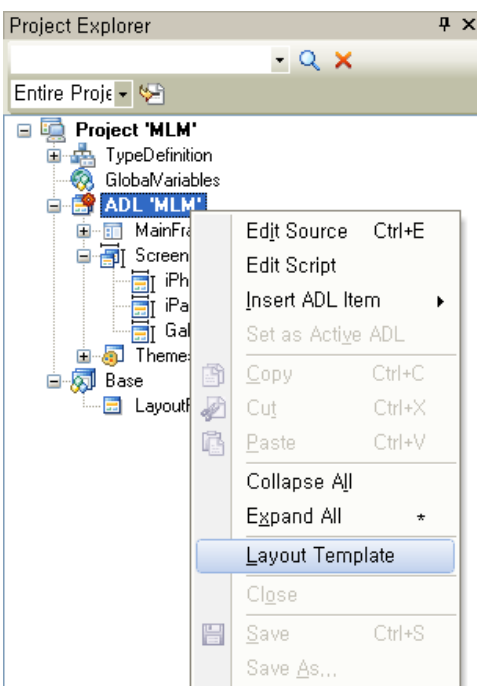


No	name	theme	systemtype	os	device	description
1	iPhone	default,xtheme				iPhone4
2	iPad	black,xtheme				iPad2
3	GalaxyTab	blue,xtheme				GalaxyTab

At the bottom right of the window, there are buttons for '+ Add' and '- Del'.

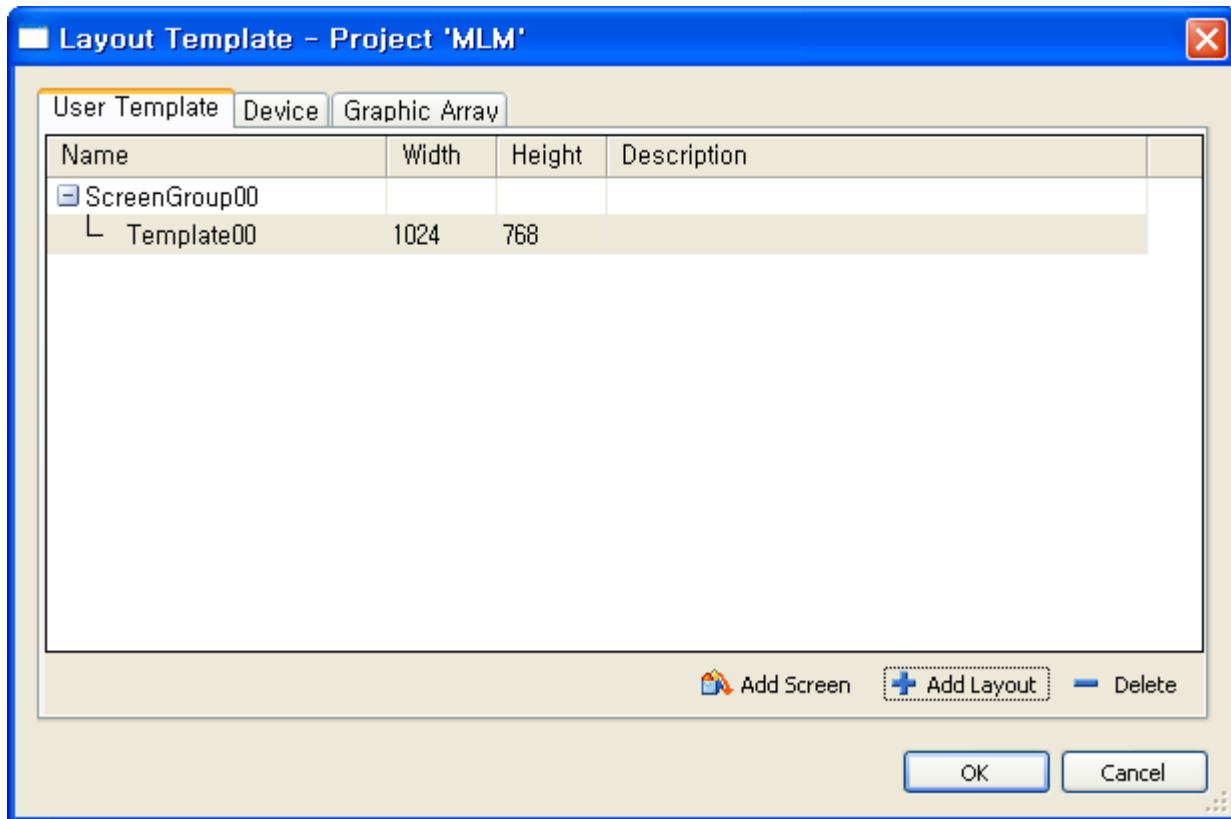
9.1.3 Layout Template등록

기본적으로 제공되는 Layout Template를 사용할 수 있습니다. 또한, 자주 사용되는 Layout정보를 사용자가 Template으로 등록 할 수 있습니다.



Layout Template Dialog는 기본적으로 제공되는 Template인 Device, Graphic Array Template과 사용자가 직접 등록하여 사용하는 User Template 3개의 Tab으로 구성되어 있습니다.

- User Template : 사용자가 직접 등록하여 사용하는 Layout Template List
- Device : 현재 사용되고 있는 주요 모바일 기기
- Graphic Array : 해상도로 사용되고 있는 Graphic Array

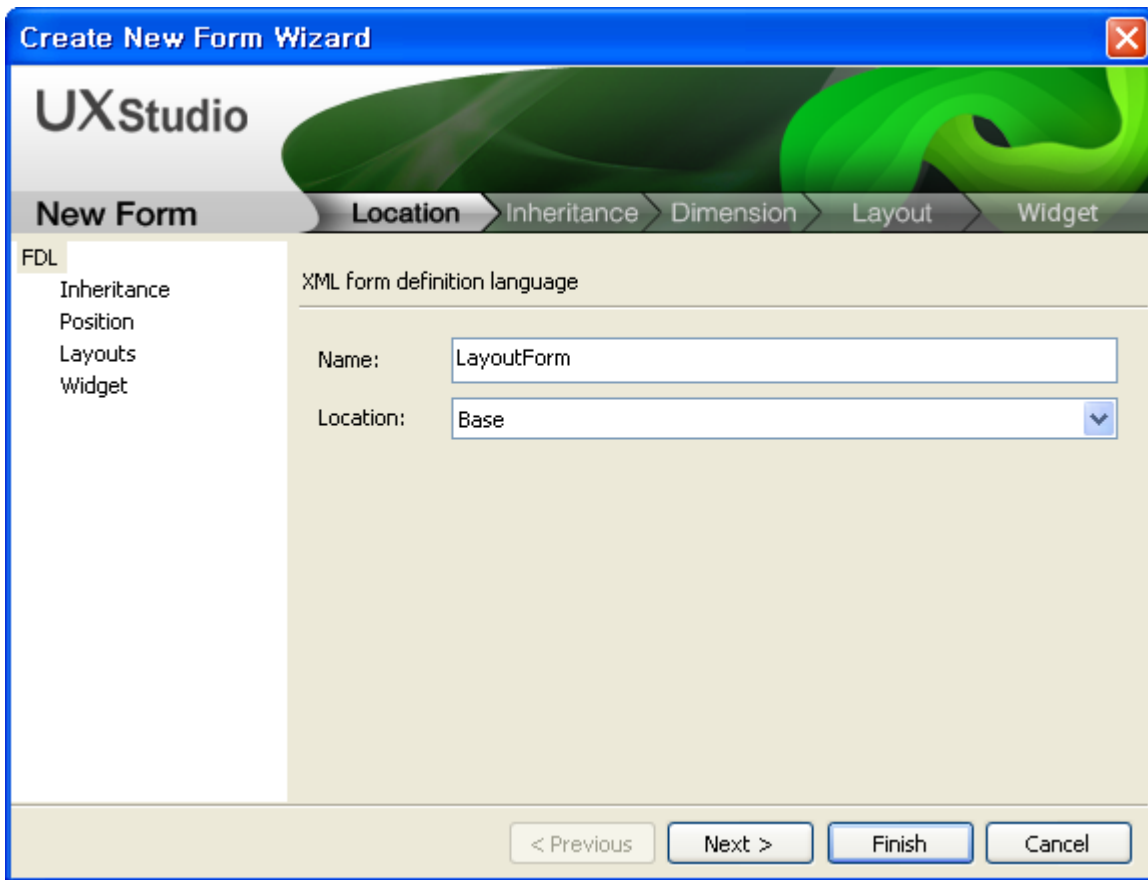


9.2 Form생성

'I-Phone', 'I-Pad', 'GalaxyTab' 3군데서 사용할 FDL파일을 만드는 방법을 예로 들어 설명합니다. File > New > Item > Form 메뉴를 선택할 경우 아래와 같이 FDL을 생성하는 Wizard가 표시되며, 각 단계별로 정보를 입력하여 생성할 수 있습니다.

1. New Form Wizard - Step1

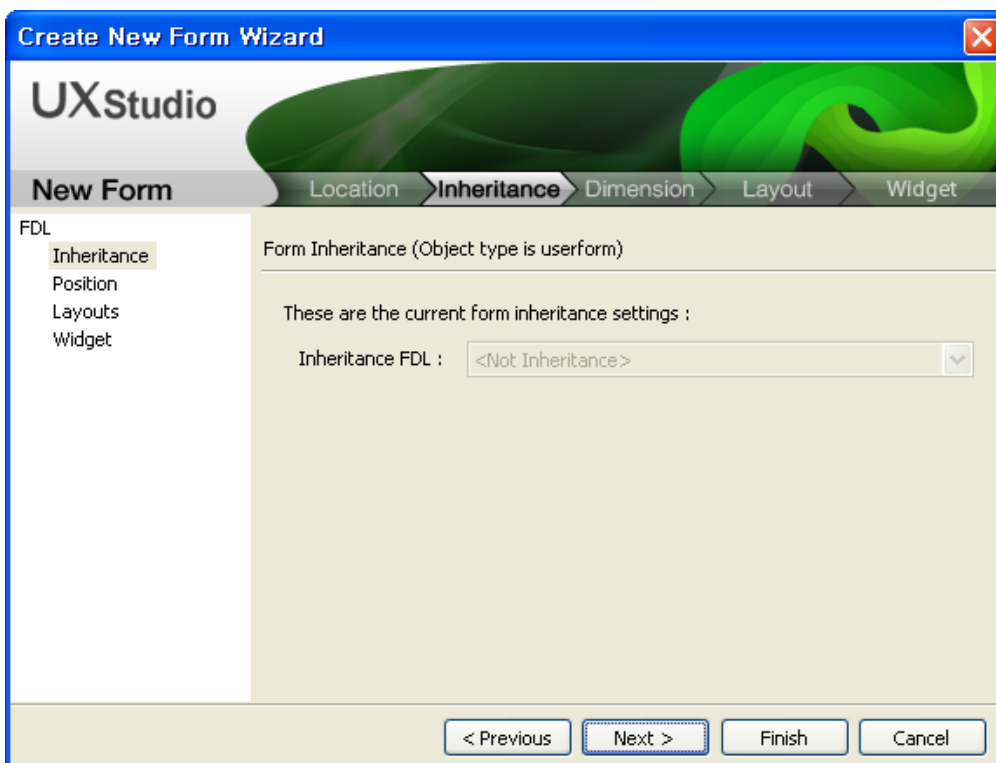
생성될 Form의 경로와 이름을 입력하는 단계입니다. Form명은 반드시 입력해야 하는 필수 항목이며, 생성될 경로에 동일한 Form명이 존재할 경우에는 생성할 수 없습니다.



2. New Form Wizard - Step2

생성될 Form의 Inheritance를 지정하는 단계입니다.

TypeDefinition에 'UserForm'이 등록되어 있는 경우에만 입력 컨트롤이 활성화 됩니다.



3. New Form Wizard - Step3

생성될 Form의 'Default' Layout Form 크기를 입력하는 단계입니다. 초기 입력값은 Option에서 변경할 수 있습니다.

Create New Form Wizard

UXStudio

New Form Location Inheritance **Dimension** Layout Widget

FDL

Inheritance

Position

Layouts

Widget

Form design window width / height.

These are the window width / height settings :

Width: 1024

Height: 768

< Previous Next > Finish Cancel

4. New Form Wizard - Step4

생성될 Form에서 Design할 Layout정보를 입력하는 단계입니다. Screen종류별, 가로, 세로의 크기에 맞춘 Layout정보를 설정합니다.

Create New Form Wizard

UXStudio

New Form Location Inheritance Dimension **Layout** Widget

FDL

Inheritance

Position

Layouts

Widget

Add Layout

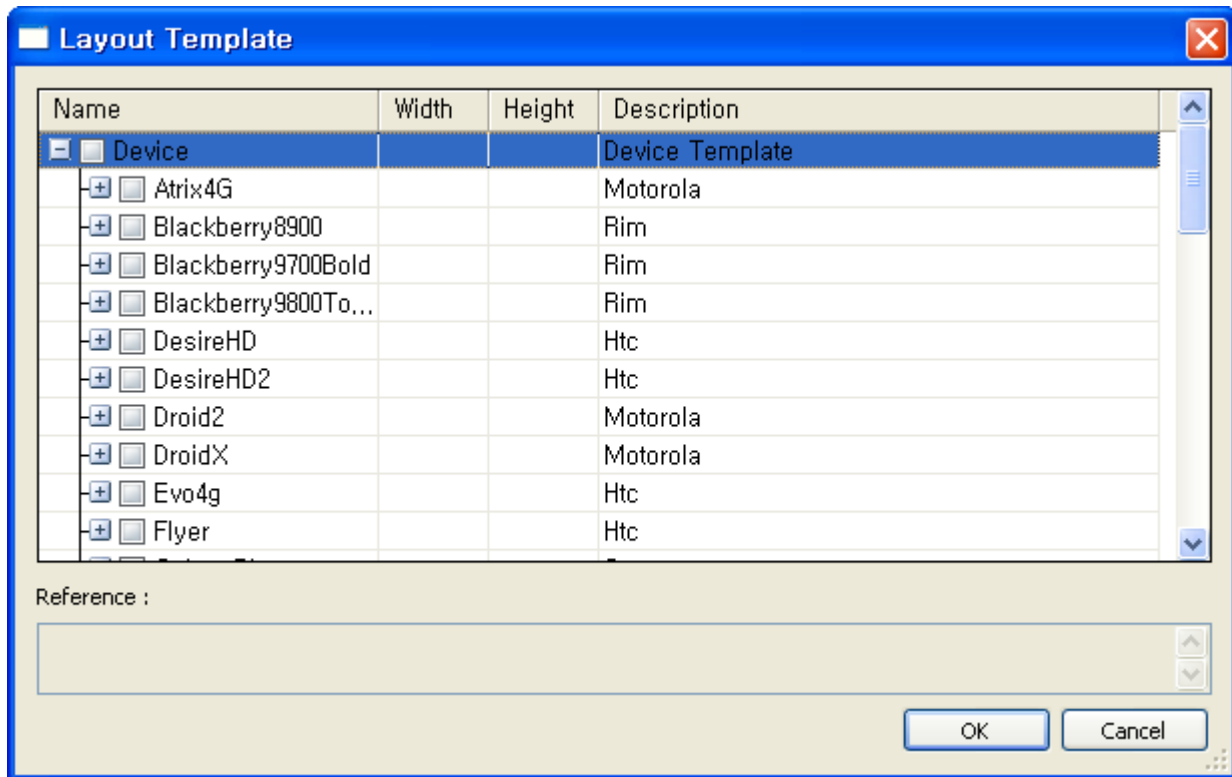
Layouts :

Template + Add Layout - Delete

No	Name	Screenid	Width	Height	De
1	iPhoneW	iPhone	640	960	
2	iPhoneH	iPhone	960	640	
3	iPadW	iPad	768	1024	
4	iPadH	iPad	1024	768	
5	GalaxyTabW	GalaxyTab	600	1024	
6	GalaxyTabH	GalaxyTab	1024	600	▼

< Previous Next > Finish Cancel

Template버튼을 누르면 아래 그림과 같은 Template Dialog가 나타나고 원하는 Layout, Layout group을 선택한 후 ok버튼을 누르면 선택한 Layout 정보가 New Form Wizard에 추가됩니다.

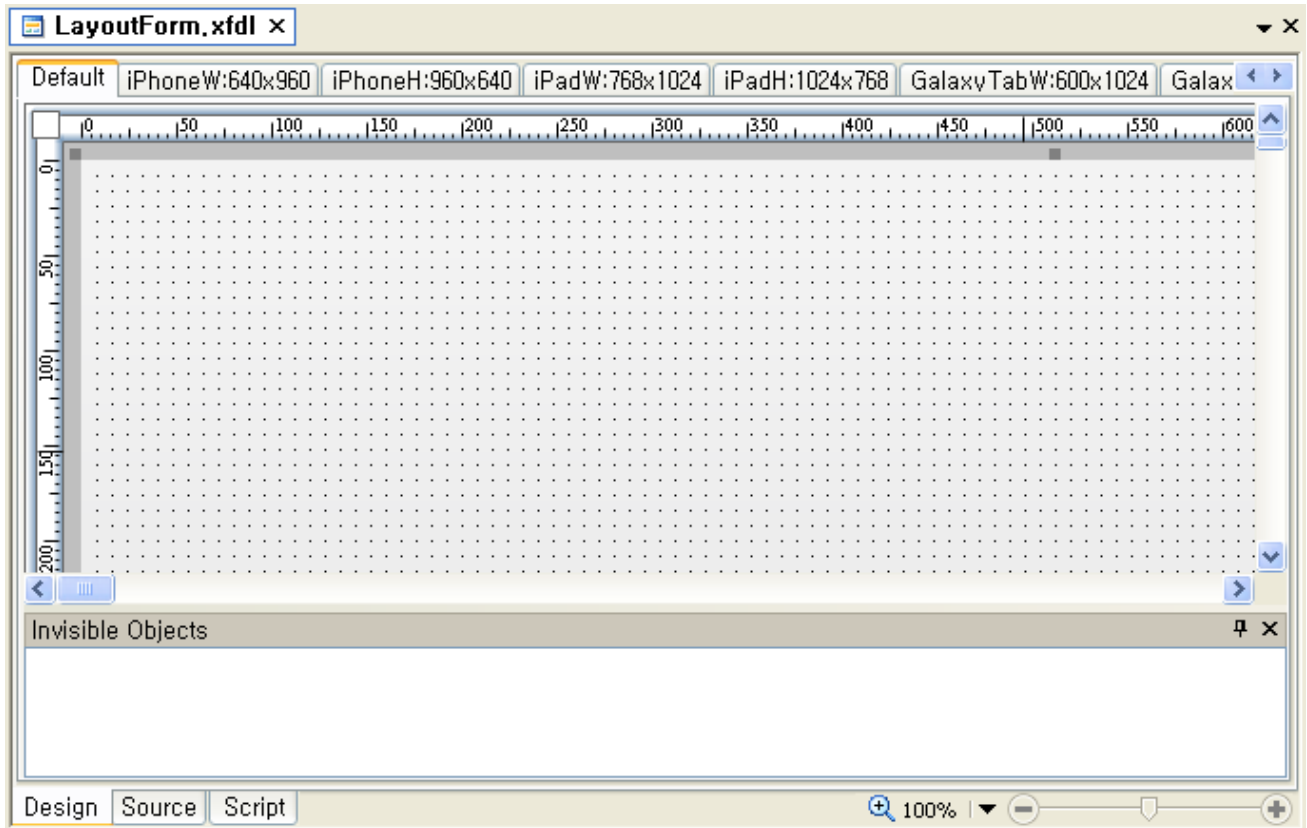


9.3 Layout

'New Form Wizard'에서 입력한 Layouts정보가 Tab형태로 표시 되도록 Form Design기능이 변경되었습니다.

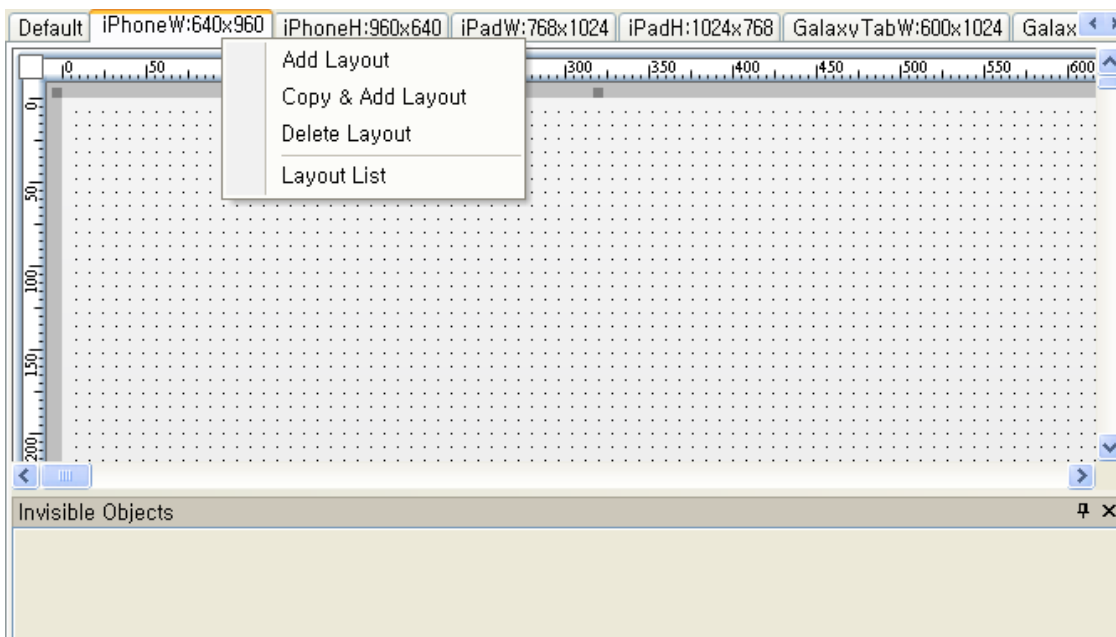
9.3.1 Layout Tab

현재 Form에서 사용되는 Layout이 탭으로 표시됩니다. 탭의 위치는 Option에서 사용자가 변경할 수 있습니다.



Tab을 전환하여 'New Form Wizard'에서 입력한 Layout으로 화면 전환이 가능하며 Layout에 대한 수정 기능을 PO PUP메뉴로 제공합니다.

- Add Layout : 'Layout Tab'의 마지막에 새 Layout을 생성하는 Dialog를 보여줍니다.
- Copy & Add Layout : 현재 선택된 Layout을 복사해 'Layouts'의 마지막에 추가
- Delete Layout : 현재 선택된 Layout을 'Layouts'에서 제거. ('Default'는 삭제할 수 없습니다.)
- Layout List : 현재 Form에서 사용중인 Layout정보를 수정할 수 있는 Dialog를 보여줍니다.

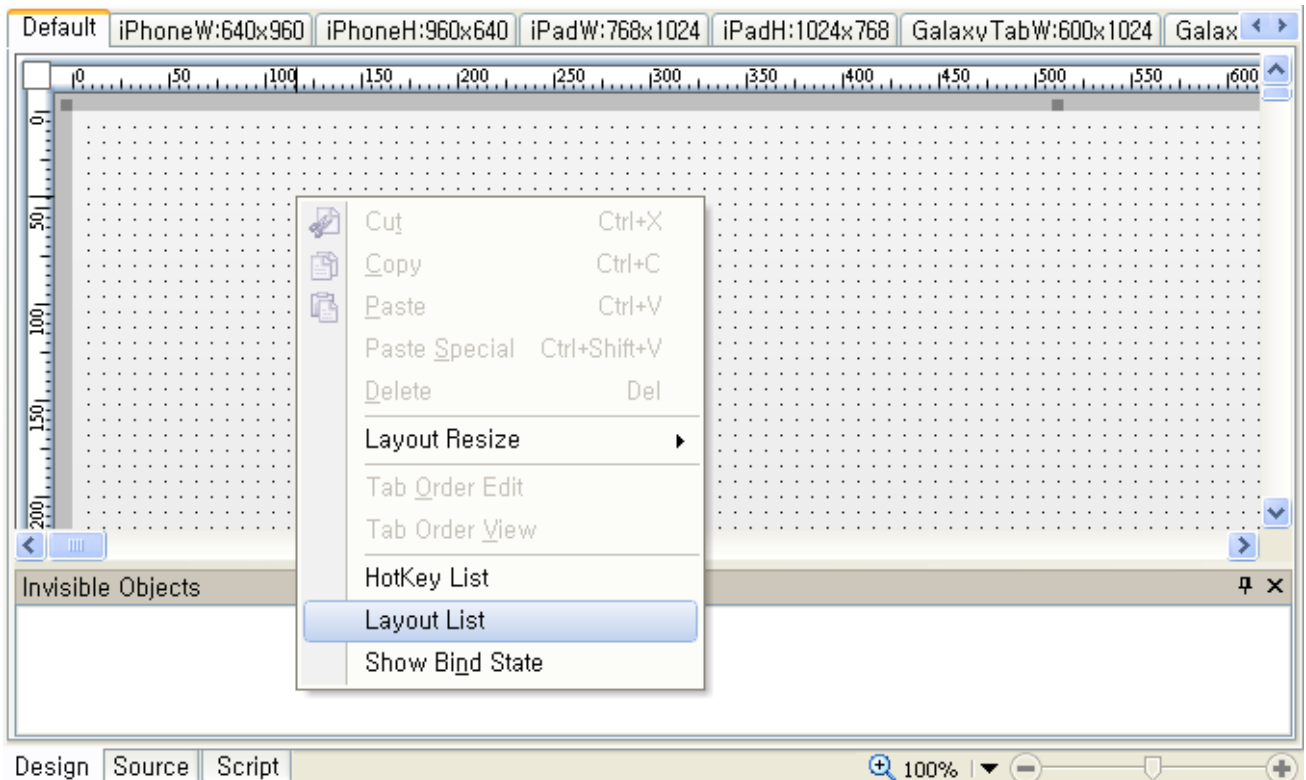


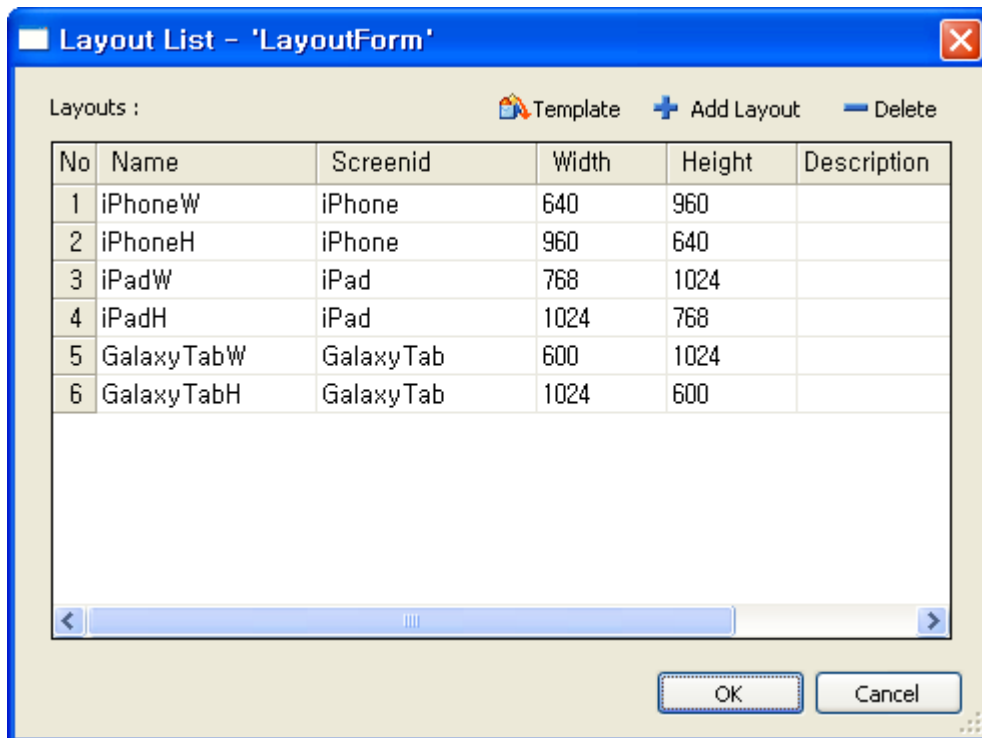
‘Default’ 탭에서는 모든 기능이 제공되지만, 추가된 ‘Layout’ 탭에서는 일부 Design 기능이 제한됩니다.

Design 기능	Default	추가 Layout
컴포넌트 Create, Delete	O	X
컴포넌트 Copy, Cut, Paste, Paste Special	O	X
TabOrder 변경	O	X
Form의 Size수정	O	X
Form의 Size를 Guide Line으로 표시	X	O
Form의 Tracker표시	O	X
Div, Tab, Popup등의 하위 컴포넌트 수정	O	X
Invisible Object 추가, 삭제, 선택	O	X

9.3.2 Layout 정보 수정

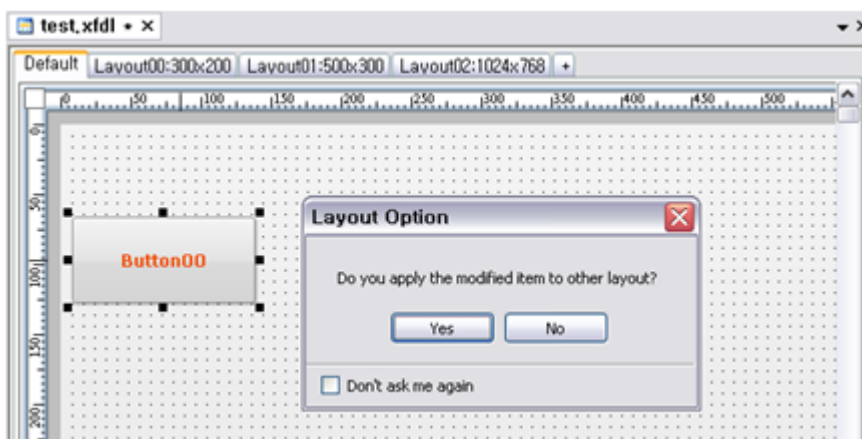
현재 Form에서 사용중인 Layout의 정보를 수정할 수 있는 기능이 POPUP메뉴로 추가되었습니다.





- Name : Layout의 이름
 - Layout의 Name으로 대소문자 구별 없이 'Default'를 사용할 수 없습니다
 - 같은 Form안에서는 중복된 Layout Name을 사용할 수 없습니다
- Screen : Layout이 사용할 Screen명
- Width : Layout의 넓이
- Height : Layout의 높이

'Default' 탭에서 컴포넌트의 속성값이 변경되었을 경우에는 변경된 내용을 다른 Layout에 적용할지 확인하는 메시지 창이 Popup됩니다.



- 위 확인하는 메시지 창은 아래의 조건을 모두 만족한 경우에만 Popup됩니다.

번호	조건
1	다른 Layout에서 변경될 속성이 Source상에 존재하는 경우
2	'Layouts Edit' Option 값이 'Ask whenever a property changed'로 되어 있는 경우

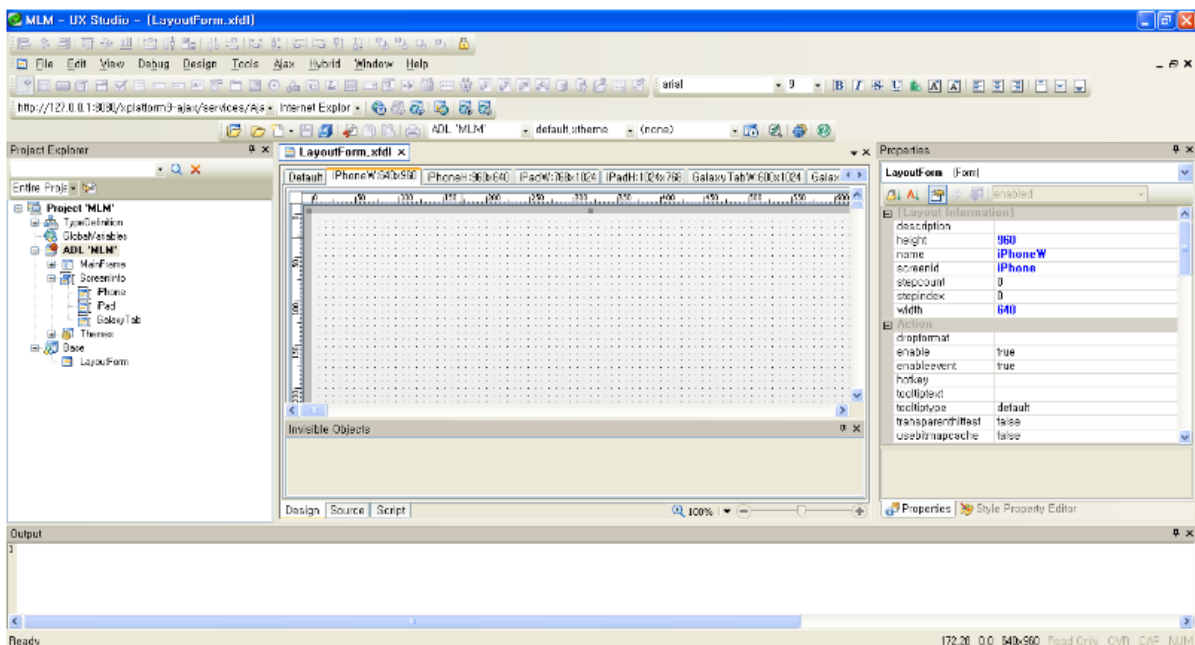
- 'Default'탭에서 수정이 되어 다른 Layout에 적용할지 확인하는 메시지 창에서 'Don't ask me again'을 체크한 상태에서 'Yes'나 'No'를 선택하는 경우 'Layouts Edit' Option값이 바뀌면서 이후에는 메시지 창이 Popup되지 않습니다



버튼	'Layouts Edit' Option
Yes	Change property Default Layout with other layouts
No	Changed property is only Default Layout applied

9.3.3 Properties Window

Form Design의 정보를 Properties Windows에서 보여주고 있는 경우, Layout 탭에 따라 표시 방법이 달라지게 됩니다.



- Default Layout탭에서는 Form Properties중 'position'정보를 수정하면 Multi Layout을 사용할 경우, 'height'와 'width'가 현재 size에 맞게 설정됩니다. Default Layout탭에서 'height'와 'width'는 hidden property입니다. Tracker를 사용하여 size 변경을 할 경우 'position'과 'height'와 'width'정보 모두가 바뀌게 됩니다.
- 추가된 Layout탭에서는 Form Properties중 'position'정보를 수정할 수는 없지만 'Layout Information'정보인 'height'와 'width'를 변경하여 크기를 변경할 수 있습니다. Tracker를 사용하여 size 변경을 할 경우 'position'정보는 저장되지 않고 'height'와 'width'정보만 저장 됩니다.
- 'Layout Information'에서 보여주는 Properties는 'Layout List Dialog'와 동일합니다.
- Form Design의 'Default'탭과 다른 정보를 가지고 있는 추가된 Layout탭에서는 Property를 굵은 파란색으로 표시하게 되었습니다. 따라서 Properties Window에서 표시해주는 정보는 굵기와 색깔에 따라서 아래와 같은 의미를 가지게 됩니다.

색상	굵게	정보
검은색	X	Theme와 css만 적용된 기본값을 가진 경우
검은색	O	'Default'탭에서 수정하여 Theme와 css가 적용된 기본값과 다른 값을 가진 경우
파란색	O	추가된 Layout탭에서 수정한 값이 'Default'탭의 정보와 다른 값을 가진 경우

- Form Design의 추가된 Layout탭에서 Property를 표시할 경우에 아래와 같은 Property는 수정할 수 없도록 disabled처리됩니다.

속성명	설명
id	모든 컴포넌트의 ID는 추가된 Layout탭에서는 수정할 수 없습니다.
taborder	모든 컴포넌트의 taborder는 추가된 Layout탭에서는 수정할 수 없습니다.
inheritanceid	Form Property중에서 inheritanceid는 수정할 수 없습니다.
position	Form Property중에서 position은 수정할 수 없습니다.

9.4 Sub Layout

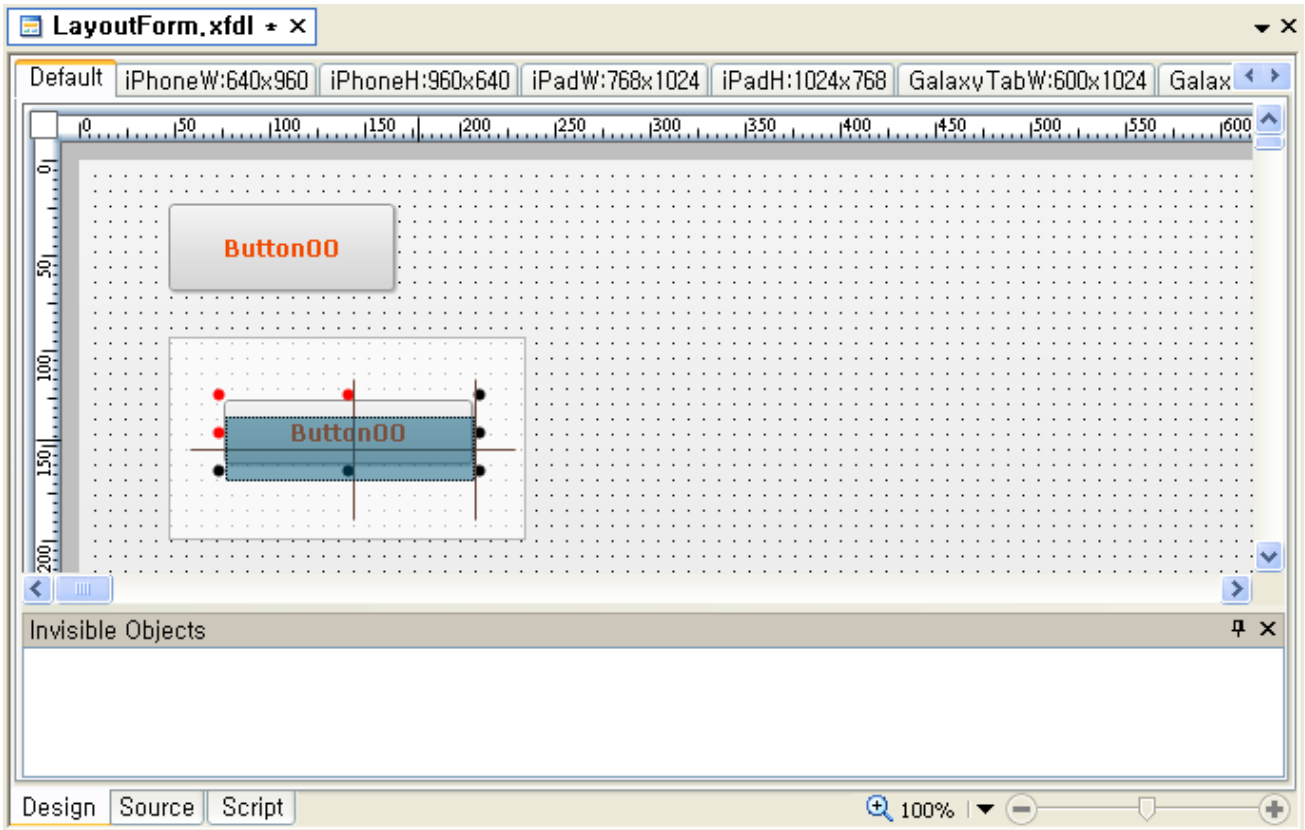
컴포넌트내부에 Contents를 가지고 있는 Div, PopupDiv, TabPage 등도 Form과 같이 Multi Layout을 가지게 되며 이를 Sub Layout이라 합니다. Sub Layout은 Form Layout과 1:1 매칭이 아니고 별도의 Multi Layout을 구성합니다. Tab의 경우, TabPage 별로 다른 Multi Layout을 구성할 수 있습니다.

- Sub Layout을 가지고 있는 컴포넌트는 자신의 Sub Layout 중에서 해당 컴포넌트의 Size에 맞는 최적화된 Sub Layout을 자동으로 표시해 줍니다.
- Target 컴포넌트의 Size가 변경될 시, 자동으로 Sub Layout Change가 일어나고 새로운 Size에 맞는 최적화된 Sub Layout을 찾아줍니다.

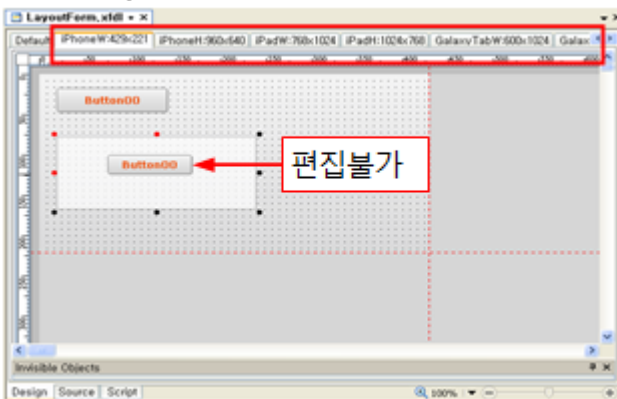
9.4.1 Default Layout에서의 편집

Sub Layout을 가지고 있는 컴포넌트의 하위 Contents는 Form의 Default Layout에서만 직접 편집이 가능합니다. (기존의 편집 방법과 동일)

직접 편집되는 내용은 해당 컴포넌트가 보여주고 있는 Sub Layout에 저장됩니다.



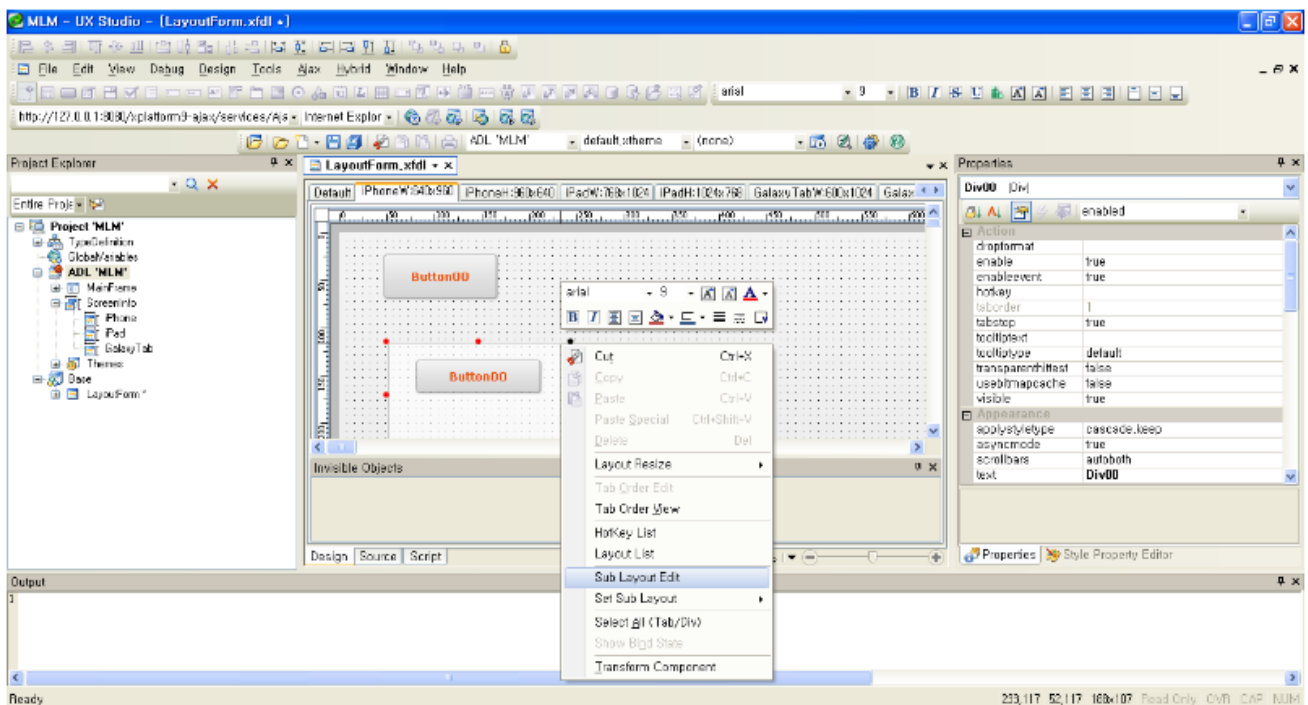
단, Default Layout이 아닌 추가 Layout (예: 위 그림에서 iPhoneW Layout)에서는 Contents를 직접 편집 할 수 없습니다. 따라서, 하위 Contents는 선택 및 수정 할 수 없으며 하위 Contents 선택 시, 최상위 컴포넌트가 선택됩니다. 추가 Layout에서 Contents를 편집 하기 위해서는 Sub Layout Editor를 사용하여 편집하여야 합니다.



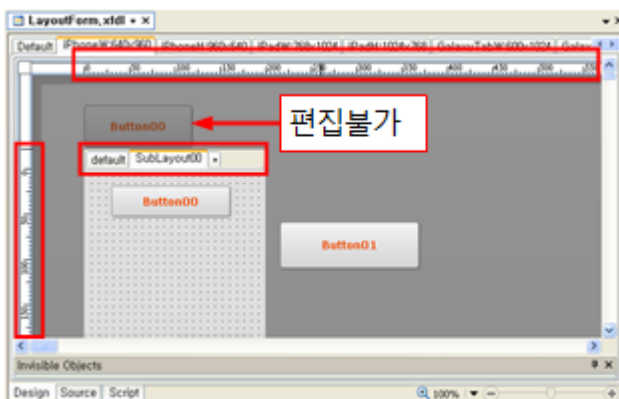
9.4.2 Sub Layout에서의 편집

추가 Layout에서 Contents를 편집 하기 위해서는 Sub Layout Editor를 사용하여 편집하여야 합니다. Sub Layout 을 가진 컴포넌트는 Sub Layout Edit, Set Sub Layout 2개의 Popup Menu를 제공합니다.

- Sub Layout Edit메뉴를 선택하면 Sub Layout을 편집할 수 있습니다.
- Set Sub Layout메뉴를 선택하면 컴포넌트의 Size를 선택한 Sub Layout의 크기로 변환해 줍니다. Set Sub Layout메뉴는 default Sub Layout이외의 Sub Layout이 존재할 경우에만 나타납니다.



Sub Layout Edit메뉴를 선택하면 아래 그림과 같은 Sub Layout Editor가 나타납니다.

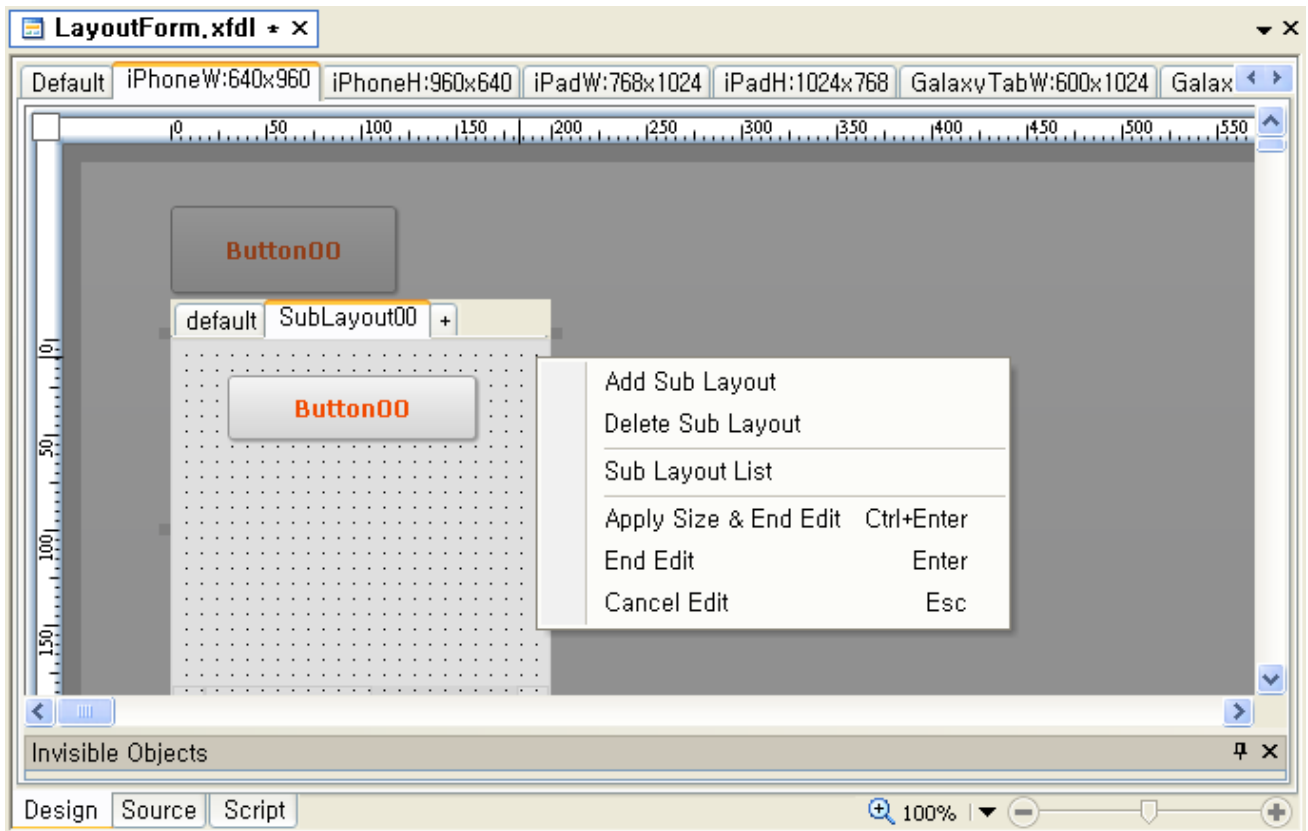


- Link Url을 가지고 있는 컴포넌트는 Sub Layout Editor를 실행 할 수 없습니다.
- Contents Editor와 마찬가지로 더블클릭으로도 실행할 수 있습니다.
- 최초 Sub Layout Editor 실행 시, 현재 Target 컴포넌트의 Default Size가 default Sub Layout의 Size가 됩니다

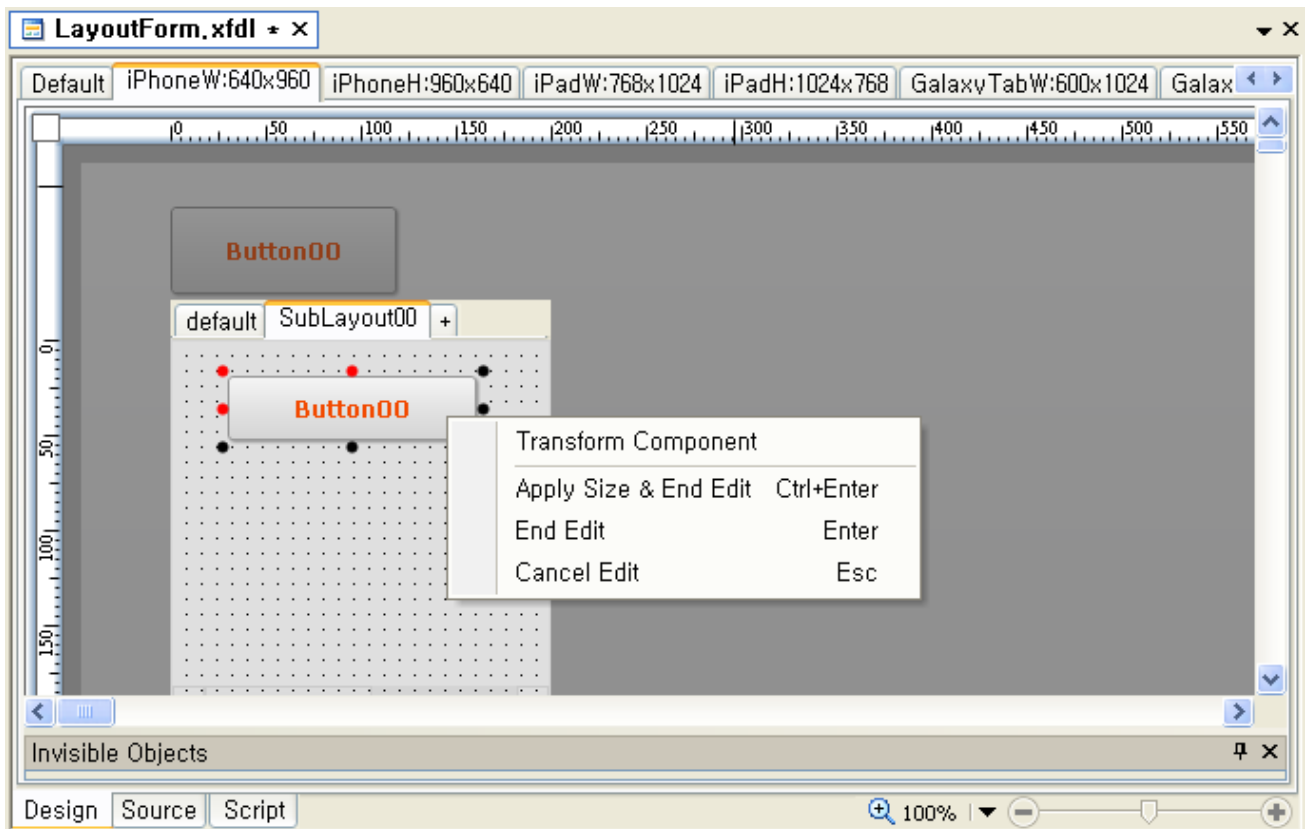
다.

- 해당 컴포넌트 및 하위 컴포넌트를 제외한 다른 컴포넌트는 선택 및 편집할 수 없습니다.
- Scroll에 가려져 보이지 않던 컴포넌트 들도 편집 가능합니다.
- 해당 컴포넌트의 Size에 맞는 Ruler가 제공됩니다.
- Sub Layout Editor 역시 Form Layout 처럼 Default를 제외한 Layout에서는 컴포넌트생성 등의 일부 기능이 제한됩니다.

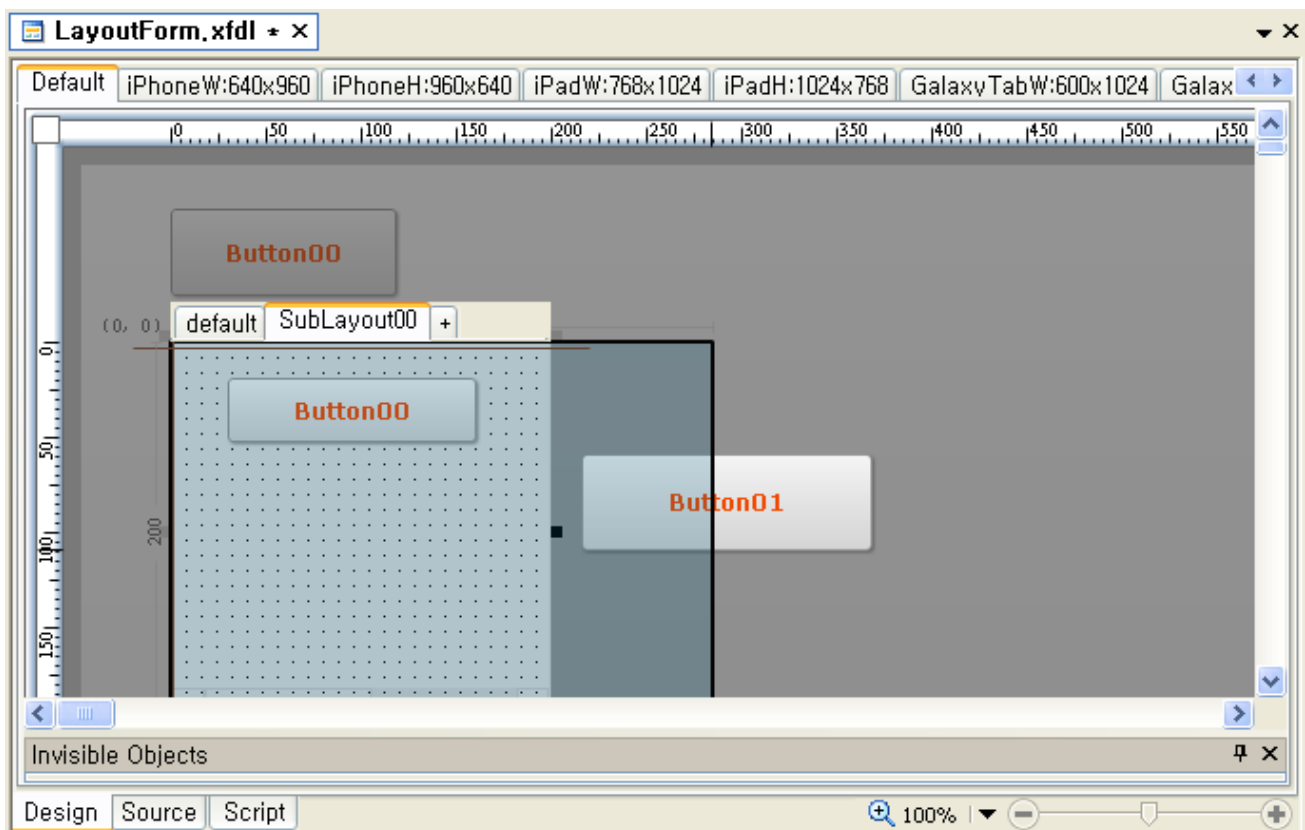
Sub Layout Editor만의 Popup Menu가 제공됩니다.



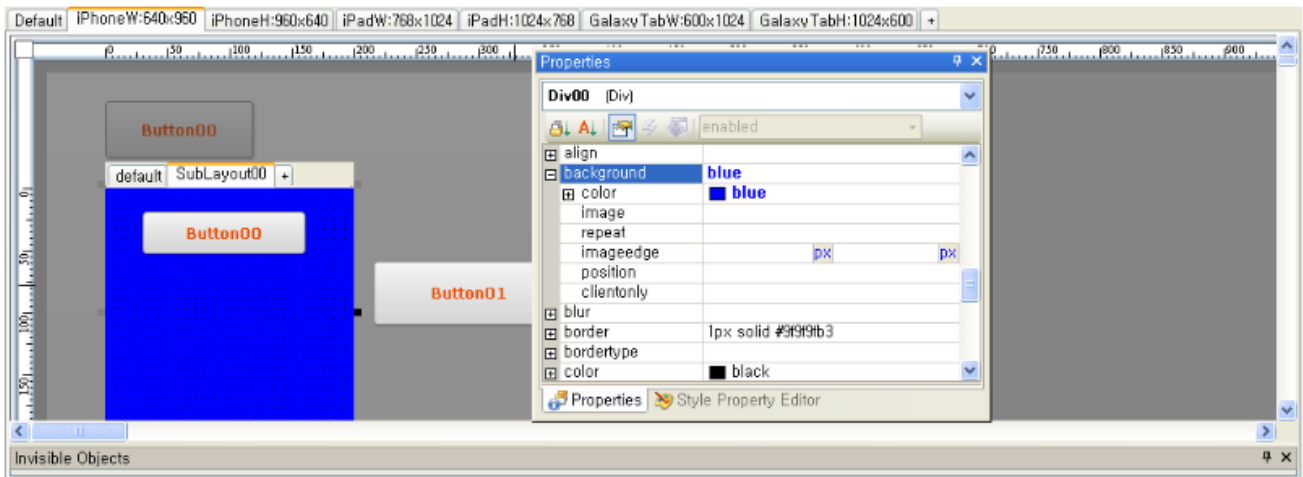
- Sub Layout List : 현재 컴포넌트에서 사용중인 Sub Layout정보를 수정할 수 있는 Dialog를 보여줍니다
- Apply Size & End Edit : 현재 edit 중인 sub Layout의 size로 target component를 설정하고 현재까지 Sub Layout Editor에서 작업한 undo buffer를 하나로 묶어서 design undo list에 추가합니다
- 컴포넌트를 선택하면 Popup Menu가 약간 다르게 제공됩니다.



Sub Layout의 Size를 변경 할 수는 있지만 Target 컴포넌트의 Position은 변하지 않습니다.



- 각 Sub Layout 마다 Target 컴포넌트의 Style 및 일부 속성을 설정 할 수 있습니다.
 - default Layout의 수정 내용은 <Div>등의 Tag내에 저장됩니다.
 - 그 외 하위 Sub Layout의 수정 내용은 해당 <Layout>에 저장됩니다.



```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <FDL version="1.3">
3    <TypeDefinition url="..\default_typedef.xml">
4    = <Form id="LayoutForm" classname="LayoutForm" inheritanceid="" position="absolute 0 0 1024 768" titletext="New Form">
5      <Layout width="1024" height="768">
6        <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:47 w:119 t:23 h:46" positiontype="position2"/>
7        = <Div id="Div00" taborder="1" position2="absolute 1:47 w:201 t:93 h:129" positiontype="position2" text="Div00">
8          <Layout width="292" height="200">
9            <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:29 w:131 t:18 h:34" positiontype="position2"/>
10           <Button id="Button01" taborder="1" text="Button01" position2="absolute 1:216 w:153 t:60 h:51" positiontype="position2"/>
11         </Layout>
12       </Div>
13     </Layout>
14     <Layout name="SubLayout00" width="200" height="200" style="background:blue;">
15       <Button id="Button01" position2="absolute 1:215 w:153 t:58 h:51"/>
16     </Layout>
17   </Form>
18   <Layout name="iPhoneW" screenid="iPhone" width="640" height="960" description="">
19   <Layout name="iPhoneH" screenid="iPhone" width="960" height="640" description="">
20   <Layout name="iPadW" screenid="iPad" width="768" height="1024" description="">
21   <Layout name="iPadH" screenid="iPad" width="1024" height="768" description="">
22   <Layout name="GalaxyTabW" screenid="GalaxyTab" width="600" height="1024" description="">
23   <Layout name="GalaxyTabH" screenid="GalaxyTab" width="1024" height="600" description="">

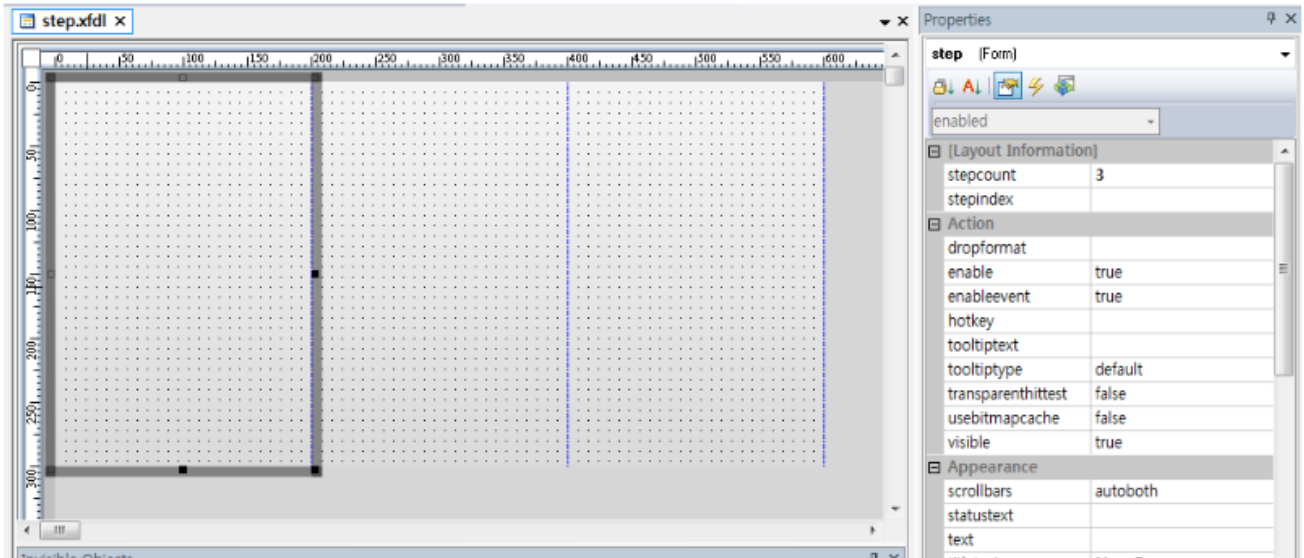
```

- 기타 Sub Layout Editor의 조작관련 사항
 - 각 Sub Layout 안에 있는 하위 Contents를 가진 컴포넌트는 Form Design과 마찬가지로 default Layout에서만 편집 가능합니다.
 - Sub Layout Editor 내에서 또 다른 Sub Layout Editor를 실행하지 못합니다.
 - Contents Editor는 실행 할 수 있습니다.
 - 그 외 컴포넌트의 조작 같은 경우, Form Design과 동일하게 동작합니다.
- Sub Layout Undo
 - Sub Layout Editor에서 Undo는 Form Design과 따로 관리됩니다.
 - Sub Layout Editor 실행 전의 Undo는 Sub Layout Editor 안에서 사용 할 수 없습니다.
 - Sub Layout Editor 안에서 누적된 Undo는 Editor 종료와 동시에 사라지거나 한꺼번에 Form Design Undo List에 추가됩니다. (Sub Layout Editor 안의 편집 내용을 하나의 Undo로 인식)

9.5 Step

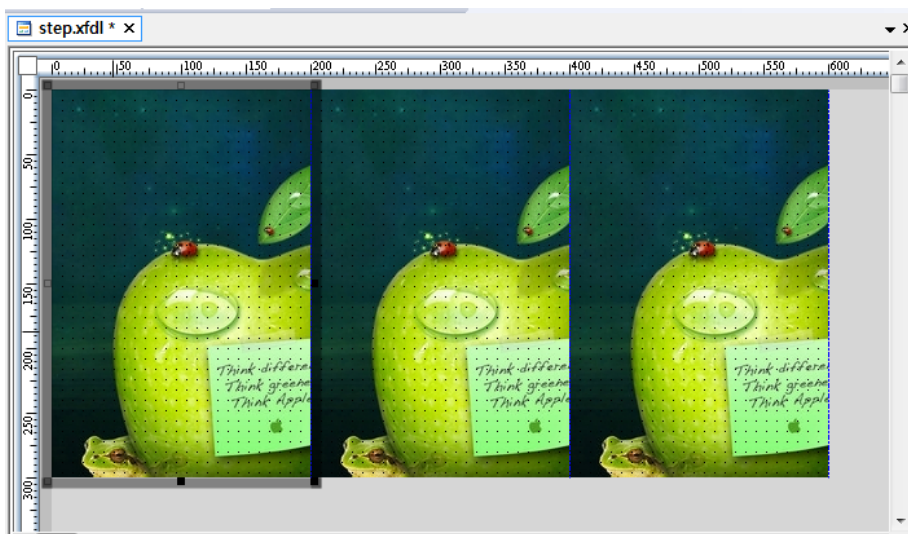
Step이란 여러 개의 step으로 이루어진 페이지를 하나의 Form에서 개발할 수 있는 기능을 의미합니다.

Form의 stepcount Property 값을 조정하면 아래그림과 같이 step이 표시됩니다.



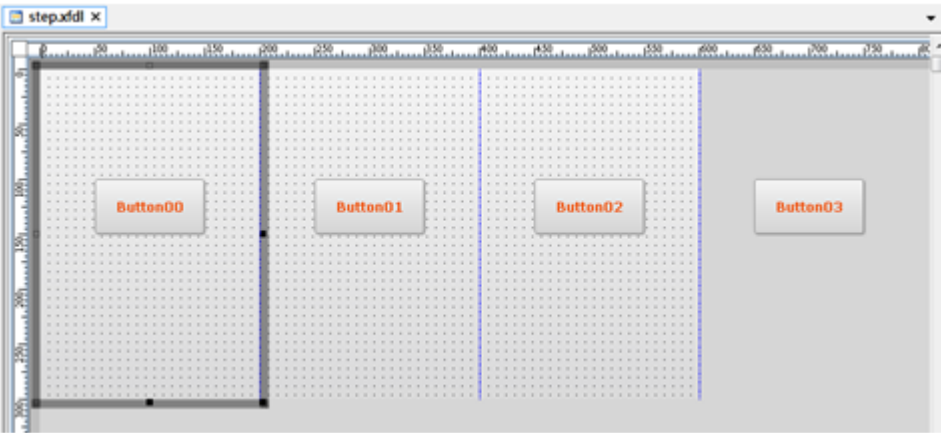
번호	설명
1	현재 편집중인 Step을 표시. 현재 편집중인 Step은 마우스포인터의 위치로 판단합니다.
2	Step과 Step 사이의 안내선

- 여러 개의 step이 있다고 해서 Form의 실질적인 Size가 변하지 않습니다.
- 실행 시에는 1개의 Step만 보여지게 됩니다.
- Form의 배경은 Step마다 적용 됩니다.



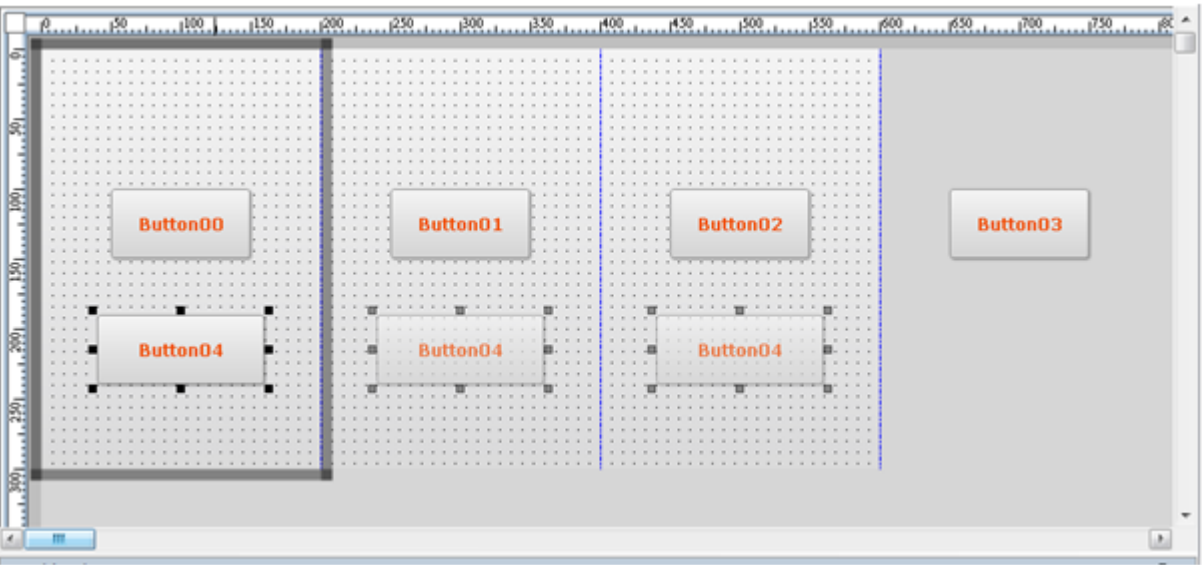
9.5.1 Positionstep Property

- 컴포넌트에 positionstep Property가 추가 되었습니다. positionstep Property는 해당 컴포넌트의 소속 Step을 나타냅니다.
- 컴포넌트의 Position이 positionstep + Position으로 적용됩니다.
- 컴포넌트의 Position은 지정된 positionstep 을 기준으로 설정됩니다.



버튼	positionstep + Position
Button00	positionstep="0" position="absolute 50 100 150 150"
Button01	positionstep ="1" position="absolute 50 100 150 150"
Button03	positionstep ="2" position="absolute 50 100 150 150"
Button03	positionstep ="0" position="absolute 650 100 750 150"

- 컴포넌트가 모든 Step영역을 벗어나면 positionstep 0으로 처리됩니다.
- positionstep이 -1일 경우에는 모든 Step에 표시됩니다.

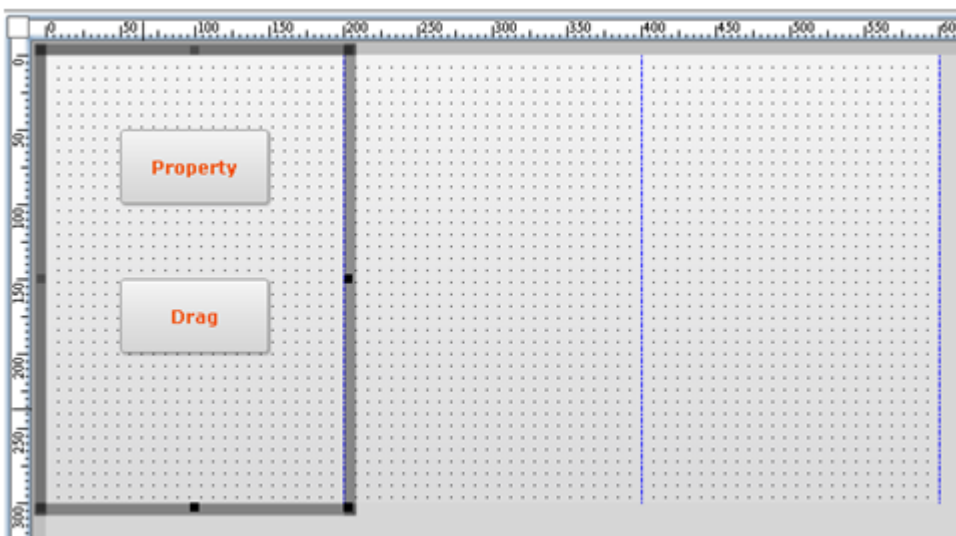


- -1 positionstep 컴포넌트는 모든 Step 영역에서 선택, 편집 할 수 있습니다
- -1 positionstep 컴포넌트는 현재 편집중인 Step이 아닌 경우, 실제 컴포넌트보다 약간 투명하게 보여집니다.
- -1 positionstep 컴포넌트는 모든 Step에서 해당 Step에서 보여지는 컴포넌트를 선택 및 편집 할 수 있습니다.

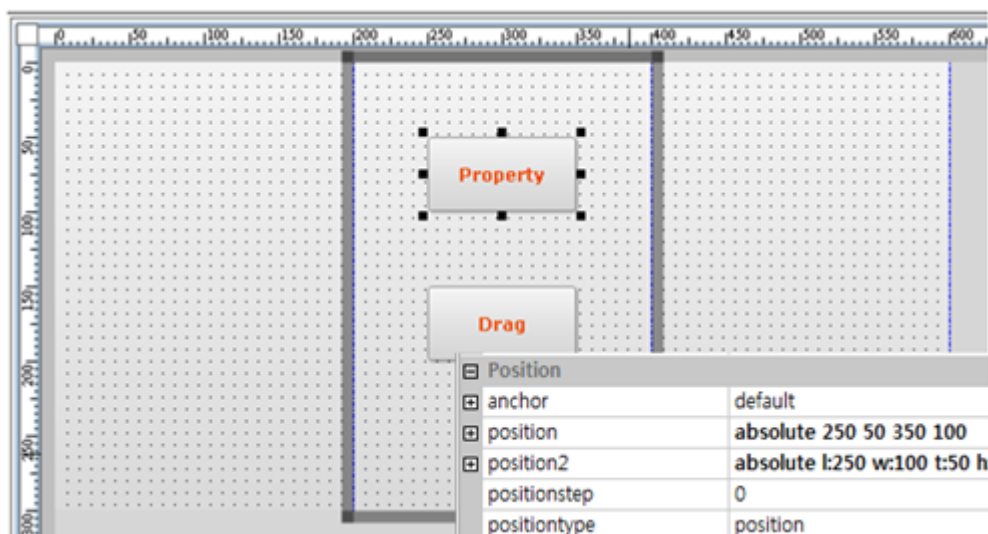
9.5.2 컴포넌트 이동 및 Position 변환

컴포넌트의 Step간 이동은 Property를 직접 입력하거나 Design화면에서 직접 이동할 수 있습니다.

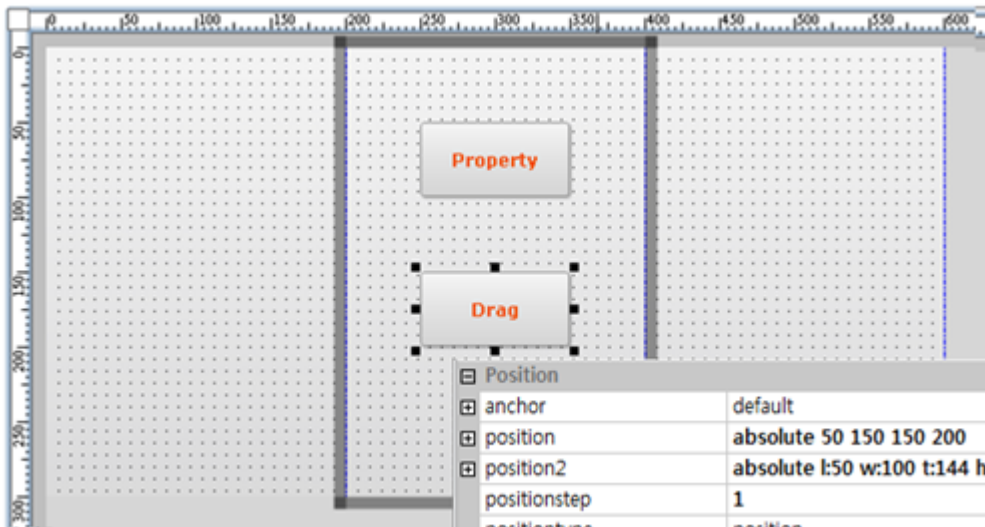
- Property 입력 - 입력 값 유지, positionstep 및 Position 재조정 없음 (현재 step에서 벗어날 수 있음)
- Design 이동 - Design 화면에서 직접 이동된 Position 정보를 현재 Step에 맞게 변환하여 재적용



(이동 전)



(Position Property 를 직접 입력하여 이동한 경우)



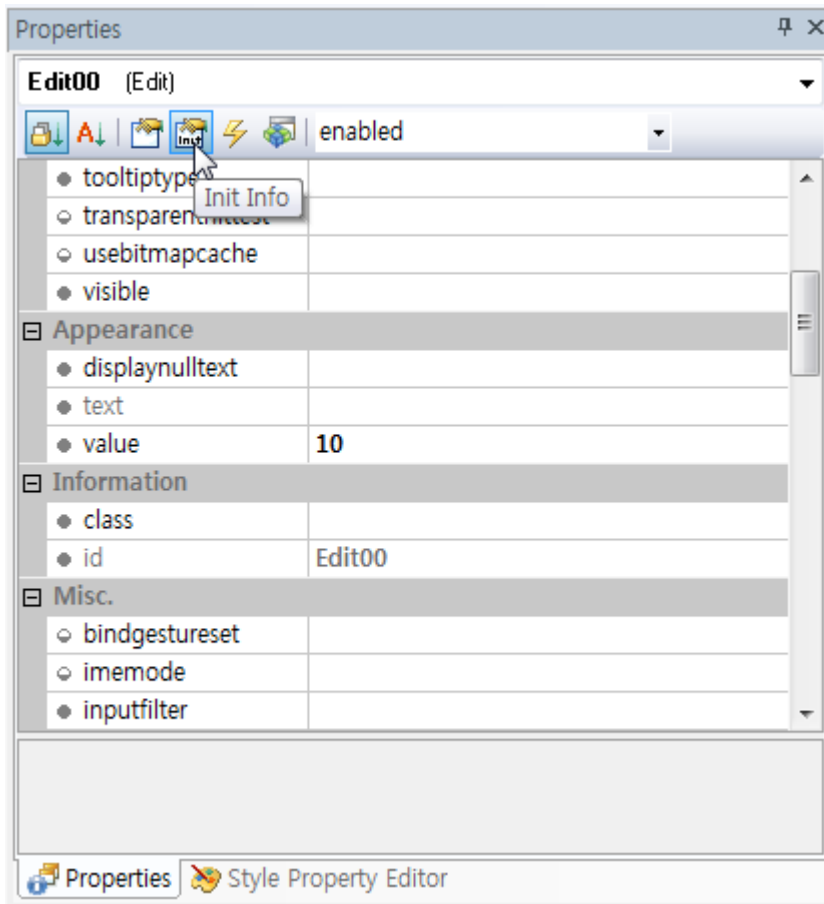
(컴포넌트를 직접 Design 화면에서 이동한 경우)

9.6 InitValue

MLM을 적용한 애플리케이션이 특정한 값을 가진 상태에서 시작해야 하는 경우가 있습니다. 예를 들어 각 레이아웃마다 다른 디자인 속성을 가지는 경우 레이아웃마다 다른 속성값을 지정합니다.

하지만 사용자가 입력하거나 변경할 수 있는 속성은 레이아웃이 변한다고 해서 입력되거나 수정된 값이 처음 설정한 값으로 변경대서는 안됩니다. 이런 항목에 속성값을 직접 지정하지 않고 InitValue를 선언해주면 처음 애플리케이션을 호출할 때만 해당 값을 사용하고 레이아웃 변경과 상관없이 수정된 값을 유지할 수 있습니다.

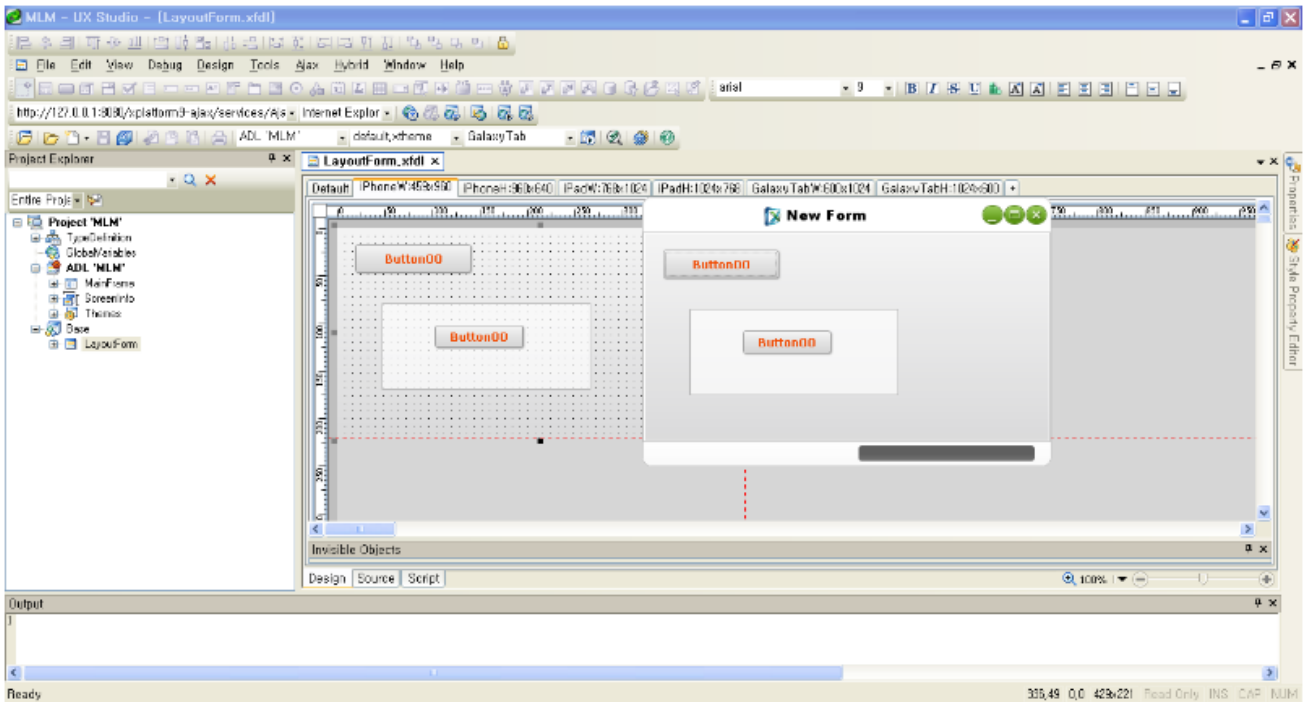
- UX스튜디오 속성창에 Init Info라는 항목이 추가되었습니다. 필요할 때 아래 그림과 같이 InitValue를 적용할 수 있습니다. 수정된 값은 xfdl 파일에 <InitValue> 태그로 추가됩니다.



- InitValue로 지정된 값은 품이 로딩되기 전에 적용되며 속성값은 품이 로딩되면서 적용됩니다. InitValue와 속성 값을 같이 지정하는 경우에는 속성값이 InitValue를 덮어쓰게 됩니다.
- 레이아웃에 따라 InitValue를 따로 지정할 수는 없습니다.
- 컴포넌트의 속성값이 임의로 수정되지 않은 상태(Default Value)일 때는 레이아웃 변경이 속성값에 영향을 미치지 않습니다.

9.7 실행

- Quick View는 현재 활성화된 Layout Tab을 기준으로 보여주도록 되어 있습니다.
- Launch Project는 Standard Toolbar에서 선택한 Screen정보를 기준으로 보여주도록 되어 있습니다.
- Screen정보를 입력하지 않거나 ADL에 없는 Screen일 경우 Active ADL의 Theme를 적용하여 보여주도록 되어 있습니다.



9.8 XML 추가/변경 사항

9.8.1 XPRJ

‘Layout Template’기능으로 생성되는 정보는 XPRJ파일에 저장하도록 되어 있습니다.

```
<?xml version="1.0" encoding="utf-8"?>
<Project active_adl="MLM" version="1.1">
  <TypeDefinition url="default_typedef.xml"/>
  <GlobalVariables url="globalvars.xml"/>
  <ADL id="MLM" url="MLM.xadl"/>
  <Options positiontype="position2"/>
  <LayoutTemplate>
    <ScreenGroup name="ScreenGroup00" description="">
      <Layout name="Template00" width="1024" height="768" description=""/>
    </ScreenGroup>
  </LayoutTemplate>
</Project>
```

9.8.2 XADL

Project생성시 또는 ScreenInfo Edit로 생성되는 정보는 XADL파일에 저장됩니다.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <ADL version="1.0">
3    <TypeDefinition url="default_typedef.xml"/>
4    <GlobalVariables url="globalvars.xml"/>
5    <Application id="MLM" codepage="utf-8" language="Korean" loginformurl="" loginformstyle="" windowopeneffect="" windowcloseeffect="" versi
6    = <Layout>
7      <MainFrame id="mainframe" title="maintitle" defaultfont="" resizable="true" showtitlebar="true" showstatusbar="true" position="absolu
8      <ChildFrame id="childframe" formurl="" showtitlebar="false" showstatusbar="false" openeffect="" closeeffect=""/>
9    </MainFrame>
10   </Layout>
11   = <ScreenInfo>
12     <Screen name="iPhone" theme="default.xtheme" description="iPhone4"/>
13     <Screen name="iPad" theme="black.xtheme" description="iPad2"/>
14     <Screen name="GalaxyTab" theme="blue.xtheme" description="GalaxyTab"/>
15   </ScreenInfo>
16 </Application>
17 </ADL>
18

```

9.8.3 XFDL

- Form의 Layout정보 및 컴포넌트의 속성값들은 XFDL에 저장됩니다.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <FDL version="1.3">
3    <TypeDefinition url="..\default_typedef.xml"/>
4    = <Form id="LayoutForm" classname="LayoutForm" inheritanceid="" position="absolute 0 0 1024 768" titletext="New Form">
5      <Layout width="1024" height="768"/>
6      <Layout name="iPhoneW" screenid="iPhone" width="640" height="960" description=""/>
7      <Layout name="iPhoneH" screenid="iPhone" width="960" height="640" description=""/>
8      <Layout name="iPadW" screenid="iPad" width="768" height="1024" description=""/>
9      <Layout name="iPadH" screenid="iPad" width="1024" height="768" description=""/>
10     <Layout name="GalaxyTabW" screenid="GalaxyTab" width="600" height="1024" description=""/>
11     <Layout name="GalaxyTabH" screenid="GalaxyTab" width="1024" height="600" description=""/>
12   </Form>
13 </FDL>
14

```

- 'Default'탭에서 Form의 속성이 변경된 경우는 <Form>태그에 저장됩니다.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <FDL version="1.3">
3    <TypeDefinition url="..\default_rvmedef.xml"/>
4    = <Form id="LayoutForm" classname="LayoutForm" inheritanceid="" position="absolute 0 0 833 768" titletext="New Form">
5      <Layout width="833" height="768">
6        <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:19 w:122 t:17 h:30" positiontype="position2"/>
7        <Div id="Div00" taborder="1" position2="absolute 1:47 w:220 t:79 h:91" positiontype="position2" text="Div00">
8          <Layout width="220" height="91">
9            <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:55 w:93 t:22 h:24" positiontype="position2"/>
10          </Layout>
11          <Layout name="SubLayout00" width="200" height="200"/>
12        </Div>
13      </Layout>
14      <Layout name="iPhoneW" screenid="iPhone" width="459" height="960" description=""/>
15      <Layout name="iPhoneH" screenid="iPhone" width="960" height="640" description=""/>
16      <Layout name="iPadW" screenid="iPad" width="768" height="1024" description=""/>

```

- 추가된 Layout 탭에서 Form의 속성이 변경된 경우 해당하는 <Layout>태그에 저장됩니다

```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <FDL version="1.3">
3    <TypeDefinition url="..\default_typedef.xml"/>
4    = <Form id="LayoutForm" classname="LayoutForm" inheritanceid="" position="absolute 0 0 833 768" titletext="New Form">
5      = <Layout width="833" height="768">
6        <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:19 w:122 t:17 h:30" positiontype="position2"/>
7      = <Div id="Div00" taborder="1" position2="absolute 1:47 w:220 t:79 h:91" positiontype="position2" text="Div00">
8        = <Layout width="220" height="91">
9          <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:55 w:93 t:22 h:24" positiontype="position2"/>
10         </Layout>
11       <Layout name="SubLayout00" width="200" height="200"/>
12     </Div>
13   </Layout>
14   <Layout name="iPhoneW" screenid="iPhone" width="459" height="960" description="" />
15   <Layout name="iPhoneH" screenid="iPhone" width="960" height="640" description="" />
16   <Layout name="iPadW" screenid="iPad" width="768" height="1024" description="" />

```

- 추가된 Layout 탭에서 컴포넌트의 속성이 변경된 경우 해당하는 <Layout>태그의 하위에 변경된 속성값만 저장됩니다.

```

1  <?xml version="1.0" encoding="utf-8"?>
2  = <FDL version="1.3">
3    <TypeDefinition url="..\default_typedef.xml"/>
4    = <Form id="LayoutForm" classname="LayoutForm" inheritanceid="" position="absolute 0 0 833 768" titletext="New Form">
5      = <Layout width="833" height="768">
6        <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:19 w:122 t:17 h:30" positiontype="position2"/>
7      = <Div id="Div00" taborder="1" position2="absolute 1:47 w:220 t:79 h:91" positiontype="position2" text="Div00">
8        = <Layout width="220" height="91">
9          <Button id="Button00" taborder="0" text="Button00" position2="absolute 1:55 w:93 t:22 h:24" positiontype="position2"/>
10        </Layout>
11      <Layout name="SubLayout00" width="200" height="200"/>
12    </Div>
13  </Layout>
14  <Layout name="iPhoneW" screenid="iPhone" width="459" height="960" description="" />
15  <Layout name="iPhoneH" screenid="iPhone" width="960" height="640" description="" />
16  <Layout name="iPadW" screenid="iPad" width="768" height="1024" description="" />

```

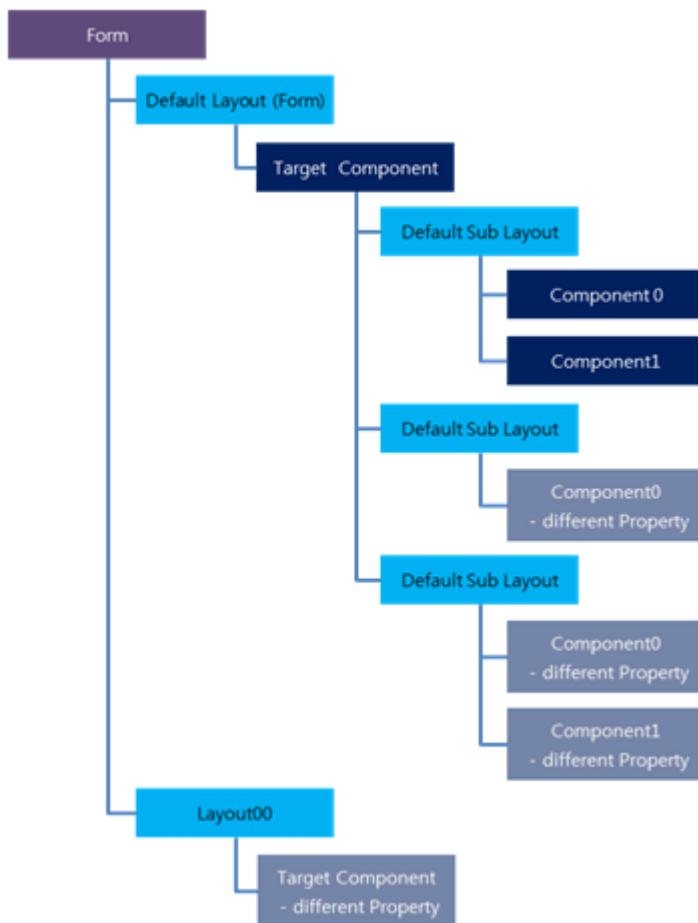
- InitValue가 추가되면 <InitValue> 태그로 저장됩니다. Form에 대한 InitValue를 지정하는 경우에는 InitValue 태그 자체 속성으로 처리되고 각 컴포넌트의 InitValue는 <InitValue> 태그 안에 변경된 속성값이 처리됩니다.

```

19      <Edit id="Edit00" value="20"/>
20    </Layout>
21  </Layouts>
22  = <InitValue position="absolute 0 0 600 400" text="DDDD" style="color:black;">
23    <Button id="Button00" style="color:black;" />
24    <Edit id="Edit00" value="10"/>
25  </InitValue>
26 </Form>
27 </FDL>

```

- Sub Layout
 - 기본적으로 Form의 XML 구조와 동일한 구조를 갑습니다. Target 컴포넌트의 하위로 default Layout이 있고 그 밑으로 Sub Layout의 Tag가 있습니다.



- 각각의 Sub Layout 에서 수정된 내용은 해당 Sub Layout Tag에 Multi Layout Manger 형식으로 저장됩니다.

```

<Div id="Div00" taborder="1" position2="absolute l:47 w:292 t:93 h:200" positiontype="position2" text="Div00">
  <Layout width="292" height="200">
    <Button id="Button00" taborder="0" text="Button00" position2="absolute l:29 w:131 t:18 h:34" positiontype="position2"/>
    <Button id="Button01" taborder="1" text="Button01" position2="absolute l:216 w:153 t:60 h:51" positiontype="position2"/>
  </Layout>
  <Layout name="SubLayout00" width="200" height="200" style="background:blue;">
    <Button id="Button01" position2="absolute l:215 w:153 t:58 h:51"/>
  </Layout>
</Div>
  
```

10.

Position

10.1 positiontype 속성

position2 속성이 추가되면서 컴포넌트에서 어떤 속성을 적용할 지 판단하는 속성으로 positiontype 속성값을 추가했습니다. positiontype 속성값에 따라 사용하지 않는 속성은 UX-Studio 속성창에서 비활성화 상태로 변경됩니다.

Position	
anchor	none
position	absolute 144 330 276 384
position2	absolute l:144 w:132 t:330 h:54
positionstep	0
positiontype	position
rtldirection	inherit

UX-Studio 메뉴[Tools > Options]에서 옵션 항목 중 [Form Design > Position] 항목에서 default 값으로 적용할 positiontype 속성값을 지정할 수 있습니다. 해당 설정에 따라 컴포넌트를 Form에 배치했을 때 positiontype 속성값이 지정됩니다. 해당 설정값은 default 값을 지정하는 것일 뿐 컴포넌트의 positiontype은 변경할 수 있습니다.

10.2 Position & Anchor

컴포넌트가 화면에 표시될 위치와 크기를 지정하기 위해 position 속성값을 지정합니다. 컴포넌트가 표시될 x, y 좌표값을 지정하고 컴포넌트의 높이(height)와 너비(width)를 지정할 수 있습니다. x, y 좌표값 대신 left, top 속성값을 지정해 부모 컴포넌트(Form, Div 등)로부터 떨어진 거리를 지정할 수도 있습니다.

부모 컴포넌트의 크기가 고정된 경우에는 컴포넌트의 위치와 크기를 변경할 필요가 없습니다. 하지만 화면 크기가 가변적인 경우에는 배치된 컴포넌트의 위치와 크기를 적절하게 재조정해주어야 합니다. position2 속성을 사용하는 경우에는 위치나 크기를 부모 컴포넌트 대비 비율(%)로 지정할 수 있어 부모 컴포넌트의 크기가 변경되었을 때 배치된 컴포넌트의 크기를 조정할 수 있습니다. 하지만 position 속성값을 지정하는 단위는 픽셀(px)만 지원합니다.

그래서 position 속성값을 사용하는 경우에도 부모 컴포넌트의 위치와 크기 조정에 대응할 수 있도록 anchor 속성을

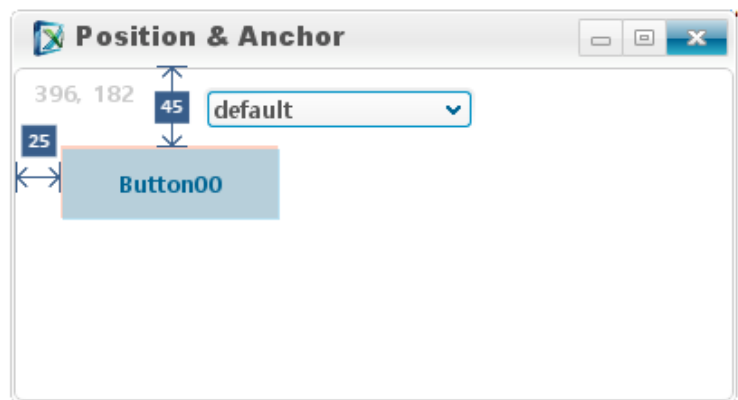
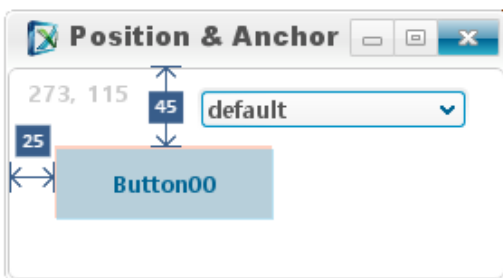
지원하고 있습니다. anchor 속성은 컴포넌트와 부모 컴포넌트 간의 거리를 고정할지 여부를 지정하는 속성입니다. left, top, right, bottom 4가지 속성을 조합해 어떤 식으로 간격을 유지할지 결정합니다.



UX-Studio 속성창에서 anchor 속성은 positiontype 속성값이 "position"인 경우에만 활성화됩니다. position2 속성값에 따라 anchor 속성값이 변경되긴 하지만 positiontype 속성값을 변경할 경우 비슷한 조건을 맞추기 위해 UX-Studio 내부에서 처리하는 것입니다.

10.2.1 default

anchor 속성값을 default로 지정한 경우에는 속성값을 "left top"으로 지정한 것과 같은 방식으로 처리합니다. 컴포넌트의 x, y 좌표값이 고정된 상태에서 컴포넌트의 다른 position 속성은 변경되지 않습니다. 그래서 부모 컴포넌트의 위치나 크기가 변경되어도 항상 같은 위치를 유지합니다.

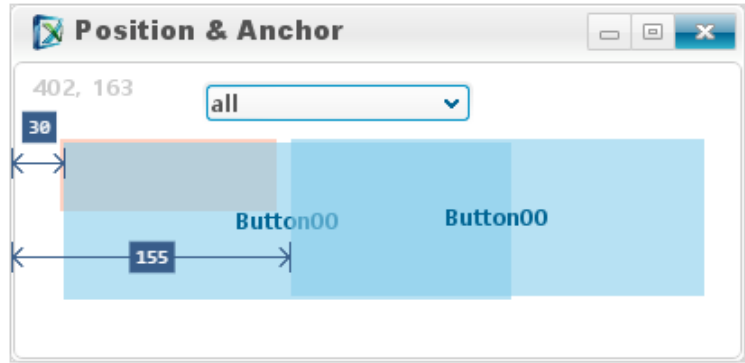
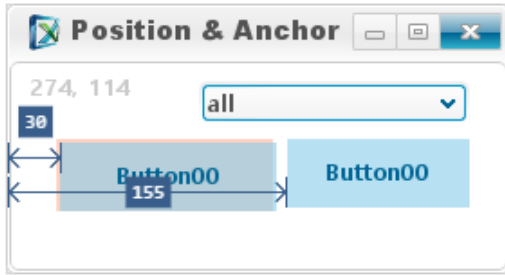


속성값	설명
default	left top 속성값으로 처리합니다.
left top	컴포넌트의 x, y 좌표값은 고정됩니다. 부모 컴포넌트의 위치, 크기와 상관없이 같은 위치, 크기에 고정됩니다.

10.2.2 left top 고정

속성값을 default로 지정했을 때처럼 "left top"이 고정된 경우에는 컴포넌트의 x, y 좌표값은 변경되지 않습니다. 추가로 지정된 속성값에 따라 컴포넌트의 너비, 높이가 커지게 됩니다.

컴포넌트의 x, y 좌표값이 고정되기 때문에 다른 컴포넌트와 가까이 붙어서 배치된 경우에는 컴포넌트가 겹쳐서 표기될 수 있습니다.

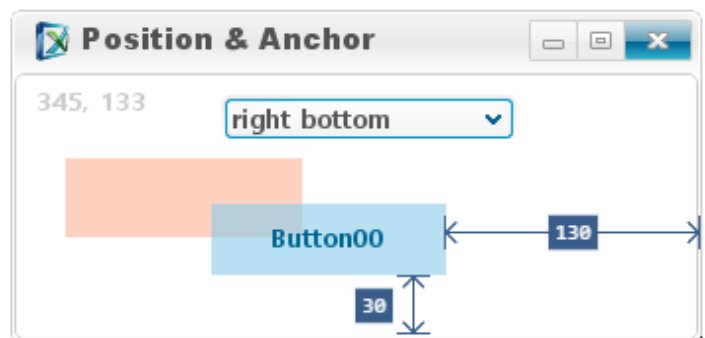
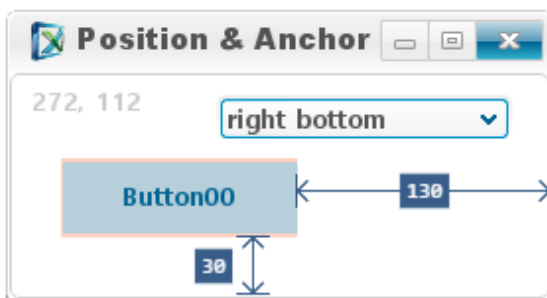


속성값	설명
all	left top right bottom 속성값으로 처리합니다. 부모 컴포넌트의 크기에 비례해 컴포넌트의 크기가 변경됩니다.
left top right	부모 컴포넌트의 너비 변경에 비례해 컴포넌트의 너비가 변경됩니다. 컴포넌트의 높이는 변경되지 않습니다.
left top bottom	부모 컴포넌트의 높이 변경에 비례해 컴포넌트의 높이가 변경됩니다. 컴포넌트의 너비는 변경되지 않습니다.

10.2.3 속성값에 따라 변경되는 요소

속성값으로 지정되는 값은 컴포넌트를 잡아당겨 고정하고 있는 면이라고 볼 수 있습니다. 예를 들어 left top 방향에서 컴포넌트를 잡고 있으면 x, y 좌표값은 고정된 상태에서 나머지 속성에 따라 컴포넌트의 크기가 변합니다. 반대로 left, top 방향에서 컴포넌트를 잡고 있지 않다면 컴포넌트는 기존 위치에서 벗어나게 됩니다.

아래 그림은 anchor 속성값을 "right bottom"으로 지정했습니다. 부모 컴포넌트의 크기 변경에 따라 컴포넌트의 x, y 좌표값이 이동하는 것을 확인할 수 있습니다. 빨간색으로 표시된 부분은 원래 컴포넌트의 위치를 표시한 것입니다.



부모 컴포넌트의 크기가 커질 때 자식 컴포넌트의 크기는 작아지는 경우도 있습니다. 예를 들어 anchor 속성값이 "left"인 경우 x 좌표값은 고정된 상태지만 나머지 방향에서는 기존 상태를 유지합니다. y 좌표값은 아래로 이동하고 bottom, right 방향으로 컴포넌트의 크기가 변경되지 않습니다. 컴포넌트의 크기는 그대로 유지하려고 하고 y 좌표값이

아래로 이동하다 보니 컴포넌트의 높이값이 감소하게 됩니다.



아래 표는 각 속성값에 따라서 변하는 x, y 좌표값과 컴포넌트의 크기 변화를 정리한 내용입니다.

속성값	x 좌표값	y 좌표값	너비	높이
default (left top)	X	X	X	X
all	X	X	O	O
none	O	O	O(감소)	O(감소)
left top right	X	X	O	X
left top bottom	X	X	X	O
top right bottom	O	X	X	O
left right	X	O	O	O(감소)
left bottom	X	O	X	X
top right	O	X	X	X
top bottom	O	X	O(감소)	O
right bottom	O	O	X	X
left	X	O	X	O(감소)
top	O	X	O(감소)	X
right	O	O	X	O(감소)
bottom	O	O	O(감소)	X

10.3 Position2 (V9.2 추가)

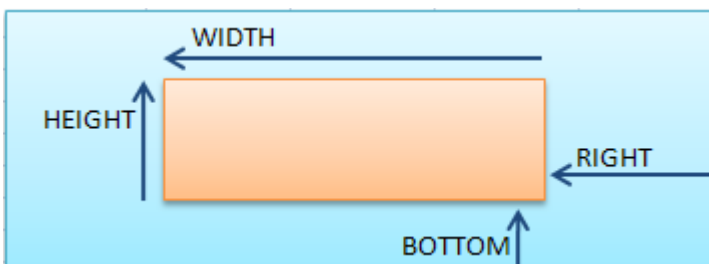
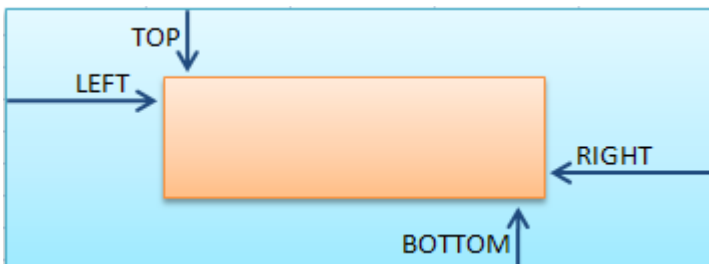
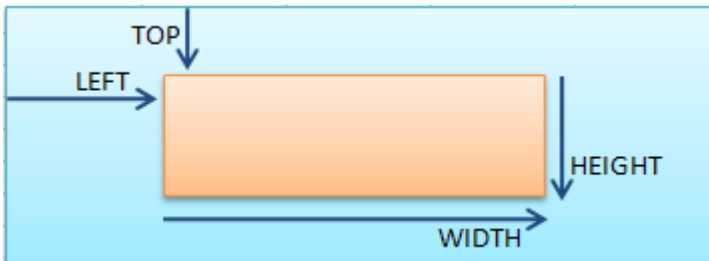
position2란 비율인 “%”로 Position을 지정할 수 있도록 하는 속성을 의미합니다. UX-Studio에서 Ruler, DotGrid (화면 바닥에 보이는 점들)등의 Position2 관련 편집기능을 제공합니다.

10.3.1 좌표계 설명

기존 position의 좌표는 화면의 left와 top을 기준으로 하여 기준점에서 떨어진 pixel 거리를 좌표 정보로 사용합니다.

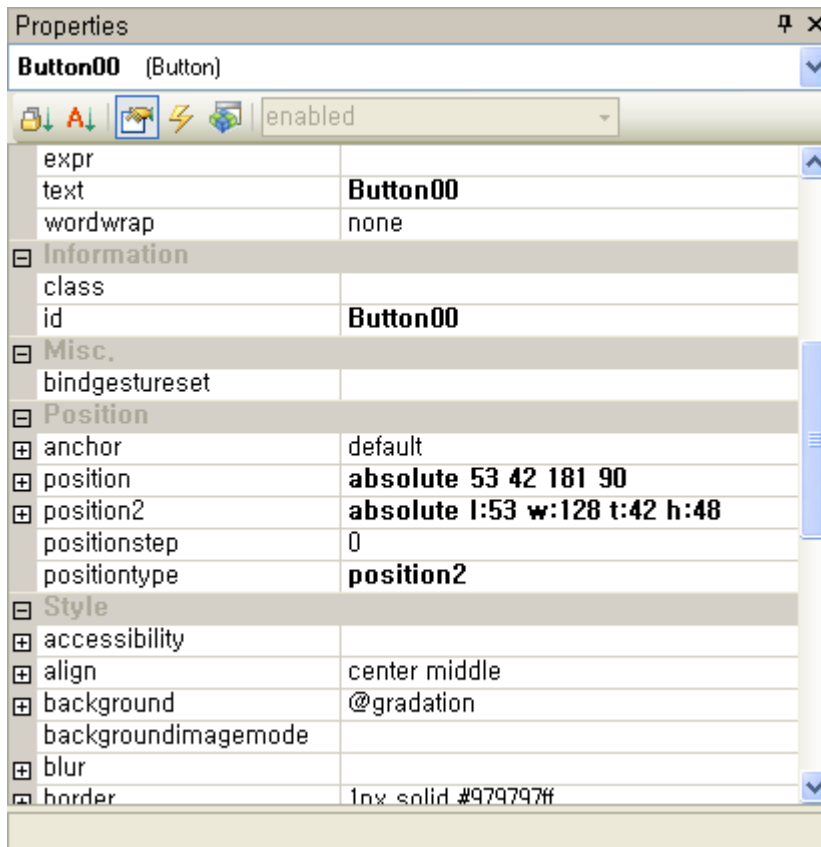


V9.2에서 추가된 position2는 기존 좌표계의 표현 한계를 극복하기 위해 도입된 좌표계로서 아래그림과 같이 사용자가 입력한 left, right, width, top, bottom, height 중 4개를 입력 받아 표현됩니다.

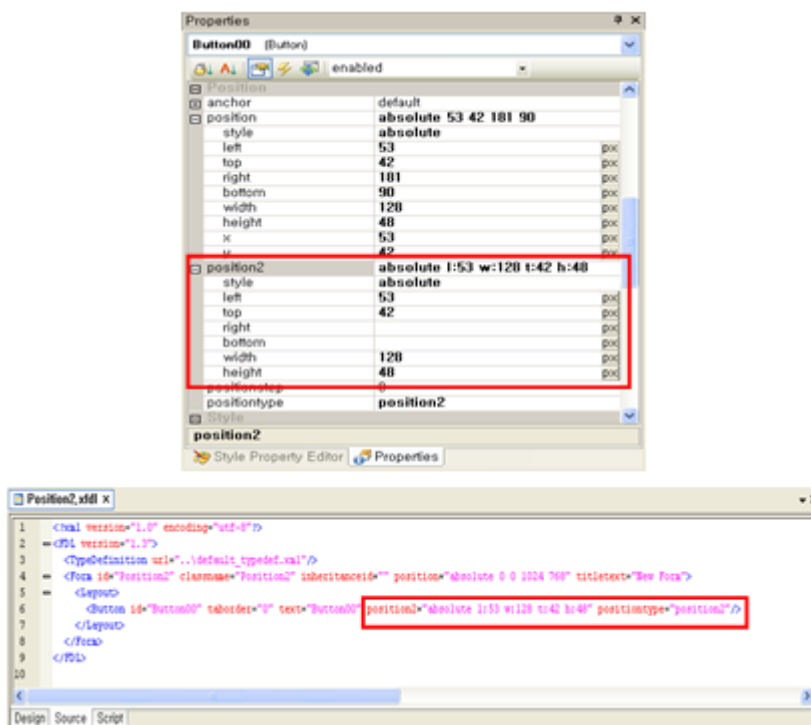


10.3.2 컴포넌트의 Position2 설정

컴포넌트의 Property로 position2와 positiontype이 추가되었으며, 컴포넌트의 좌표계는 positiontype으로 결정됩니다.



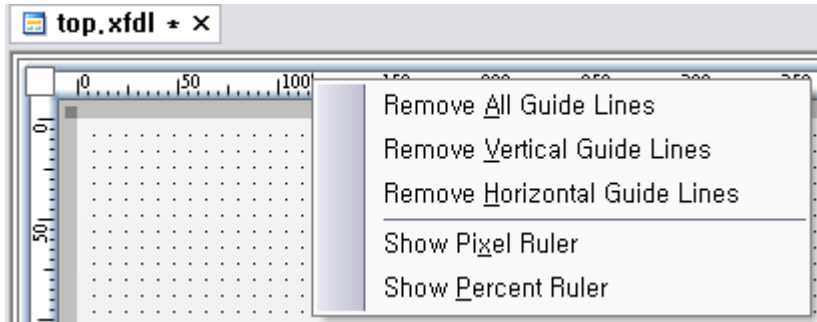
Properties Window에서는 position, position2 양쪽 모두 수정이 가능하지만, XFDL의 Source에는 positiontype에서 지정된 좌표계로 저장됩니다.



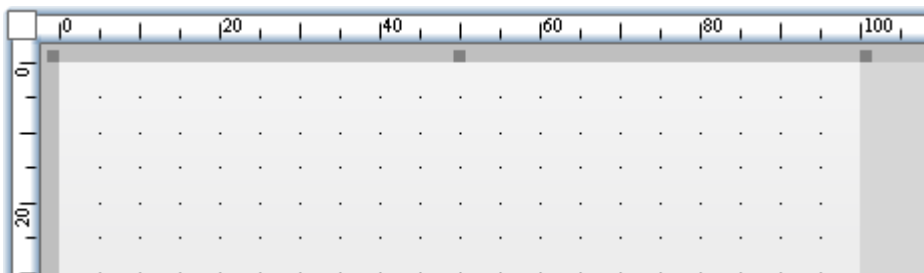
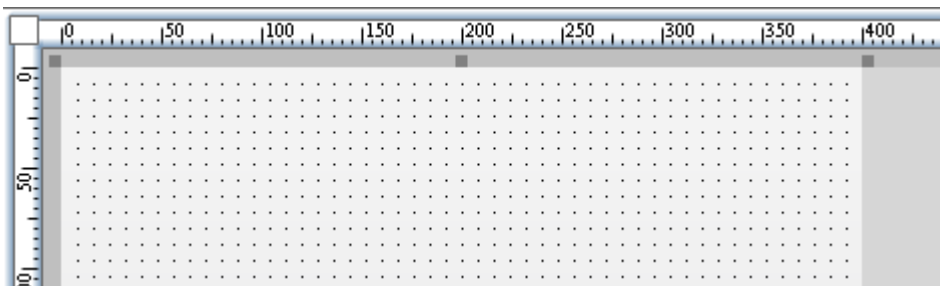
Properties Window에서는 추가된 Position2의 단위를 쉽게 변경할 수 있도록 px, %를 지원합니다.

Ruler / Dot Grid

Position2는 Percent입력을 지원하기 때문에, Form Design상에서 손쉽게 확인할 수 있도록 Percent단위로 정보를 표시해 주는 기능이 추가되었습니다.

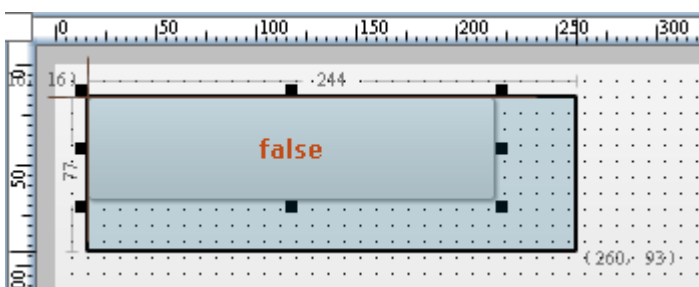


Ruler의 Popup Menu에서 Show Pixel Ruler / Show Percent Ruler를 선택할 경우 해당 단위로 Ruler와 Dot Grid의 표시 방법이 변경됩니다.

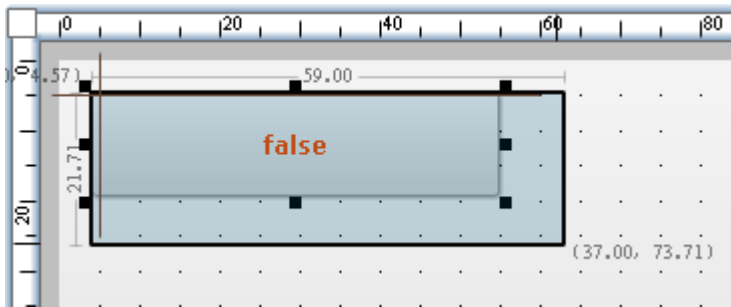


컴포넌트 Resize Info

컴포넌트를 한 개만 선택한 상태에서 Tracker로 Resize를 하는 경우, Resize되는 정보를 표시해 줍니다.

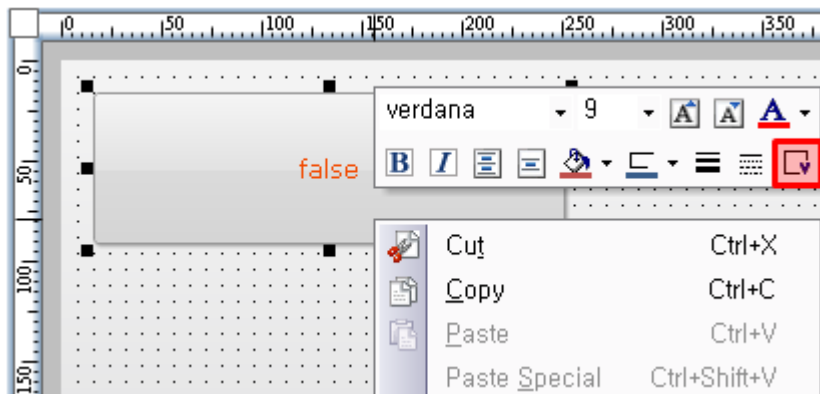


만약 Ruler단위가 Percent로 표시 중이라면 Resize Info도 Percent 단위로 표시됩니다.

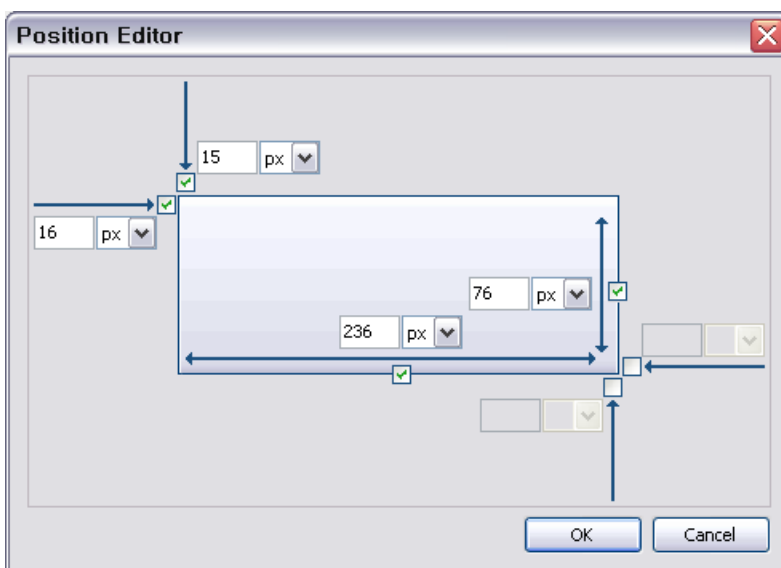


Position Editor

사용자가 쉽게 Position정보를 설정할 수 있도록 UI를 가진 Editor를 제공합니다. Position Editor는 Form Design의 Minitoolbar에서 호출할 수 있습니다.



Minitoolbar는 XPLATFORM 9.2 버전에서 추가된 기능으로 자주 사용되는 Style Property를 메뉴를 사용하여 손쉽게 수정할 수 있는 기능입니다.

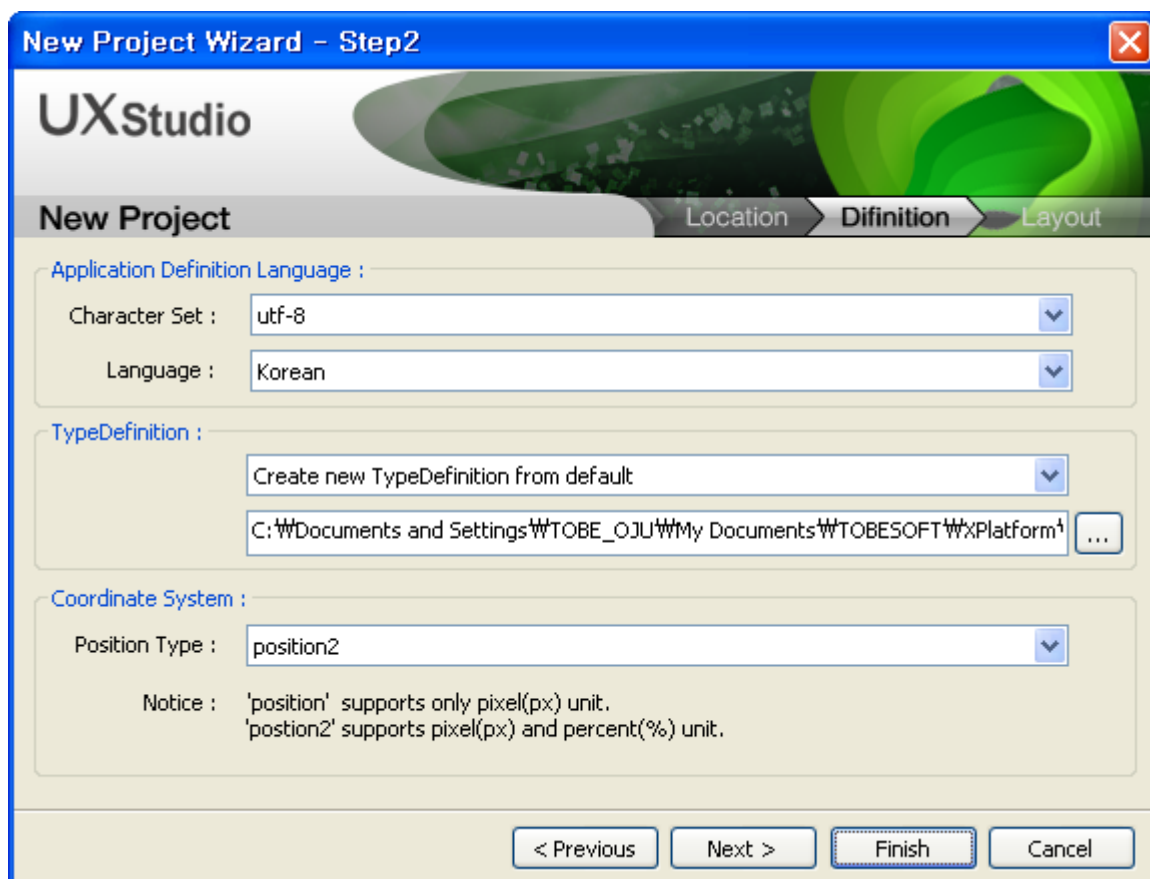


Position Editor에서 수정된 값은 대상 컴포넌트가 사용중인 positiontype에 맞는 position, position2값으로 변경되어 반영됩니다.

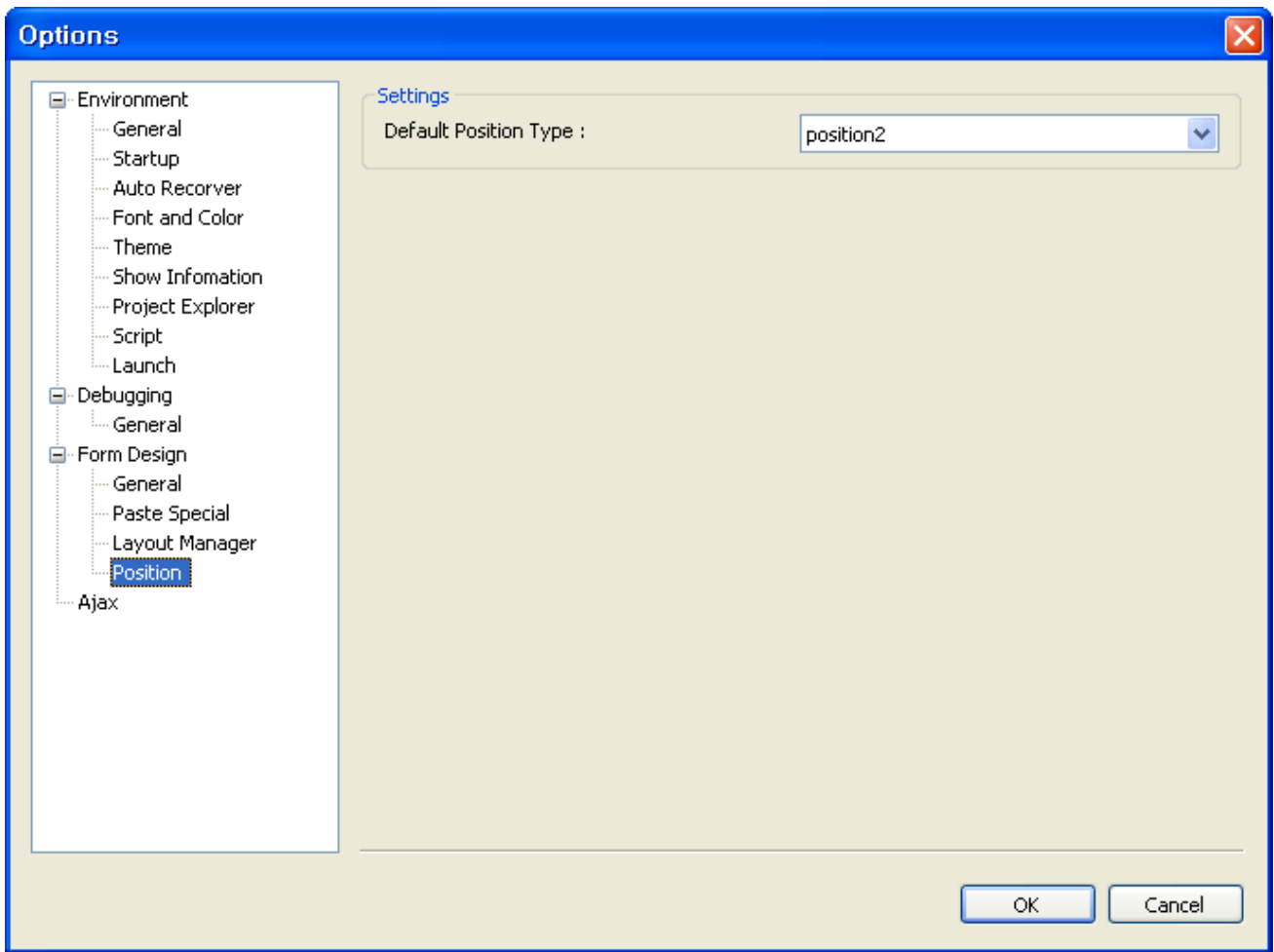
10.3.4 New Project Wizard

Form Design에서 컴포넌트를 생성할 경우, 생성될 컴포넌트의 positiontype의 기본값을 New Project Wizard에서 입력할 수 있도록 기능이 추가되었습니다.

입력된 PositionType은 XPRJ파일에 저장되어 Project단위로 관리됩니다.



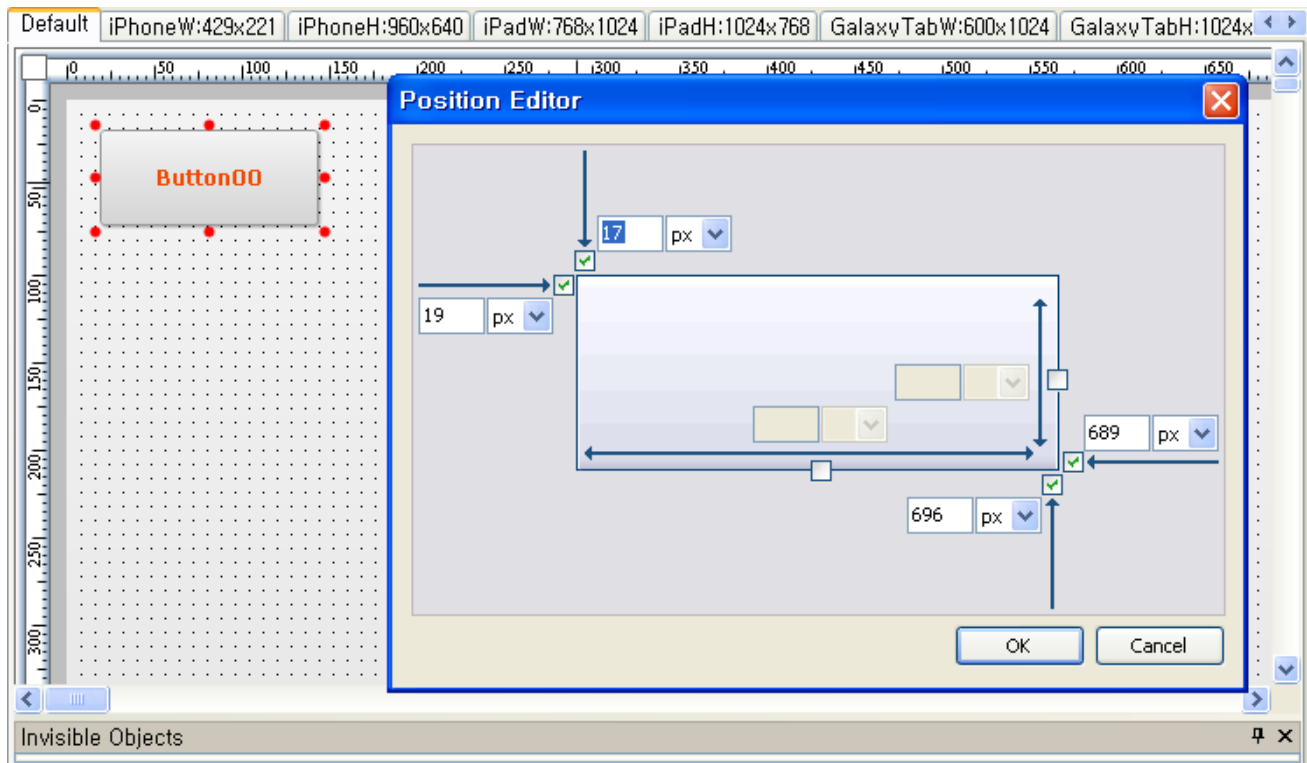
New Project Wizard에서 선택된 PositionType은 Tools > Options에서 변경 가능합니다.



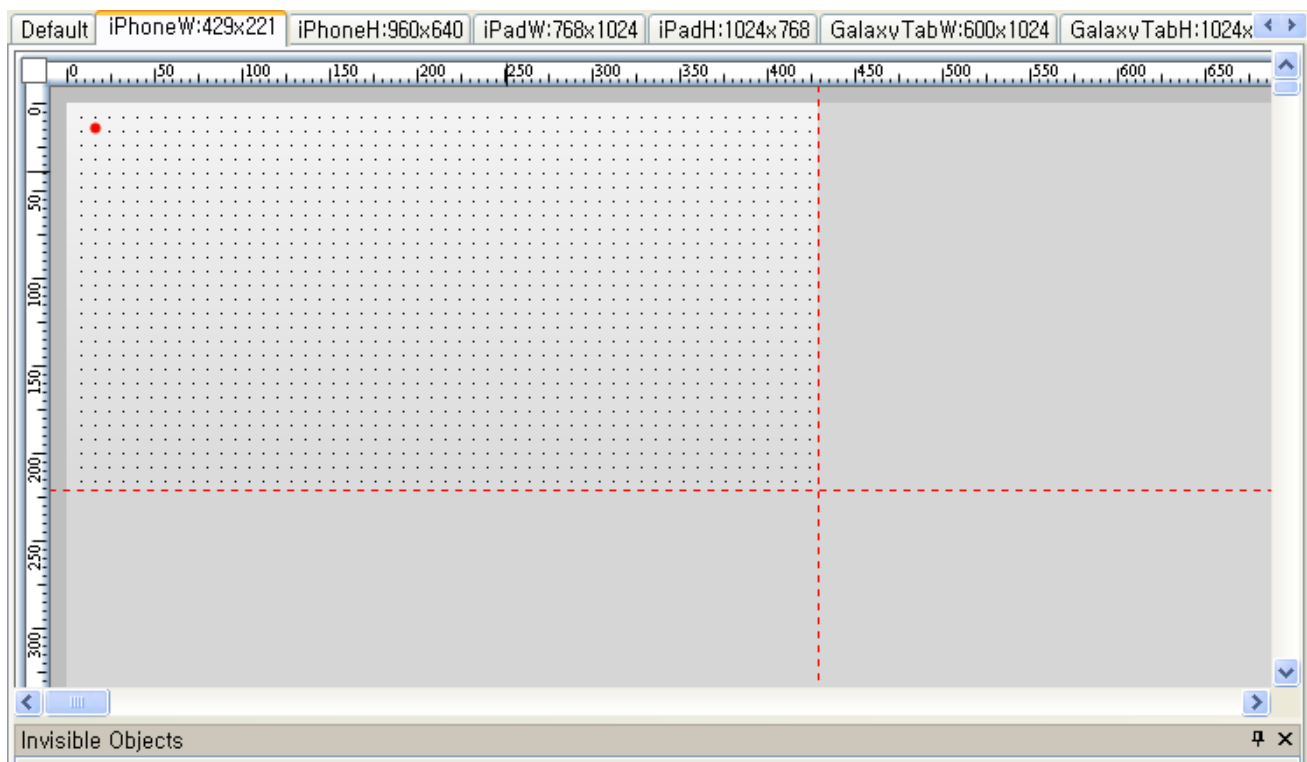
10.3.5 Position2관련 주의사항

Layout Manager 기능을 이용하여 Form을 Design하는 경우, Left, Top, Width, Height를 이용하여 Position2를 지정하지 않으면 추가 Layout에서 컴포넌트가 사라지는 현상이 발생하므로 주의하십시오.

예를 들어, 아래 그림과 같이 Left, Top, Right, Bottom을 이용하여 Position2를 지정하면 Default Layout에서는 Button이 정상으로 보입니다.



하지만, 아래 그림과 같이 추가 Layout에서는 Button이 사라져서 보이지 않게 됩니다.



이 현상이 발생하는 이유는 Form의 Left, Top 위치가 고정되어 있기 때문입니다.

따라서, Layout Manager기능을 이용할 경우 반드시 Left, Top, Width, Height를 이용하여 Position2를 설정하십시오.